Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

CSS3
Styling, Layout and Modern CSS

Complete notes for the Learn Being Forward CSS3 syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

21	3	39	412
modules	levels	minimum days	examples rendered

How to use these notes

These notes follow the **Learn Being Forward CSS3 syllabus** exactly: the same 21 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 8	11 days	Style pages with selectors, colours, units, typography, the box model, backgrounds, positioning and an understanding of the cascade
2. Intermediate	9 to 15	16 days	Build layouts with Flexbox and Grid, make sites responsive, and add transitions, animations and theming with custom properties
3. Advanced	16 to 21	12 days	Use container queries, nesting, layers and modern visual features, organise CSS at scale, use Sass and tooling, and ship fast, accessible projects
Total	1 to 21	39 days	About 7 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every CSS example in these notes was applied to its HTML in a real browser (Chromium 141), and the picture under the code, framed as a browser window, is a screenshot of what that browser displayed. Where an example states a computed value (a specificity winner, a box size, a resolved colour), the value was read back from the browser with JavaScript and is shown in a COMPUTED IN CHROMIUM or BROWSER CONSOLE box. Sass, PostCSS and tooling commands in Modules 19 and 20 show their real or representative terminal output as labelled. Fonts and form controls can differ slightly between browsers and operating systems.

HOW TO READ THE BOXES
Dark boxes with a gold top line are **CSS and HTML code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to CSS	Beginner	1
02	Adding CSS and syntax	Beginner	1
03	Selectors basics	Beginner	1
04	Colours, units and typography	Beginner	2
05	The box model	Beginner	2

Module	Topic	Level	Min days
06	Backgrounds, borders and shadows	Beginner	1
07	Display, positioning and overflow	Beginner	2
08	The cascade, specificity and inheritance	Beginner	1
09	Advanced selectors, pseudo-classes and pseudo-elements	Intermediate	2
10	Flexbox	Intermediate	3
11	CSS Grid	Intermediate	3
12	Responsive design and media queries	Intermediate	3
13	Transforms and transitions	Intermediate	2
14	Animations	Intermediate	2
15	Custom properties and CSS functions	Intermediate	1
16	Modern CSS layout and architecture features	Advanced	3
17	Modern colour, visual effects and scroll features	Advanced	2
18	CSS architecture and methodologies	Advanced	1
19	Preprocessors, frameworks and tooling	Advanced	2
20	Performance, accessibility and debugging	Advanced	1
21	Projects and interview preparation	Advanced	3

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 8

Module 01 · Introduction to CSS 7

1.1 What is CSS? 7
1.2 History of CSS 10
1.3 Why do we need CSS? 11
1.4 What CSS3 brought 13
1.5 Where is CSS used? 16

Module 02 · Adding CSS and syntax 18

2.1 Ways to add CSS 18
2.2 CSS syntax 22
2.3 Browser default styles 24
2.4 Developer tools 26

Module 03 · Selectors basics 28

3.1 Basic selectors 28
3.2 Grouping and combining 31
3.3 Combinators 33
3.4 Attribute selectors 36
3.5 Naming classes 39

Module 04 · Colours, units and typography 41

4.1 Colours 41
4.2 Units 47
4.3 Fonts 50
4.4 Text styling 54
4.5 Styling links and lists 58

Module 05 · The box model 62

5.1 Parts of the box 62
5.2 Sizing 63
5.3 Padding and margin 67
5.4 Borders 71
5.5 Logical properties (introduction) 74
5.6 Inspecting the box model in DevTools 75

Module 06 · Backgrounds, borders and shadows 78

6.1 Backgrounds 78
6.2 Gradients 83
6.3 Shadows 87
6.4 Border images (overview) 89

Module 07 · Display, positioning and overflow 91

7.1 The display property 91
7.2 Normal flow 94
7.3 Positioning 94
7.4 Stacking 99
7.5 Overflow 101
7.6 Floats (legacy layout) 102
7.7 Centring techniques 104

Module 08 · The cascade, specificity and inheritance 108

8.1 The cascade 108
8.2 Specificity 111
8.3 Inheritance 115
8.4 Debugging conflicts 117

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 9 TO 15

Module 09 · Advanced selectors, pseudo-classes and pseudo-elements 121

9.1 User-action pseudo-classes 121
9.2 Structural pseudo-classes 123
9.3 Form pseudo-classes 127
9.4 Logical pseudo-classes 129
9.5 Other pseudo-classes 131
9.6 Pseudo-elements 132
9.7 Practical uses 136

Module 10 · Flexbox 140

10.1 Flexbox basics 140
10.2 Container properties 141
10.3 Item properties 144
10.4 Common flexbox patterns 149
10.5 Practice with Flexbox Froggy 152

Module 11 · CSS Grid 156

11.1 Grid basics 156
11.2 Defining the grid 158
11.3 Placing items 161
11.4 Implicit grid 164
11.5 Alignment in grid 165
11.6 Common grid patterns 167
11.7 Practice with Grid Garden 171

Module 12 · Responsive design and media queries 173

12.1 Responsive design principles 173
12.2 Media queries 176
12.3 User preference queries 179
12.4 Responsive techniques 181
12.5 Testing 184

Module 13 · Transforms and transitions 187

13.1 2D transforms 187
13.2 3D transforms 190
13.3 Transitions 191
13.4 Practical effects 195

Module 14 · Animations 199

14.1 Keyframes 199
14.2 Animation properties 200
14.3 Common animations 203
14.4 Performance and accessibility 206

Module 15 · Custom properties and CSS functions 209

15.1 Custom properties 209

15.2 Maths functions 214
15.3 Other useful functions 216
15.4 Registered properties 217

LEVEL 3 · ADVANCED LEVEL · MODULES 16 TO 21

Module 16 · Modern CSS layout and architecture features 221

16.1 Container queries 221
16.2 CSS nesting 224
16.3 Cascade layers 226
16.4 Scoping 228
16.5 Subgrid 229
16.6 Logical properties in depth 230
16.7 Feature queries 232

Module 17 · Modern colour, visual effects and scroll features 235

17.1 Modern colour 235
17.2 Visual effects 239
17.3 Scrolling 244
17.4 Other modern features 249

Module 18 · CSS architecture and methodologies 253

18.1 Common problems at scale 253
18.2 Methodologies 254
18.3 Organising files 258
18.4 Scoped styles in frameworks (overview) 260

Module 19 · Preprocessors, frameworks and tooling 263

19.1 Sass/SCSS 263
19.2 PostCSS 271
19.3 Build tools 272
19.4 CSS frameworks (overview) 273
19.5 Linting and formatting 275

Module 20 · Performance, accessibility and debugging 279

20.1 Performance 279
20.2 Accessibility in CSS 283
20.3 Cross-browser compatibility 285
20.4 Debugging CSS 286

Module 21 · Projects and interview preparation 290

21.1 Capstone projects 290
21.2 Practice challenges 295
21.3 Interview preparation 296
21.4 Next steps 300

LEVEL 1

Beginner level

Outcome: style pages with selectors, colours, units, typography, the box model, backgrounds, positioning and an understanding of the cascade.

8 MODULES · 11 DAYS MINIMUM

Module 01 · Introduction to CSS	**1 day**
Module 02 · Adding CSS and syntax	**1 day**
Module 03 · Selectors basics	**1 day**
Module 04 · Colours, units and typography	**2 days**
Module 05 · The box model	**2 days**
Module 06 · Backgrounds, borders and shadows	**1 day**
Module 07 · Display, positioning and overflow	**2 days**
Module 08 · The cascade, specificity and inheritance	**1 day**

MODULE 01 · BEGINNER

Introduction to CSS

Minimum duration **1 day** | Level **Beginner**

This module explains what CSS is, why it exists and how it works with HTML, before any styles are written. It covers the definition of Cascading Style Sheets, the separation of content and presentation, what "cascading" means, the history of CSS from Håkon Wium Lie's 1994 proposal through CSS1, CSS2, CSS2.1 and the modular CSS3 to today's yearly feature growth and Baseline support, why we need CSS (branding, layouts for every screen, one stylesheet for many pages, animation without JavaScript, accessibility), what CSS3 brought, and where CSS is used.

IN THIS MODULE

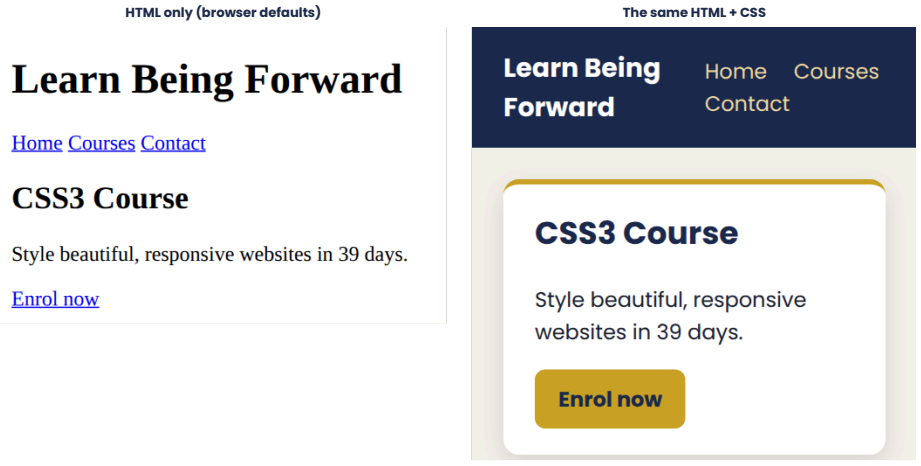
1.1 What is CSS?
1.2 History of CSS
1.3 Why do we need CSS?
1.4 What CSS3 brought
1.5 Where is CSS used?

1.1 What is CSS?

1.1.1 Definition: Cascading Style Sheets, the language that styles and lays out web pages

CSS (**Cascading Style Sheets**) is the language that describes how HTML elements are **presented**: their colours, fonts, sizes, spacing, borders, backgrounds, positions on the page, how they rearrange on different screens, and how they animate. HTML says *what* each piece of content is (a heading, a paragraph, a navigation menu); CSS says *how it looks*. CSS is a **style sheet language**, not a programming language: it has no loops or variables in the traditional sense (custom properties come close, Module 15), but a set of **rules** that the browser applies to matching elements.

The same HTML below is shown twice: first with only the browser's default styles, then with about fifteen lines of CSS. Nothing in the HTML changed.

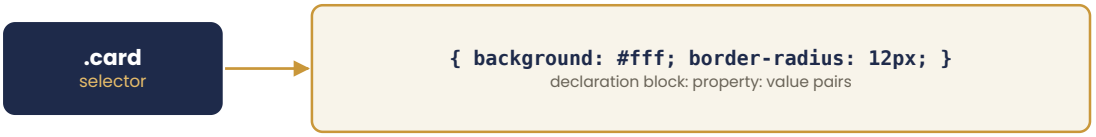


CSS CHANGES ONLY THE PRESENTATION; THE CONTENT AND STRUCTURE STAY THE SAME

STYLE.CSS: THE CSS USED ON THE RIGHT CSS

```css
body { font-family: "Poppins", sans-serif; margin: 0; background: #f4f1ea; color: #1d2433; }
header { background: #1a2a4f; color: #fff; padding: 16px 24px; display: flex; justify-content: space-
between; align-items: center; }
header h1 { font-size: 20px; margin: 0; }
nav a { color: #f0d9a0; margin-left: 16px; text-decoration: none; }
main { padding: 24px; }
.card { background: #fff; border-radius: 12px; padding: 20px 24px; max-width: 360px;
        box-shadow: 0 6px 20px rgb(0 0 0 / 0.12); border-top: 4px solid #c9a227; }
.card h2 { margin-top: 0; color: #1a2a4f; }
.btn { display: inline-block; background: #c9a227; color: #1a2a4f; padding: 10px 18px;
       border-radius: 8px; font-weight: 700; text-decoration: none; }
```

A CSS **rule** has two parts: a **selector** that chooses elements (`.card`, `nav a`) and a **declaration block** of `property: value;` pairs (Module 2 covers the syntax in detail).



A RULE = SELECTOR + DECLARATION BLOCK

1.1.2 Separation of content (HTML) and presentation (CSS)

Before CSS, HTML pages were styled with presentational tags and attributes such as `<font color="red" size="5">`, `<center>`, `bgcolor` and layout tables. Every page repeated the same formatting, a redesign meant editing every file, and the markup became unreadable for people and machines. CSS introduced the principle of **separation of concerns**:

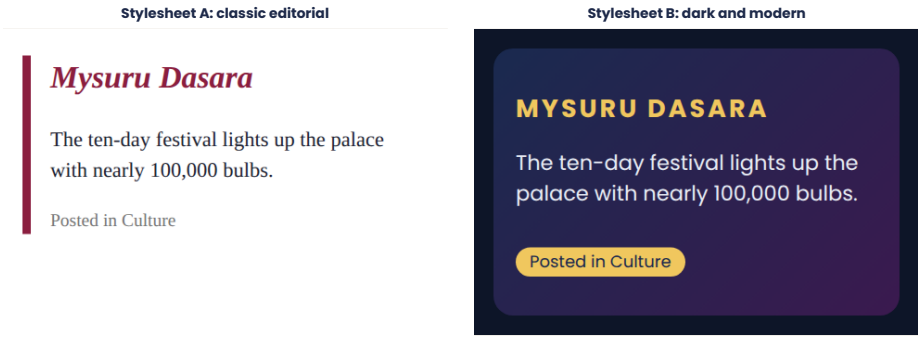
Layer	Language	Responsible for	Example
Content and structure	HTML	what the content is and how it is organised	`<h1>`, `<nav>`, `<button>`
Presentation	CSS	how it looks and where it goes	`h1 { color: navy; }`
Behaviour	JavaScript	what happens when users interact	`button.addEventListener(...)`



THE THREE LAYERS OF THE FRONT END

Benefits: one stylesheet styles a whole site; the HTML stays clean, semantic and accessible; different style sheets can present the same content for screens, print or dark mode; designers and developers can work in parallel; and pages load faster because the stylesheet is downloaded once and cached.

The famous demonstration is the **CSS Zen Garden** (csszengarden.com, 2003): one HTML file, hundreds of completely different designs, each created only by changing the CSS. Here is the same small article with two different stylesheets:



ONE HTML ARTICLE, TWO STYLESHEETS

1.1.3 What "cascading" means

Several rules can target the same element, and they can come from different places: the browser's built-in styles, the user's settings, and the author's (your) stylesheets. The **cascade** is the algorithm that decides which value wins for each property of each element. Styles "cascade" down from many sources, and conflicts are resolved in a fixed order:

1. **Origin and importance**: author styles beat browser defaults; `!important` declarations reverse the order.
2. **Cascade layers** (`@layer`, Module 16): later layers beat earlier ones.
3. **Specificity**: a more specific selector (an id beats a class, which beats a type) wins.
4. **Source order**: if everything else is equal, the rule that appears **last** wins.

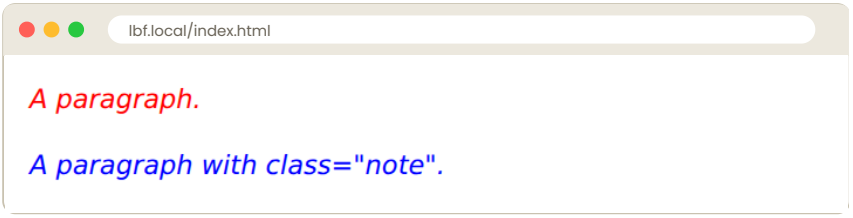
Separately, many properties (colour, font) are **inherited** by child elements from their parents, so setting a font on `body` styles the whole page. Module 8 explains the cascade, specificity and inheritance in full. A first taste:

WHICH COLOUR WINS? CSS

```css
p { color: green; }          /* type selector                    */
.note { color: blue; }      /* class selector: more specific    */
p { color: red; }           /* later, but still only a type     */
body { font-style: italic; } /* inherited by every paragraph    */
```

INDEX.HTML HTML

```html
<p>A paragraph.</p>
<p class="note">A paragraph with class="note".</p>
```

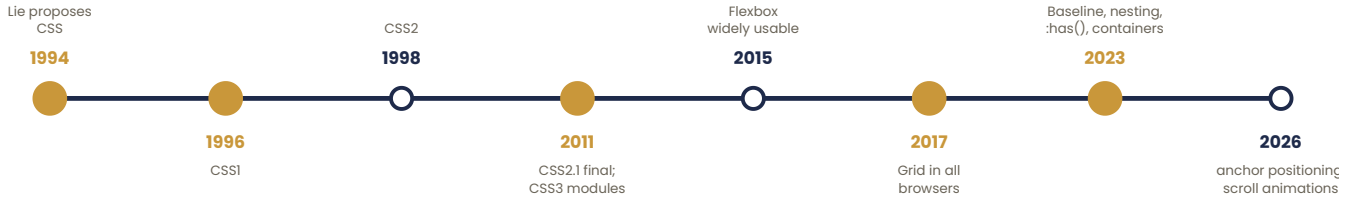


THE CLASS RULE BEATS BOTH TYPE RULES; THE ITALIC STYLE IS INHERITED FROM BODY

COMPUTED IN CHROMIUM

```
paragraph 1: color = rgb(255, 0, 0), font-style = italic
paragraph 2: color = rgb(0, 0, 255), font-style = italic
```

1.2 History of CSS



A TIMELINE OF CSS

1.2.1 Håkon Wium Lie's proposal (1994) and CSS1 (1996)

On 10 October 1994, **Håkon Wium Lie**, working with Tim Berners-Lee at CERN, published "Cascading HTML style sheets", a proposal for a style language in which both authors and readers could influence presentation, with their preferences "cascading" together. **Bert Bos** was working on a similar language (Stream-based Style Sheet Proposal) and joined him; together they shaped CSS. The new **World Wide Web Consortium (W3C)** published **CSS level 1** as a Recommendation in **December 1996**: fonts, colours, text properties, backgrounds, margins, borders, padding and simple selectors. Internet Explorer 3 (1996) was the first commercial browser with (partial) CSS support; full, correct CSS1 support took browsers years.

1.2.2 CSS2 and CSS2.1

CSS2 (May 1998) added positioning (absolute, relative, fixed), z-index, media types (screen, print), more selectors (child, adjacent sibling, attribute), generated content with `::before` / `::after`, and tables. Browsers implemented it inconsistently (the "browser wars" era of Internet Explorer 5/6 and Netscape), so the W3C produced **CSS2.1**: a cleaned-up revision that removed poorly supported features and fixed errors, matching what browsers actually did. CSS2.1 became a Recommendation in June 2011 and is still the foundation of the box model, the visual formatting model and the cascade.

1.2.3 CSS3 and the move to independent modules

CSS2 was a single large specification, which made progress slow: the whole document had to be finished before anything was official. "CSS3" therefore split CSS into **independent modules** (Selectors, Backgrounds and Borders, Color, Fonts, Media Queries, Flexbox, Grid, Transitions, Animations…), each with its own **level** number and maturity. Selectors Level 3 and Media Queries became Recommendations in 2011-2012; Flexbox was widely usable by about 2015 and Grid arrived in all major browsers in 2017. Because modules evolve separately (Selectors Level 4, Color Level 5…), there will never be a "CSS4"; "CSS3" is now simply the name for modern CSS as a whole.

Module (examples)	Current level	Brings
Selectors	Level 4	:is(), :where(), :has(), :focus-visible
Color	Levels 4 and 5	oklch(), color-mix(), relative colours
Flexible Box Layout	Level 1	Flexbox
Grid Layout	Level 2	Grid and subgrid
Media Queries	Levels 4 and 5	range syntax, prefers-color-scheme
Cascading and Inheritance	Levels 5 and 6	@layer, @scope
Containment	Level 3	container queries

1.2.4 Modern CSS: yearly feature growth and "Baseline" browser support

Today, Chrome, Edge, Firefox and Safari release updates every four to six weeks, and CSS gains major features every year. The browser makers coordinate through the yearly **Interop** project to make the same features work identically. Recent additions include native nesting, `:has()`, container queries, cascade layers, subgrid, `oklch()` colours, `color-mix()`, view transitions, scroll-driven animations, anchor positioning and `@starting-style`.

To tell whether a feature is safe to use, MDN, Can I Use and web.dev show a **Baseline** label: **Newly available** means it works in the latest version of all the core browsers (Chrome, Edge, Firefox, Safari on desktop and mobile); **Widely available** means it has been newly available for 30 months, so almost all users have it.

Baseline: widely available

Flexbox, Grid, custom properties, clamp(), aspect-ratio, gap. Use freely.

Baseline: newly available

Nesting, :has(), container queries, @layer, subgrid, oklch(). Use, with a fallback for old browsers if needed.

Limited availability

Some anchor positioning or scroll-driven animation features. Use as progressive enhancement.

1.3 Why do we need CSS?

1.3.1 Visual design and branding

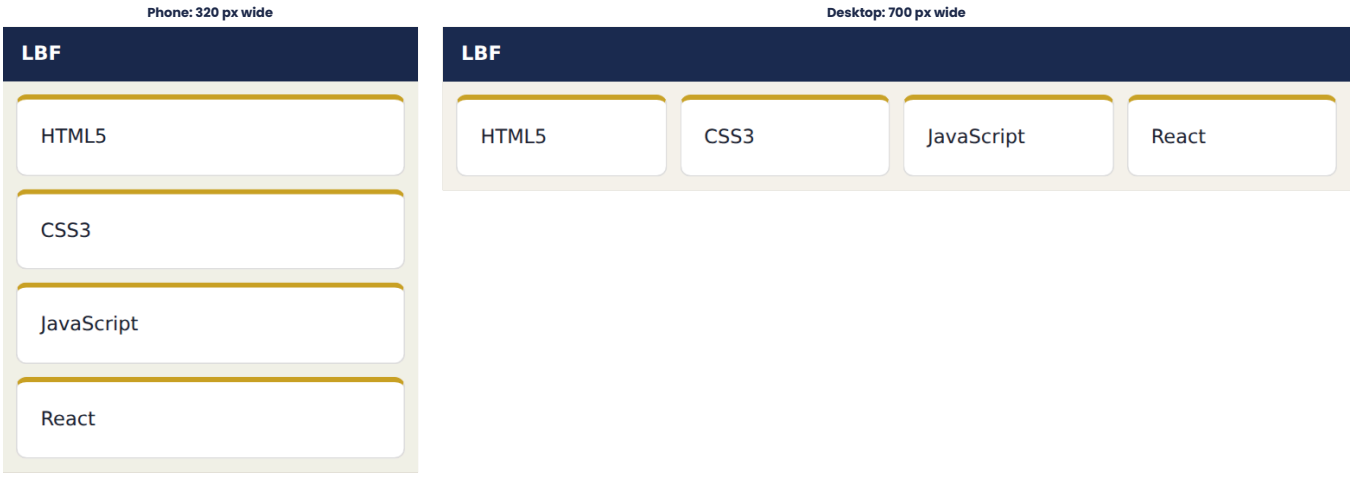
Every brand has an identity: colours, typefaces, spacing, shapes and tone. CSS turns a designer's mock-up into a real interface, keeps the brand consistent on every page, and makes small changes (a new brand colour) a one-line edit. Good visual design is not decoration: it guides attention, builds trust and makes information easier to find.

1.3.2 Layouts that work on every screen size

Visitors arrive on phones 320 pixels wide, tablets, laptops and 4K monitors. CSS layout systems (Flexbox and Grid) and media and container queries let one HTML page rearrange itself for each screen. The same page below is shown at phone and desktop widths using one stylesheet:

ONE RESPONSIVE STYLESHEET — CSS

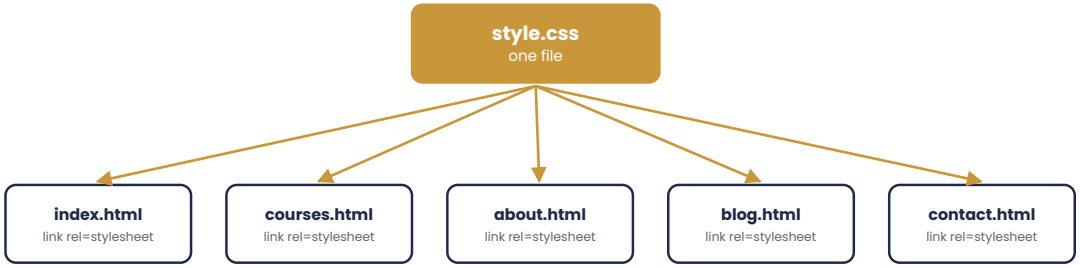
```
header { background: #1a2a4f; color: #fff; padding: 10px 14px; font-weight: 700; }
.grid { display: grid; gap: 10px; padding: 10px; grid-template-columns: 1fr; }
@media (min-width: 600px) { .grid { grid-template-columns: repeat(4, 1fr); } }
article { background: #fff; border: 1px solid #ddd; border-top: 4px solid #c9a227; padding: 18px;
border-radius: 8px; }
body { background: #f4f1ea; margin: 0; }
```



THE SAME PAGE AND STYLESHEET AT TWO WIDTHS: ONE COLUMN ON THE PHONE, FOUR COLUMNS ON THE DESKTOP

1.3.3 One stylesheet controlling many pages

A site with 200 pages links every page to the same `style.css`. Change one rule and all 200 pages update; the browser downloads the file once and reuses it from its cache as the user moves between pages.



EVERY PAGE LINKS TO THE SAME STYLESHEET

1.3.4 Animations and interactivity without JavaScript

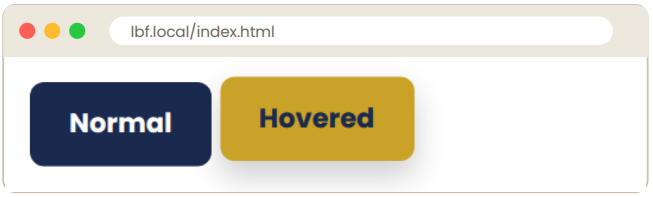
Hover effects, smooth colour changes, dropdown menus, accordions, carousels with scroll snapping, loading spinners and complex keyframe animations can all be done in CSS, often more smoothly than with JavaScript because the browser can run them on the GPU. Modules 13, 14 and 17 cover transitions, animations and scroll-driven effects.

A HOVER EFFECT IN PURE CSS CSS

```
.btn { background: #1a2a4f; color: #fff; border: 0; padding: 12px 22px; border-radius: 8px;
       font: 600 15px "Poppins", sans-serif; transition: transform .2s, background .2s, box-shadow
.2s; }
.btn:hover { background: #c9a227; color: #1a2a4f; transform: translateY(-3px);
             box-shadow: 0 8px 18px rgb(0 0 0 / .2); }
```

INDEX.HTML HTML

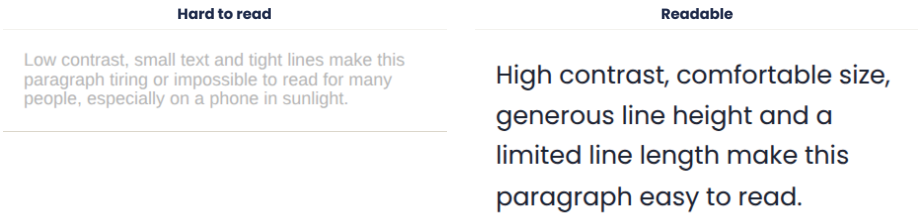
```
<button class="btn">Normal</button>
<button class="btn" id="h">Hovered</button>
```



THE SECOND BUTTON IS BEING HOVERED: IT LIFTS, CHANGES COLOUR AND CASTS A SHADOW, ANIMATED BY THE TRANSITION

1.3.5 Accessibility and readability

CSS directly affects whether people can use a site: text size and line spacing, colour contrast, visible focus outlines for keyboard users, large enough touch targets, layouts that survive 200 percent zoom, honouring the user's "reduce motion" and "dark mode" preferences, and hiding decorative content while keeping it available to screen readers. Module 20 covers accessibility in CSS in depth.



THE SAME KIND OF TEXT WITH POOR AND GOOD TYPOGRAPHIC CSS

1.4 What CSS3 brought

The CSS3 modules replaced most of the tricks of the 2000s (images for every rounded corner, spacer GIFs, Flash animations, float-based layouts, JavaScript for hover menus) with native features. The following demo shows six of them in one screenshot.

1.4.1 Rounded corners, shadows and gradients

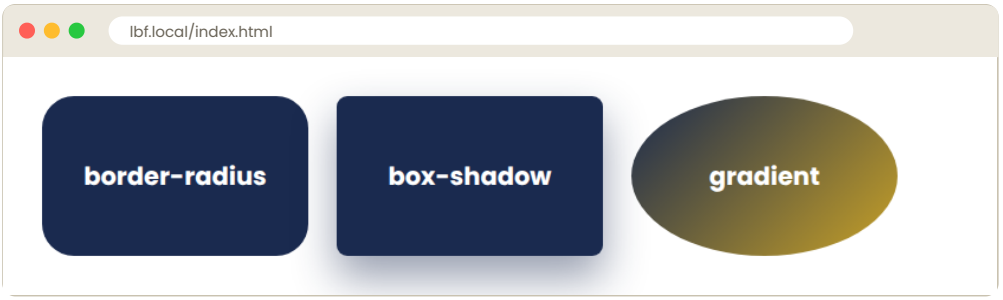
`border-radius`, `box-shadow`, `text-shadow` and `linear-gradient()` / `radial-gradient()` replaced sliced background images. They scale to any size and cost no downloads.

ROUNDED CORNERS, SHADOWS AND GRADIENTS CSS

```css
.box { width: 150px; height: 90px; display: inline-grid; place-items: center; margin: 8px;
       color: #fff; font: 600 14px "Poppins", sans-serif; }
.round { background: #1a2a4f; border-radius: 18px; }
.shadow { background: #1a2a4f; border-radius: 6px; box-shadow: 0 12px 24px rgb(26 42 79 / .45); }
.gradient { background: linear-gradient(135deg, #1a2a4f, #c9a227); border-radius: 50%; }
```

INDEX.HTML HTML

```html
<div class="box round">border-radius</div><div class="box shadow">box-shadow</div><div class="box
gradient">gradient</div>
```



HOW THE BROWSER DISPLAYS IT

1.4.2 Flexbox and Grid layouts

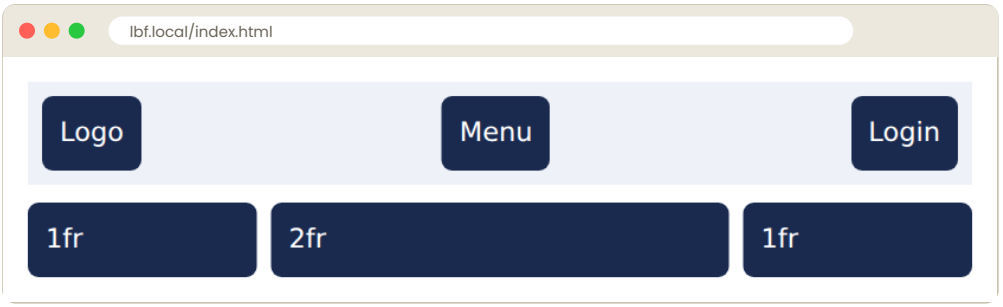
Flexbox (one dimension: a row or a column) and **Grid** (two dimensions: rows and columns) are real layout systems. They replaced float hacks and table layouts, and made centring, equal-height columns and reordering trivial (Modules 10 and 11).

FLEXBOX AND GRID IN FOUR LINES EACH CSS

```
.flex { display: flex; justify-content: space-between; align-items: center; background: #eef1f8;
padding: 8px; }
.grid { display: grid; grid-template-columns: 1fr 2fr 1fr; gap: 8px; margin-top: 10px; }
.flex div, .grid div { background: #1a2a4f; color: #fff; padding: 10px; border-radius: 6px; }
```

INDEX.HTML HTML

```
<div class="flex"><div>Logo</div><div>Menu</div><div>Login</div></div>
<div class="grid"><div>1fr</div><div>2fr</div><div>1fr</div></div>
```



HOW THE BROWSER DISPLAYS IT

1.4.3 Media queries and responsive design

`@media` rules apply styles only when conditions are true: the viewport is at least 768 pixels wide, the device is in landscape, the user prefers dark mode, the page is being printed. Ethan Marcotte named the approach **responsive web design** in 2010 (Module 12).

1.4.4 Transitions, transforms and animations

`transform` moves, rotates, scales and skews elements in 2D and 3D; `transition` animates property changes smoothly; `@keyframes` with `animation` defines multi-step animations (Modules 13 and 14).

FOUR TRANSFORMS CSS

```
.t { display: inline-block; width: 90px; height: 60px; margin: 16px; background: #c9a227; border-
radius: 8px; }
.rotate { transform: rotate(15deg); } .scale { transform: scale(1.25); } .skew { transform:
skewX(-15deg); }
.three { transform: perspective(300px) rotateY(40deg); }
```

INDEX.HTML HTML

```
<div class="t rotate"></div><div class="t scale"></div><div class="t skew"></div><div class="t three">
</div>
```



ROTATE, SCALE, SKEW AND A 3D ROTATEY WITH PERSPECTIVE

1.4.5 Custom properties (variables)

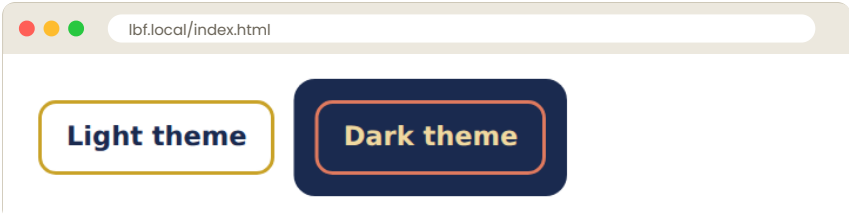
Custom properties (`--brand: #1a2a4f;`, read with `var(--brand)`) store values once and reuse them everywhere. Unlike preprocessor variables they are live in the browser: they inherit, can change in media queries or with JavaScript, and make theming and dark mode simple (Module 15).

ONE COMPONENT, TWO THEMES, CHANGED ONLY THROUGH VARIABLES CSS

```
:root { --brand: #1a2a4f; --accent: #c9a227; --radius: 10px; }
.dark { --brand: #f0d9a0; --accent: #e07a5f; }
.tag { display: inline-block; padding: 8px 14px; margin: 6px; border-radius: var(--radius);
       color: var(--brand); border: 2px solid var(--accent); font-weight: 600; }
.dark { background: #1a2a4f; padding: 6px; border-radius: 12px; display: inline-block; }
```

INDEX.HTML HTML

```
<span class="tag">Light theme</span> <span class="dark"><span class="tag">Dark theme</span></span>
```



HOW THE BROWSER DISPLAYS IT

1.4.6 Web fonts

`@font-face` lets a page download and use any font, instead of only those installed on the visitor's computer. Services such as Google Fonts host thousands of free fonts; variable fonts put every weight in a single file (Module 4).

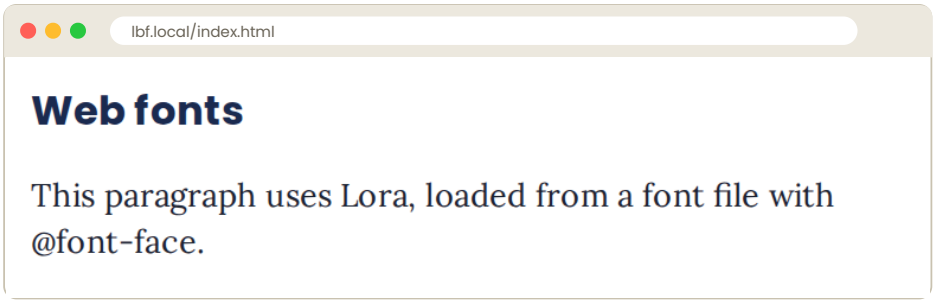
LOADING FONTS WITH @FONT-FACE CSS

```
@font-face { font-family: "LBF Lora"; src: url("fonts/Lora-Variable.ttf"); font-weight: 400 700; }
@font-face { font-family: "LBF Poppins"; src: url("fonts/Poppins-Bold.ttf"); font-weight: 700; }
h2 { font-family: "LBF Poppins", sans-serif; color: #1a2a4f; margin: 0; }
p { font-family: "LBF Lora", serif; font-size: 18px; }
```

INDEX.HTML

HTML

```
<h2>Web fonts</h2><p>This paragraph uses Lora, loaded from a font file with @font-face.</p>
```



HOW THE BROWSER DISPLAYS IT

1.5 Where is CSS used?

1.5.1 Every website and web app

Every website styles its HTML with CSS: news sites, online shops, banking portals, government services, and applications such as Gmail, YouTube, Google Docs, Figma, Notion and Zomato. Whatever the back-end language (Java, Python, PHP, Node.js) or front-end framework (React, Angular, Vue), the browser ultimately receives HTML, CSS and JavaScript.

1.5.2 Email templates

HTML emails (newsletters, order confirmations, OTP mails) are styled with CSS, but email clients support only an old subset: no external stylesheets in many clients, limited or no Flexbox and Grid, and Outlook on Windows uses Microsoft Word to render. Email CSS is therefore written inline, with tables for layout. Module 21 gives an overview.

1.5.3 Mobile and desktop apps built with web technology

Many desktop apps are web pages inside a browser engine: Visual Studio Code, Slack, Discord, Microsoft Teams and Postman are built with **Electron**; mobile apps built with Ionic, Capacitor or Cordova and Progressive Web Apps are styled with CSS too. Even React Native and Flutter borrow CSS ideas (Flexbox for layout, style objects with CSS-like names).

1.5.4 Frameworks and libraries (Bootstrap, Tailwind CSS)

CSS frameworks package ready-made styles. **Bootstrap** provides a grid system and pre-styled components (buttons, navbars, cards, modals); **Tailwind CSS** provides small utility classes (`p-4`, `text-center`, `bg-blue-600`) that you combine in HTML. Both generate plain CSS, so understanding CSS itself is essential to using them well (Module 19, and the LBF TailwindCSS course).

THE SAME BUTTON WITH TWO FRAMEWORKS

HTML

```
<!-- Bootstrap: component classes -->
<button class="btn btn-primary btn-lg">Enrol</button>

<!-- Tailwind CSS: utility classes -->
<button class="rounded-lg bg-blue-600 px-5 py-3 font-semibold text-white hover:bg-blue-
700">Enrol</button>
```

PRACTICE CHECKPOINT

Explain in your own words the difference between HTML and CSS, and what "cascading" means.

Question	Model answer
Difference between HTML and CSS	HTML marks up content and gives it meaning and structure: headings, paragraphs, links, forms. CSS describes how that content is presented: colour, typography, spacing, layout, responsiveness and animation. The same HTML can be given completely different looks by different CSS, and the HTML stays readable by browsers, search engines and screen readers.
What "cascading" means	Several style rules from different sources (browser defaults, user settings, the author's stylesheets) can apply to the same element. The cascade is the set of rules that decides which declaration wins for each property: origin and importance first, then cascade layers, then specificity, then source order (the last one wins). Inheritance additionally passes some property values (such as colour and font) from parents to children.

MODULE 02 · BEGINNER

Adding CSS and syntax

Minimum duration **1 day** | Level **Beginner**

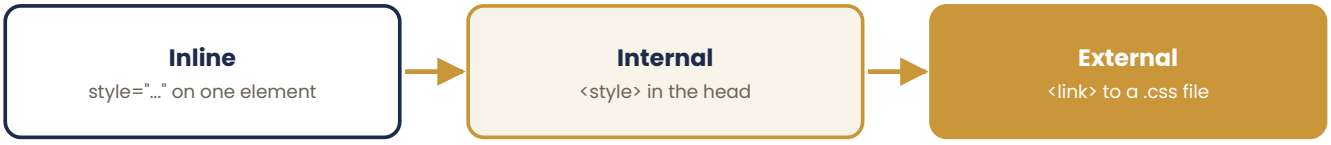
By the end of this module the learner can connect CSS to HTML in all three ways and write correct CSS rules. It covers inline styles, internal style elements, external stylesheets with link, @import and why to avoid it, which method to use, the anatomy of a rule set (selector, declaration block, property, value), multiple declarations and semicolons, comments, whitespace and formatting, how the browser parses invalid CSS, browser default (user-agent) styles, CSS resets versus normalize, the box-sizing reset, and using DevTools to inspect, edit and read computed styles.

IN THIS MODULE

2.1 Ways to add CSS
2.2 CSS syntax
2.3 Browser default styles
2.4 Developer tools

2.1 Ways to add CSS

There are three ways to attach CSS to HTML, plus `@import` for loading one stylesheet from another. All three can be combined on one page; the cascade (Module 8) decides the winner when they conflict.



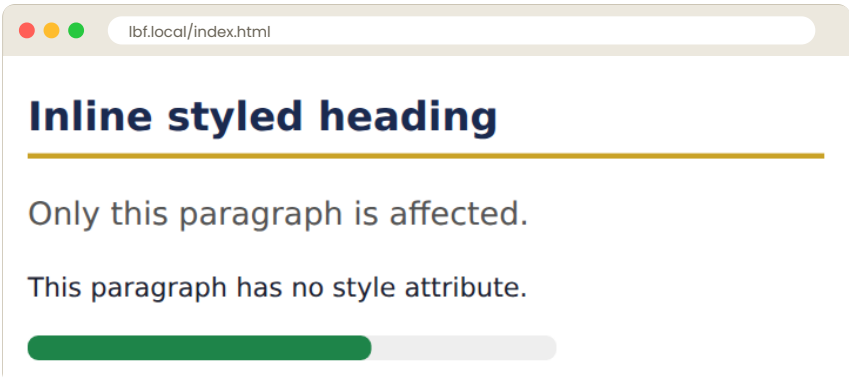
THREE WAYS TO ADD CSS, FROM LEAST TO MOST REUSABLE

2.1.1 Inline styles with the style attribute

The `style` attribute holds declarations for **that one element**: no selector and no braces, just `property: value;` pairs. It is quick for experiments, for values computed by JavaScript or a template engine (a progress bar width, a user's chosen colour), and in HTML emails. It is a poor choice for normal styling: it cannot be reused, cannot use media queries, pseudo-classes (`:hover`) or pseudo-elements, mixes presentation into content, and has very high priority so it is hard to override.

INDEX.HTML — HTML

```
<h2 style="color: #1a2a4f; border-bottom: 3px solid #c9a227; padding-bottom: 4px;">Inline styled
heading</h2>
<p style="font-size: 18px; color: #555;">Only this paragraph is affected.</p>
<p>This paragraph has no style attribute.</p>
<div style="background: #eee; border-radius: 6px; height: 14px; width: 300px;">
  <div style="background: #1e8449; height: 100%; width: 65%; border-radius: 6px;"></div>
</div>
```



INLINE STYLES, INCLUDING A PROGRESS BAR WHOSE WIDTH A TEMPLATE WOULD COMPUTE

2.1.2 Internal styles with <style>

A `<style>` element, normally inside `<head>`, contains full CSS rules that apply to **that page only**. It is useful for single-page demos, prototypes, pages with unique styles, and **critical CSS** inlined for speed (Module 20). Its rules cannot be shared with other pages, and they are downloaded again with every page.

INTERNAL.HTML HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Internal styles</title>
  <style>
    body { font-family: system-ui, sans-serif; margin: 16px; }
    h1 { color: #1a2a4f; }
    p { line-height: 1.6; max-width: 40ch; }
    .highlight { background: #fff3c4; padding: 2px 4px; }
  </style>
</head>
<body>
  <h1>Internal stylesheet</h1>
  <p>All rules live in the <span class="highlight">style element</span> in the head of this page.</p>
</body>
</html>
```



RULES FROM THE STYLE ELEMENT APPLY TO THE WHOLE PAGE

2.1.3 External stylesheets with <link>

An **external stylesheet** is a plain text file with a `.css` extension, connected in the head with `<link rel="stylesheet" href="css/style.css">`. This is the standard method for real websites: one file styles every page, the browser caches it, HTML and CSS stay separate, and teams can work on them independently. A page can link several stylesheets; they apply in the order they are linked.

INDEX.HTML: LINKING STYLESHEETS HTML

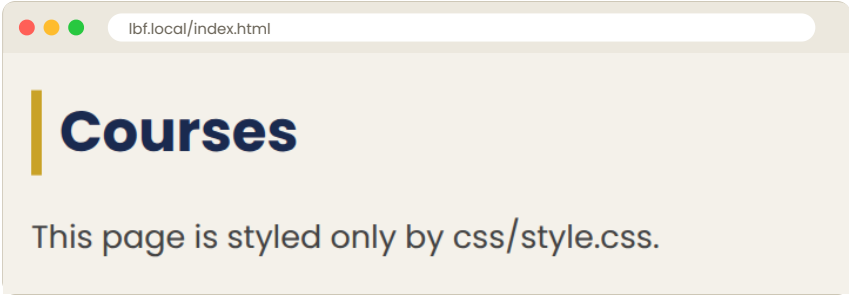
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>LBF Courses</title>
  <link rel="stylesheet" href="css/reset.css">
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/print.css" media="print">
</head>
```

TYPICAL FOLDER STRUCTURE text

```
my-site/
├── index.html
├── courses.html
└── css/
    ├── reset.css
    ├── style.css
    └── print.css
```

INDEX.HTML HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>External stylesheet</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Courses</h1>
  <p class="lead">This page is styled only by css/style.css.</p>
</body>
</html>
```



THE PAGE AFTER THE BROWSER DOWNLOADED AND APPLIED CSS/STYLE.CSS

CSS/STYLE.CSS CSS

```css
body { font-family: "Poppins", sans-serif; background: #f4f1ea; margin: 16px; }
h1 { color: #1a2a4f; border-left: 6px solid #c9a227; padding-left: 10px; }
.lead { font-size: 18px; color: #444; }
```

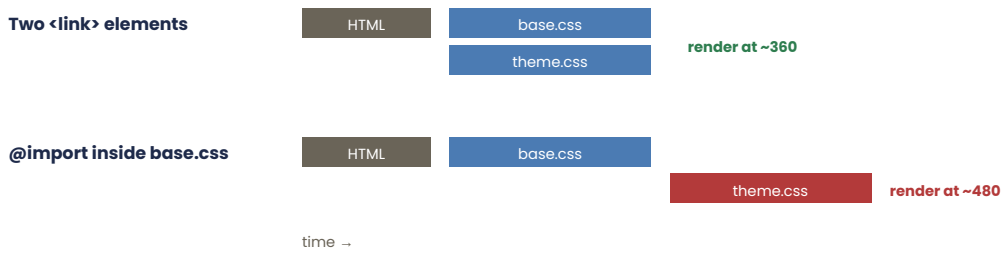
WATCH OUT

The `href` is a URL relative to the HTML file (HTML Module 4). The most common beginner bug is a wrong path: `href="style.css"` when the file is in a `css` folder. Open DevTools > Network or Console: a 404 error for the CSS file confirms it. Also remember that URLs inside a CSS file (images, fonts) are relative to the **CSS file**, not the page.

2.1.4 @import (and why to avoid it for performance)

`@import url("other.css");` loads one stylesheet from inside another (or from a style element). It must appear at the top of the file, before any other rules except `@charset` and `@layer` statements. It can include a media condition or a layer: `@import url("print.css") print;`, `@import url("theme.css") layer(theme);`.

The problem is **performance**: the browser discovers an imported file only after downloading and reading the file that imports it, so requests happen **one after another** (a request chain) instead of in parallel. Several `<link>` elements, or a build tool that combines files (Module 19), avoid this.



@IMPORT CREATES A REQUEST CHAIN; SEPARATE LINK ELEMENTS DOWNLOAD IN PARALLEL (SCHEMATIC)

2.1.5 Which method to use and why

Method	Scope	Reusable	Cached	Use for
Inline `style=""`	one element	no	no	dynamic values from JS or templates, emails, quick tests
Internal `<style>`	one page	no	with the page	demos, one-off pages, critical CSS
External `<link>`	every page that links it	yes	yes	**all normal site styling**
`@import`	the importing sheet	yes	yes	cascade layers or build-time combining; avoid at run time

When the same property is set by several methods, the **inline style wins** over rules in internal and external stylesheets (it behaves like the highest specificity), and between internal and external sheets the one that comes **later in the document** wins if specificity is equal. Chromium confirms the result:

COMPUTED IN CHROMIUM

```
#a color: rgb(255, 165, 0)
#b color: rgb(128, 0, 128)
```

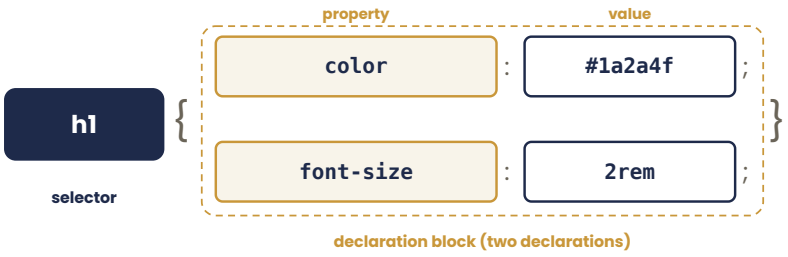
Order in the document: the demo stylesheet (green), then the linked sheet (blue), then the style element (orange). #a gets the last one, orange (255, 165, 0); #b gets its inline purple (128, 0, 128).

2.2 CSS syntax

2.2.1 Rule sets: selector, declaration block, property, value

A CSS file is a list of **rules** (also called rule sets) and **at-rules** (beginning with `@`, such as `@media`, `@font-face`, `@import`). A rule set has:

- a **selector** that chooses the elements (`h1`, `.card`, `nav a`);
- a **declaration block** in curly braces `{ }`;
- inside it, **declarations**: a **property** name, a colon, a **value**, and a semicolon.



ANATOMY OF A CSS RULE

Property names are fixed keywords defined by the CSS specifications (there are over 500), always lower-case by convention and not case-sensitive. Values can be keywords (`bold`, `auto`, `flex`), numbers with units (`16px`, `1.5rem`, `50%`), colours (`#1a2a4f`, `rgb(26 42 79)`), strings in quotes (`"Poppins"`), URLs (`url("img.png")`) or functions (`calc(100% - 2rem)`). Some properties accept several space-separated values (`margin: 10px 20px;`) or comma-separated lists (`font-family: "Poppins", sans-serif;`).

2.2.2 Multiple declarations and semicolons

Each declaration ends with a **semicolon**. The semicolon after the last declaration is optional, but always writing it prevents bugs when you add another line later. A missing semicolon in the middle glues two declarations together into one invalid value, and the browser drops **both**.

A MISSING SEMICOLON CSS

```css
.ok     { color: #1a2a4f; background: #fff3c4; padding: 6px; }
.broken { color: #1a2a4f   /* missing semicolon here */
          background: #fff3c4; padding: 6px; }
```

INDEX.HTML HTML

```html
<p class="ok">Correct declarations</p>
<p class="broken">Missing semicolon</p>
```



THE SECOND PARAGRAPH LOST ITS COLOUR AND ITS BACKGROUND: "#1A2A4F BACKGROUND: #FFF3C4" IS ONE INVALID VALUE

The browser applies a strict but forgiving error-handling rule: it **ignores** any declaration it cannot understand (a misspelt property, an invalid value, an unsupported new feature) and carries on with the next one. This is what makes it safe to write fallbacks:

INVALID DECLARATIONS ARE SKIPPED, NOT FATAL CSS

```css
.title {
  colr: red;                        /* misspelt property: ignored             */
  font-size: 20pxx;                 /* invalid value: ignored                 */
  background: #1a2a4f;              /* fallback for old browsers              */
  background: oklch(30% 0.08 265);  /* newer browsers use this; old ones skip it */
}
```

COMPUTED IN CHROMIUM

```
color: rgb(29, 36, 51)   (default: colr was ignored)
font-size: 22.5px   (default h2 size: 20pxx was ignored)
background-color: oklch(0.3 0.08 265)   (oklch value used)
```

2.2.3 Comments /* */

Comments start with `/*` and end with `*/`; they can span many lines and are ignored by the browser. CSS has **no** single-line `//` comment (that is Sass and JavaScript): a `//` in CSS makes the following rule invalid. Use comments to label sections, explain non-obvious values ("matches header height") and temporarily disable code. Comments cannot be nested.

COMMENT STYLES CSS

```css
/* ========================================
   Header
   ======================================== */
.site-header {
  height: 64px;          /* must match .hero padding-top */
  /* background: red; */ /* temporarily disabled         */
}
```

2.2.4 Whitespace and formatting

Spaces, tabs and new lines between tokens are ignored, so all of these are identical to the browser. Teams pick one readable style and enforce it with Prettier (Module 19): one declaration per line, two-space indentation, a space after the colon, a blank line between rules, lower-case hex colours.

THREE FORMATTING STYLES, ONE MEANING CSS

```css
/* Expanded (recommended for source code) */
.card {
  padding: 16px;
  border-radius: 8px;
}

/* Compact: one rule per line */
.card { padding: 16px; border-radius: 8px; }

/* Minified (generated by build tools for production) */
.card{padding:16px;border-radius:8px}
```

NOTE

Whitespace does matter inside values and selectors: `margin: 10px 20px` is two values but `10px20px` is invalid; `div p` (descendant) is a different selector from `div.p` (a div with class p) and `divp` (an element called divp).

2.3 Browser default styles

2.3.1 User-agent stylesheets

Even with no CSS at all, headings are big and bold, links are blue and underlined, and the body has an 8-pixel margin. These styles come from the browser's built-in **user-agent stylesheet**. Each browser has its own (they are similar but not identical), and they are the lowest-priority origin in the cascade, so any author rule overrides them. A few of Chromium's defaults, read back from the browser:

COMPUTED IN CHROMIUM

```
body    margin: 8px; fontFamily: "Times New Roman"; fontSize: 16px
h1      fontSize: 32px; fontWeight: 700; marginTop: 21.44px
h2      fontSize: 24px; marginTop: 19.92px
p       marginTop: 16px; marginBottom: 16px
ul      paddingLeft: 40px; listStyleType: disc
a       color: rgb(0, 0, 238); textDecorationLine: underline
button  fontSize: 13.3333px; paddingLeft: 6px
```

h1 is 2em (32px with the 16px default), h2 is 1.5em, paragraphs have 1em top and bottom margins, lists have a 40px left padding. You can see the full sheet in Chromium's source (html.css) or in DevTools: rules marked "user agent stylesheet".

2.3.2 CSS resets vs normalize

Because defaults differ slightly between browsers and rarely match a design, projects start with one of two strategies:

	Reset (e.g. Eric Meyer's reset, modern minimal resets)	Normalize (normalize.css)
Idea	remove almost all default styling, then style everything yourself	keep useful defaults, fix only the inconsistencies between browsers
Headings and lists after it	same size as body text, no bullets	still bigger and bulleted
Size	very small	a few kilobytes
Used by	many design systems; Tailwind's "Preflight" is a reset	Bootstrap's "Reboot" builds on it

RESET.CSS CSS

```css
/* A modern minimal reset (based on widely used community resets) */
*, *::before, *::after { box-sizing: border-box; }
* { margin: 0; }
html { -webkit-text-size-adjust: 100%; }
body { line-height: 1.5; -webkit-font-smoothing: antialiased; }
img, picture, video, canvas, svg { display: block; max-width: 100%; }
input, button, textarea, select { font: inherit; }
p, h1, h2, h3, h4, h5, h6 { overflow-wrap: break-word; }
h1, h2, h3 { text-wrap: balance; }
```



MARGINS REMOVED, LINE HEIGHT INCREASED, FORM CONTROLS INHERIT THE PAGE FONT

2.3.3 box-sizing: border-box reset

By default, `width` sets the width of the **content** only; padding and border are added on top (the content-box model). A 300px-wide box with 20px padding and a 2px border is really 344px wide. Almost every modern project therefore starts with the rule below, which makes `width` include padding and border (Module 5 explains it fully).

THE BORDER-BOX RESET — CSS

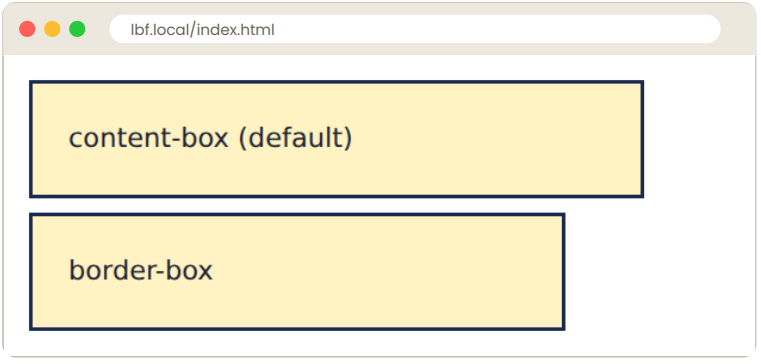
```css
*, *::before, *::after {
  box-sizing: border-box;
}
```

THE SAME WIDTH, TWO BOX MODELS — CSS

```css
.box { width: 300px; padding: 20px; border: 2px solid #1a2a4f; margin-bottom: 8px; background:
#fff3c4; }
.border-box { box-sizing: border-box; }
```

INDEX.HTML — HTML

```html
<div class="box">content-box (default)</div>
<div class="box border-box">border-box</div>
```



HOW THE BROWSER DISPLAYS IT

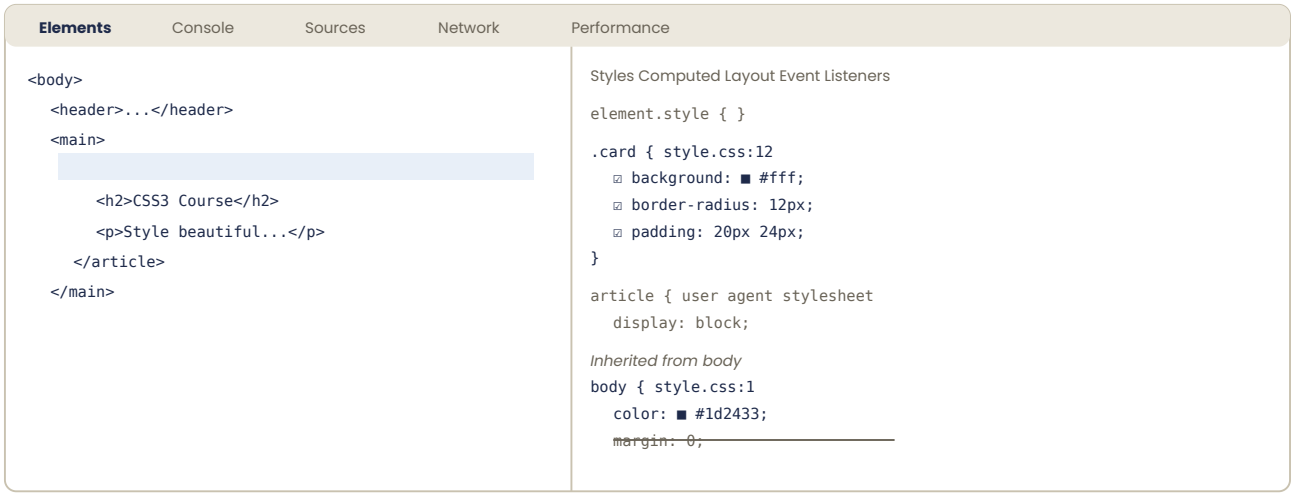
COMPUTED IN CHROMIUM

```
content-box rendered width: 344px
border-box  rendered width: 300px
```

2.4 Developer tools

2.4.1 Inspecting and editing styles live in DevTools

Every browser has developer tools. In Chrome or Edge press **F12** or **Ctrl+Shift+I** (Cmd+Option+I on Mac), or right-click an element and choose **Inspect**. The **Elements** panel shows the DOM tree on the left and, for the selected element, the **Styles** pane on the right:



DEVTOOLS: THE DOM TREE (LEFT) AND THE STYLES PANE FOR THE SELECTED ELEMENT (RIGHT)

- Rules are listed from **most to least important**; the file name and line number link to the source.
- Click any value to edit it, or click after the last declaration to add a new one; changes appear immediately (but are lost on reload unless you copy them into your file, or use Workspaces).
- Untick the checkbox beside a declaration to switch it off temporarily.
- Click a colour swatch for a colour picker with contrast information; hover a value like `12px` and use the arrow keys to nudge it (Shift for steps of 10).
- The **:hov** button forces states such as `:hover` and `:focus`; **.cls** toggles classes; **+** adds a new rule for the selected element.
- Struck-through declarations were overridden by a more important rule (Module 8); a yellow warning icon marks an invalid property or value.

2.4.2 Computed styles panel

The **Computed** tab shows the **final value of every property** for the selected element after the cascade, inheritance and unit conversion: `2em` becomes `32px`, `bold` becomes `700`, colours become `rgb()`. Expand a property to see every rule that tried to set it and which one won. The box-model diagram at the top shows the content size, padding, border and margin with real pixel values (Module 5). JavaScript reads the same values with `getComputedStyle()`, which is how the "COMPUTED IN CHROMIUM" boxes in this book were produced:

COMPUTED IN CHROMIUM

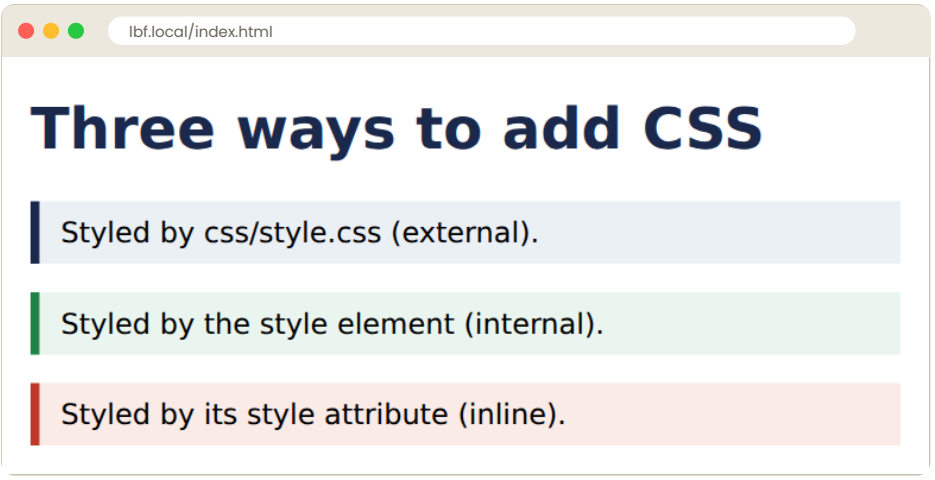
```
font-size: 2em   -> 30px
font-weight: bold -> 700
color: hsl(220 50% 20%) -> rgb(26, 43, 77)
margin: 1rem 0  -> 16px 0px
```

PRACTICE CHECKPOINT

Style one HTML page **three ways** (inline, internal, external) and use DevTools to change a colour live.

THREE-WAYS.HTML: SAMPLE SOLUTION HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Three ways</title>
  <link rel="stylesheet" href="css/style.css">          <!-- external -->
  <style>                                                  /* internal */
    .internal { background: #eef8f1; border-left: 5px solid #1e8449; padding: 8px 12px; }
  </style>
</head>
<body>
  <h1>Three ways to add CSS</h1>
  <p class="external">Styled by css/style.css (external).</p>
  <p class="internal">Styled by the style element (internal).</p>
  <p style="background: #fdecea; border-left: 5px solid #c0392b; padding: 8px 12px;">Styled by its
style attribute (inline).</p>
</body>
</html>
```



EACH PARAGRAPH IS STYLED BY A DIFFERENT METHOD

Then open DevTools, select the h1, click its colour value in the Styles pane and choose a new colour: the heading changes instantly, which proves the rule is applied. Reload the page and the change is gone, because DevTools edits the live page, not the file.

MODULE 03 · BEGINNER

Selectors basics

Minimum duration **1 day** | Level **Beginner**

Selectors choose which elements a rule applies to; choosing well keeps CSS simple and maintainable. This module covers the universal, type, class and id selectors and when to use a class or an id, grouping and chaining selectors, the four combinators (descendant, child, adjacent sibling, general sibling), attribute selectors with every operator (=, ^=, $=, *=, ~=, |=) and case-insensitive matching, and how to name classes well. Every example highlights exactly the elements the browser matched.

IN THIS MODULE

3.1 Basic selectors
3.2 Grouping and combining
3.3 Combinators
3.4 Attribute selectors
3.5 Naming classes

3.1 Basic selectors

A **selector** is a pattern. The browser tests it against every element in the document and applies the rule to the elements that match. In this module, matched elements are highlighted with a gold outline, and the list under each screenshot is what `document.querySelectorAll()` (which uses exactly the same selector engine) returned.

Selector	Example	Matches
Universal	`*`	every element
Type	`p`	every <p>
Class	`.card`	elements whose class attribute contains the word card
ID	`#header`	the element with id="header"
Attribute	`[type="email"]`	elements with that attribute value
Pseudo-class	`a:hover`	elements in a state (Module 9)
Pseudo-element	`p::first-line`	part of an element (Module 9)

3.1.1 Universal selector *

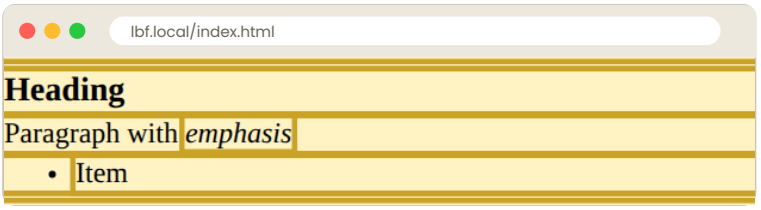
`*` matches **every element**. Its main legitimate uses are global resets (`*, *::before, *::after { box-sizing: border-box; }`) and, combined with other selectors, "every child" (`.stack > * + *`). It has zero specificity (Module 8), so any other rule overrides it. Applying heavy styles to `*` is wasteful because they reach every element, including ones you did not mean.

STYLE.CSS CSS

```css
* { outline: 3px solid #c9a227; background: #fff3c4; margin: 4px 0; }
```

INDEX.HTML HTML

```html
<h3>Heading</h3><p>Paragraph with <em>emphasis</em></p><ul><li>Item</li></ul>
```



EVERYTHING MATCHED, INCLUDING HTML, BODY, EM AND LI

3.1.2 Type (element) selector

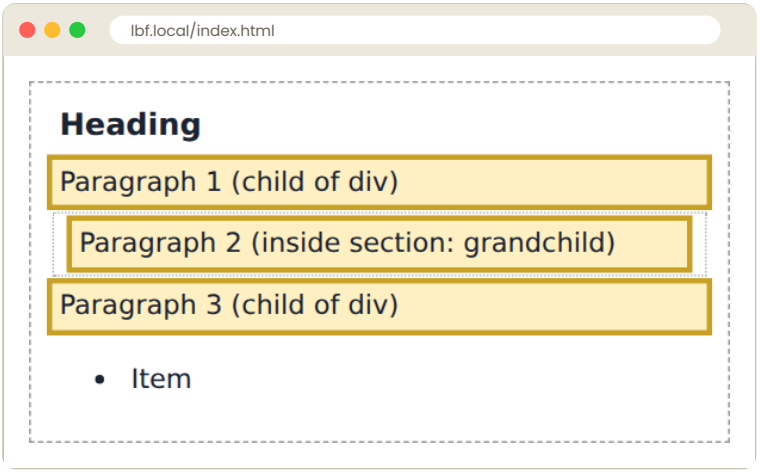
A **type selector** is an element name: `p`, `h1`, `a`, `table`. It matches every element of that type, wherever it is, so it is ideal for base styles (typography for all paragraphs, all links) and resets. Type selectors are case-insensitive in HTML.

STYLE.CSS CSS

```css
p { outline: 3px solid #c9a227; background: #fff3c4; }
.box { border: 1px dashed #999; padding: 6px 12px; } section { border: 1px dotted #bbb; padding: 0
10px; margin: 4px 0; } p, h2, li { margin: 4px 0; padding: 2px 4px; } h2 { font-size: 17px; }
```

INDEX.HTML HTML

```html
<div class="box">
  <h2>Heading</h2>
  <p>Paragraph 1 (child of div)</p>
  <section>
    <p>Paragraph 2 (inside section: grandchild)</p>
  </section>
  <p>Paragraph 3 (child of div)</p>
  <ul><li>Item</li></ul>
</div>
```



P MATCHED ALL THREE PARAGRAPHS AT EVERY DEPTH

3.1.3 Class selector .card

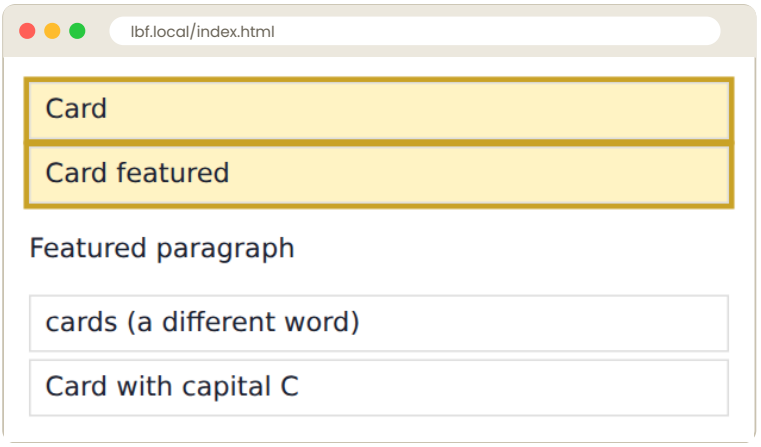
A **class selector** is a dot followed by a class name. It matches every element whose `class` attribute contains that name as one of its space-separated words. Classes are the main tool of CSS: reusable, combinable (an element can have many classes: `class="card featured"`) and of moderate specificity. Class names are **case-sensitive**: `.Card` does not match `class="card"`.

STYLE.CSS CSS

```css
.card { outline: 3px solid #c9a227; background: #fff3c4; }
div { margin: 4px 0; padding: 4px 8px; border: 1px solid #ddd; }
```

INDEX.HTML HTML

```html
<div class="card">Card</div>
<div class="card featured">Card featured</div>
<p class="featured">Featured paragraph</p>
<div class="cards">cards (a different word)</div>
<div class="Card">Card with capital C</div>
```



.CARD MATCHED ONLY THE ELEMENTS WHOSE CLASS LIST CONTAINS THE EXACT WORD "CARD"

MATCHED BY CHROMIUM

```
document.querySelectorAll(".card") matched 2:
  <div class="card"> Card
  <div class="card featured"> Card featured
```

3.1.4 ID selector #header

An **ID selector** is a hash followed by an id: `#header` matches `<header id="header">`. An id must be unique in the document, so the selector matches at most one element. IDs are also used as link targets (`href="#pricing"`), for label `for`, ARIA references and JavaScript (`getElementById`).

STYLE.CSS CSS

```css
#pricing { outline: 3px solid #c9a227; background: #fff3c4; }
section { margin: 4px 0; padding: 4px 8px; border: 1px solid #ddd; }
```

INDEX.HTML HTML

```html
<section id="features">Features</section>
<section id="pricing">Pricing</section>
<section id="faq">FAQ</section>
```



#PRICING MATCHED THE ONE ELEMENT WITH THAT ID

3.1.5 Class vs ID: when to use each

	Class	ID
Syntax	`.name`	`#name`
Per page	any number of elements	one element
Per element	many classes	one id
Specificity	(0,1,0): moderate	(1,0,0): very high, hard to override
Use for	**styling**: almost all of it	fragment links, form labels, ARIA, JavaScript hooks

Professional CSS styles with classes and keeps ids for non-styling purposes. One id selector outweighs any number of classes, so a single `#sidebar a` rule forces every later override to use ids or `!important`: the start of a "specificity war" (Module 8).

COMPUTED IN CHROMIUM

```
Four classes vs one id -> computed color: rgb(255, 0, 0)  (red: the id rule still wins)
```

3.2 Grouping and combining

3.2.1 Grouping selectors h1, h2

A comma-separated **selector list** applies one declaration block to several selectors, avoiding repetition. Each selector in the list is independent (and keeps its own specificity). Put each on its own line when the list is long.

GROUPED SELECTORS — CSS

```css
h1, h2, h3 { font-family: Georgia, serif; color: #1a2a4f; margin: 6px 0; }
.btn,
button,
input[type="submit"] { background: #c9a227; border: 0; padding: 6px 14px; border-radius: 6px; font:
inherit; }
```

INDEX.HTML — HTML

```html
<h1>Heading 1</h1><h2>Heading 2</h2><h3>Heading 3</h3><a class="btn">.btn link</a>
<button>button</button> <input type="submit" value="submit">
```



HOW THE BROWSER DISPLAYS IT

WATCH OUT

If **one** selector in a list is invalid (a typo, or a new pseudo-class an old browser does not know), the **whole rule** is dropped. `:is()` and `:where()` (Module 9) are "forgiving" and avoid this.

3.2.2 Chaining p.intro

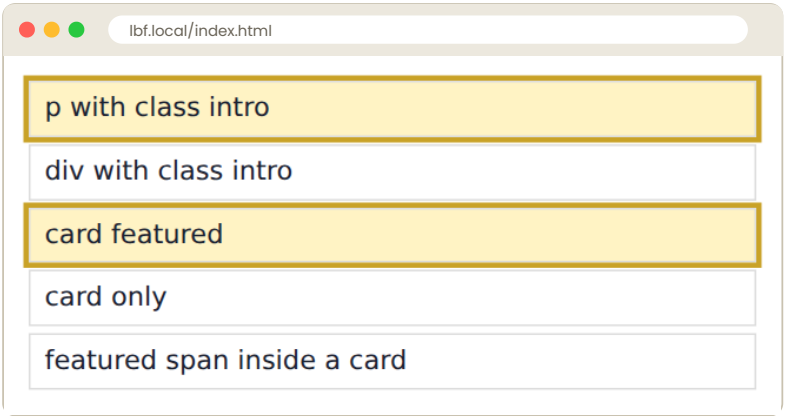
Writing simple selectors together **without spaces** creates a **compound selector** that matches elements meeting **all** the conditions: `p.intro` is a paragraph with class intro; `.card.featured` is an element with both classes; `a.btn[target="_blank"]` combines a type, a class and an attribute. A type selector must come first. Compare with the space, which means "descendant" (3.3.1):

STYLE.CSS CSS

```css
p.intro, .card.featured { outline: 3px solid #c9a227; background: #fff3c4; }
div, p { margin: 4px 0; padding: 4px 8px; border: 1px solid #ddd; }
```

INDEX.HTML HTML

```html
<p class="intro">p with class intro</p>
<div class="intro">div with class intro</div>
<div class="card featured">card featured</div>
<div class="card">card only</div>
<div class="card"><span class="featured">featured span inside a card</span></div>
```



P.INTRO AND .CARD.FEATURED: BOTH CONDITIONS ON THE SAME ELEMENT

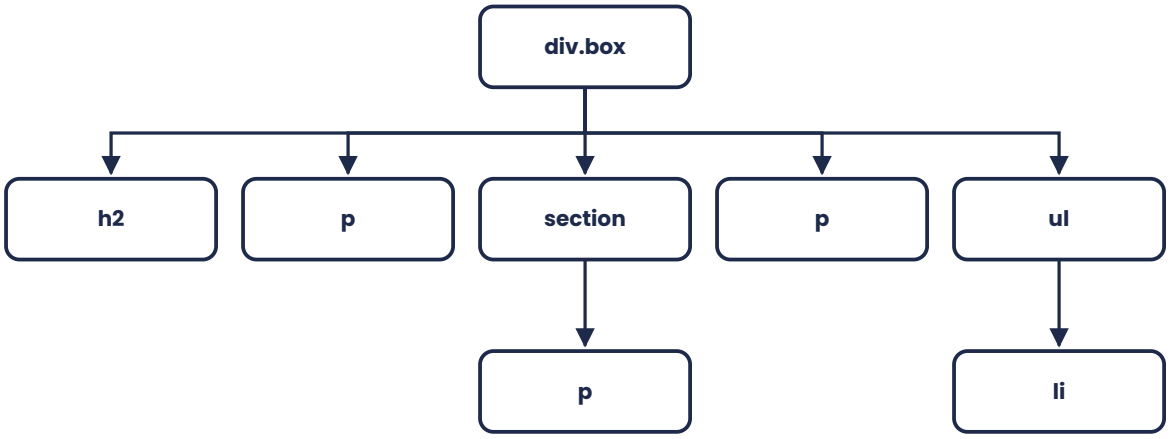
MATCHED BY CHROMIUM

```
document.querySelectorAll(".card .featured") matched 1:
  <span class="featured"> featured span inside a card
```

With a space, `.card .featured` matched the **span inside** a card instead: a completely different meaning.

3.3 Combinators

Combinators express relationships between elements in the document tree. The four combinators use the tree below:



THE DOCUMENT TREE USED IN THIS SECTION

3.3.1 Descendant div p

A **space** between two selectors matches the second when it is **anywhere inside** the first: child, grandchild or deeper. `.box p` matches all three paragraphs. It is the most common combinator ("links inside the nav": `nav a`).

STYLE.CSS

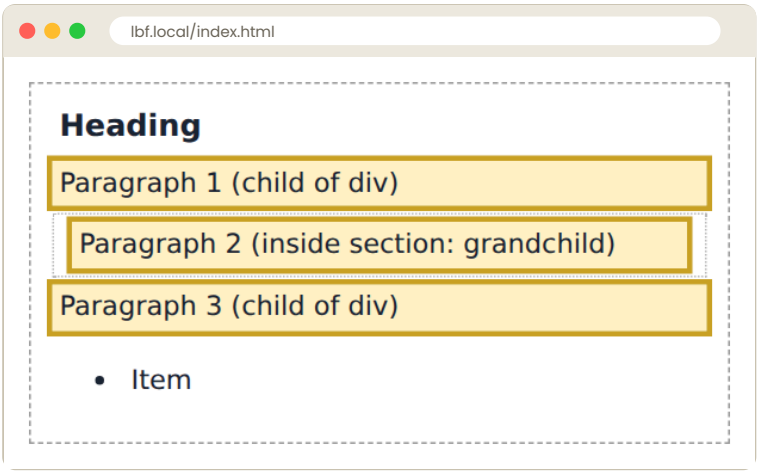
CSS

```
.box p { outline: 3px solid #c9a227; background: #fff3c4; }
.box { border: 1px dashed #999; padding: 6px 12px; } section { border: 1px dotted #bbb; padding: 0
10px; margin: 4px 0; } p, h2, li { margin: 4px 0; padding: 2px 4px; } h2 { font-size: 17px; }
```

INDEX.HTML

HTML

```
<div class="box">
  <h2>Heading</h2>
  <p>Paragraph 1 (child of div)</p>
  <section>
    <p>Paragraph 2 (inside section: grandchild)</p>
  </section>
  <p>Paragraph 3 (child of div)</p>
  <ul><li>Item</li></ul>
</div>
```



.BOX P: EVERY PARAGRAPH AT ANY DEPTH

3.3.2 Child ul > li

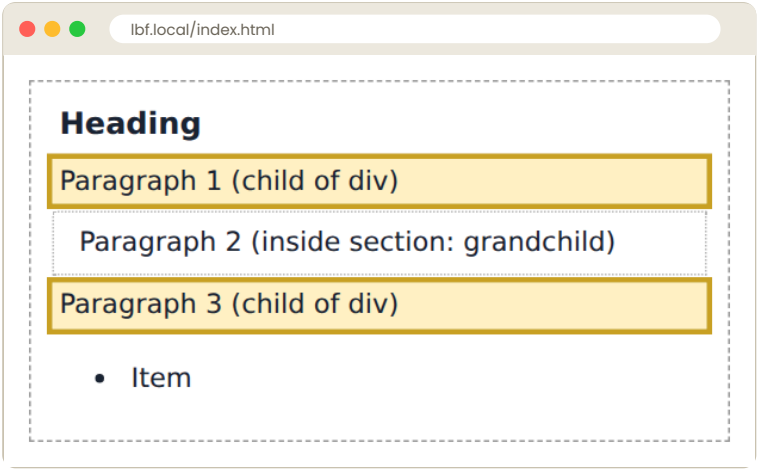
`>` matches only **direct children**. `.box > p` matches paragraphs 1 and 3 but not paragraph 2, which is a child of the section. It is useful for nested lists and menus, where `.menu > li` styles only the top level.

STYLE.CSS CSS

```
.box > p { outline: 3px solid #c9a227; background: #fff3c4; }
.box { border: 1px dashed #999; padding: 6px 12px; } section { border: 1px dotted #bbb; padding: 0
10px; margin: 4px 0; } p, h2, li { margin: 4px 0; padding: 2px 4px; } h2 { font-size: 17px; }
```

INDEX.HTML HTML

```
<div class="box">
  <h2>Heading</h2>
  <p>Paragraph 1 (child of div)</p>
  <section>
    <p>Paragraph 2 (inside section: grandchild)</p>
  </section>
  <p>Paragraph 3 (child of div)</p>
  <ul><li>Item</li></ul>
</div>
```



.BOX > P: ONLY DIRECT CHILDREN

3.3.3 Adjacent sibling h2 + p

`+` matches an element that comes **immediately after** another sibling (same parent). `h2 + p` styles the first paragraph after each heading (an intro). It is often used as "an element that follows another of its

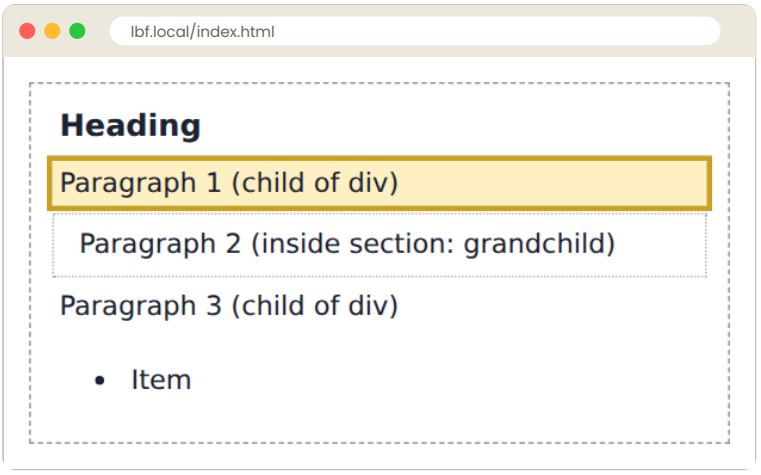
kind": `li + li { border-top: 1px solid #ddd; }` draws lines **between** list items.

STYLE.CSS CSS

```css
h2 + p { outline: 3px solid #c9a227; background: #fff3c4; }
.box { border: 1px dashed #999; padding: 6px 12px; } section { border: 1px dotted #bbb; padding: 0
10px; margin: 4px 0; } p, h2, li { margin: 4px 0; padding: 2px 4px; } h2 { font-size: 17px; }
```

INDEX.HTML HTML

```html
<div class="box">
  <h2>Heading</h2>
  <p>Paragraph 1 (child of div)</p>
  <section>
    <p>Paragraph 2 (inside section: grandchild)</p>
  </section>
  <p>Paragraph 3 (child of div)</p>
  <ul><li>Item</li></ul>
</div>
```



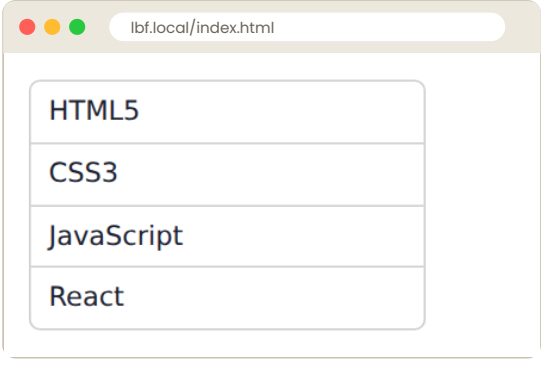
H2 + P: ONLY PARAGRAPH 1, WHICH IMMEDIATELY FOLLOWS THE H2

SEPARATORS WITH LI + LI CSS

```css
ul { list-style: none; padding: 0; width: 220px; border: 1px solid #ccc; border-radius: 6px; }
li { padding: 6px 10px; }
li + li { border-top: 1px solid #ccc; }   /* a line between items, not above the first */
```

INDEX.HTML HTML

```html
<ul><li>HTML5</li><li>CSS3</li><li>JavaScript</li><li>React</li></ul>
```



HOW THE BROWSER DISPLAYS IT

3.3.4 General sibling h2 ~ p

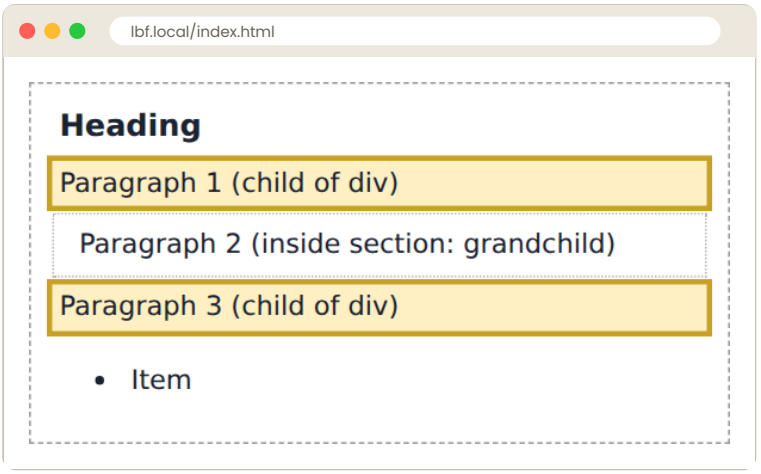
`~` matches every following sibling, not just the next one. `h2 ~ p` matches paragraphs 1 and 3 (both siblings after the h2), but not paragraph 2 (it is not a sibling). Neither sibling combinator can look **backwards** (for "previous" or "parent" selection use `:has()`, Module 9). A classic use is CSS-only toggles: `input:checked ~ .panel { display: block; }`.

STYLE.CSS CSS

```
h2 ~ p { outline: 3px solid #c9a227; background: #fff3c4; }
.box { border: 1px dashed #999; padding: 6px 12px; } section { border: 1px dotted #bbb; padding: 0
10px; margin: 4px 0; } p, h2, li { margin: 4px 0; padding: 2px 4px; } h2 { font-size: 17px; }
```

INDEX.HTML HTML

```
<div class="box">
  <h2>Heading</h2>
  <p>Paragraph 1 (child of div)</p>
  <section>
    <p>Paragraph 2 (inside section: grandchild)</p>
  </section>
  <p>Paragraph 3 (child of div)</p>
  <ul><li>Item</li></ul>
</div>
```



H2 ~ P: PARAGRAPHS 1 AND 3

Combinator	Syntax	Matches B when B is…	In the tree above
Descendant	`A B`	anywhere inside A	p #1, #2, #3
Child	`A > B`	a direct child of A	p #1, #3
Adjacent sibling	`A + B`	the very next sibling after A	p #1 (after h2)
General sibling	`A ~ B`	any later sibling of A	p #1, #3 (after h2)

3.4 Attribute selectors

3.4.1 [type="email"]

Square brackets select by attribute. `[disabled]` matches elements that **have** the attribute (any value); `[type="email"]` matches an exact value. They are perfect for styling form controls by type, links by

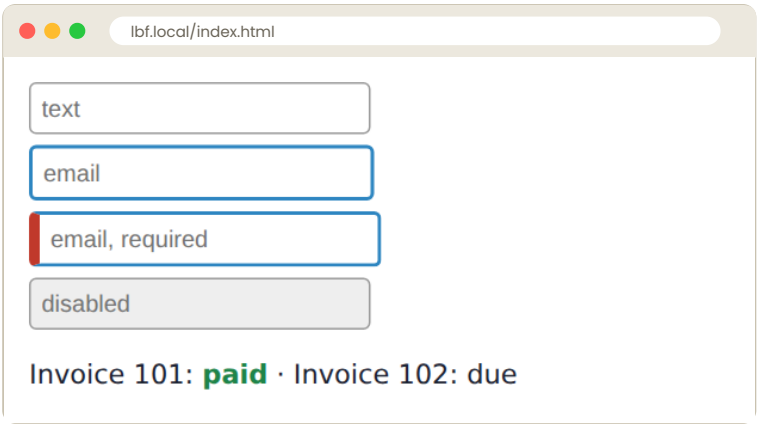
destination and elements by data attributes, without adding classes.

SELECTING BY ATTRIBUTE CSS

```css
input { display: block; margin: 6px 0; padding: 6px; border: 1px solid #999; border-radius: 4px; }
input[type="email"] { border: 2px solid #2e86c1; }
input[required] { border-left: 6px solid #c0392b; }
input[disabled] { background: #eee; color: #999; }
[data-status="paid"] { color: #1e8449; font-weight: 700; }
```

INDEX.HTML HTML

```html
<input type="text" placeholder="text">
<input type="email" placeholder="email">
<input type="email" placeholder="email, required" required>
<input type="text" placeholder="disabled" disabled>
<p>Invoice 101: <span data-status="paid">paid</span> · Invoice 102: <span data-status="due">due</span>
</p>
```



HOW THE BROWSER DISPLAYS IT

3.4.2 ^=, $=, *=, ~=, |=

Selector	Matches when the attribute value…	Example
`[attr]`	exists (any value)	`a[title]`
`[attr="v"]`	is exactly v	`[type="checkbox"]`
`[attr^="v"]`	**starts** with v	`a[href^="https"]` external links
`[attr$="v"]`	**ends** with v	`a[href$=".pdf"]` PDF links
`[attr*="v"]`	**contains** v anywhere	`a[href*="youtube"]`
`[attr~="v"]`	contains v as a whole **space-separated word**	`[data-tags~="css"]`
`[attr|="v"]`	is v or starts with **v-**	`[lang|="en"]`: en, en-IN, en-US

STYLING LINKS BY DESTINATION CSS

```css
a[href^="http"]::after   { content: " ↗"; }
a[href$=".pdf"]::before   { content: "📄 "; }
a[href^="mailto:"]        { color: #1e8449; }
a[href*="youtube"]        { color: #c0392b; font-weight: 600; }
a:not([href^="http"])     { text-decoration-style: dotted; }
```

INDEX.HTML

HTML

```
<ul>
  <li><a href="https://developer.mozilla.org">MDN (external, https)</a></li>
  <li><a href="http://example.com">Example (external, http)</a></li>
  <li><a href="/courses/css3">CSS3 course (internal)</a></li>
  <li><a href="syllabus.pdf">Syllabus (PDF)</a></li>
  <li><a href="mailto:info@learnbeingforward.in">Email us</a></li>
  <li><a href="https://www.youtube.com/watch?v=abc">Intro video</a></li>
</ul>
```



EXTERNAL LINKS GET AN ARROW, PDFS AN ICON, EMAIL AND YOUTUBE LINKS A COLOUR; INTERNAL LINKS ARE DOTTED

`~=` and `|=` are about words and language codes:

~= MATCHES WHOLE WORDS; |= MATCHES A LANGUAGE AND ITS SUB-CODES

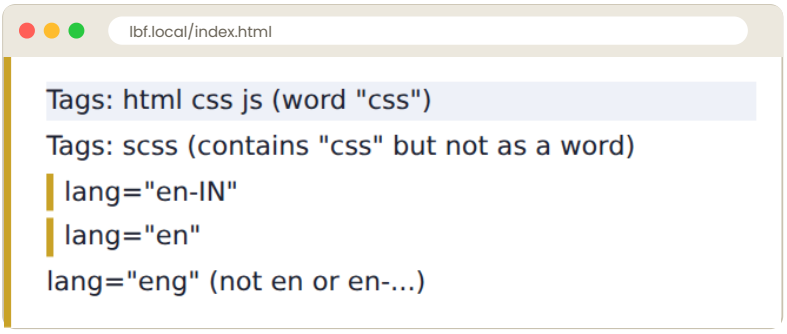
CSS

```
[data-tags~="css"] { background: #eef1f8; }
[lang|="en"] { border-left: 4px solid #c9a227; padding-left: 6px; }
p { margin: 4px 0; }
```

INDEX.HTML

HTML

```
<p data-tags="html css js">Tags: html css js (word "css")</p>
<p data-tags="scss">Tags: scss (contains "css" but not as a word)</p>
<p lang="en-IN">lang="en-IN"</p>
<p lang="en">lang="en"</p>
<p lang="eng">lang="eng" (not en or en-...)</p>
```



HOW THE BROWSER DISPLAYS IT

3.4.3 Case-insensitive matching i

Attribute values are matched case-sensitively (except for some HTML attributes such as `type`, which HTML defines as case-insensitive). Adding `i` before the closing bracket makes any comparison case-insensitive, useful for file extensions and user-entered data. `s` forces case-sensitive matching.

[href$=".pdf"]: 1 match	[href$=".pdf" i]: 3 matches
report.pdf SCAN.PDF notes.Pdf	report.pdf SCAN.PDF notes.Pdf

WITHOUT AND WITH THE I FLAG

3.5 Naming classes

3.5.1 Meaningful, reusable class names

Class names are an interface between HTML and CSS that whole teams read for years. Good names describe **what something is or does**, not how it looks today.

Avoid	Prefer	Why
`.red-text`, `.big-blue-box`	`.error-message`, `.promo-banner`	the design will change; the purpose will not
`.left-col`	`.sidebar`	on phones the sidebar is not on the left
`.div1`, `.box2`, `.asdf`	`.course-card`, `.price-tag`	meaningless names cannot be searched or reused
`.CourseCard`, `.course_card`	`.course-card`	lower-case kebab-case is the convention
`.c`, `.hdr`	`.header`	saving letters costs readability
`.mt-20` everywhere (by hand)	a spacing system or utility framework	one-off utilities become unmanageable

- Use **lower-case words separated by hyphens** (kebab-case). Class names cannot start with a digit (without escaping).
- Name **components** (`.card`, `.navbar`), their **parts** (`.card-title`, or `.card__title` in BEM) and their **variants** (`.card--featured`). Module 18 covers BEM and other methodologies.
- Prefer one class per component over long descendant chains (`.card-title` rather than `main section div.card h3`): shorter selectors are faster to read, have lower specificity and do not break when the HTML is restructured.
- Use `is-` / `has-` prefixes for states set by JavaScript: `.is-open`, `.is-active`, `.has-error`.

PRACTICE CHECKPOINT

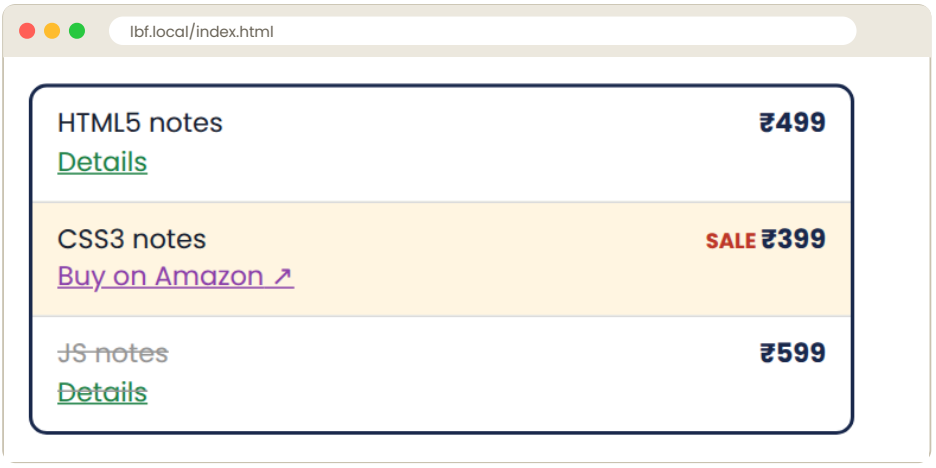
Style a **product list** using type, class, ID, combinator and attribute selectors, with **external links** marked differently from internal ones.

PRODUCTS.CSS: SAMPLE SOLUTION CSS

```css
/* type */
body { font-family: "Poppins", sans-serif; }
ul { list-style: none; padding: 0; max-width: 460px; }
/* id */
#products { border: 2px solid #1a2a4f; border-radius: 10px; }
/* class + child combinator */
.product-list > li { padding: 10px 14px; }
/* adjacent sibling */
.product-list > li + li { border-top: 1px solid #ddd; }
/* chained class */
.product.on-sale { background: #fff8e1; }
/* descendant */
.product .price { float: right; font-weight: 700; color: #1a2a4f; }
.product.on-sale .price::before { content: "SALE "; color: #c0392b; font-size: 12px; }
/* attribute: data and links */
[data-stock="0"] { color: #999; text-decoration: line-through; }
a[href^="http"] { color: #8e44ad; }
a[href^="http"]::after { content: " ↗"; }
a:not([href^="http"]) { color: #1e8449; }
```

INDEX.HTML HTML

```html
<ul id="products" class="product-list">
  <li class="product">HTML5 notes <span class="price">₹499</span>
      <br><a href="/books/html5">Details</a></li>
  <li class="product on-sale">CSS3 notes <span class="price">₹399</span>
      <br><a href="https://www.amazon.in/">Buy on Amazon</a></li>
  <li class="product" data-stock="0">JS notes <span class="price">₹599</span>
      <br><a href="/books/js">Details</a></li>
</ul>
```



INTERNAL LINKS ARE GREEN, EXTERNAL LINKS PURPLE WITH AN ARROW; THE OUT-OF-STOCK ITEM IS STRUCK THROUGH

MODULE 04 · BEGINNER

Colours, units and typography

Minimum duration **2 days** | Level **Beginner**

This module covers how to express colours and sizes in CSS and how to style text beautifully. It explains named, hexadecimal, rgb() and hsl() colours, transparency and opacity, currentColor and colour contrast; absolute, relative, viewport, dynamic viewport and character units and when to use px, rem or %; font families, generic families, Google Fonts, @font-face with font-display, variable fonts, font size, weight, style and the font shorthand; text colour, alignment, decoration, transformation, spacing, indentation, shadows, wrapping and overflow including text-wrap: balance and pretty; and styling links in every state and list markers.

IN THIS MODULE

4.1 Colours
4.2 Units
4.3 Fonts
4.4 Text styling
4.5 Styling links and lists

4.1 Colours

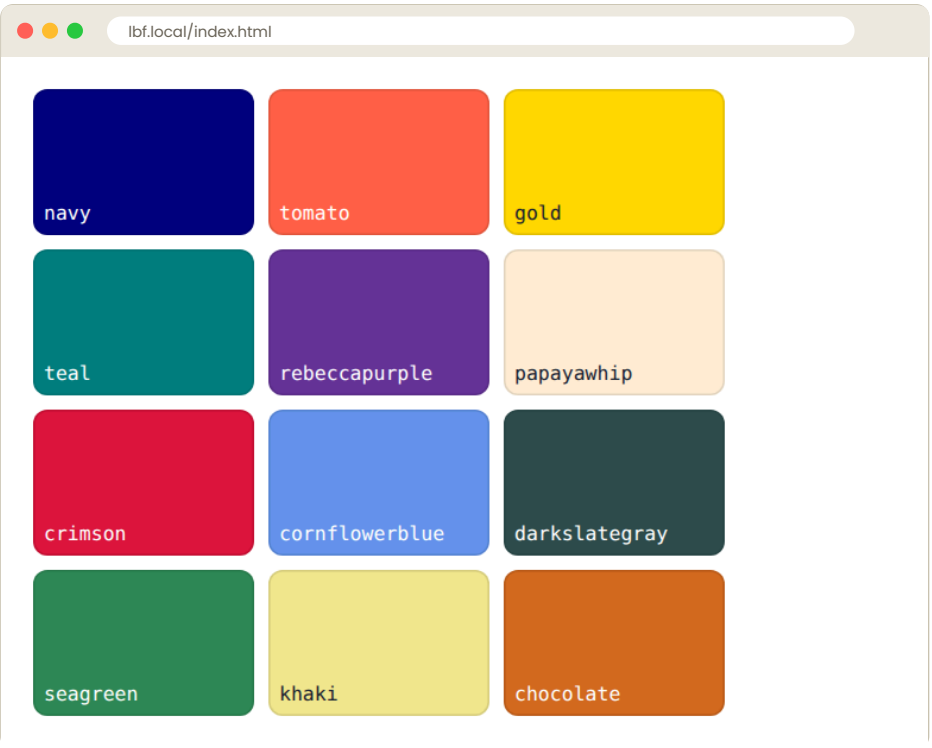
Colours appear in many properties: `color` (text), `background-color`, `border-color`, `outline-color`, `box-shadow`, `text-decoration-color`, `accent-color`, gradients and SVG `fill` / `stroke`. CSS accepts many notations for the same colour; the browser converts them all to one internal value.

4.1.1 Named colours

CSS defines **148 named colours**, from `black`, `white`, `red` and `navy` to `rebeccapurple`, `tomato`, `cornflowerblue` and `papayawhip`, plus the special keyword `transparent`. Names are case-insensitive and handy for quick tests, but real designs need exact brand colours, so they use the numeric formats below.

STYLE.CSS — CSS

```
.sw { display: inline-flex; align-items: end; width: 112px; height: 70px; margin: 4px; padding: 6px;
border-radius: 8px; font: 11px/1.2 ui-monospace, monospace; color: #fff; box-shadow: inset 0 0 0 1px
rgb(0 0 0 / .1); }
```



TWELVE OF THE 148 NAMED COLOURS

4.1.2 Hexadecimal #1C2B4A and short hex

A **hex colour** is `#` followed by three pairs of hexadecimal digits (0-9, a-f) for **red, green and blue**, each from `00` (0) to `ff` (255). `#1c2b4a` means red 0x1c = 28, green 0x2b = 43, blue 0x4a = 74: a dark navy. When each pair has two identical digits, the **short form** uses one: `#ffcc00` = `#fc0`. An optional fourth pair (or fourth short digit) is **alpha** (opacity): `#1c2b4a80` is 50 percent opaque. Hex is case-insensitive; lower case is the common convention.



READING A HEX COLOUR

STYLE.CSS CSS

```css
.sw { display: inline-flex; align-items: end; width: 112px; height: 70px; margin: 4px; padding: 6px;
border-radius: 8px; font: 11px/1.2 ui-monospace, monospace; color: #fff; box-shadow: inset 0 0 0 1px
rgb(0 0 0 / .1); }
```

INDEX.HTML HTML

```html
<span class="sw" style="background:#1c2b4a">#1c2b4a</span>
<span class="sw" style="background:#c9a227">#c9a227</span>
<span class="sw" style="background:#f00">#f00 = #ff0000</span>
<span class="sw" style="background:#000">#000</span>
<span class="sw" style="background:#1c2b4a80">#1c2b4a80</span>
<span class="sw" style="background:#1c2b4a33;color:#1d2433">#1c2b4a33</span>
```



HEX COLOURS, SHORT HEX AND HEX WITH ALPHA (ON A WHITE PAGE)

COMPUTED IN CHROMIUM

```
color:#1c2b4a       -> rgb(28, 43, 74)
color:#fc0          -> rgb(255, 204, 0)
color:#1c2b4a80     -> rgba(28, 43, 74, 0.5)
```

4.1.3 rgb() and rgba()

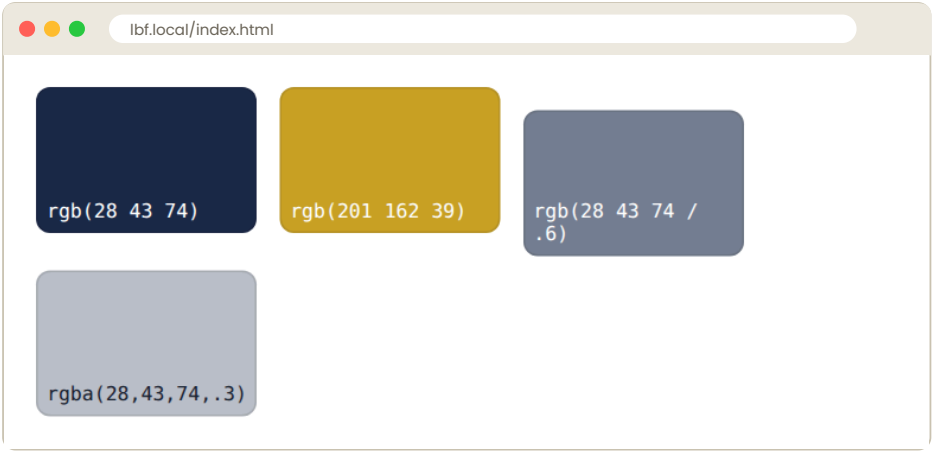
`rgb()` gives the red, green and blue channels as numbers from 0 to 255 (or percentages). The modern syntax separates them with spaces and adds alpha after a slash: `rgb(28 43 74 / 0.5)`. The older comma syntax `rgb(28, 43, 74)` and `rgba(28, 43, 74, 0.5)` still works everywhere; `rgba()` is now just an alias of `rgb()`.

STYLE.CSS CSS

```css
.sw { display: inline-flex; align-items: end; width: 112px; height: 70px; margin: 4px; padding: 6px;
border-radius: 8px; font: 11px/1.2 ui-monospace, monospace; color: #fff; box-shadow: inset 0 0 0 1px
rgb(0 0 0 / .1); }
```

INDEX.HTML HTML

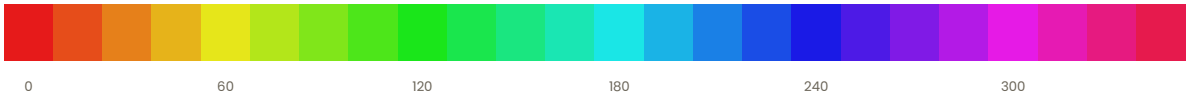
```html
<span class="sw" style="background: rgb(28 43 74)">rgb(28 43 74)</span>
<span class="sw" style="background: rgb(201 162 39)">rgb(201 162 39)</span>
<span class="sw" style="background: rgb(28 43 74 / 0.6)">rgb(28 43 74 / .6)</span>
<span class="sw" style="background: rgba(28, 43, 74, 0.3); color:#1d2433">rgba(28,43,74,.3)</span>
```



RGB() WITH AND WITHOUT ALPHA

4.1.4 hsl() and hsla()

`hsl(hue saturation lightness)` describes colour the way people think about it. **Hue** is an angle on the colour wheel (0 red, 120 green, 240 blue); **saturation** is how vivid (0% grey to 100% full colour); **lightness** goes from 0% (black) through 50% (pure colour) to 100% (white). It makes colour **variations** easy: keep the hue and change lightness for hover states, borders and tints. Alpha works as in rgb: `hsl(220 45% 20% / 0.5)`.

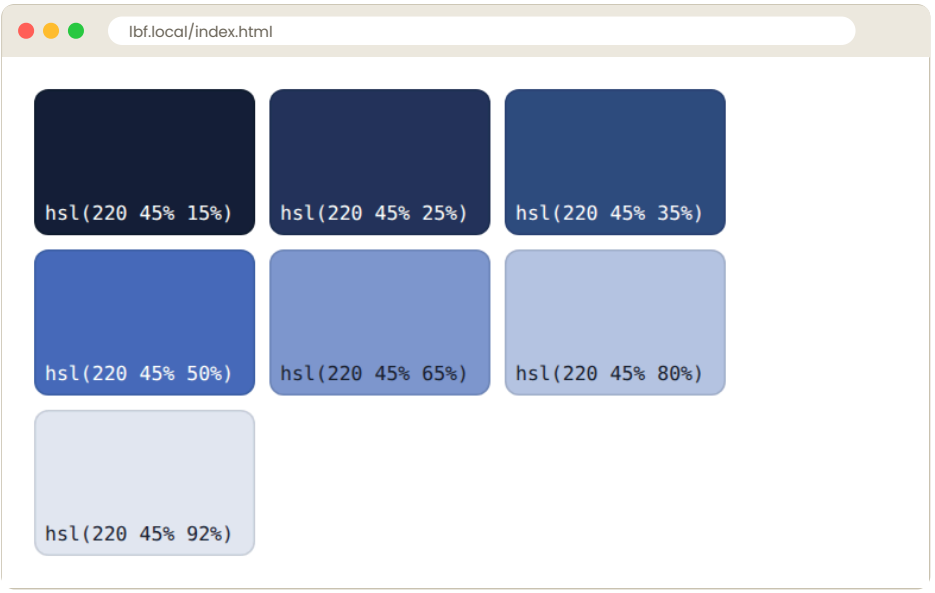


HUE FROM 0 TO 345 DEGREES AT SATURATION 80%, LIGHTNESS 50%

STYLE.CSS

CSS

```
.sw { display: inline-flex; align-items: end; width: 112px; height: 70px; margin: 4px; padding: 6px;
border-radius: 8px; font: 11px/1.2 ui-monospace, monospace; color: #fff; box-shadow: inset 0 0 0 1px
rgb(0 0 0 / .1); }
```



ONE HUE, SEVEN LIGHTNESS STEPS: A COLOUR SCALE

VARIATIONS FROM ONE HUE

CSS

```
.btn       { background: hsl(220 45% 25%); }
.btn:hover { background: hsl(220 45% 35%); }   /* same hue, lighter */
.btn-light { background: hsl(220 45% 92%); color: hsl(220 45% 25%); }
```

4.1.5 Transparency and opacity

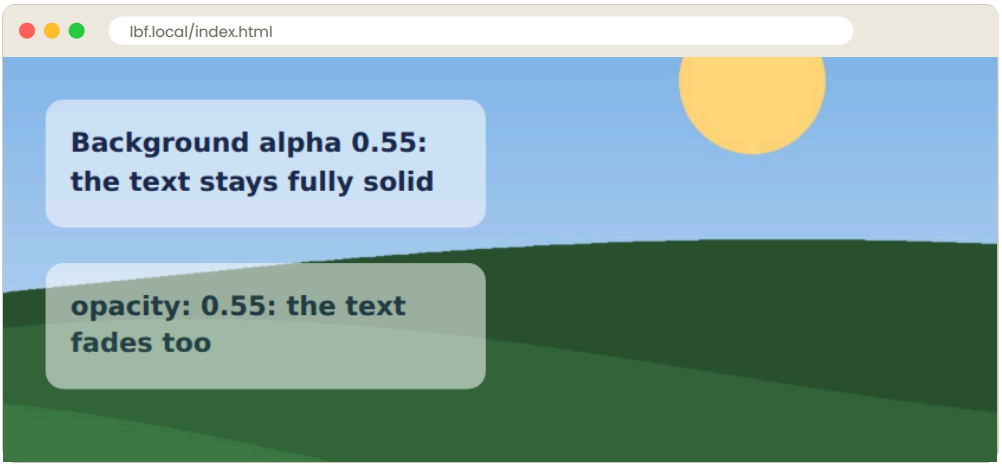
There are two different ways to make something see-through. An **alpha channel** in a colour affects only that colour (just the background, or just the text). The `opacity` property (0 to 1) makes the **whole element**, including its text, borders, images and children, transparent. Use alpha for translucent backgrounds behind readable text; use opacity for fading whole elements (disabled states, animations).

ALPHA VS OPACITY CSS

```css
body { background: url("hills.jpg") center / cover; min-height: 200px; }
.card { width: 220px; padding: 14px; margin: 10px; display: inline-block; border-radius: 10px; color:
#1a2a4f; font-weight: 700; vertical-align: top; }
.alpha   { background: rgb(255 255 255 / 0.55); }
.opacity { background: #fff; opacity: 0.55; }
```

INDEX.HTML HTML

```html
<div class="card alpha">Background alpha 0.55: the text stays fully solid</div>
<div class="card opacity">opacity: 0.55: the text fades too</div>
```



HOW THE BROWSER DISPLAYS IT

4.1.6 currentColor

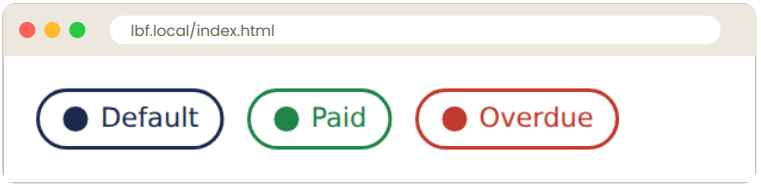
The keyword `currentColor` means "the value of this element's `color` property". Borders, outlines, text decorations and SVG icons using it automatically follow the text colour, so changing one `color` (for example on hover, or a different variant) recolours everything. Borders and text decoration already default to currentColor.

ONE COLOR VALUE DRIVES THE BORDER AND ICON TOO CSS

```css
.chip { color: #1a2a4f; border: 2px solid currentColor; border-radius: 20px; padding: 4px 12px;
display: inline-flex; gap: 6px; align-items: center; margin: 4px; }
.chip svg { width: 16px; height: 16px; fill: currentColor; }
.chip.success { color: #1e8449; }
.chip.danger { color: #c0392b; }
```

INDEX.HTML HTML

```html
<span class="chip"><svg viewBox="0 0 16 16"><circle cx="8" cy="8" r="7"/></svg>Default</span>
<span class="chip success"><svg viewBox="0 0 16 16"><circle cx="8" cy="8" r="7"/></svg>Paid</span>
<span class="chip danger"><svg viewBox="0 0 16 16"><circle cx="8" cy="8" r="7"/></svg>Overdue</span>
```



HOW THE BROWSER DISPLAYS IT

4.1.7 Colour contrast for readability

Text must stand out from its background. WCAG defines a **contrast ratio** from 1:1 (same colour) to 21:1 (black on white), computed from the relative luminance of the two colours. Level **AA** requires at least **4.5:1** for normal text and **3:1** for large text (24px, or 18.66px bold) and for UI components such as input borders and icons; AAA requires 7:1. DevTools shows the ratio in the colour picker. This script computes the WCAG ratio in the browser for several pairs:

WCAG CONTRAST RATIOS main.js

```
function lum(hex) {
  const c = hex.match(/\w\w/g).map(h => parseInt(h, 16) / 255)
    .map(v => v <= 0.03928 ? v / 12.92 : ((v + 0.055) / 1.055) ** 2.4);
  return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2];
}
const ratio = (a, b) => { const [x, y] = [lum(a), lum(b)].sort((m, n) => n - m); return (x + 0.05) /
(y + 0.05); };
for (const [fg, bg] of [["1a2a4f", "ffffff"], ["c9a227", "ffffff"], ["777777", "ffffff"], ["999999",
"ffffff"], ["c9a227", "1a2a4f"], ["ffffff", "c0392b"]]) {
  const r = ratio(fg, bg);
  console.log(`#${fg} on #${bg}: ${r.toFixed(2)}:1  ${r >= 7 ? "AAA" : r >= 4.5 ? "AA" : r >= 3 ? "AA
large text only" : "FAIL"}`);
}
```

BROWSER CONSOLE

```
#1a2a4f on #ffffff: 14.11:1  AAA
#c9a227 on #ffffff: 2.42:1  FAIL
#777777 on #ffffff: 4.48:1  AA large text only
#999999 on #ffffff: 2.85:1  FAIL
#c9a227 on #1a2a4f: 5.83:1  AA
#ffffff on #c0392b: 5.44:1  AA
```

CONTRAST IN PRACTICE CSS

```
p { padding: 8px 12px; margin: 4px 0; font-size: 16px; }
```

INDEX.HTML HTML

```
<p style="color:#1a2a4f;background:#fff">#1a2a4f on white: 14:1</p>
<p style="color:#777;background:#fff">#777 on white: 4.48:1 (just fails AA)</p>
<p style="color:#999;background:#fff">#999 on white: 2.85:1 (fails)</p>
<p style="color:#c9a227;background:#1a2a4f">gold on navy: comfortable</p>
<p style="color:#c9a227;background:#fff">gold on white: too faint for body text</p>
```



HOW THE BROWSER DISPLAYS IT

TIP

Never rely on colour alone to convey meaning (red for errors, green for success): add text or an icon, because about 1 in 12 men and 1 in 200 women have some form of colour vision deficiency.

4.2 Units

Lengths in CSS are a number followed immediately by a unit (no space): `16px`, `1.5rem`, `50%`. The number `0` may be written without a unit. Units fall into absolute units, font-relative units, viewport units and percentages, each relative to something different.

4.2.1 Absolute units: px, pt, cm

px (CSS pixel) is the standard absolute unit. It is not a physical screen pixel: a CSS pixel is defined as 1/96 of an inch at arm's length, and on high-density (Retina) screens one CSS pixel covers 2 or 3 device pixels. The other absolute units are fixed multiples of it and are meant for **print**: 1in = 96px, 1cm = 37.8px, 1mm = 3.78px, 1pt = 1/72in = 1.333px, 1pc = 12pt.

COMPUTED IN CHROMIUM

```
1px  = 1.000 CSS px
1pt  = 1.328 CSS px
1pc  = 16.000 CSS px
1mm  = 3.766 CSS px
1cm  = 37.781 CSS px
1in  = 96.000 CSS px
1Q   = 0.938 CSS px
```

4.2.2 Relative units: %, em, rem

Unit	Relative to	Typical use
`%`	usually the parent: the containing block's **width** for width, padding and margin (even vertical!), the parent's font size for font-size	fluid widths, positions
`em`	the **element's own font-size** (for font-size itself: the parent's font-size)	padding and margins that scale with the text of a component
`rem`	the **root** (html) font-size, 16px by default	font sizes and spacing scales across the site

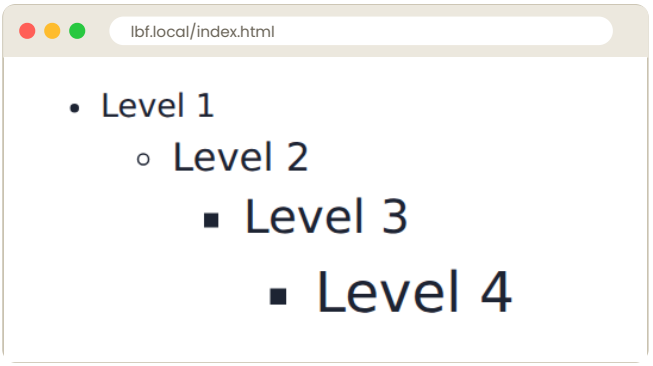
`em` **compounds** when nested: a list at 1.2em inside a list at 1.2em is 1.44 times the base. `rem` never compounds, which is why it is preferred for font sizes:

EM COMPOUNDS IN NESTED LISTS CSS

```css
li { font-size: 1.2em; }   /* try 1.2rem instead */
```

INDEX.HTML HTML

```html
<ul><li>Level 1
  <ul><li>Level 2
    <ul><li>Level 3
      <ul><li>Level 4</li></ul></li></ul></li></ul></li></ul>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
level-4 item with em:  31.104px  (15 x 1.2^4)
level-4 item with rem: 19.2px  (16 x 1.2, root size)
```

Because rem and em follow the user's browser font-size setting (people with low vision often raise it from 16px to 20px or more), text sized in rem respects their choice, while px-sized text ignores it.

4.2.3 Viewport units: vw, vh, vmin, vmax

Viewport units are percentages of the browser window: `1vw` = 1% of the viewport width, `1vh` = 1% of its height, `vmin` the smaller of the two, `vmax` the larger. `height: 100vh` makes a full-screen hero section; `font-size: 5vw` makes text scale with the window (usually combined with clamp(), Module 15, so it cannot become tiny or huge).

COMPUTED IN CHROMIUM

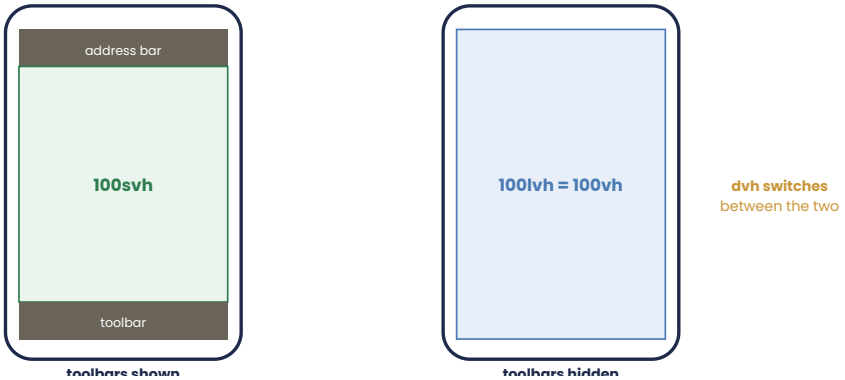
```
viewport: 800 x 600
50vw  = 400px
25vh  = 150px
10vmin = 60px (10% of the smaller side)
```

4.2.4 Modern viewport units: svh, lvh, dvh

On phones, browser toolbars slide in and out as you scroll, so the viewport height changes. `100vh` is the **largest** possible height, so a 100vh hero is partly hidden behind the toolbar when it is shown. The newer units fix this:

Unit	Means	Use
`svh` (small)	height with all toolbars **shown**	content that must always be fully visible

Unit	Means	Use
`lvh` (large)	height with toolbars **hidden** (same as vh)	backgrounds
`dvh` (dynamic)	the **current** height, changing as toolbars move	full-screen layouts (may cause re-layout while scrolling)
also	`svw lvw dvw`, `svmin` … and logical `vi vb`	



SMALL, LARGE AND DYNAMIC VIEWPORT HEIGHTS ON A PHONE

4.2.5 Character units: ch, ex

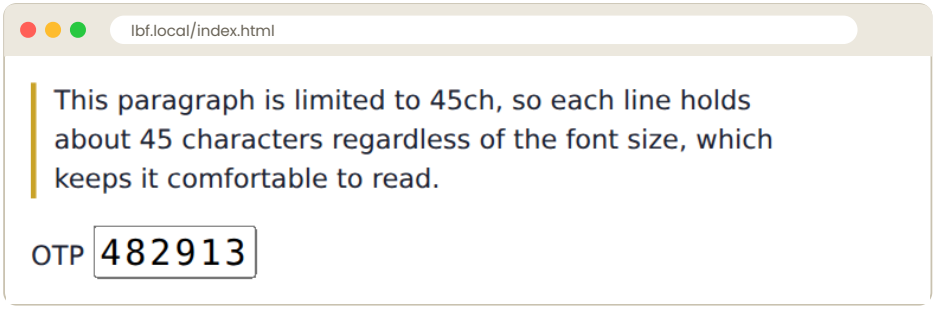
`ch` is the width of the "0" character in the current font; `ex` is the height of a lower-case "x". `max-width: 65ch` limits a paragraph to roughly 65 characters per line, the comfortable range for reading (45 to 75); `width: 6ch` fits a 6-digit OTP input. `lh` (the line height) and `cap` (capital height) are newer font-relative units.

SIZING WITH CH CSS

```css
p { max-width: 45ch; border-left: 3px solid #c9a227; padding-left: 10px; }
input { width: 7ch; font: 20px ui-monospace, monospace; letter-spacing: 2px; }
```

INDEX.HTML HTML

```html
<p>This paragraph is limited to 45ch, so each line holds about 45 characters regardless of the font size, which keeps it comfortable to read.</p>
<label>OTP <input value="482913"></label>
```



HOW THE BROWSER DISPLAYS IT

4.2.6 When to use px vs rem vs %

Use	Unit	Why
Font sizes	`rem`	respects the user's font-size setting; no compounding

Use	Unit	Why
Spacing (margin, padding, gap)	`rem` (or `em` inside components)	scales with text; em keeps a button's padding proportional to its font
Borders, hairlines, shadows	`px`	should stay crisp and thin at any text size
Layout widths	`%`, `fr`, `max-width` in rem or ch	fluid with the container
Line height	unitless number (`1.5`)	scales with each element's font size (4.4.2)
Full-screen sections	`dvh` / `svh` (fallback `vh`)	mobile toolbars
Media query breakpoints	`em` or `px`	em-based breakpoints follow text zoom
Print	`pt` , `cm` , `mm`	physical sizes on paper

4.3 Fonts

4.3.1 font-family and font stacks

`font-family` lists fonts in order of preference, separated by commas: a **font stack**. The browser uses the first one available on the device (or downloaded), falling back character by character, and should always end with a **generic family**. Font names containing spaces or digits should be quoted.

FONT STACKS CSS

```css
body { font-family: "Poppins", "Segoe UI", Roboto, Helvetica, Arial, sans-serif; }
code { font-family: ui-monospace, "Cascadia Code", Consolas, "Courier New", monospace; }
.article { font-family: Georgia, "Times New Roman", serif; }

/* The "system font stack": uses each operating system's own UI font, no download */
.app { font-family: system-ui, -apple-system, "Segoe UI", Roboto, "Noto Sans", sans-serif; }
```

4.3.2 Generic families: serif, sans-serif, monospace

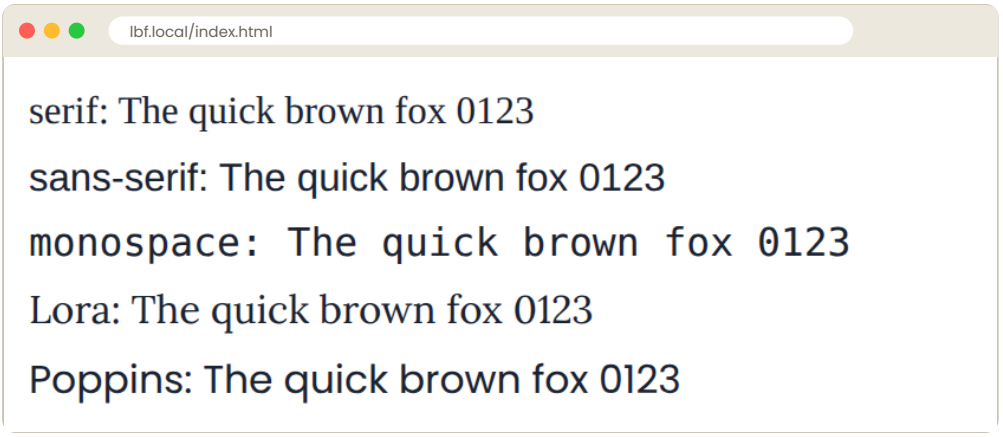
Generic family	Look	Typical use
`serif`	small strokes ("serifs") at the ends of letters	long-form reading, editorial, formal
`sans-serif`	clean strokes without serifs	interfaces and most websites
`monospace`	every character the same width	code, tables of numbers, OTPs
`cursive` , `fantasy`	handwriting-like / decorative	rarely; unpredictable across systems
`system-ui` , `ui-serif` , `ui-monospace` , `ui-rounded`	the operating system's own fonts	native-looking apps

GENERIC FAMILIES AND TWO SPECIFIC FONTS CSS

```css
p { font-size: 22px; margin: 6px 0; }
```

INDEX.HTML HTML

```html
<p style="font-family: serif">serif: The quick brown fox 0123</p>
<p style="font-family: sans-serif">sans-serif: The quick brown fox 0123</p>
<p style="font-family: monospace">monospace: The quick brown fox 0123</p>
<p style="font-family: 'Lora', serif">Lora: The quick brown fox 0123</p>
<p style="font-family: 'Poppins', sans-serif">Poppins: The quick brown fox 0123</p>
```



HOW THE BROWSER DISPLAYS IT

4.3.3 Web fonts with Google Fonts

Google Fonts (fonts.google.com) hosts over 1,500 free, open-licence families. Choose a family and the styles you need, then copy the generated code: two preconnect hints and a stylesheet link into the head, and the font-family into your CSS. Request **only the weights you use**: every weight is a separate download.

IN THE HEAD HTML

```html
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?
family=Poppins:wght@400;600;700&family=Lora:ital@0;1&display=swap">
```

IN THE CSS CSS

```css
body { font-family: "Poppins", sans-serif; }
blockquote { font-family: "Lora", serif; font-style: italic; }
```

NOTE

Self-hosting the font files (download them, for example with the google-webfonts-helper tool, and use @font-face) avoids a third-party connection, can be faster, and avoids sending visitors' IP addresses to Google, which matters for GDPR compliance in Europe.

4.3.4 @font-face and font-display

`@font-face` defines a font family from font files. Each rule maps a family name plus a weight and style to a file. **WOFF2** is the best format (compressed, supported everywhere). `font-display` controls what users see while the font downloads:

FONTS.CSS: ONE FAMILY, TWO WEIGHTS CSS

```css
@font-face {
  font-family: "Poppins";
  src: url("../fonts/poppins-400.woff2") format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}
@font-face {
  font-family: "Poppins";
  src: url("../fonts/poppins-700.woff2") format("woff2");
  font-weight: 700;
  font-style: normal;
  font-display: swap;
}
```

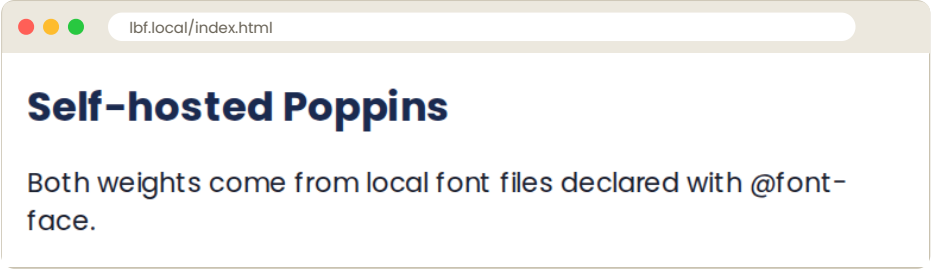
font-display	While loading	If the font arrives late
`auto`	browser default (usually like block)	
`block`	invisible text for up to about 3 s ("FOIT")	swaps in
`swap`	fallback font shown immediately ("FOUT")	swaps in whenever it arrives: best for body text
`fallback`	invisible about 100 ms, then fallback	swaps only if it arrives within about 3 s
`optional`	invisible about 100 ms, then fallback	used only if already cached or very fast: no layout shift

SELF-HOSTING A FONT CSS

```css
@font-face { font-family: "Brand"; src: url("fonts/Poppins-Bold.ttf"); font-weight: 700; font-display: swap; }
@font-face { font-family: "Brand"; src: url("fonts/Poppins-Regular.ttf"); font-weight: 400; font-display: swap; }
h2 { font-family: "Brand", sans-serif; font-weight: 700; color: #1a2a4f; margin: 0 0 6px; }
p  { font-family: "Brand", sans-serif; font-weight: 400; }
```

INDEX.HTML HTML

```html
<h2>Self-hosted Poppins</h2><p>Both weights come from local font files declared with @font-face.</p>
```



HOW THE BROWSER DISPLAYS IT

4.3.5 Variable fonts (overview)

A **variable font** is a single file containing a continuous range of styles along **axes**: weight (`wght`), width (`wdth`), slant, optical size and custom axes. One file replaces many static files and allows any weight, such as 450 or 650, and smooth weight animations. Declare the range in `@font-face`; use standard

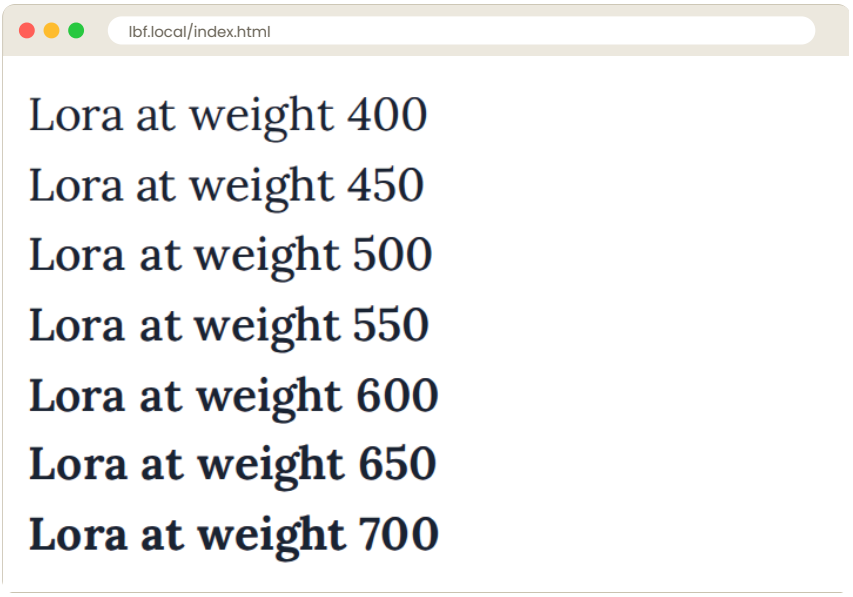
properties (`font-weight` , `font-stretch`) or `font-variation-settings` for custom axes. Lora, used below, is a variable font with a weight axis from 400 to 700:

SEVEN WEIGHTS FROM ONE VARIABLE FONT FILE CSS

```css
@font-face { font-family: "Lora VF"; src: url("fonts/Lora-Variable.ttf"); font-weight: 400 700; }
p { font-family: "Lora VF", serif; font-size: 26px; margin: 2px 0; }
```

INDEX.HTML HTML

```html
<p style="font-weight:400">Lora at weight 400</p><p style="font-weight:450">Lora at weight 450</p><p
style="font-weight:500">Lora at weight 500</p><p style="font-weight:550">Lora at weight 550</p><p
style="font-weight:600">Lora at weight 600</p><p style="font-weight:650">Lora at weight 650</p><p
style="font-weight:700">Lora at weight 700</p>
```



HOW THE BROWSER DISPLAYS IT

4.3.6 font-size, font-weight, font-style

Property	Values
`font-size`	lengths (`1rem` , `18px`), percentages, keywords `small` , `medium` (default 16px), `large` , `x-large` , and relative `smaller` / `larger`
`font-weight`	`100` (thin) to `900` (black); `400` = `normal` , `700` = `bold` ; `bolder` / `lighter` relative to the parent. Only weights the font provides look right; others are approximated or synthesised.
`font-style`	`normal` , `italic` (a true italic design if the font has one), `oblique` (slanted)
`font-variant`	`small-caps` , `font-variant-numeric: tabular-nums` (aligned digits in tables)

WEIGHTS AVAILABLE IN THE POPPINS FILES INSTALLED HERE CSS

```css
p { font-family: "Poppins", sans-serif; margin: 3px 0; font-size: 18px; }
```

INDEX.HTML HTML

```
<p style="font-weight:300">font-weight: 300</p><p style="font-weight:400">font-weight: 400</p><p
style="font-weight:500">font-weight: 500</p><p style="font-weight:700">font-weight: 700</p><p
style="font-style: italic">font-style: italic</p><p style="font-variant: small-caps">font-variant:
small-caps</p>
```



POPPINS PROVIDES 300, 400, 500 AND 700 (AND THEIR ITALICS), SO EACH WEIGHT IS A REAL DESIGN

4.3.7 font shorthand

`font` sets style, variant, weight, stretch, size, line height and family in one declaration, in this order: `font: [style] [variant] [weight] size[/line-height] family`. Size and family are **required**; anything omitted is **reset** to its initial value. Common uses: `font: inherit` on form controls, and compact type definitions.

THE FONT SHORTHAND CSS

```
h1     { font: 700 2.5rem/1.2 "Poppins", sans-serif; }
.quote { font: italic 1.25rem/1.6 Georgia, serif; }
button { font: inherit; }   /* use the page's font instead of the system's */
```

COMPUTED IN CHROMIUM

```
font-style: normal
font-weight: 700
font-size: 40px
line-height: 48px
font-family: Poppins, sans-serif
```

4.4 Text styling

4.4.1 color, text-align, text-decoration, text-transform

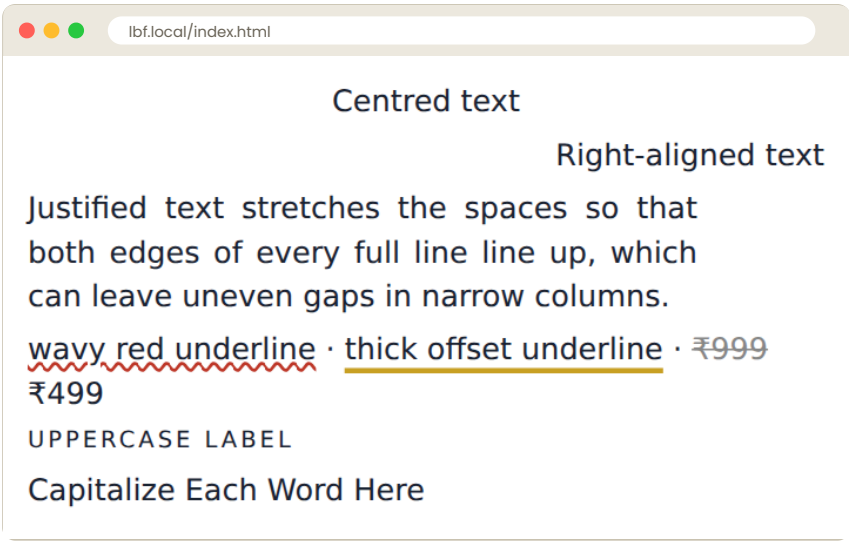
Property	Common values
`color`	any colour; inherited, so set it once on body
`text-align`	`start` (default), `end`, `left`, `right`, `center`, `justify`
`text-decoration`	shorthand for line (`underline`, `overline`, `line-through`, `none`), style (`solid`, `wavy`, `dotted`, `dashed`, `double`), colour and thickness; plus `text-underline-offset`
`text-transform`	`uppercase`, `lowercase`, `capitalize`, `none` (changes display only: copied text and screen readers keep the original)

ALIGNMENT, DECORATION AND TRANSFORMATION CSS

```css
p { margin: 6px 0; font-size: 17px; }
.center { text-align: center; } .right { text-align: right; }
.justify { text-align: justify; max-width: 380px; }
.wavy { text-decoration: underline wavy #c0392b; }
.fancy { text-decoration: underline 3px #c9a227; text-underline-offset: 5px; }
.through { text-decoration: line-through; color: #888; }
.upper { text-transform: uppercase; letter-spacing: 2px; font-size: 13px; }
.cap { text-transform: capitalize; }
```

INDEX.HTML HTML

```html
<p class="center">Centred text</p>
<p class="right">Right-aligned text</p>
<p class="justify">Justified text stretches the spaces so that both edges of every full line line up, which can leave uneven gaps in narrow columns.</p>
<p><span class="wavy">wavy red underline</span> · <span class="fancy">thick offset underline</span> · <span class="through">₹999</span> ₹499</p>
<p class="upper">uppercase label</p><p class="cap">capitalize each word here</p>
```



HOW THE BROWSER DISPLAYS IT

4.4.2 line-height, letter-spacing, word-spacing

`line-height` sets the height of each line box. Use a **unitless number** such as `1.5`: it multiplies each element's own font size and is inherited as a ratio. A length (`24px`) or em value is inherited as a fixed computed length, which breaks for headings with bigger text. Body text reads best at 1.4 to 1.7; large headings need less (1.1 to 1.3). `letter-spacing` adds space between characters (often slightly positive for uppercase labels, slightly negative for large headings); `word-spacing` between words.

line-height: 1.1

Line height 1.1: the lines are cramped and the eye struggles to find the start of the next line in long paragraphs.

line-height: 1.6

Line height 1.6: generous spacing makes long paragraphs comfortable to read on screen.

line-height: 2.4

Line height 2.4: so much space that the lines no longer read as one paragraph.

LINE HEIGHT AND READABILITY

COMPUTED IN CHROMIUM

```
h1 (40px) inside line-height: 1.5  -> 60px
h1 (40px) inside line-height: 24px -> 24px  (lines overlap!)
```

4.4.3 text-indent, text-shadow

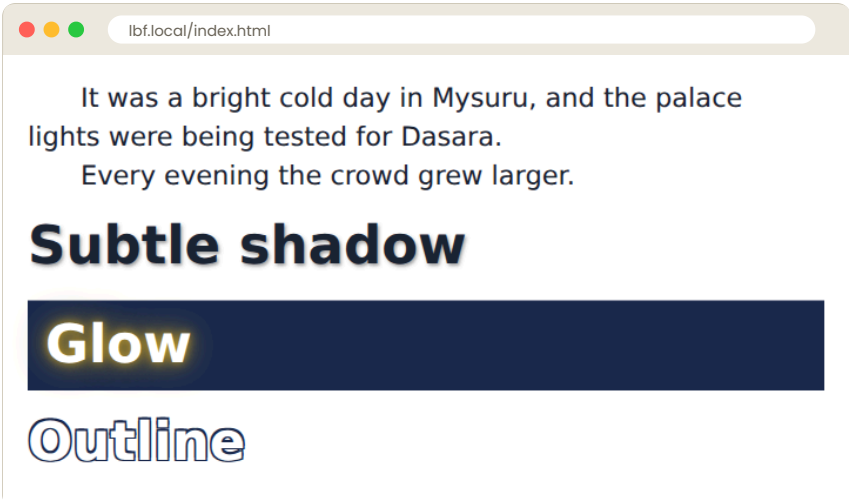
`text-indent` indents the first line of a block (book-style paragraphs; a negative value with padding creates a hanging indent). `text-shadow` takes x offset, y offset, blur and colour, and accepts several comma-separated shadows: subtle depth, glows, outlines or legibility over images.

TEXT-INDENT AND TEXT-SHADOW CSS

```css
.book p { text-indent: 2em; margin: 0; max-width: 420px; }
.s1 { text-shadow: 1px 1px 2px rgb(0 0 0 / .35); }
.s2 { color: #fff; text-shadow: 0 0 6px #c9a227, 0 0 18px #c9a227; }
.s3 { color: #fff; text-shadow: -1px -1px 0 #1a2a4f, 1px -1px 0 #1a2a4f, -1px 1px 0 #1a2a4f, 1px 1px 0 #1a2a4f; }
h2 { font-size: 30px; margin: 8px 0; }
```

INDEX.HTML HTML

```html
<div class="book"><p>It was a bright cold day in Mysuru, and the palace lights were being tested for Dasara.</p><p>Every evening the crowd grew larger.</p></div>
<h2 class="s1">Subtle shadow</h2><h2 class="s2" style="background:#1a2a4f;padding:4px 10px">Glow</h2>
<h2 class="s3">Outline</h2>
```



HOW THE BROWSER DISPLAYS IT

4.4.4 white-space, text-overflow: ellipsis, word-break, overflow-wrap

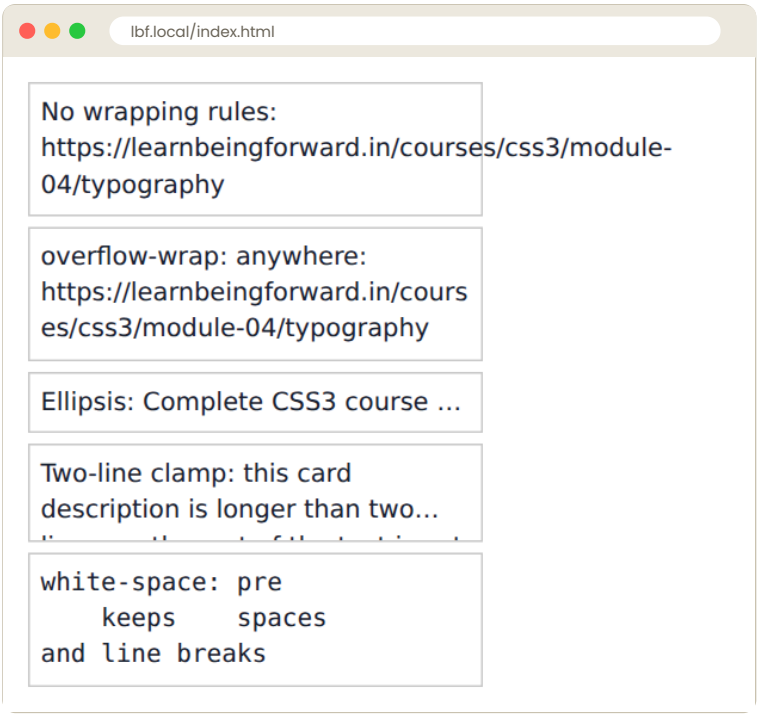
Property	Values and effect
`white-space`	`normal` (collapse spaces, wrap), `nowrap` (never wrap), `pre` (keep spaces and line breaks, no wrapping), `pre-wrap` (keep and wrap), `pre-line` (keep line breaks only)
`text-overflow`	`ellipsis` shows "…" when text is cut off; needs `overflow: hidden` and `white-space: nowrap` (single line) on a block with a width
`overflow-wrap`	`break-word` / `anywhere`: break long words or URLs only when they would overflow
`word-break`	`break-all`: break between any characters; `keep-all` for CJK text
`hyphens`	`auto`: hyphenate according to the `lang` attribute
`-webkit-line-clamp`	with `display: -webkit-box; -webkit-box-orient: vertical; overflow: hidden`: limit to N lines with an ellipsis (works in all browsers)

CONTROLLING WRAPPING AND OVERFLOW OF TEXT CSS

```css
.box { width: 240px; border: 1px solid #ccc; padding: 6px; margin: 6px 0; font-size: 14px; }
.ellipsis { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.clamp { display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 2; overflow: hidden;
}
.url { overflow-wrap: anywhere; }
.pre { white-space: pre; font-family: monospace; }
```

INDEX.HTML HTML

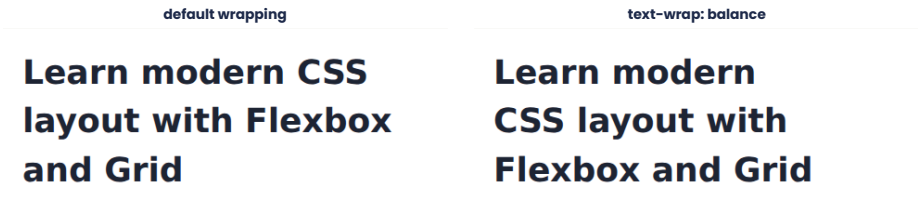
```html
<div class="box">No wrapping rules: https://learnbeingforward.in/courses/css3/module-
04/typography</div>
<div class="box url">overflow-wrap: anywhere: https://learnbeingforward.in/courses/css3/module-
04/typography</div>
<div class="box ellipsis">Ellipsis: Complete CSS3 course from basics to advanced with projects</div>
<div class="box clamp">Two-line clamp: this card description is longer than two lines, so the rest of
the text is cut off with an ellipsis at the end of the second line.</div>
<div class="box pre">white-space: pre
    keeps    spaces
and line breaks</div>
```



HOW THE BROWSER DISPLAYS IT

4.4.5 text-wrap: balance and pretty

`text-wrap: balance` (Baseline 2024) distributes words evenly across the lines of short multi-line text such as headings and captions, avoiding a long line followed by a single word. `text-wrap: pretty` optimises paragraphs to avoid an **orphan** (a very short last line), at a small performance cost, so it is meant for body text.



THE SAME HEADING AT THE SAME WIDTH

4.5 Styling links and lists

4.5.1 Link states: :link, :visited, :hover, :focus, :active

Links have states, each with a pseudo-class. Because several can apply at once and have equal specificity, the order matters: **LoVe HAte**, that is `:link`, `:visited`, `:hover`, `:active`, with `:focus` (or `:focus-visible`) styled alongside hover. For privacy, `:visited` can only change colours.

Pseudo-class	When
`:link`	a link not yet visited
`:visited`	a link to a URL in the browser history
`:hover`	the pointer is over it

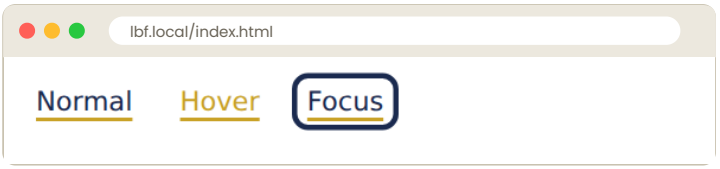
Pseudo-class	When
`:focus` / `:focus-visible`	it has keyboard focus (focus-visible: only when focus should be shown, e.g. via the keyboard)
`:active`	while it is being clicked/pressed

LINK STATES CSS

```css
a { color: #1a2a4f; text-decoration: underline 2px #c9a227; text-underline-offset: 4px; padding: 2px 4px; margin-right: 14px; border-radius: 4px; }
a:visited { color: #6b3fa0; }
a:hover { color: #c9a227; text-decoration-color: currentColor; }
a:focus-visible { outline: 3px solid #1a2a4f; outline-offset: 2px; }
a:active { color: #c0392b; }
```

INDEX.HTML HTML

```html
<p><a href="#a" class="l1">Normal</a> <a href="#b" class="l2">Hover</a> <a href="#c" class="l3">Focus</a></p>
```



SECOND LINK HOVERED, THIRD FOCUSED WITH THE KEYBOARD

4.5.2 list-style-type, list-style-position, custom markers

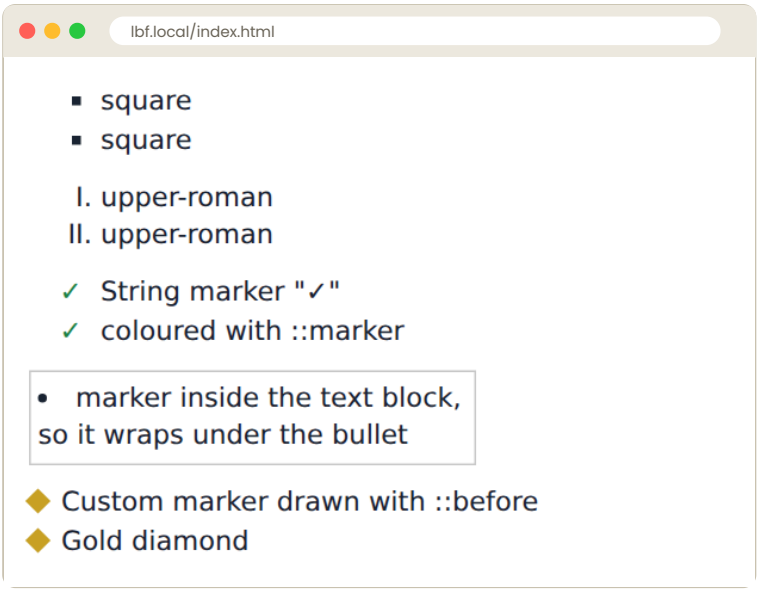
`list-style-type` chooses the marker: `disc`, `circle`, `square`, `decimal`, `decimal-leading-zero`, `lower-alpha`, `upper-roman`, `none`, other languages' numbering (`devanagari`, `kannada`) or **any string** such as `"✓ "`. `list-style-position: inside` puts the marker inside the text block. The `::marker` pseudo-element styles the marker's colour and font, and `@counter-style` defines entirely custom numbering. For full control, remove the marker and draw your own with `::before`.

LIST MARKERS CSS

```css
ul, ol { margin: 4px 0 10px; }
.square { list-style-type: square; }
.roman { list-style-type: upper-roman; }
.check { list-style-type: "✓  "; }
.check li::marker { color: #1e8449; font-weight: 700; }
.inside { list-style-position: inside; border: 1px solid #ccc; padding: 4px; width: 240px; }
.custom { list-style: none; padding: 0; }
.custom li::before { content: ""; display: inline-block; width: 10px; height: 10px; margin-right: 8px;
                     background: #c9a227; transform: rotate(45deg); }
```

INDEX.HTML HTML

```html
<ul class="square"><li>square</li><li>square</li></ul>
<ol class="roman"><li>upper-roman</li><li>upper-roman</li></ol>
<ul class="check"><li>String marker "✓"</li><li>coloured with ::marker</li></ul>
<ul class="inside"><li>marker inside the text block, so it wraps under the bullet</li></ul>
<ul class="custom"><li>Custom marker drawn with ::before</li><li>Gold diamond</li></ul>
```



HOW THE BROWSER DISPLAYS IT

PRACTICE CHECKPOINT

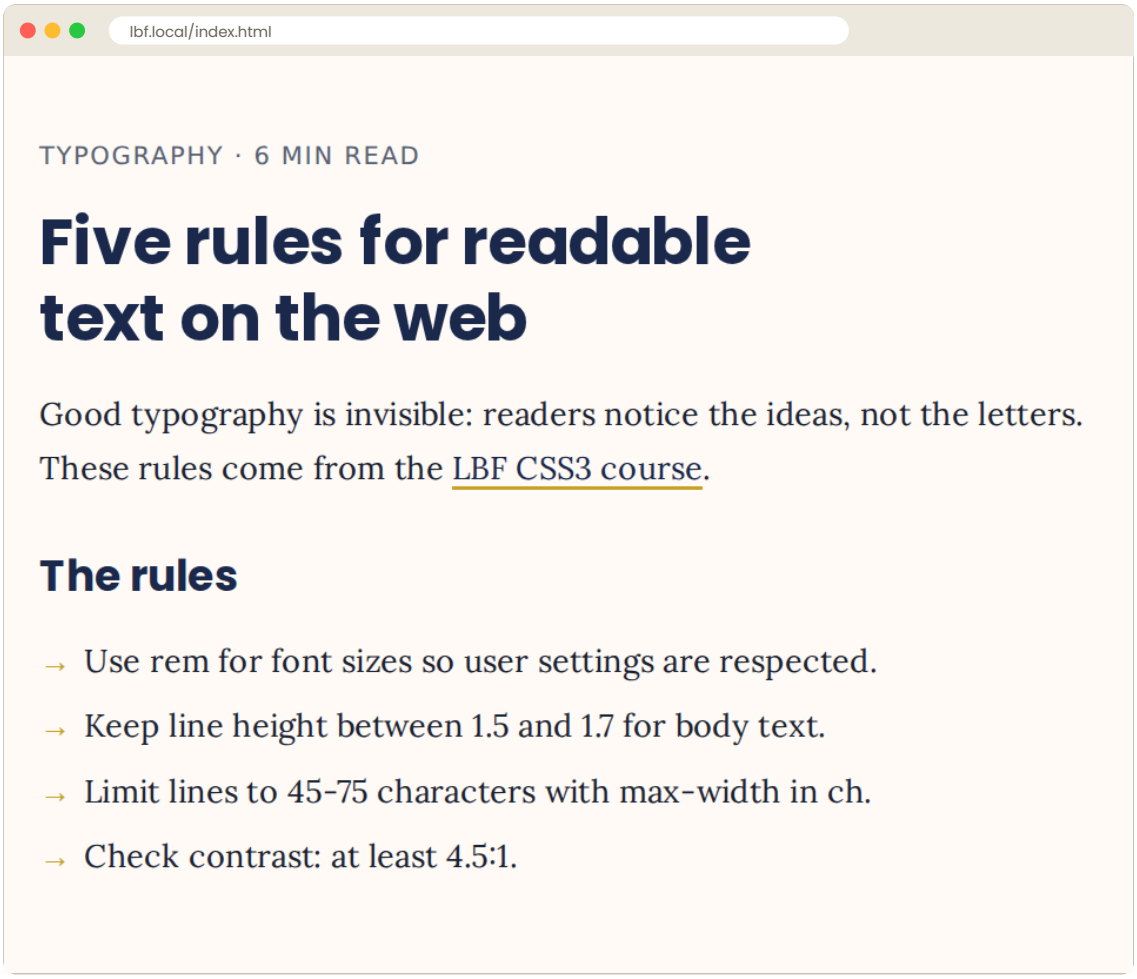
A styled **blog article** with a Google Font, rem-based typography scale, comfortable line height, styled links and custom list markers.

BLOG.CSS: SAMPLE SOLUTION CSS

```
@font-face { font-family: "Lora"; src: url("fonts/Lora-Variable.ttf"); font-weight: 400 700; font-display: swap; }
@font-face { font-family: "Poppins"; src: url("fonts/Poppins-Bold.ttf"); font-weight: 700; font-display: swap; }
html { font-size: 100%; }                          /* 16px, or the user's setting */
body { font-family: "Lora", Georgia, serif; font-size: 1.125rem; line-height: 1.7;
       color: #1d2433; background: #fffdf8; margin: 0; }
article { max-width: 62ch; margin: 0 auto; padding: 2rem 1.25rem; }
h1, h2 { font-family: "Poppins", sans-serif; line-height: 1.2; color: #1a2a4f; text-wrap: balance; }
h1 { font-size: 2.25rem; margin: 0 0 .5rem; letter-spacing: -0.01em; }
h2 { font-size: 1.5rem; margin: 2rem 0 .5rem; }
.meta { font-family: system-ui, sans-serif; font-size: .875rem; color: #5c6475; text-transform: uppercase; letter-spacing: .08em; }
a { color: #1a2a4f; text-decoration: underline 2px #c9a227; text-underline-offset: .2em; }
a:hover { color: #9a7a14; }
a:focus-visible { outline: 3px solid #1a2a4f; outline-offset: 2px; }
ul { list-style: none; padding-left: 0; }
li { padding-left: 1.6rem; position: relative; margin: .4rem 0; }
li::before { content: "→"; position: absolute; left: 0; color: #c9a227; font-weight: 700; }
```

INDEX.HTML HTML

```
<article>
  <p class="meta">Typography · 6 min read</p>
  <h1>Five rules for readable text on the web</h1>
  <p>Good typography is invisible: readers notice the ideas, not the letters. These rules come from
     the <a href="#">LBF CSS3 course</a>.</p>
  <h2>The rules</h2>
  <ul>
    <li>Use rem for font sizes so user settings are respected.</li>
    <li>Keep line height between 1.5 and 1.7 for body text.</li>
    <li>Limit lines to 45-75 characters with max-width in ch.</li>
    <li>Check contrast: at least 4.5:1.</li>
  </ul>
</article>
```



THE STYLED ARTICLE (FONTS SELF-HOSTED HERE; WITH GOOGLE FONTS, USE THE LINK FROM 4.3.3)

MODULE 05 · BEGINNER

The box model

Minimum duration **2 days** | Level **Beginner**

Every element is a rectangular box; understanding the box model is the foundation of all layout. This module covers the four parts of the box (content, padding, border, margin), width and height with their minimum and maximum limits, content-box versus border-box sizing, intrinsic sizing with min-content, max-content and fit-content, aspect-ratio, padding and margin shorthands with one to four values, centring with margin: 0 auto, negative margins, margin collapsing, borders and border-radius (including circles), outline versus border, an introduction to logical properties, and inspecting all of it in DevTools. Every size quoted was measured in Chromium.

IN THIS MODULE

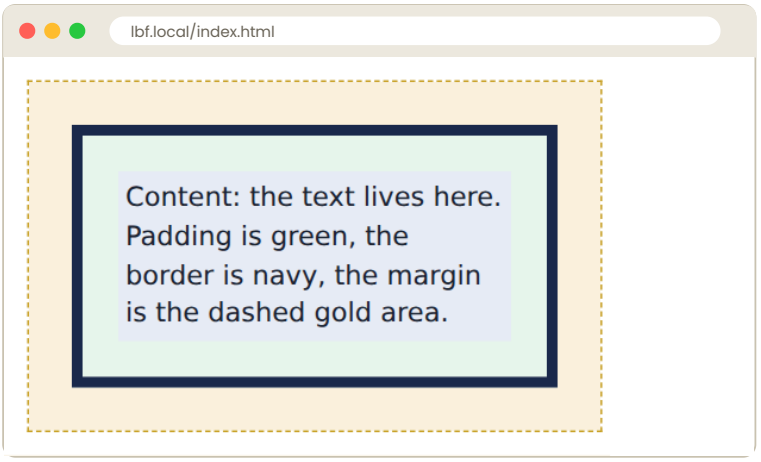
5.1	Parts of the box	5.4	Borders
5.2	Sizing	5.5	Logical properties (introduction)
5.3	Padding and margin	5.6	Inspecting the box model in DevTools

5.1 Parts of the box

The browser lays out every element as a rectangular **box**, even if it looks round or the text inside wraps across lines. From the inside out, each box has four layers. Together they decide how much space the element takes and how far it sits from its neighbours.



THE CSS BOX MODEL: CONTENT, PADDING, BORDER, MARGIN



THE FOUR LAYERS DRAWN WITH REAL CSS

5.1.1 Content

The **content box** holds the element's content: text, images, child elements. Its size comes from `width` and `height` if set, otherwise from the content and the layout (a block fills its container's width; its height grows with its content).

5.1.2 Padding

Padding is the space between the content and the border, **inside** the element. It takes the element's background colour or image, and it makes clickable areas bigger. Padding cannot be negative.

5.1.3 Border

The **border** surrounds the padding. It has a width, a style and a colour (5.4), and it is part of the element's visible size.

5.1.4 Margin

Margin is transparent space **outside** the border, pushing other elements away. It never shows the element's background. Vertical margins between blocks can **collapse** into one (5.3.4), and margins can be negative or `auto`.

Layer	Inside or outside	Shows background	Can be negative	Clickable
Content	inside	yes	-	yes
Padding	inside	yes	no	yes
Border	edge	its own colour	no	yes
Margin	outside	no (transparent)	yes	no

5.2 Sizing

5.2.1 width and height

`width` and `height` accept lengths (`300px`, `20rem`), percentages of the **containing block**, the keyword `auto` (the default) and intrinsic keywords (5.2.4). They apply to block and inline-block elements, images and form controls, but **not** to plain inline elements such as `<span>` or `<a>`, whose size comes from their text.

	width: auto (default)	height: auto (default)
Block element	fills the container's width (minus margins)	just tall enough for the content
Percentage	% of the container's width	% of the container's height, **only if that height is definite**; otherwise it behaves as auto

WATCH OUT

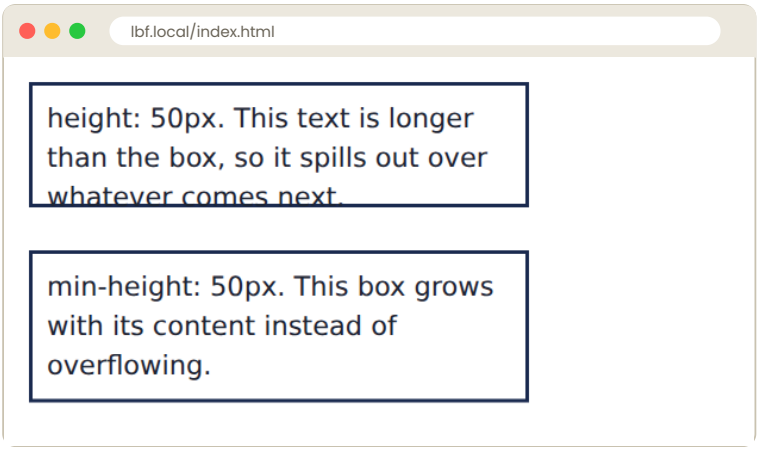
Avoid fixed `height` on boxes that contain text: when the text grows (translation, user font size, more content), it overflows the box. Use `min-height` instead, so the box can grow.

HEIGHT VS MIN-HEIGHT CSS

```css
.fixed, .min { width: 260px; border: 2px solid #1a2a4f; padding: 8px; margin: 6px 0 24px; background: #fff; }
.fixed { height: 50px; }
.min { min-height: 50px; }
```

INDEX.HTML HTML

```html
<div class="fixed">height: 50px. This text is longer than the box, so it spills out over whatever comes next.</div>
<div class="min">min-height: 50px. This box grows with its content instead of overflowing.</div>
```



HOW THE BROWSER DISPLAYS IT

5.2.2 min-width, max-width, min-height, max-height

The `min-*` and `max-*` properties set limits; they win over `width` / `height` (and `min` wins over `max` if they conflict). The single most useful layout rule in CSS is a fluid container with a maximum width:

FLUID WIDTH WITH A LIMIT CSS

```css
.container {
  width: 100%;          /* fill small screens            */
  max-width: 1100px;    /* but never wider than 1100px   */
  margin-inline: auto;  /* centre it when there is room  */
  padding-inline: 1rem; /* breathing room at the edges   */
}
img { max-width: 100%; height: auto; }   /* images never overflow their container */
```

COMPUTED IN CHROMIUM

```
viewport 900px
.container (100%, max 600px): 600px
.narrow (50%, min 300px):     436px
```

COMPUTED IN CHROMIUM

```
viewport 428px
.container (100%, max 600px): 400px
.narrow (50%, min 300px):     300px  (50% would be 200px)
```

5.2.3 box-sizing: content-box vs border-box

By default (`box-sizing: content-box`), `width` sets only the content box; padding and border are **added**. With `border-box`, `width` is the width of the whole visible box, and padding and border are taken from inside it. Border-box makes layout arithmetic intuitive (two 50% columns with padding still fit), which is why the reset in Module 2 applies it to everything.



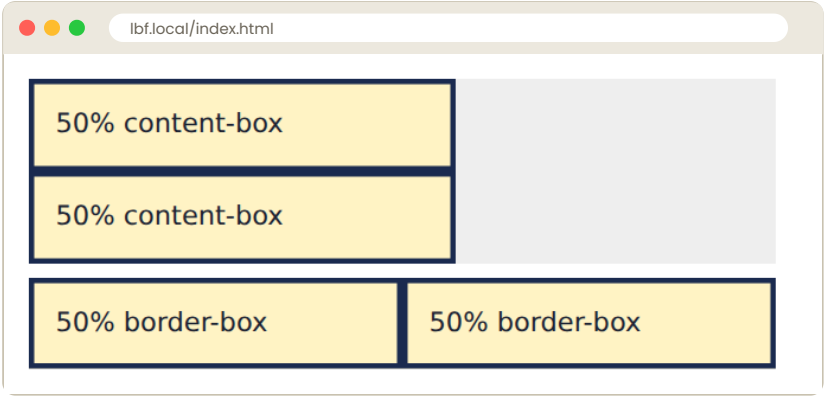
THE SAME WIDTH: 200PX UNDER EACH BOX-SIZING VALUE (SCHEMATIC)

TWO 50% COLUMNS — CSS

```css
.row { width: 420px; background: #eee; margin-bottom: 8px; }
.col { width: 50%; padding: 12px; border: 3px solid #1a2a4f; float: left; background: #fff3c4; }
.row::after { content: ""; display: block; clear: both; }
.bb .col { box-sizing: border-box; }
```

INDEX.HTML — HTML

```html
<div class="row"><div class="col">50% content-box</div><div class="col">50% content-box</div></div>
<div class="row bb"><div class="col">50% border-box</div><div class="col">50% border-box</div></div>
```



WITH CONTENT-BOX THE SECOND COLUMN NO LONGER FITS (IT WRAPS UNDERNEATH); WITH BORDER-BOX BOTH FIT

5.2.4 Intrinsic sizing: min-content, max-content, fit-content

Instead of a fixed size, a box can be sized by its content:

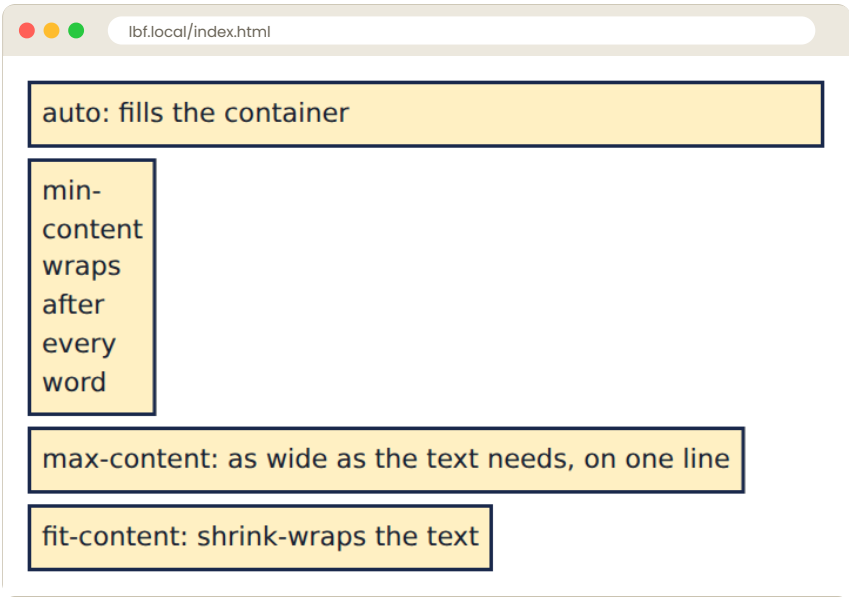
Keyword	Size
`min-content`	the narrowest the box can be without overflowing: the width of its longest word or widest image
`max-content`	the width the content would take with no wrapping at all (can overflow its container)
`fit-content`	max-content, but never wider than the available space (then it wraps): `min(max-content, max(min-content, available))`
`fit-content(300px)`	like fit-content, with 300px as the limit (in grid tracks)

INTRINSIC WIDTHS CSS

```css
div { border: 2px solid #1a2a4f; background: #fff3c4; padding: 6px; margin: 6px 0; }
.min { width: min-content; } .max { width: max-content; } .fit { width: fit-content; }
.auto { width: auto; }
```

INDEX.HTML HTML

```html
<div class="auto">auto: fills the container</div>
<div class="min">min-content wraps after every word</div>
<div class="max">max-content: as wide as the text needs, on one line</div>
<div class="fit">fit-content: shrink-wraps the text</div>
```



HOW THE BROWSER DISPLAYS IT

`width: fit-content` with `margin-inline: auto` is a neat way to centre a button or a tag whose width depends on its text.

5.2.5 aspect-ratio

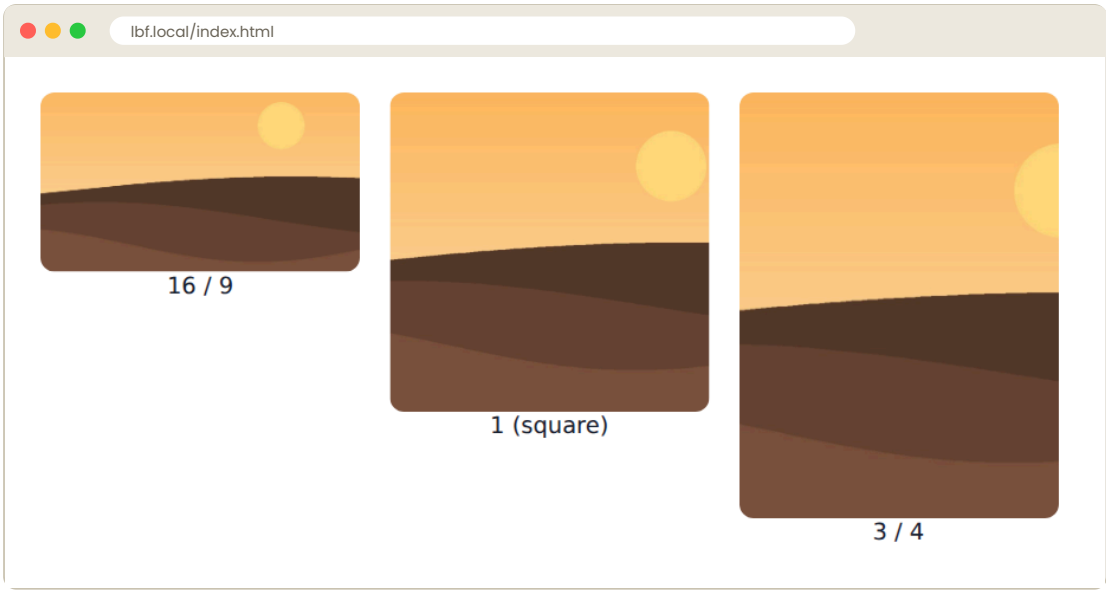
`aspect-ratio: 16 / 9` keeps a box at a fixed shape: set one dimension (usually width) and the other is calculated. It is ideal for video embeds, cards, thumbnails and avatars, and replaces the old "padding-top: 56.25%" hack. Combine with `object-fit: cover` (Module 17) for images so they fill the box without distortion.

ONE PHOTO, THREE ASPECT RATIOS CSS

```css
.thumb { width: 180px; display: inline-block; margin: 6px; vertical-align: top; text-align: center;
font-size: 13px; }
.thumb img { width: 100%; object-fit: cover; display: block; border-radius: 8px; }
.wide img { aspect-ratio: 16 / 9; }
.square img { aspect-ratio: 1; }
.portrait img { aspect-ratio: 3 / 4; }
```

INDEX.HTML HTML

```html
<div class="thumb wide"><img src="mysuru-palace.jpg" alt="">16 / 9</div>
<div class="thumb square"><img src="mysuru-palace.jpg" alt="">1 (square)</div>
<div class="thumb portrait"><img src="mysuru-palace.jpg" alt="">3 / 4</div>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM
`width 320px, aspect-ratio 16/9 -> height 180px`

5.3 Padding and margin

5.3.1 Individual sides and shorthand (1, 2, 3 and 4 values)

Each side has its own property (`padding-top`, `margin-left`…). The shorthands `padding` and `margin` take one to four values, applied **clockwise from the top**: top, right, bottom, left (remember "TRouBLe").

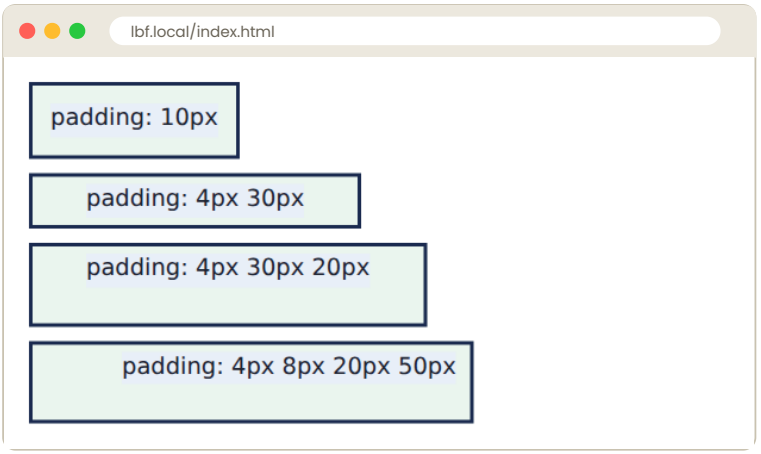
Shorthand	Top	Right	Bottom	Left
`margin: 10px;`	10	10	10	10
`margin: 10px 20px;`	10	20	10	20
`margin: 10px 20px 30px;`	10	20	30	20
`margin: 10px 20px 30px 40px;`	10	20	30	40

PADDING SHORTHANDS (GREEN = PADDING) CSS

```css
div { background: #eaf5ee; border: 2px solid #1a2a4f; margin-bottom: 8px; font-size: 13px; width: max-content; }
div span { background: #e8eff8; display: block; }
```

INDEX.HTML HTML

```html
<div style="padding: 10px"><span>padding: 10px</span></div>
<div style="padding: 4px 30px"><span>padding: 4px 30px</span></div>
<div style="padding: 4px 30px 20px"><span>padding: 4px 30px 20px</span></div>
<div style="padding: 4px 8px 20px 50px"><span>padding: 4px 8px 20px 50px</span></div>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
margin: 10px 20px 30px ->
  top 10px, right 20px, bottom 30px, left 20px
```

5.3.2 margin: 0 auto for centring

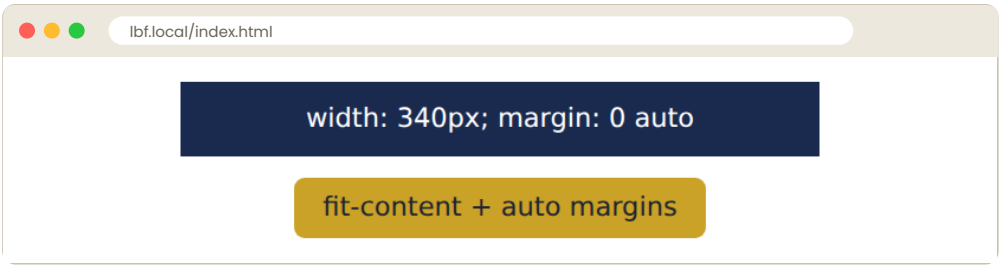
When a block element has a **width smaller than its container**, setting the left and right margins to `auto` splits the leftover space equally, centring it horizontally. It does not work without a width (an auto-width block already fills the container) and it does not centre vertically in normal flow (use Flexbox or Grid, Module 7.7).

HORIZONTAL CENTRING WITH AUTO MARGINS CSS

```css
.page { width: 340px; margin: 0 auto; background: #1a2a4f; color: #fff; padding: 10px; text-align: center; }
.btn { display: block; width: fit-content; margin: 12px auto 0; background: #c9a227; padding: 6px 16px; border-radius: 6px; }
```

INDEX.HTML HTML

```html
<div class="page">width: 340px; margin: 0 auto</div><span class="btn">fit-content + auto margins</span>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
container 772px (800 - 2x14 body margin): margin-left 216px, margin-right 216px
```

5.3.3 Negative margins

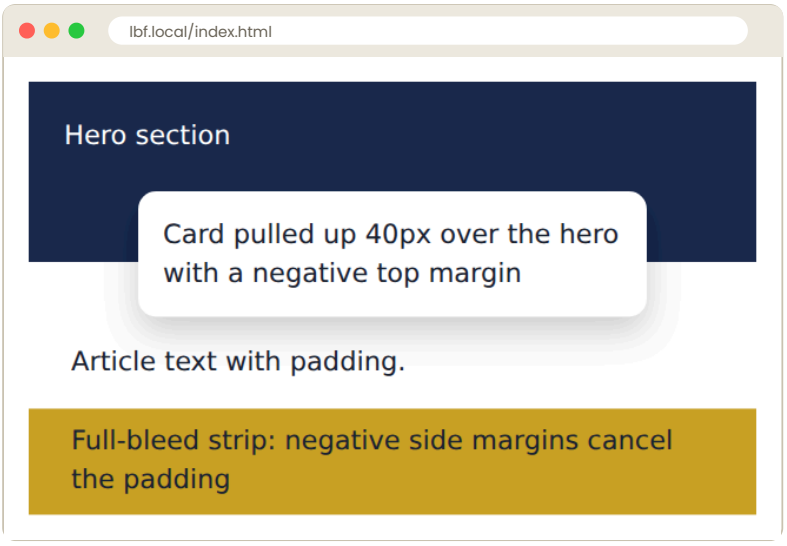
Margins can be negative: `margin-top: -40px` pulls an element up over the previous one; a negative left or right margin pulls it outward or lets it extend beyond its container ("full-bleed" images inside a padded article). Use them sparingly, since they make layouts harder to reason about; `gap`, transforms and Grid often express the intent better.

TWO USES OF NEGATIVE MARGINS CSS

```css
.hero { background: #1a2a4f; color: #fff; padding: 20px 20px 60px; }
.card { background: #fff; width: 260px; margin: -40px auto 0; padding: 14px; border-radius: 10px; box-shadow: 0 8px 20px rgb(0 0 0 / .2); position: relative; }
.article { padding: 0 24px; } .bleed { margin: 10px -24px; background: #c9a227; padding: 8px 24px; }
```

INDEX.HTML HTML

```html
<div class="hero">Hero section</div><div class="card">Card pulled up 40px over the hero with a negative top margin</div><div class="article"><p>Article text with padding.</p><div class="bleed">Full-bleed strip: negative side margins cancel the padding</div></div>
```



HOW THE BROWSER DISPLAYS IT

5.3.4 Margin collapsing

In normal block flow, **vertical** margins of adjacent blocks do not add up: they **collapse** into a single margin equal to the **larger** of the two (or the sum of the most positive and most negative when one is negative). Horizontal margins never collapse. It happens in three situations:

1. **Adjacent siblings**: a paragraph with margin-bottom 30px followed by one with margin-top 20px are 30px apart, not 50px.
2. **Parent and first/last child**: if nothing separates them (no border, padding, inline content or new formatting context), a child's top margin "escapes" through the parent and becomes the parent's margin.
3. **Empty blocks**: the top and bottom margins of an empty element collapse with each other.

SIBLING COLLAPSE AND PARENT-CHILD "ESCAPE" CSS

```css
.a { margin-bottom: 30px; background: #e8eff8; }
.b { margin-top: 20px; background: #fff3c4; }
.parent { background: #1a2a4f; color: #fff; }
.parent h3 { margin-top: 30px; color: #c9a227; }
p, h3 { margin-left: 0; padding: 4px; }
```

INDEX.HTML HTML

```html
<p class="a">margin-bottom: 30px</p>
<p class="b">margin-top: 20px. The gap above me is 30px, not 50px</p>
<div class="parent"><h3>Child with margin-top: 30px</h3></div>
```

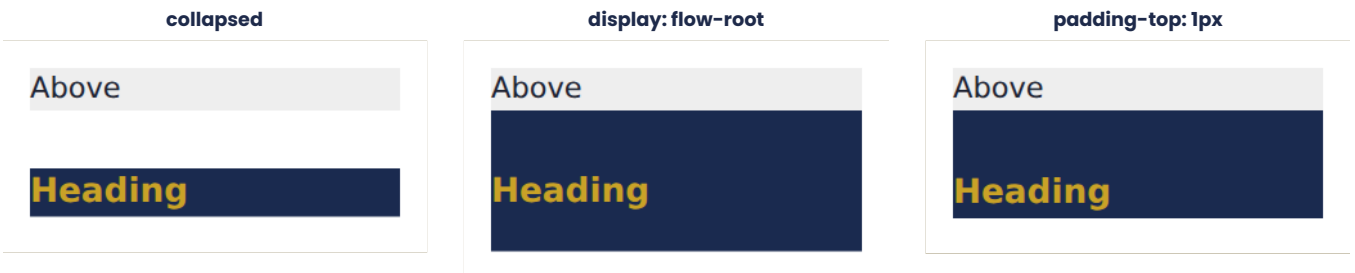


THE HEADING'S TOP MARGIN ESCAPED THE NAVY PARENT AND PUSHED THE WHOLE PARENT DOWN

COMPUTED IN CHROMIUM

```
gap between .a and .b: 30px  (30px and 20px collapsed to the larger)
```

Ways to prevent collapsing between a parent and its child: give the parent padding or a border, or make it a new block formatting context (`display: flow-root`, Module 7), or make it a flex or grid container (margins never collapse inside Flexbox and Grid). Many modern layouts avoid the issue by spacing children with `gap` or by using only bottom margins.



STOPPING THE PARENT-CHILD COLLAPSE

5.4 Borders

5.4.1 border-width, border-style, border-color

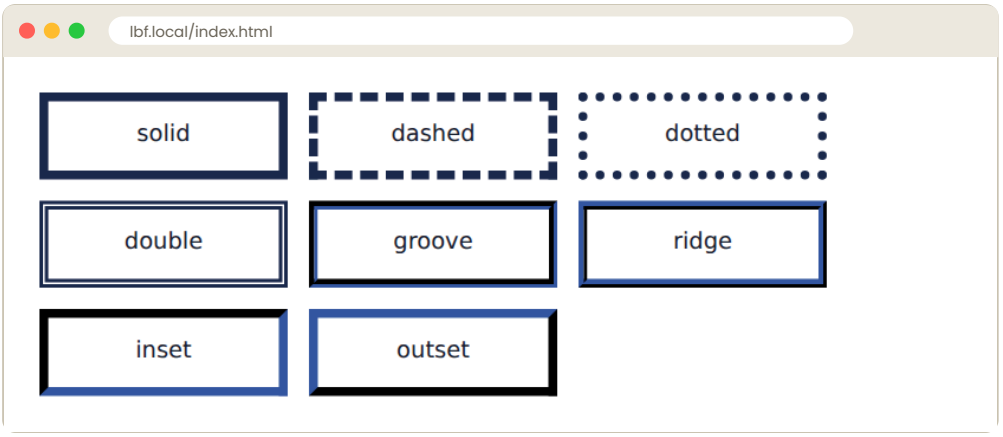
A border is drawn only when `border-style` is set (the default is `none`). `border-width` accepts lengths or `thin` / `medium` / `thick`; `border-color` defaults to currentColor.

BORDER-STYLE VALUES CSS

```
div { display: inline-block; width: 110px; padding: 10px; margin: 6px; text-align: center; font-size:
13px;
      border-width: 5px; border-color: #1a2a4f; }
```

INDEX.HTML HTML

```
<div style="border-style:solid">solid</div><div style="border-style:dashed">dashed</div><div
style="border-style:dotted">dotted</div><div style="border-style:double">double</div><div
style="border-style:groove">groove</div><div style="border-style:ridge">ridge</div><div style="border-
style:inset">inset</div><div style="border-style:outset">outset</div>
```



HOW THE BROWSER DISPLAYS IT

5.4.2 Individual sides and shorthand

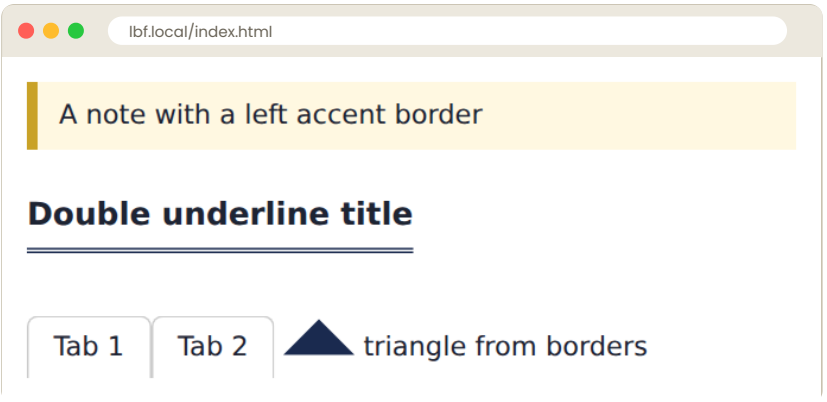
`border: 2px solid #1a2a4f;` sets width, style and colour for all four sides in any order. Each side has its own shorthand (`border-top`, `border-left`…) and longhands (`border-left-color`). Setting one side is a common way to create accents:

SINGLE-SIDE BORDERS CSS

```
.note { border-left: 6px solid #c9a227; background: #fff8e1; padding: 8px 12px; margin: 8px 0; }
.title { border-bottom: 3px double #1a2a4f; padding-bottom: 4px; display: inline-block; }
.tab { border: 1px solid #ccc; border-bottom: none; border-radius: 6px 6px 0 0; padding: 6px 14px;
display: inline-block; }
.triangle { width: 0; height: 0; border: 20px solid transparent; border-bottom-color: #1a2a4f;
display: inline-block; }
```

INDEX.HTML HTML

```
<p class="note">A note with a left accent border</p><h3 class="title">Double underline title</h3><br>
<span class="tab">Tab 1</span><span class="tab">Tab 2</span> <span class="triangle"></span> triangle
from borders
```



HOW THE BROWSER DISPLAYS IT

5.4.3 border-radius and circles

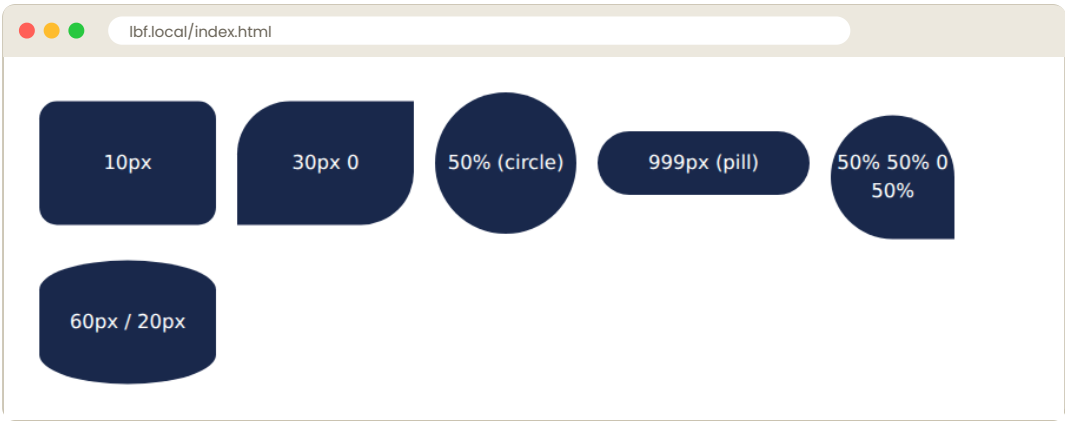
`border-radius` rounds the corners (it clips the background even without a border). One to four values work clockwise from the top-left; a slash gives elliptical corners (`40px / 20px`). `50%` on a square makes a **circle**; a very large value (`999px`) on a rectangle makes a **pill**.

BORDER-RADIUS SHAPES CSS

```css
.s { display: inline-grid; place-items: center; width: 100px; height: 70px; margin: 6px; background: #1a2a4f; color: #fff; font-size: 11px; text-align: center; }
.r1 { border-radius: 10px; } .r2 { border-radius: 30px 0; } .r3 { border-radius: 50%; width: 80px; height: 80px; }
.r4 { border-radius: 999px; height: 36px; width: 120px; } .r5 { border-radius: 50% 50% 0 50%; width: 70px; height: 70px; }
.r6 { border-radius: 60px / 20px; }
```

INDEX.HTML HTML

```html
<div class="s r1">10px</div><div class="s r2">30px 0</div><div class="s r3">50% (circle)</div><div class="s r4">999px (pill)</div><div class="s r5">50% 50% 0 50%</div><div class="s r6">60px / 20px</div>
```



HOW THE BROWSER DISPLAYS IT

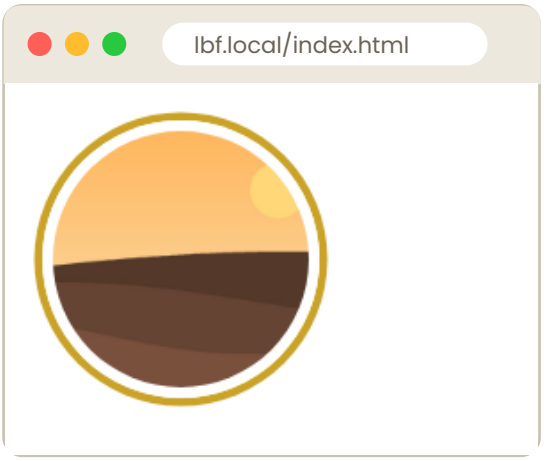
A CIRCULAR AVATAR CSS

```css
.avatar { width: 96px; height: 96px; border-radius: 50%; object-fit: cover; border: 4px solid #fff; box-shadow: 0 0 0 3px #c9a227; }
```

INDEX.HTML

HTML

```
<img class="avatar" src="mysuru-palace.jpg" alt="Asha Rao">
```



WORKS WITH ANY PHOTO: ASPECT-RATIO-INDEPENDENT THANKS TO OBJECT-FIT

5.4.4 outline vs border

An **outline** is drawn outside the border and **does not take up space**: adding or changing it never moves anything. That makes it ideal for focus indicators and debugging. `outline-offset` moves it away from (or into) the border. Outlines follow border-radius in all modern browsers but cannot be set per side.

	border	outline
Takes up space (affects layout)	yes	no
Per-side control	yes	no
Offset from the box	no	`outline-offset`
Typical use	design	focus rings, debugging

OUTLINE ADDS NO SIZE; BORDER DOES

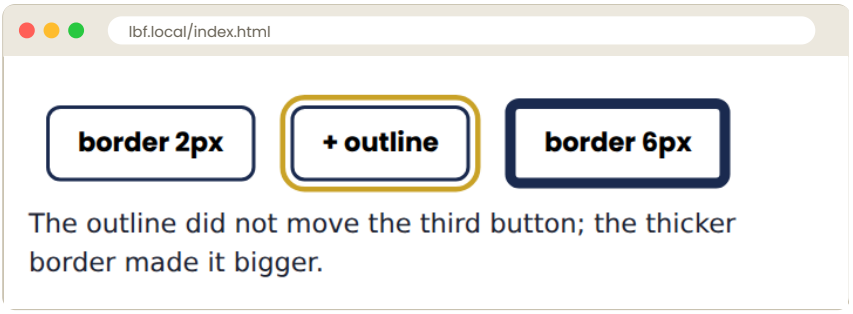
CSS

```
button { font: 600 15px "Poppins", sans-serif; padding: 8px 16px; margin: 10px; border-radius: 8px;
background: #fff; }
.b1 { border: 2px solid #1a2a4f; }
.b2 { border: 2px solid #1a2a4f; outline: 3px solid #c9a227; outline-offset: 3px; }
.b3 { border: 6px solid #1a2a4f; }
```

INDEX.HTML

HTML

```
<button class="b1">border 2px</button><button class="b2">+ outline</button><button class="b3">border
6px</button><p style="margin:0">The outline did not move the third button; the thicker border made it
bigger.</p>
```



HOW THE BROWSER DISPLAYS IT

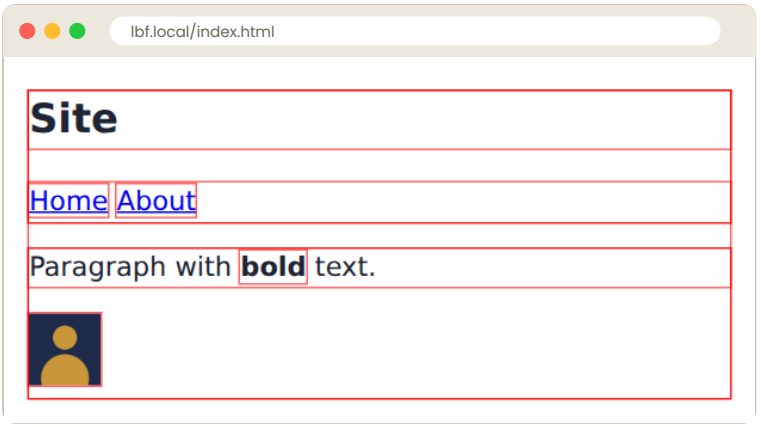
Debugging trick: `* { outline: 1px solid red; }` shows every box on a page without changing the layout.

OUTLINING EVERY ELEMENT CSS

```css
* { outline: 1px solid rgb(255 0 0 / .6); }
```

INDEX.HTML HTML

```html
<header><h2>Site</h2><nav><a href="#">Home</a> <a href="#">About</a></nav></header><main><p>Paragraph with <b>bold</b> text.</p><img src="avatar.png" width="40" alt=""></main>
```



HOW THE BROWSER DISPLAYS IT

5.5 Logical properties (introduction)

5.5.1 margin-inline, padding-block, inline-size

Physical properties (`left`, `right`, `top`, `bottom`, `width`, `height`) are tied to the screen. **Logical properties** are tied to the **writing direction**: the **inline** direction is the way text runs (left to right in English, right to left in Arabic and Urdu, top to bottom in vertical Japanese) and the **block** direction is the way paragraphs stack. In English they map as below; in right-to-left text, `margin-inline-start` automatically becomes the right margin. Module 16 goes deeper.

Logical property	In left-to-right horizontal text	Shorthand
`inline-size` / `block-size`	width / height	
`margin-inline-start` / `-end`	margin-left / margin-right	`margin-inline: 1rem 2rem`
`margin-block-start` / `-end`	margin-top / margin-bottom	`margin-block: 1rem`

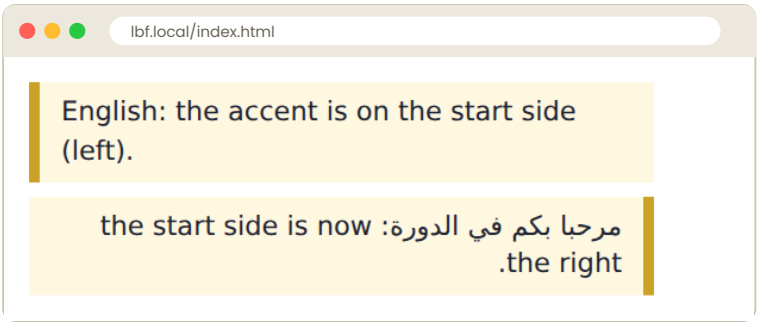
Logical property	In left-to-right horizontal text	Shorthand
`padding-inline`, `padding-block`	left+right / top+bottom	
`border-inline-start`	border-left	
`inset-inline-start`	left (positioning)	`inset-inline: 0`
`text-align: start`	left	

ONE RULE, TWO WRITING DIRECTIONS CSS

```
.note { border-inline-start: 6px solid #c9a227; padding-inline: 12px; padding-block: 6px; margin-block: 8px; background: #fff8e1; inline-size: 320px; }
```

INDEX.HTML HTML

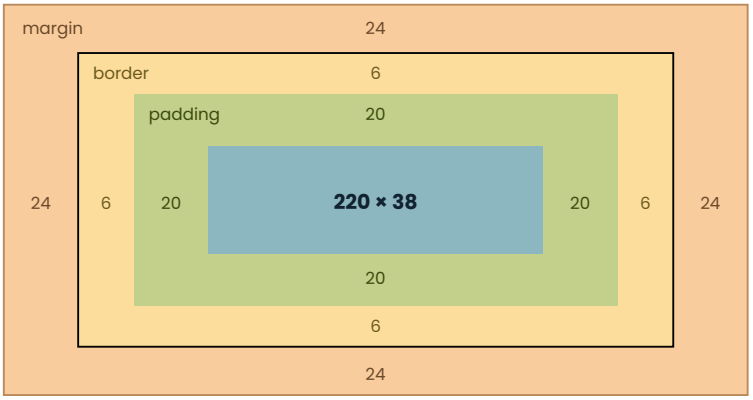
```
<p class="note" dir="ltr">English: the accent is on the start side (left).</p><p class="note" dir="rtl" lang="ar">مرحبا بكم في الدورة: the start side is now the right.</p>
```



HOW THE BROWSER DISPLAYS IT

5.6 Inspecting the box model in DevTools

Select an element in DevTools and open the **Computed** tab (or scroll the Styles pane): the box-model diagram shows the content size and every padding, border and margin value. Hovering the element in the DOM tree highlights it on the page with colour-coded overlays: **blue** content, **green** padding, **yellow** border and **orange** margin. You can double-click any number in the diagram to edit it.



THE DEVTOOLS BOX-MODEL DIAGRAM (CHROME COLOURS) FOR A BOX WITH WIDTH 220PX, PADDING 20PX, BORDER 6PX, MARGIN 24PX

JavaScript exposes the same measurements, which is how you can verify a layout precisely:

COMPUTED IN CHROMIUM

```
clientWidth  (content + padding):          260
offsetWidth  (content + padding + border): 272
getBoundingClientRect().width:             272
margin-left: 24px  -> the box starts at x = 38 (14px body margin + 24px)
```

PRACTICE CHECKPOINT

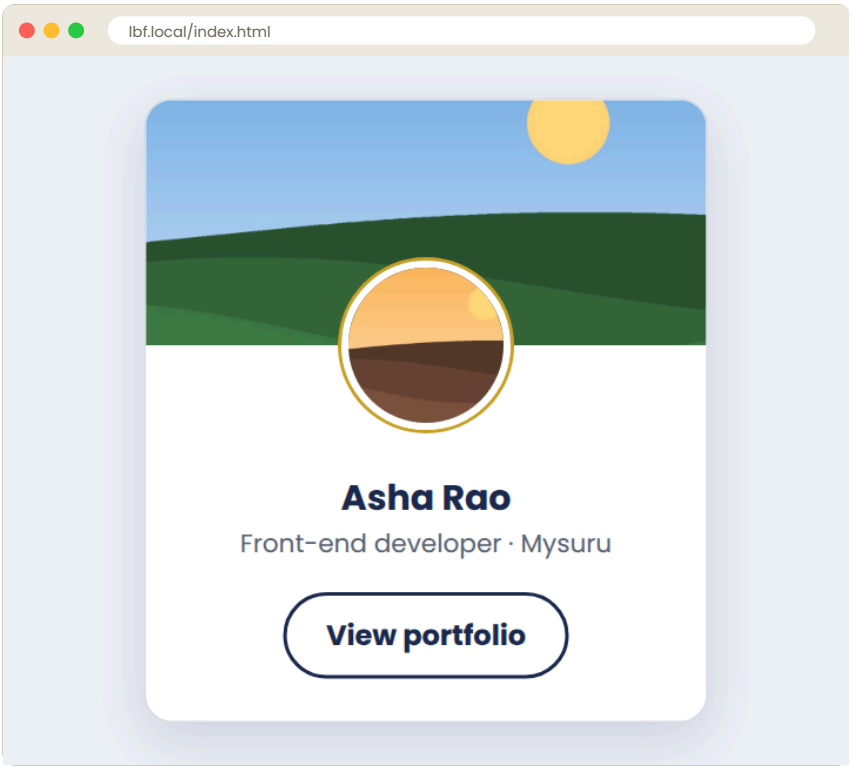
A **profile card** with padding, borders, rounded corners, a circular avatar, a fixed aspect ratio image and a centred layout.

PROFILE-CARD.CSS: SAMPLE SOLUTION CSS

```css
*, *::before, *::after { box-sizing: border-box; }
body { background: #eef1f8; font-family: "Poppins", sans-serif; margin: 0; padding: 24px; }
.profile {
  max-width: 320px; margin: 0 auto;              /* centred */
  background: #fff; border: 1px solid #d8dde8; border-radius: 16px;
  overflow: hidden;                               /* clip the cover image to the corners */
  text-align: center; box-shadow: 0 10px 30px rgb(26 42 79 / .15);
}
.profile__cover { width: 100%; aspect-ratio: 16 / 7; object-fit: cover; display: block; }
.profile__avatar {
  width: 96px; height: 96px; border-radius: 50%; object-fit: cover;
  border: 4px solid #fff; margin-top: -48px;     /* overlap the cover */
  box-shadow: 0 0 0 2px #c9a227;
}
.profile__body { padding: 8px 24px 24px; }
.profile__name { margin: 8px 0 2px; font-size: 1.25rem; color: #1a2a4f; }
.profile__role { margin: 0 0 16px; color: #5c6475; font-size: .9rem; }
.profile__btn { display: inline-block; padding: 10px 22px; border: 2px solid #1a2a4f; border-radius:
999px;
                color: #1a2a4f; text-decoration: none; font-weight: 700; }
```

INDEX.HTML HTML

```html
<article class="profile">
  <img class="profile__cover" src="hills.jpg" alt="">
  <img class="profile__avatar" src="mysuru-palace.jpg" alt="Asha Rao">
  <div class="profile__body">
    <h2 class="profile__name">Asha Rao</h2>
    <p class="profile__role">Front-end developer · Mysuru</p>
    <a class="profile__btn" href="#">View portfolio</a>
  </div>
</article>
```



THE PROFILE CARD

MODULE 06 · BEGINNER

Backgrounds, borders and shadows

Minimum duration **1 day** | Level **Beginner**

These properties add depth and visual style to elements. This module covers background colour and images, repeat, position and size (cover and contain), background-attachment for parallax, background-clip and background-origin, multiple backgrounds and the background shorthand; linear, radial, conic and repeating gradients and gradient text; box-shadow with offsets, blur, spread, colour and inset, layered "elevation" shadows and text-shadow; and an overview of border images.

IN THIS MODULE

6.1 Backgrounds
6.2 Gradients
6.3 Shadows
6.4 Border images (overview)

6.1 Backgrounds

Every box can paint a background behind its content and padding (and, by default, under its border). A background is made of a colour at the bottom and any number of image layers on top. Each aspect is a separate property, and the `background` shorthand sets them all.

6.1.1 background-color

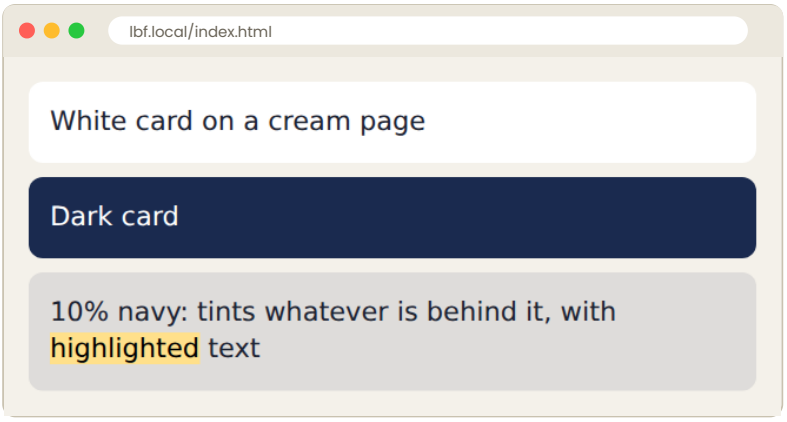
`background-color` fills the box with a colour; the default is `transparent`, so a parent's background shows through. Use it for page, section and component backgrounds, and always set one under background images so text stays readable while the image loads or if it fails.

BACKGROUND COLOURS — CSS

```css
body { background-color: #f4f1ea; }
.card { background-color: #fff; padding: 12px; border-radius: 8px; margin-bottom: 8px; }
.dark { background-color: #1a2a4f; color: #fff; }
mark { background-color: #ffe08a; }
.translucent { background-color: rgb(26 42 79 / 0.1); }
```

INDEX.HTML — HTML

```html
<div class="card">White card on a cream page</div><div class="card dark">Dark card</div><div
class="card translucent">10% navy: tints whatever is behind it, with <mark>highlighted</mark>
text</div>
```



HOW THE BROWSER DISPLAYS IT

6.1.2 background-image

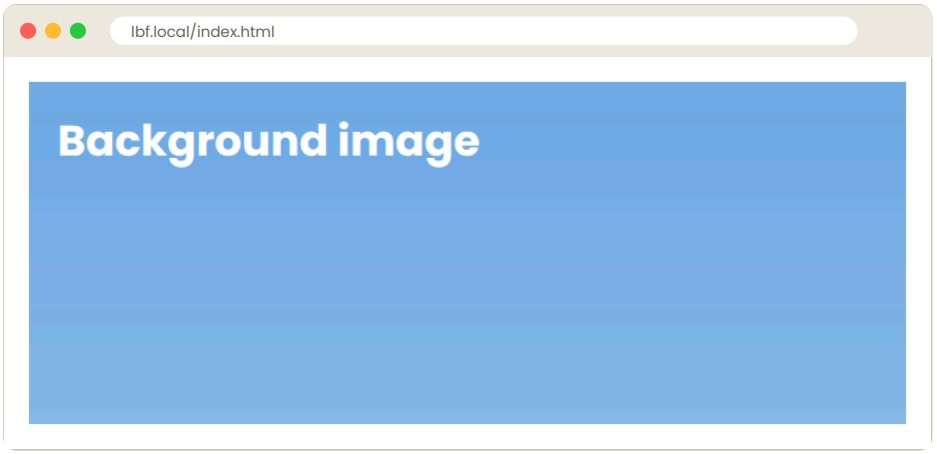
`background-image: url("path/to/image.jpg")` paints an image (or a gradient, 6.2) behind the content. Paths are relative to the **CSS file**. Background images are **decorative**: they have no alt text and are invisible to screen readers, so anything meaningful (a product photo, a chart) belongs in an `<img>`.

STYLE.CSS CSS

```css
.hero { background-image: url("hills.jpg"); background-color: #7cb342; height: 160px; color: #fff;
padding: 16px; font: 700 24px "Poppins", sans-serif; }
```

INDEX.HTML HTML

```html
<div class="hero">Background image</div>
```



BY DEFAULT THE IMAGE IS DRAWN AT ITS NATURAL SIZE FROM THE TOP-LEFT CORNER AND REPEATED

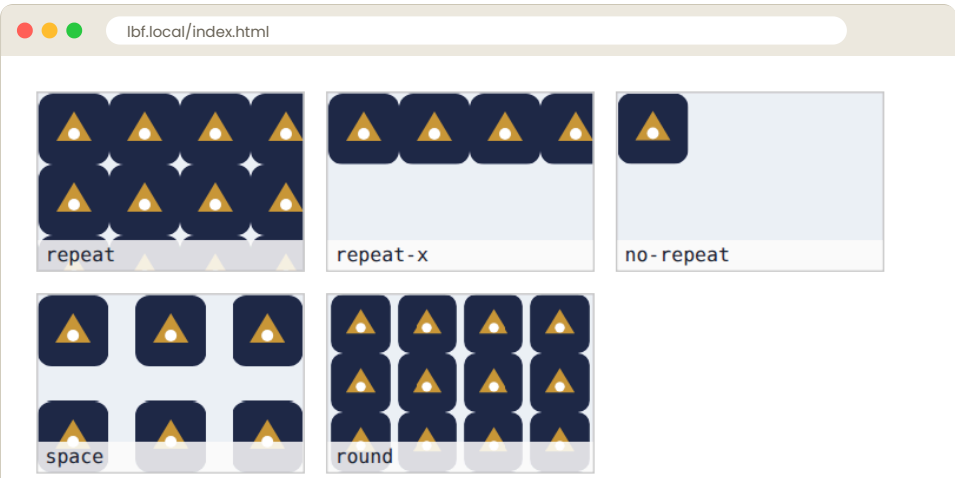
6.1.3 background-repeat, background-position, background-size (cover, contain)

Property	Values
`background-repeat`	`repeat` (default, tile in both directions), `repeat-x`, `repeat-y`, `no-repeat`, `space` (whole tiles, gaps between), `round` (whole tiles, stretched to fit)
`background-position`	keywords (`center`, `top right`), lengths or percentages (`20px 50%`), edge offsets (`right 20px bottom 10px`)
`background-size`	`auto` (natural), lengths (`200px 100px`), percentages, `cover` (fill the box, crop overflow), `contain` (fit entirely, may leave gaps)

BACKGROUND-REPEAT VALUES WITH A 40PX TILE

CSS

```
.t { display: inline-block; width: 150px; height: 100px; margin: 6px; border: 1px solid #ccc;
vertical-align: top; position: relative; } .t span { position: absolute; left: 0; bottom: 0; right: 0;
background: rgb(255 255 255 / .85); font: 11px ui-monospace, monospace; padding: 2px 4px; }
.t { background: #eef1f8 url("logo.svg"); background-size: 40px; }
```

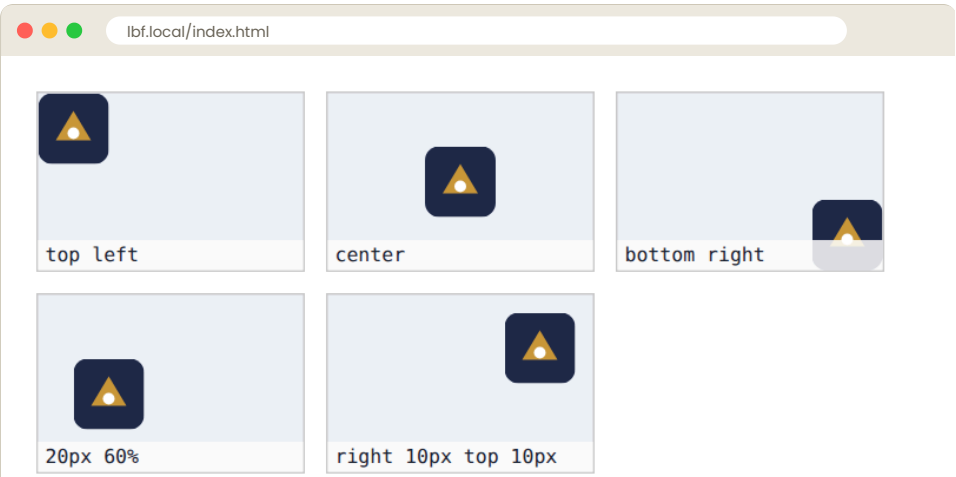


HOW THE BROWSER DISPLAYS IT

BACKGROUND-POSITION VALUES

CSS

```
.t { display: inline-block; width: 150px; height: 100px; margin: 6px; border: 1px solid #ccc;
vertical-align: top; position: relative; } .t span { position: absolute; left: 0; bottom: 0; right: 0;
background: rgb(255 255 255 / .85); font: 11px ui-monospace, monospace; padding: 2px 4px; }
.t { background: #eef1f8 url("logo.svg") no-repeat; background-size: 40px; }
```



HOW THE BROWSER DISPLAYS IT

BACKGROUND-SIZE VALUES ON A 200 × 110 BOX

CSS

```
.t { display: inline-block; width: 150px; height: 100px; margin: 6px; border: 1px solid #ccc;
vertical-align: top; position: relative; } .t span { position: absolute; left: 0; bottom: 0; right: 0;
background: rgb(255 255 255 / .85); font: 11px ui-monospace, monospace; padding: 2px 4px; }
.t { width: 200px; height: 110px; background: #1a2a4f url("mysuru-palace.jpg") no-repeat center; }
```



COVER FILLS AND CROPS; CONTAIN SHOWS THE WHOLE IMAGE WITH EMPTY BANDS; 100% 100% STRETCHES AND DISTORTS

6.1.4 background-attachment (parallax effect)

`background-attachment` decides what the background scrolls with: `scroll` (default: with the element), `fixed` (stays fixed to the viewport while the element scrolls over it, a simple **parallax** effect) or `local` (scrolls with the element's own scrolling content). `fixed` is ignored by most mobile browsers and can be expensive to repaint, so treat it as an enhancement.

A PARALLAX SECTION CSS

```css
.parallax {
  min-height: 60vh;
  background: url("mysuru-palace.jpg") center / cover fixed;   /* fixed = attached to the viewport */
}
```

COMPUTED IN CHROMIUM

```
background-attachment: fixed
background-size: cover
background-position: 50% 50%
```

6.1.5 background-clip and background-origin

`background-origin` decides where the image's position 0 0 starts (`padding-box` by default, or `border-box` / `content-box`). `background-clip` decides how far the background is **painted**: `border-box` (default, under the border), `padding-box`, `content-box`, or `text` (only inside the letters, used for gradient text in 6.2.5).

BACKGROUND-CLIP: THE GOLD BACKGROUND STOPS AT DIFFERENT EDGES CSS

```css
.c { display: inline-block; width: 140px; padding: 18px; margin: 6px; border: 10px dashed #1a2a4f;
background: #c9a227; font: 12px ui-monospace, monospace; }
```



HOW THE BROWSER DISPLAYS IT

6.1.6 Multiple backgrounds

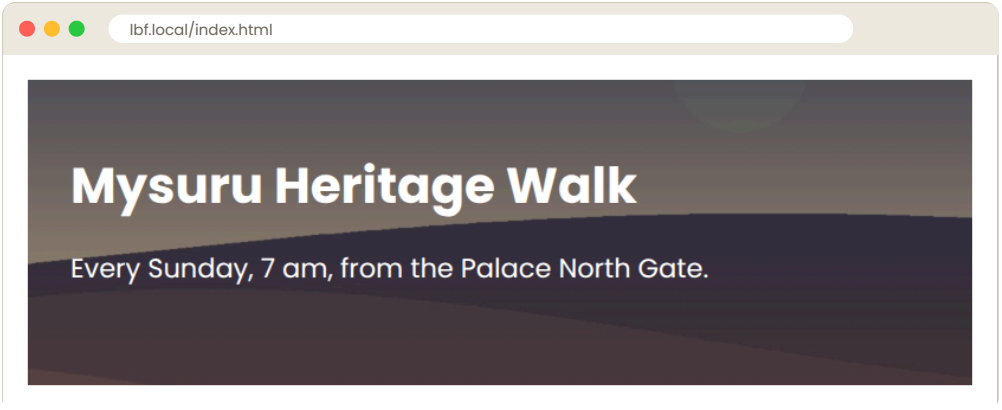
`background-image` (and the shorthand) accept a **comma-separated list** of layers. The **first** layer is drawn on **top**; the background colour always sits under all of them. Each other background property takes a matching list. A common pattern is a semi-transparent gradient over a photo, so white text stays readable:

A GRADIENT OVERLAY ON A PHOTO CSS

```css
.hero {
  background:
    linear-gradient(rgb(26 42 79 / .75), rgb(26 42 79 / .35)),   /* layer 1: overlay (top) */
    url("mysuru-palace.jpg") center / cover;                      /* layer 2: photo         */
  color: #fff; padding: 40px 24px; font-family: "Poppins", sans-serif;
}
.hero h2 { margin: 0 0 6px; font-size: 28px; }
```

INDEX.HTML HTML

```html
<section class="hero"><h2>Mysuru Heritage Walk</h2><p>Every Sunday, 7 am, from the Palace North Gate.
</p></section>
```



HOW THE BROWSER DISPLAYS IT



BACKGROUND LAYERS ARE PAINTED FROM THE LAST TO THE FIRST

6.1.7 background shorthand

The shorthand accepts the values in any order, except that `background-size` must follow `background-position` after a slash: `center / cover`. Like all shorthands it resets everything you leave out, so writing `background: red` after a background image removes the image.

SHORTHAND AND LONGHANDS CSS

```
/*          color    image            repeat     position / size  attachment */
.banner { background: #1a2a4f url("bg.jpg") no-repeat  center / cover   scroll; }

/* the same with longhands */
.banner {
  background-color: #1a2a4f;
  background-image: url("bg.jpg");
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover;
  background-attachment: scroll;
}
```

6.2 Gradients

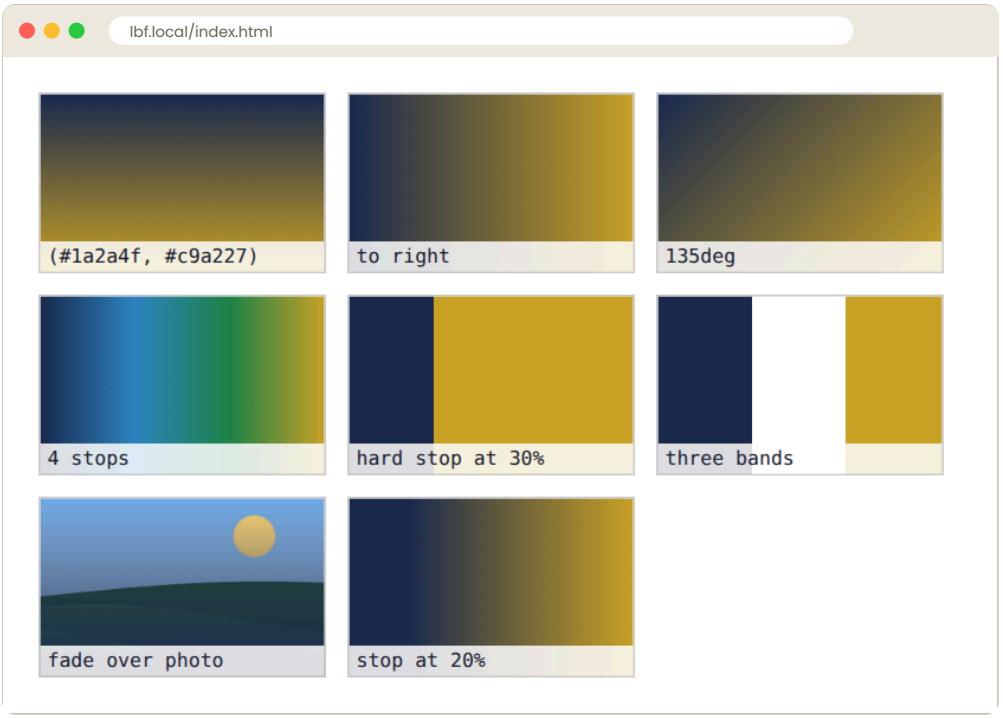
Gradients are images generated by the browser: they are used anywhere an image can be (`background-image`, `border-image`, `mask-image`, `list-style-image`), cost no download and stay sharp at any size.

6.2.1 linear-gradient()

`linear-gradient(direction, colour stops)` blends colours along a straight line. The direction is an angle (`0deg` = upwards, `90deg` = to the right, default `180deg` = downwards) or a keyword (`to right`, `to bottom left`). Each colour stop may have a position; two stops at the same position make a **hard edge**.

LINEAR-GRADIENT() VARIATIONS CSS

```
.t { display: inline-block; width: 150px; height: 100px; margin: 6px; border: 1px solid #ccc;
vertical-align: top; position: relative; } .t span { position: absolute; left: 0; bottom: 0; right: 0;
background: rgb(255 255 255 / .85); font: 11px ui-monospace, monospace; padding: 2px 4px; } .t {
width: 160px; }
```



HOW THE BROWSER DISPLAYS IT

SYNTAX OF THE EXAMPLES CSS

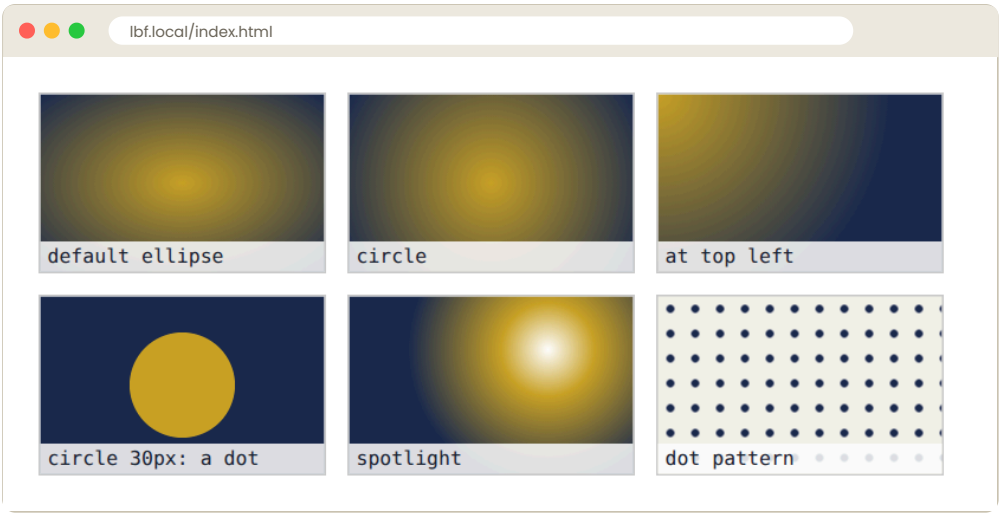
```
.a { background: linear-gradient(to right, #1a2a4f, #c9a227); }
.b { background: linear-gradient(135deg, #1a2a4f, #c9a227); }
.c { background: linear-gradient(to right, #1a2a4f 30%, #c9a227 30%); }          /* hard edge */
.d { background: linear-gradient(to right, #1a2a4f 0 33%, #fff 0 66%, #c9a227 0); } /* bands    */
```

6.2.2 radial-gradient()

`radial-gradient(shape size at position, stops)` radiates from a point. The shape is `ellipse` (default) or `circle`; the size is a length or a keyword such as `closest-side` or `farthest-corner` (default); the position defaults to the centre.

RADIAL-GRADIENT() VARIATIONS CSS

```
.t { display: inline-block; width: 150px; height: 100px; margin: 6px; border: 1px solid #ccc;
vertical-align: top; position: relative; } .t span { position: absolute; left: 0; bottom: 0; right: 0;
background: rgb(255 255 255 / .85); font: 11px ui-monospace, monospace; padding: 2px 4px; } .t {
width: 160px; }
```



HOW THE BROWSER DISPLAYS IT

6.2.3 conic-gradient()

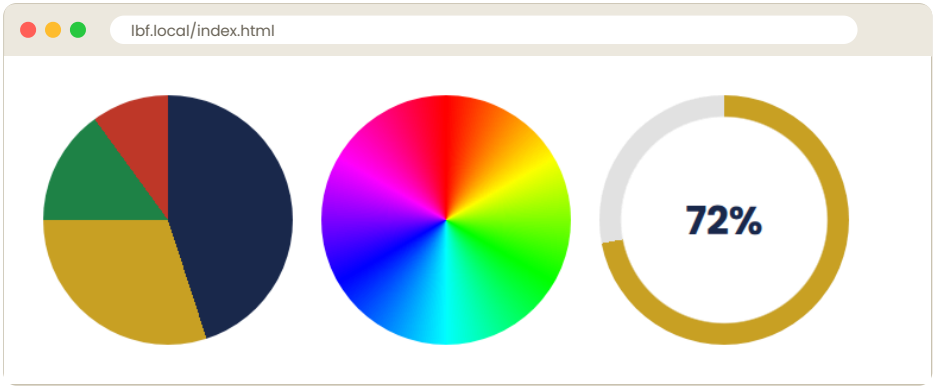
`conic-gradient(from angle at position, stops)` sweeps colours **around** a centre, like the hands of a clock. Stops are angles or percentages. With hard stops it draws **pie charts**; with `border-radius: 50%` and a mask it draws progress rings; with `repeating-conic-gradient` it draws checkerboards and sunbursts.

PIE CHART, COLOUR WHEEL AND PROGRESS RING FROM CONIC GRADIENTS CSS

```css
.pie { width: 140px; height: 140px; border-radius: 50%; display: inline-block; margin: 8px; vertical-
align: middle;
       background: conic-gradient(#1a2a4f 0 45%, #c9a227 0 75%, #1e8449 0 90%, #c0392b 0); }
.wheel { width: 140px; height: 140px; border-radius: 50%; display: inline-block; margin: 8px;
vertical-align: middle;
         background: conic-gradient(red, yellow, lime, aqua, blue, magenta, red); }
.ring { width: 140px; height: 140px; border-radius: 50%; display: inline-grid; place-items: center;
margin: 8px; vertical-align: middle;
        background: radial-gradient(#fff 58%, transparent 59%), conic-gradient(#c9a227 72%, #e5e5e5
0);
        font: 700 22px "Poppins", sans-serif; color: #1a2a4f; }
```

INDEX.HTML HTML

```html
<div class="pie"></div><div class="wheel"></div><div class="ring">72%</div>
```



HOW THE BROWSER DISPLAYS IT

6.2.4 Repeating gradients

`repeating-linear-gradient()`, `repeating-radial-gradient()` and `repeating-conic-gradient()` repeat their colour stops forever, which creates stripes, rings and patterns in one line, with no image files.

PATTERNS FROM REPEATING GRADIENTS CSS

```
.t { display: inline-block; width: 150px; height: 100px; margin: 6px; border: 1px solid #ccc;
vertical-align: top; position: relative; } .t span { position: absolute; left: 0; bottom: 0; right: 0;
background: rgb(255 255 255 / .85); font: 11px ui-monospace, monospace; padding: 2px 4px; } .t {
width: 160px; }
```



HOW THE BROWSER DISPLAYS IT

6.2.5 Gradient text effect

To fill text with a gradient, paint the gradient as the background, clip it to the text with `background-clip: text`, and make the text itself transparent. Keep a solid `color` fallback before it for very old browsers, and check contrast at both ends of the gradient.

GRADIENT TEXT CSS

```
.gradient-text {
  font: 800 44px/1.1 "Poppins", sans-serif;
  color: #1a2a4f;                                  /* fallback */
  background: linear-gradient(90deg, #1a2a4f, #2e86c1 50%, #c9a227);
  -webkit-background-clip: text; background-clip: text;
  -webkit-text-fill-color: transparent;
}
```

INDEX.HTML HTML

```
<h1 class="gradient-text">Learn Being Forward</h1>
```



HOW THE BROWSER DISPLAYS IT

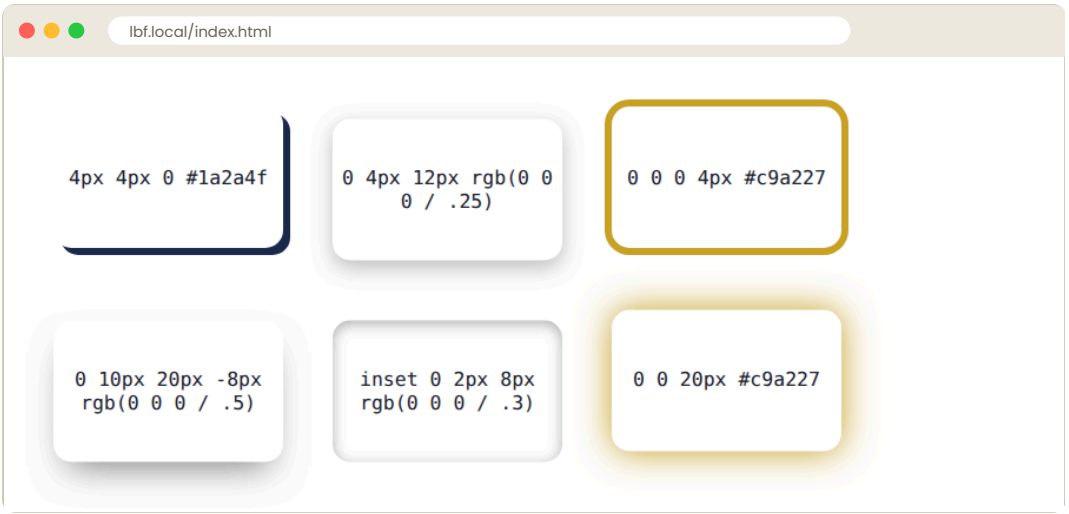
6.3 Shadows

6.3.1 box-shadow: offset, blur, spread, colour, inset

`box-shadow: [inset] x-offset y-offset blur spread colour;` draws a shadow following the box's border-radius. Positive x moves it right and positive y moves it down; **blur** softens the edge (0 is sharp); **spread** grows or shrinks it; `inset` draws it inside the box. Shadows do not affect layout.

BOX-SHADOW VALUES CSS

```css
.s { display: inline-grid; place-items: center; width: 130px; height: 80px; margin: 14px; background: #fff; border-radius: 10px; font: 11px ui-monospace, monospace; text-align: center; }
```



SHARP OFFSET, SOFT DROP SHADOW, SPREAD-ONLY RING, NEGATIVE SPREAD, INSET, GLOW



then the colour; repeat with commas for more shadows

THE PARTS OF A BOX-SHADOW VALUE

6.3.2 Multiple shadows and soft "elevation" shadows

Comma-separated shadows are layered, first on top. Real objects cast a tight, darker shadow close to them and a large, faint one further away, so design systems (such as Material Design) define **elevation levels** with two or three layered shadows. Coloured shadows tinted with the background hue look more natural than pure black.

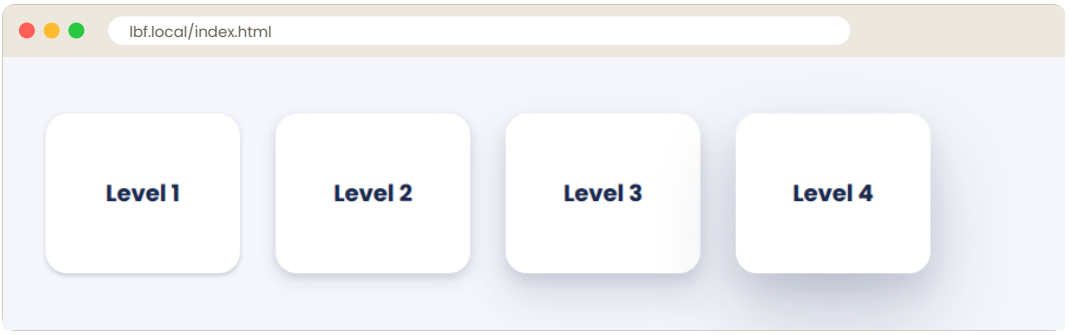
AN ELEVATION SCALE WITH LAYERED SHADOWS CSS

```css
:root {
  --shadow-1: 0 1px 2px rgb(26 42 79 / .12), 0 1px 3px rgb(26 42 79 / .08);
  --shadow-2: 0 2px 4px rgb(26 42 79 / .10), 0 4px 10px rgb(26 42 79 / .10);
  --shadow-3: 0 4px 8px rgb(26 42 79 / .10), 0 12px 24px rgb(26 42 79 / .14);
  --shadow-4: 0 8px 16px rgb(26 42 79 / .10), 0 24px 48px rgb(26 42 79 / .20);
}
body { background: #f4f6fb; }
.card { display: inline-grid; place-items: center; width: 110px; height: 90px; margin: 18px 10px; background: #fff; border-radius: 12px; font: 600 13px "Poppins", sans-serif; color: #1a2a4f; }
```

INDEX.HTML

HTML

```
<div class="card" style="box-shadow:var(--shadow-1)">Level 1</div><div class="card" style="box-shadow:var(--shadow-2)">Level 2</div><div class="card" style="box-shadow:var(--shadow-3)">Level 3</div><div class="card" style="box-shadow:var(--shadow-4)">Level 4</div>
```



HOW THE BROWSER DISPLAYS IT

6.3.3 text-shadow

`text-shadow: x y blur colour` (no spread, no inset) adds shadows to the glyphs. Typical uses: legibility of white text over photos, subtle embossing, neon glows and retro offset effects (examples in Module 4.4.3).

TEXT-SHADOW EFFECTS

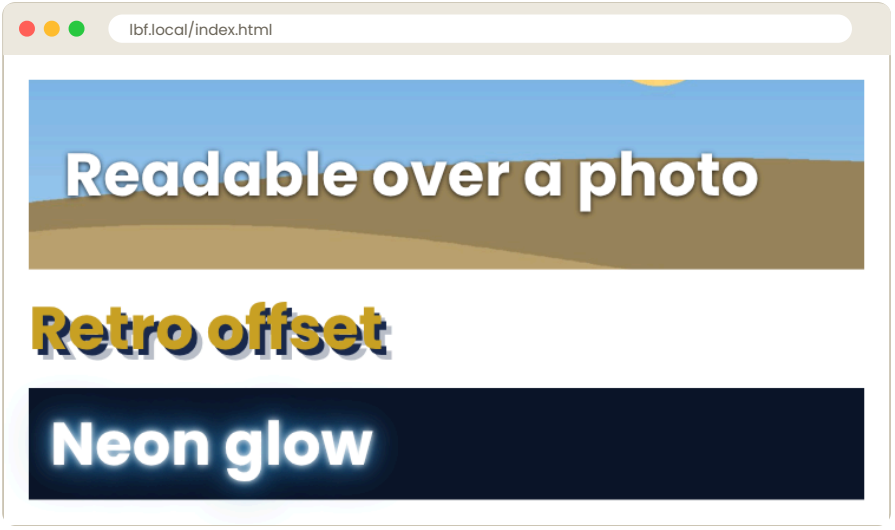
CSS

```
h2 { font: 800 34px "Poppins", sans-serif; margin: 8px 0; }
.photo { background: url("beach.jpg") center / cover; padding: 20px; }
.legible { color: #fff; text-shadow: 0 1px 3px rgb(0 0 0 / .8); }
.retro { color: #c9a227; text-shadow: 3px 3px 0 #1a2a4f, 6px 6px 0 rgb(26 42 79 / .3); }
.neon { color: #fff; background: #0d1528; padding: 6px 12px; text-shadow: 0 0 4px #fff, 0 0 12px #2e86c1, 0 0 24px #2e86c1; }
```

INDEX.HTML

HTML

```
<div class="photo"><h2 class="legible">Readable over a photo</h2></div><h2 class="retro">Retro offset</h2><h2 class="neon">Neon glow</h2>
```



HOW THE BROWSER DISPLAYS IT

6.4 Border images (overview)

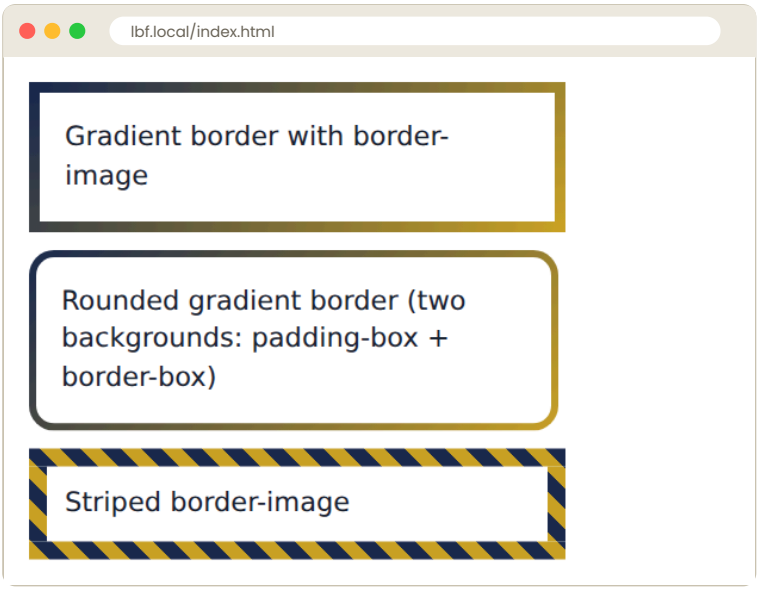
`border-image` paints the border area with an image or gradient instead of a colour. The image is sliced into nine regions (four corners, four edges, the middle) by `border-image-slice`; corners are drawn as-is and edges are stretched, repeated or rounded. The element needs a `border-style` and width. Note that border-image ignores `border-radius`. Its most common modern use is a **gradient border**:

GRADIENT BORDERS CSS

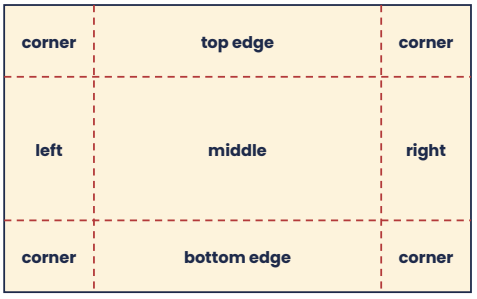
```
.gb { border: 6px solid; border-image: linear-gradient(135deg, #1a2a4f, #c9a227) 1; padding: 14px;
margin: 10px 0; width: 260px; }
.round-gb { border: 4px solid transparent; border-radius: 14px; padding: 14px; width: 260px;
            background: linear-gradient(#fff, #fff) padding-box, linear-gradient(135deg, #1a2a4f,
#c9a227) border-box; }
.dashes { border: 10px solid; border-image: repeating-linear-gradient(45deg, #c9a227 0 8px, #1a2a4f
8px 16px) 10; padding: 10px; width: 260px; margin-top: 10px; }
```

INDEX.HTML HTML

```
<div class="gb">Gradient border with border-image</div><div class="round-gb">Rounded gradient border
(two backgrounds: padding-box + border-box)</div><div class="dashes">Striped border-image</div>
```



HOW THE BROWSER DISPLAYS IT



BORDER-IMAGE-SLICE DIVIDES THE IMAGE INTO NINE REGIONS

PRACTICE CHECKPOINT

A **hero section** with a background image and gradient overlay, **gradient text**, and **cards with layered soft shadows**.

HERO.CSS: SAMPLE SOLUTION CSS

```css
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: "Poppins", sans-serif; background: #f4f6fb; color: #1d2433; }
.hero {
  padding: 56px 24px 72px; text-align: center; color: #fff;
  background:
    linear-gradient(160deg, rgb(26 42 79 / .92), rgb(26 42 79 / .55)),
    url("mysuru-palace.jpg") center / cover no-repeat;
}
.hero h1 {
  margin: 0 0 8px; font-size: 40px; line-height: 1.1;
  background: linear-gradient(90deg, #fff, #f0d9a0 60%, #c9a227);
  -webkit-background-clip: text; background-clip: text; -webkit-text-fill-color: transparent;
}
.hero p { margin: 0; opacity: .9; }
.cards { display: flex; gap: 16px; justify-content: center; padding: 0 16px; margin-top: -40px; }
.card {
  background: #fff; border-radius: 14px; padding: 18px; width: 170px;
  box-shadow: 0 2px 4px rgb(26 42 79 / .08), 0 12px 28px rgb(26 42 79 / .16);
}
.card h3 { margin: 0 0 4px; color: #1a2a4f; font-size: 16px; }
.card p { margin: 0; font-size: 13px; color: #5c6475; }
```

INDEX.HTML HTML

```html
<section class="hero">
  <h1>Code your future</h1>
  <p>Live courses in HTML, CSS, JavaScript and React</p>
</section>
<div class="cards">
  <article class="card"><h3>39 days</h3><p>From basics to advanced CSS</p></article>
  <article class="card"><h3>21 modules</h3><p>Every topic with examples</p></article>
  <article class="card"><h3>Projects</h3><p>Portfolio-ready work</p></article>
</div>
```



HERO WITH OVERLAY, GRADIENT HEADING AND FLOATING ELEVATED CARDS

MODULE 07 · BEGINNER

Display, positioning and overflow

Minimum duration **2 days** Level **Beginner**

This module covers how elements flow on the page and how to take them out of normal flow. It explains the display property (block, inline, inline-block, none versus visibility: hidden, a first look at flex and grid, contents and flow-root), normal flow and formatting contexts, the five position values (static, relative, absolute with its containing block, fixed and sticky) with top, right, bottom, left and inset, stacking with z-index and stacking contexts, overflow in all its forms, legacy floats with clear, clearfix and text wrapping, and finally every common horizontal and vertical centring technique compared side by side.

IN THIS MODULE

7.1 The display property
7.2 Normal flow
7.3 Positioning
7.4 Stacking
7.5 Overflow
7.6 Floats (legacy layout)
7.7 Centring techniques

7.1 The display property

`display` sets two things: the element's **outer** display type (how it takes part in its parent's layout: block or inline) and its **inner** display type (how it lays out its own children: normal flow, flex, grid, table). Every element has a default from the user-agent stylesheet: `div`, `p`, `h1`, `section` are block; `span`, `a`, `strong`, `img` are inline; `li` is list-item; `head` and `script` are none.

7.1.1 block, inline, inline-block

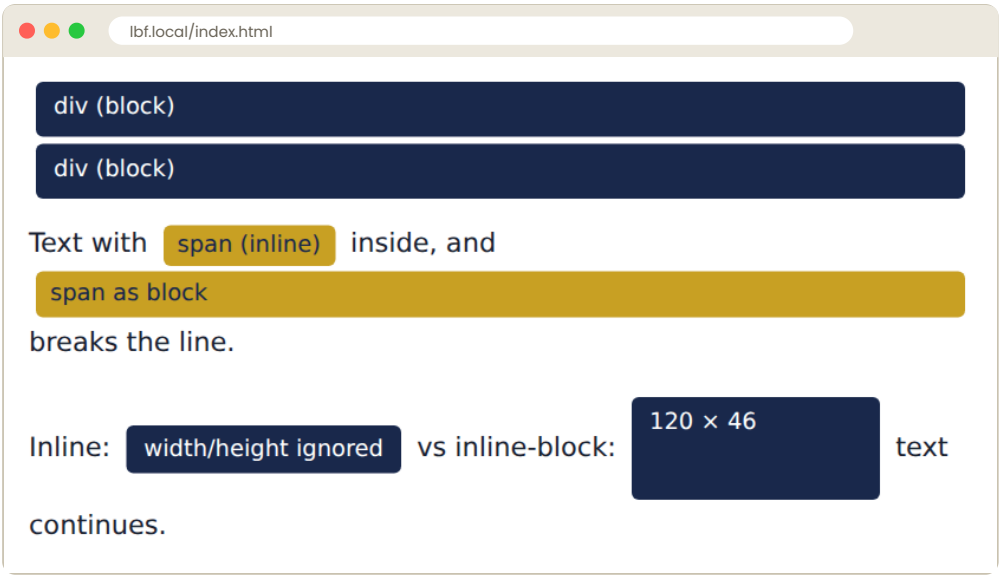
	block	inline	inline-block
Starts on a new line	yes	no: flows within text	no: flows within text
Default width	fills the container	as wide as its content	as wide as its content
width / height	respected	**ignored**	respected
Vertical margin and padding	push other boxes	padding is drawn but does not push lines apart; vertical margin ignored	push other lines
Examples	div, p, h1-h6, ul, section, form	span, a, em, strong, code	buttons, badges, nav links you want to size

BLOCK, INLINE AND INLINE-BLOCK CSS

```css
.b { background: #1a2a4f; color: #fff; padding: 6px 10px; margin: 4px; border-radius: 4px; font-size: 13px; } .i { background: #c9a227; color: #1a2a4f; padding: 4px 8px; margin: 4px; border-radius: 4px; font-size: 13px; }
.as-inline { display: inline; width: 200px; height: 60px; }   /* width/height ignored */
.as-block { display: block; }
.as-ib { display: inline-block; width: 120px; height: 46px; vertical-align: middle; }
```

INDEX.HTML HTML

```html
<div class="b">div (block)</div><div class="b">div (block)</div>
<p>Text with <span class="i">span (inline)</span> inside, and <span class="i as-block">span as block</span> breaks the line.</p>
<p>Inline: <span class="b as-inline">width/height ignored</span> vs inline-block: <span class="b as-ib">120 × 46</span> text continues.</p>
```



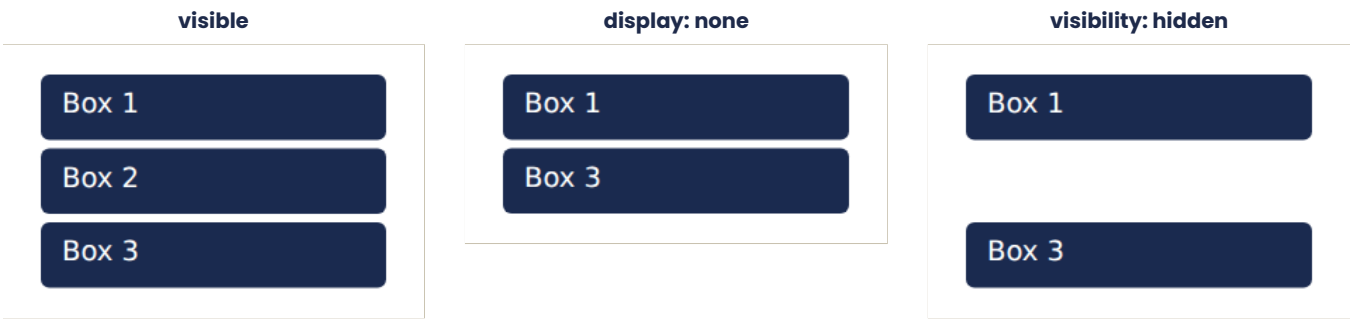
HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
.as-inline width:200/120 requested -> rendered 27.7 x 27.0
.as-ib     width:200/120 requested -> rendered 140.0 x 58.0
```

7.1.2 none vs visibility: hidden

`display: none` removes the element **completely**: it takes no space, is not rendered, cannot be focused and is hidden from screen readers. `visibility: hidden` makes it invisible but it **keeps its space** (and is also hidden from assistive technology). `opacity: 0` keeps both the space and the interactivity (it can still be clicked and focused), which is a trap. The HTML `hidden` attribute is equivalent to `display: none`.



BOX 2 HIDDEN TWO WAYS: THE SPACE DISAPPEARS WITH DISPLAY: NONE AND REMAINS WITH VISIBILITY: HIDDEN

	display: none	visibility: hidden	opacity: 0
Takes space	no	yes	yes
Clickable / focusable	no	no	**yes**
Read by screen readers	no	no	yes
Can be transitioned	not directly (see @starting-style, Module 13)	yes (discrete)	yes (smoothly)

7.1.3 flex and grid (introduced; detailed in Modules 10 and 11)

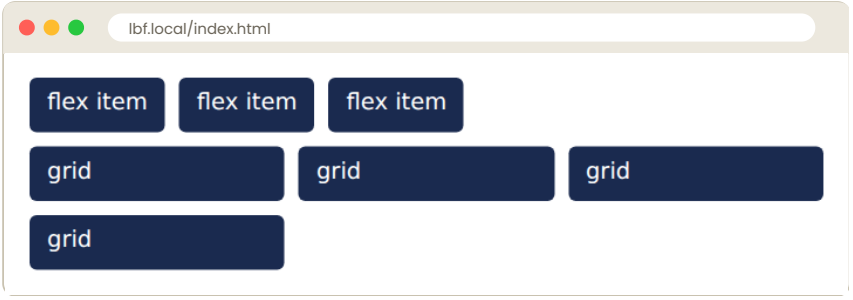
`display: flex` and `display: grid` make the element a block-level box whose **children** are laid out by the Flexbox or Grid algorithm; `inline-flex` and `inline-grid` do the same while the container itself flows inline. The values can also be written with the two-keyword syntax: `display: block flex`, `display: inline grid`.

A FIRST LOOK AT FLEX AND GRID CSS

```
.b { background: #1a2a4f; color: #fff; padding: 6px 10px; margin: 4px; border-radius: 4px; font-size:
13px; } .i { background: #c9a227; color: #1a2a4f; padding: 4px 8px; margin: 4px; border-radius: 4px;
font-size: 13px; }
.row { display: flex; gap: 8px; }
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; margin-top: 8px; }
.row .b, .grid .b { margin: 0; }
```

INDEX.HTML HTML

```
<div class="row"><div class="b">flex item</div><div class="b">flex item</div><div class="b">flex
item</div></div><div class="grid"><div class="b">grid</div><div class="b">grid</div><div
class="b">grid</div><div class="b">grid</div></div>
```

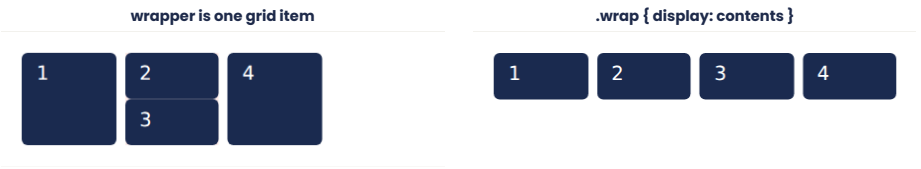


HOW THE BROWSER DISPLAYS IT

7.1.4 contents and flow-root

`display: contents` makes the element's own box **disappear** while its children remain, as if they were children of the grandparent. It lets a wrapper element's children take part in a parent flex or grid layout. Use with care on semantic elements: some browsers historically removed buttons and lists from the accessibility tree.

`display: flow-root` makes a block that creates a new **block formatting context**: it contains its floats (the modern clearfix, 7.6.2) and stops child margins from collapsing through it (5.3.4).

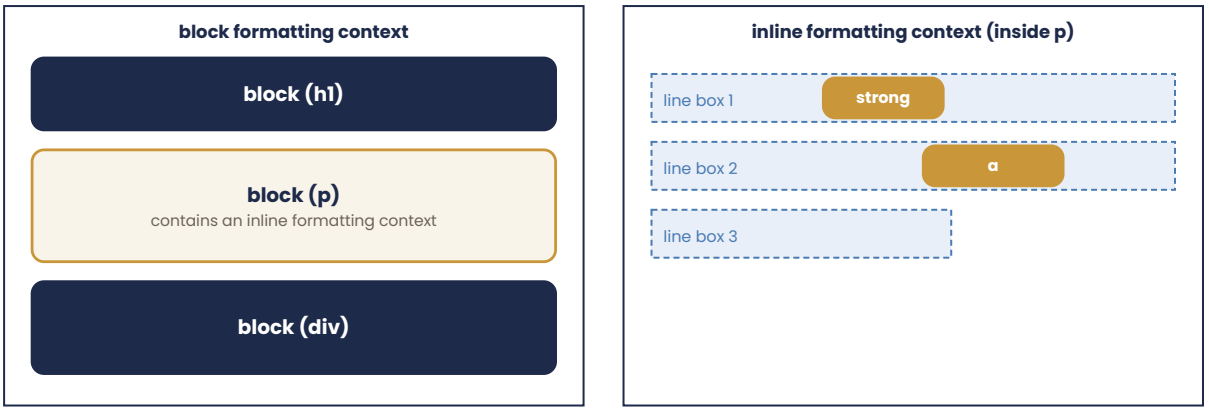


DISPLAY: CONTENTS LETS THE WRAPPER'S CHILDREN BECOME GRID ITEMS

7.2 Normal flow

7.2.1 Block and inline formatting contexts

Without any layout CSS, the browser uses **normal flow**. Inside a **block formatting context** (BFC), block boxes stack vertically, one after another, each as wide as its container, with vertical margins between them (which may collapse). Inside a block, text and inline boxes form an **inline formatting context**: they flow horizontally in **line boxes**, wrapping to the next line when the line is full, aligned by `text-align` and `vertical-align`.



BLOCKS STACK VERTICALLY; INLINE CONTENT FLOWS INTO LINE BOXES

A new BFC is created by the root element, floats, absolutely positioned elements, inline-blocks, `overflow` other than visible/clip, `display: flow-root`, flex and grid items, and table cells. Inside a BFC, floats are contained and margins of its children do not collapse with the BFC's own margins. Flexbox and Grid create their own formatting contexts with different rules (no margin collapse, children laid out by the algorithm).

7.3 Positioning

The `position` property takes an element partly or fully out of normal flow and places it with the **offset properties** `top`, `right`, `bottom` and `left` (or `inset`).

position	In normal flow?	Offsets relative to	Scrolls with page	Typical use
`static` (default)	yes	offsets ignored	yes	everything
`relative`	yes (space kept)	its own normal position	yes	nudging; containing block for absolute children
`absolute`	**no**	nearest positioned ancestor	yes	badges, overlays, dropdowns, tooltips
`fixed`	**no**	the viewport	no	sticky headers, chat buttons, modals
`sticky`	yes	its scroll container, once a threshold is reached	until it sticks	table headers, section headings, side menus

7.3.1 static

`static` is the default: the element sits in normal flow, and `top` / `left` / `z-index` have no effect. You set it explicitly only to undo another position value (for example in a media query).

7.3.2 relative

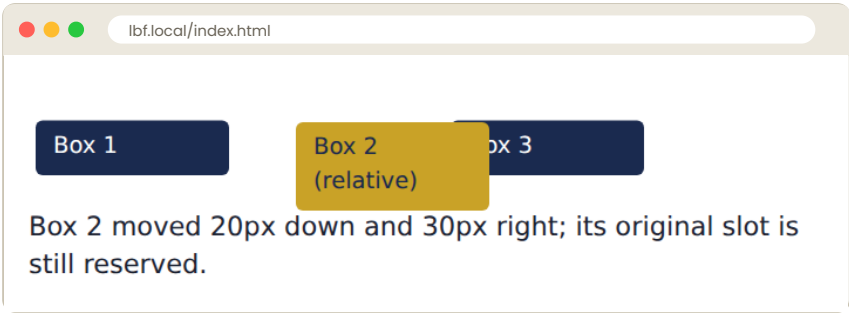
`position: relative` keeps the element in normal flow (its original space stays reserved) and shifts it **visually** by the offsets, relative to where it would have been. Other elements do not move to fill the gap. Its most important use is not shifting at all, but making the element a **containing block** for absolutely positioned children (7.3.3), and enabling z-index.

POSITION: RELATIVE CSS

```
.b { background: #1a2a4f; color: #fff; padding: 6px 10px; margin: 4px; border-radius: 4px; font-size: 13px; } .i { background: #c9a227; color: #1a2a4f; padding: 4px 8px; margin: 4px; border-radius: 4px; font-size: 13px; }
.box { display: inline-block; width: 90px; }
.moved { position: relative; top: 20px; left: 30px; background: #c9a227; color: #1a2a4f; }
```

INDEX.HTML HTML

```
<div class="b box">Box 1</div><div class="b box moved">Box 2 (relative)</div><div class="b box">Box 3</div><p>Box 2 moved 20px down and 30px right; its original slot is still reserved.</p>
```



HOW THE BROWSER DISPLAYS IT

7.3.3 absolute and the containing block

`position: absolute` removes the element from normal flow (other elements behave as if it were not there) and positions it relative to its **containing block**: the padding box of the **nearest ancestor whose**

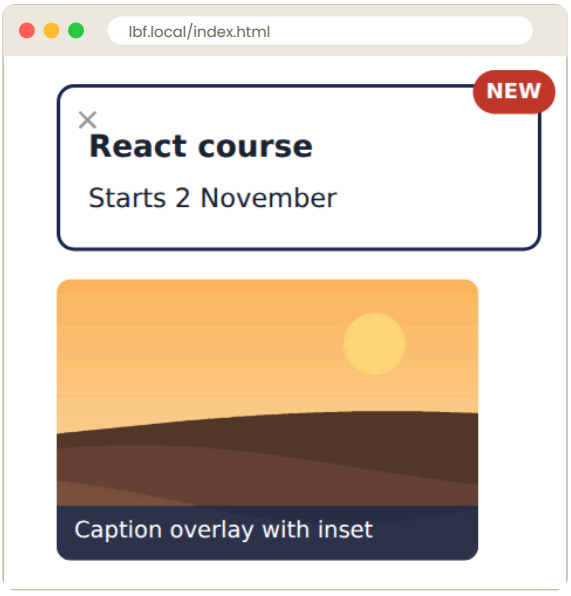
position is not static (or that has a transform, filter or container-type). If there is none, it is the initial containing block (the first screen of the page). Its width shrinks to fit its content unless set. This is the pattern behind badges, close buttons, image captions and dropdown menus: **parent relative, child absolute**.

BADGES, A CLOSE BUTTON AND A CAPTION: PARENT RELATIVE, CHILD ABSOLUTE CSS

```
.card { position: relative; width: 240px; padding: 16px; margin: 16px; border: 2px solid #1a2a4f;
border-radius: 10px; background: #fff; }
.badge { position: absolute; top: -10px; right: -10px; background: #c0392b; color: #fff; font-size:
12px; font-weight: 700;
         padding: 4px 8px; border-radius: 999px; }
.close { position: absolute; top: 6px; left: 8px; font-size: 18px; color: #999; }
.caption { position: absolute; inset: auto 0 0 0; background: rgb(26 42 79 / .8); color: #fff;
padding: 6px 10px; font-size: 13px; border-radius: 0 0 8px 8px; }
.photo { position: relative; width: 240px; margin: 16px; }
.photo img { width: 100%; display: block; border-radius: 8px; }
```

INDEX.HTML HTML

```
<div class="card"><span class="close">×</span><span class="badge">NEW</span>
  <h3 style="margin:6px 0">React course</h3><p style="margin:0">Starts 2 November</p></div>
<figure class="photo" style="margin-left:16px"><img src="mysuru-palace.jpg" alt=""><figcaption
class="caption">Caption overlay with inset</figcaption></figure>
```



HOW THE BROWSER DISPLAYS IT

What happens without a positioned ancestor? The badge is placed against the page instead of the card:



THE CONTAINING BLOCK MAKES THE DIFFERENCE

COMPUTED IN CHROMIUM

```
badge A: offsetParent = card, right edge at x = 244
badge B: offsetParent = body, right edge at x = 800
(viewport width 800)
```

7.3.4 fixed

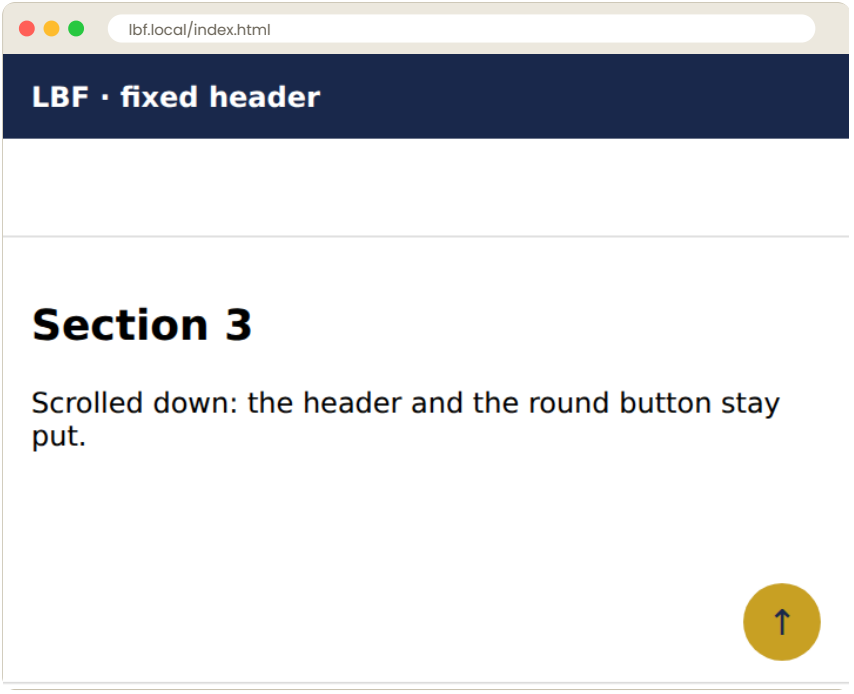
`position: fixed` removes the element from flow and positions it relative to the **viewport**, so it stays in the same place on screen while the page scrolls: headers, cookie banners, "back to top" and chat buttons, modals. Remember to reserve space (for example `padding-top` on the body equal to a fixed header's height) so it does not cover content. Exception: if an ancestor has a `transform`, `filter` or `will-change: transform`, that ancestor becomes the containing block instead.

A FIXED HEADER AND A FIXED "BACK TO TOP" BUTTON CSS

```css
body { margin: 0; padding-top: 48px; font-family: system-ui, sans-serif; }
.topbar { position: fixed; top: 0; left: 0; right: 0; height: 48px; background: #1a2a4f; color: #fff;
          display: flex; align-items: center; padding: 0 16px; font-weight: 700; z-index: 10; }
.to-top { position: fixed; right: 16px; bottom: 16px; width: 44px; height: 44px; border-radius: 50%;
          background: #c9a227; color: #1a2a4f; display: grid; place-items: center; font-size: 20px;
text-decoration: none; }
section { padding: 16px; height: 220px; border-bottom: 1px solid #ddd; }
```

INDEX.HTML HTML

```html
<header class="topbar">LBF · fixed header</header>
<section><h2>Section 1</h2></section><section><h2>Section 2</h2></section><section><h2>Section 3</h2>
<p>Scrolled down: the header and the round button stay put.</p></section><section><h2>Section 4</h2>
</section>
<a class="to-top" href="#" aria-label="Back to top">↑</a>
```



THE PAGE SCROLLED 450PX DOWN: BOTH FIXED ELEMENTS STAY IN THE VIEWPORT

7.3.5 sticky

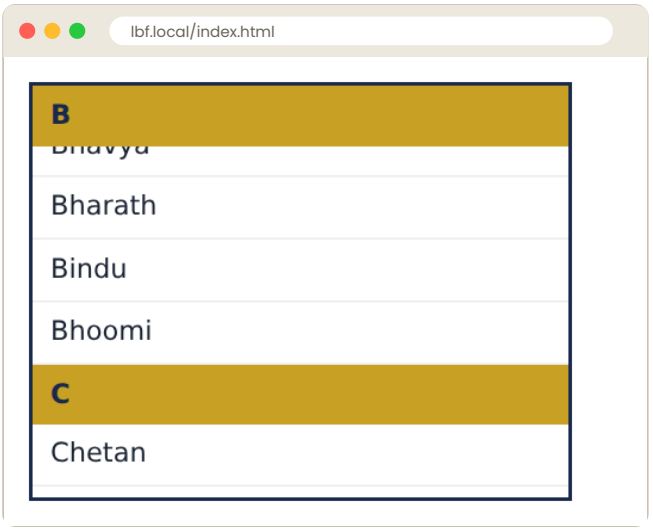
`position: sticky` is a hybrid: the element stays in normal flow until scrolling would move it past a threshold (`top: 0`), then it sticks there, **but only while its parent is visible**; when the parent scrolls away, the sticky element leaves with it. It needs an offset (top, bottom, left or right). Common pitfalls: an ancestor with `overflow: hidden` or `auto` becomes the scroll container (so it sticks within that box, or never), and the parent must be taller than the sticky element.

STICKY SECTION HEADINGS IN A CONTACT LIST CSS

```
.scroller { height: 230px; overflow-y: auto; border: 2px solid #1a2a4f; width: 300px; font-family:
system-ui, sans-serif; }
h3 { position: sticky; top: 0; margin: 0; background: #c9a227; color: #1a2a4f; padding: 6px 10px;
font-size: 15px; }
p { margin: 0; padding: 6px 10px; border-bottom: 1px solid #eee; }
```

INDEX.HTML HTML

```
<div class="scroller"><h3>A</h3><p>Asha</p><p>Ajay</p><p>Anita</p><p>Arjun</p><h3>B</h3><p>Bhavya</p>
<p>Bharath</p><p>Bindu</p><p>Bhoomi</p><h3>C</h3><p>Chetan</p><p>Chaitra</p><p>Charan</p><h3>D</h3>
<p>Deepa</p><p>Dinesh</p></div>
```



SCROLLED 190PX INSIDE THE LIST: "B" STICKS TO THE TOP, AND WILL BE PUSHED AWAY BY "C"

7.3.6 top, right, bottom, left and inset

The offset properties apply to positioned elements. Setting **both** left and right (or top and bottom) on an absolute element with `width: auto` stretches it between them. `inset` is the shorthand, with one to four values like margin: `inset: 0` covers the containing block completely (overlays), `inset: auto 0 0` pins to the bottom edge. Percentages refer to the containing block's size. Logical versions: `inset-inline-start`, `inset-block`.

OFFSETS AND INSET CSS

```
.frame { position: relative; height: 160px; border: 2px solid #1a2a4f; background: #eef1f8; font-size:
12px; }
.frame > div { position: absolute; background: #1a2a4f; color: #fff; padding: 4px 6px; }
.a { top: 8px; left: 8px; } .b { top: 8px; right: 8px; } .c { bottom: 8px; left: 50%; transform:
translateX(-50%); }
.d { inset: 50px 120px 50px 120px; background: rgb(201 162 39 / .85) !important; color: #1a2a4f
!important; }
```

INDEX.HTML HTML

```
<div class="frame"><div class="a">top: 8px; left: 8px</div><div class="b">top: 8px; right: 8px</div>
<div class="d">inset: 50px 120px (stretches)</div><div class="c">bottom: 8px; left: 50% +
translate</div></div>
```



HOW THE BROWSER DISPLAYS IT

7.4 Stacking

7.4.1 z-index

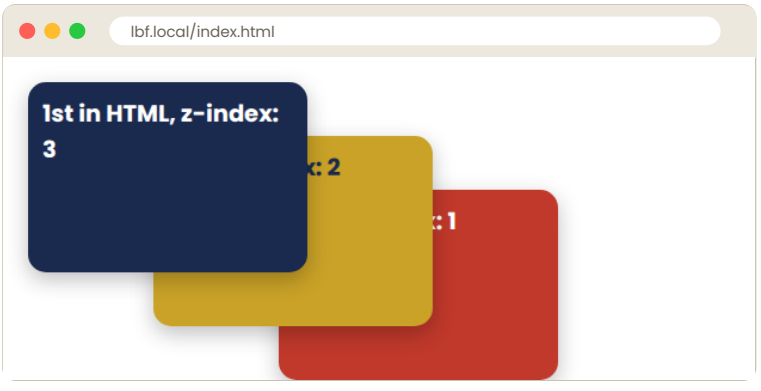
When boxes overlap, later elements in the HTML are painted on top. `z-index` changes the order along the z-axis (towards the viewer) for **positioned** elements (any position except static) and for flex and grid items. Higher numbers are on top; negative values go behind their parent's content. Without z-index, positioned elements are painted above non-positioned ones.

Z-INDEX REVERSES THE DEFAULT PAINTING ORDER CSS

```
.stack { position: relative; height: 150px; }
.card { position: absolute; width: 140px; height: 90px; border-radius: 10px; color: #fff; font: 600
13px "Poppins", sans-serif; padding: 8px; box-shadow: 0 4px 12px rgb(0 0 0 / .25); }
.c1 { left: 0; top: 0; background: #1a2a4f; z-index: 3; }
.c2 { left: 70px; top: 30px; background: #c9a227; color: #1a2a4f; z-index: 2; }
.c3 { left: 140px; top: 60px; background: #c0392b; z-index: 1; }
```

INDEX.HTML HTML

```
<div class="stack"><div class="card c1">1st in HTML, z-index: 3</div><div class="card c2">2nd, z-
index: 2</div><div class="card c3">3rd, z-index: 1</div></div>
```



HOW THE BROWSER DISPLAYS IT

7.4.2 Stacking contexts

z-index values are only compared **within the same stacking context**. An element creates a new stacking context when it is positioned with a z-index other than auto, or has `opacity` less than 1, a `transform`, `filter`, `isolation: isolate`, `will-change`, `position: fixed/sticky`, or is a flex/grid item with a z-index. Everything inside a stacking context is painted together as one layer: a child with `z-index: 9999` can

never rise above an element that is on top of its **parent's** context. This explains the classic bug "my dropdown is behind the header even with z-index 9999".

Z-INDEX: 9999 TRAPPED IN A STACKING CONTEXT CSS

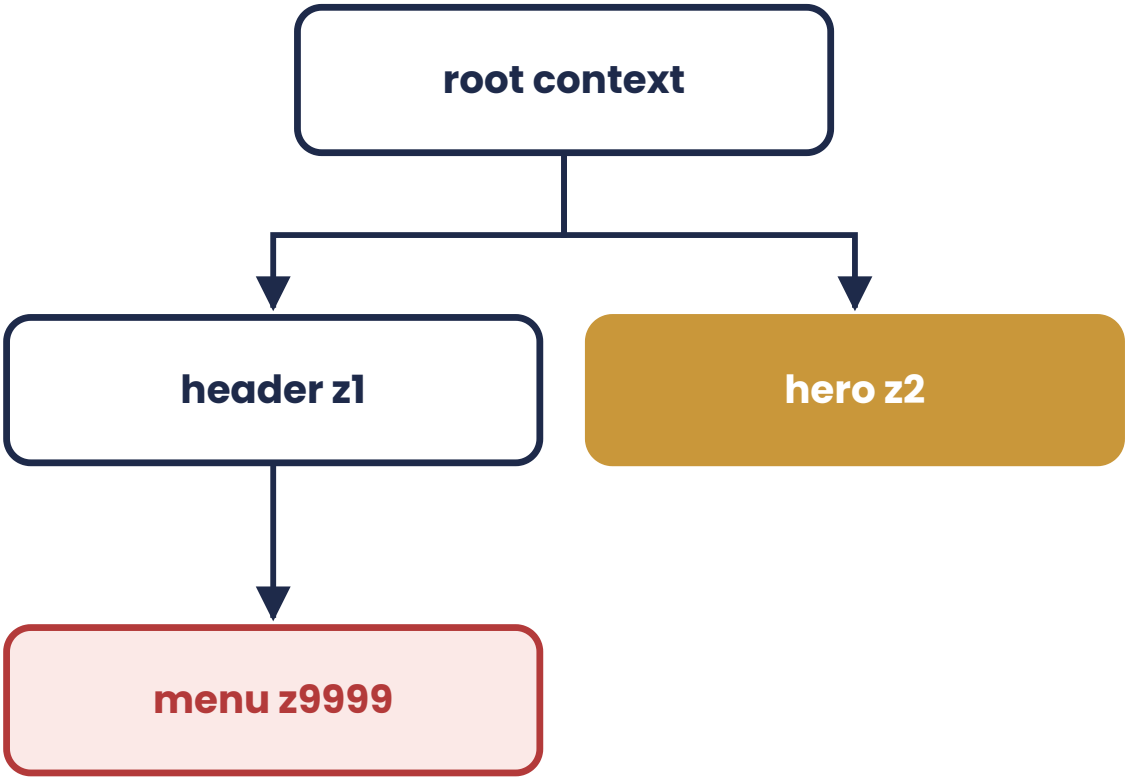
```css
.header { position: relative; z-index: 1; background: #1a2a4f; color: #fff; padding: 10px; }
.menu { position: absolute; top: 30px; left: 10px; z-index: 9999; background: #c9a227; color: #1a2a4f;
padding: 8px; width: 170px; height: 90px; }
.hero { position: relative; z-index: 2; background: rgb(192 57 43 / .92); color: #fff; padding: 30px
10px; }
```

INDEX.HTML HTML

```html
<div class="header">Header (z-index: 1)<div class="menu">Dropdown z-index: 9999</div></div><div
class="hero">Hero (z-index: 2) covers the dropdown</div>
```



THE DROPDOWN IS INSIDE THE HEADER'S CONTEXT (1), WHICH IS BELOW THE HERO (2)



STACKING CONTEXTS FORM A TREE: THE MENU COMPETES ONLY INSIDE THE HEADER

Fixes: raise the **header's** z-index above the hero, or remove the property that creates the unwanted context. `isolation: isolate` deliberately creates a context to contain a component's internal z-indexes.

7.5 Overflow

7.5.1 overflow: visible, hidden, scroll, auto, clip

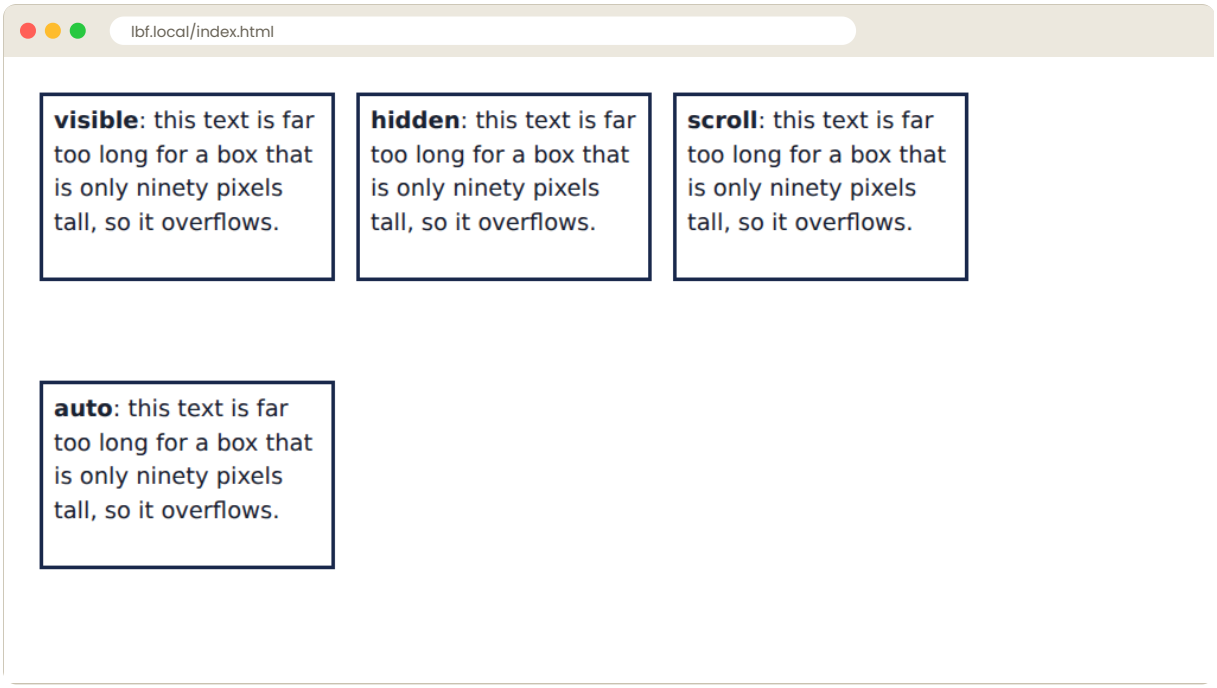
When content is larger than its box (a fixed height, a long word, a big image), `overflow` decides what happens:

Value	Effect
`visible` (default)	content spills outside the box and may overlap other content
`hidden`	cut off at the padding edge; still scrollable by script; creates a BFC
`clip`	cut off like hidden but no scrolling at all and no BFC (sticky still works inside)
`scroll`	cut off with scrollbars always shown
`auto`	scrollbars only when needed: the usual choice for scrollable areas

OVERFLOW VALUES

CSS

```
.o { display: inline-block; vertical-align: top; width: 150px; height: 90px; margin: 6px 6px 50px;
border: 2px solid #1a2a4f; padding: 6px; font-size: 13px; background: #fff; }
```



HOW THE BROWSER DISPLAYS IT

7.5.2 overflow-x and overflow-y

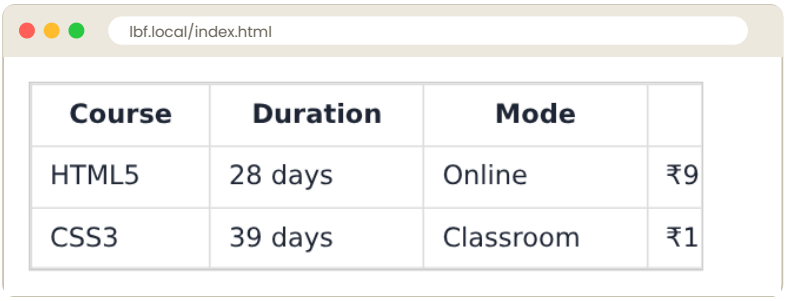
The two axes can be controlled separately: `overflow-x: auto` for a horizontally scrollable table or code block, `overflow-y: auto` for a sidebar with its own scrollbar. (If one axis is visible and the other is not, the visible one is computed as auto.) Horizontal page overflow, usually caused by a fixed-width element or `100vw` plus a scrollbar, is a frequent mobile bug; find the culprit with the outline trick from Module 5.

A TABLE THAT SCROLLS HORIZONTALLY INSIDE ITS OWN BOX CSS

```
.table-wrap { overflow-x: auto; max-width: 380px; border: 1px solid #ccc; }
table { border-collapse: collapse; min-width: 600px; } td, th { border: 1px solid #ddd; padding: 6px
10px; white-space: nowrap; }
```

INDEX.HTML HTML

```
<div class="table-wrap"><table><tr><th>Course</th><th>Duration</th><th>Mode</th><th>Fee</th><th>Next
batch</th></tr><tr><td>HTML5</td><td>28 days</td><td>Online</td><td>₹9,999</td><td>2 Nov 2026</td>
</tr><tr><td>CSS3</td><td>39 days</td><td>Classroom</td><td>₹11,999</td><td>9 Nov 2026</td></tr>
</table></div>
```



HOW THE BROWSER DISPLAYS IT

7.6 Floats (legacy layout)

Floats were designed to let text wrap around images, as in print. From about 2005 to 2017 they were also misused to build whole page layouts, which needed hacks such as clearfix. Today Flexbox and Grid handle layout; floats remain the right tool only for wrapping text around something.

7.6.1 float and clear

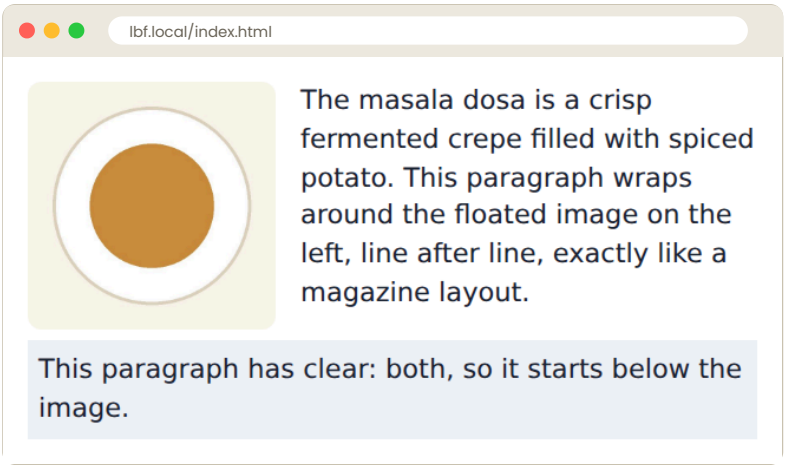
`float: left` or `right` moves the element to that side of its container, shrinks it to fit (unless it has a width), and lets inline content (text) flow around it. Following block boxes ignore it, but their text wraps. `clear: left | right | both` on a later element moves it down below the floats.

TEXT WRAPPING AROUND A FLOATED IMAGE CSS

```
.float { float: left; width: 140px; margin: 0 14px 6px 0; border-radius: 8px; }
.clear { clear: both; background: #eef1f8; padding: 6px; }
p { margin: 0 0 8px; }
```

INDEX.HTML HTML

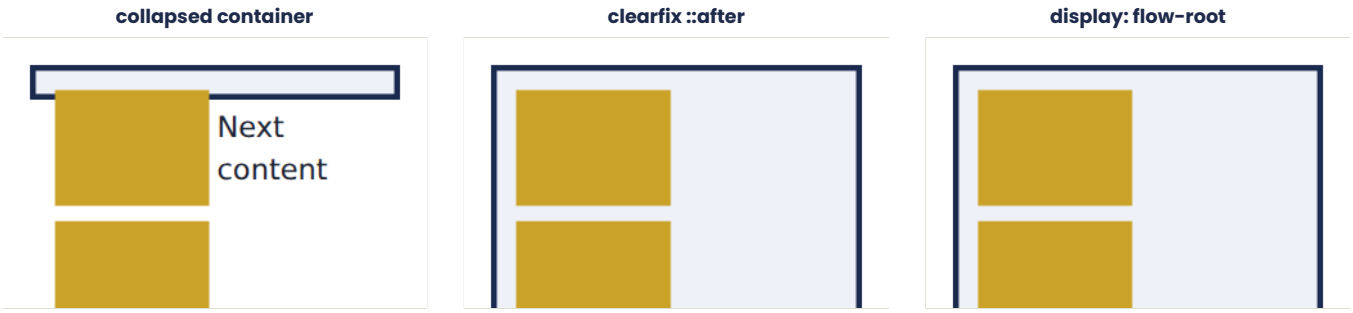
```
<img class="float" src="food-dosa.jpg" alt="Masala dosa">
<p>The masala dosa is a crisp fermented crepe filled with spiced potato. This paragraph wraps around
the floated image on the left, line after line, exactly like a magazine layout.</p>
<p class="clear">This paragraph has clear: both, so it starts below the image.</p>
```



HOW THE BROWSER DISPLAYS IT

7.6.2 Clearfix

A container whose children are all floated has **zero height**, because floats are out of normal flow: its background and border collapse. The old fix was the **clearfix** hack (a pseudo-element that clears the floats); the modern fix is `display: flow-root` on the container, which creates a BFC that contains floats.



CONTAINING FLOATS: THE OLD CLEARFIX AND THE MODERN FLOW-ROOT

CLEARFIX VS FLOW-ROOT CSS

```css
/* Legacy clearfix (still found in older code, Bootstrap 3 etc.) */
.clearfix::after { content: ""; display: block; clear: both; }

/* Modern equivalent */
.container { display: flow-root; }
```

7.6.3 Wrapping text around images

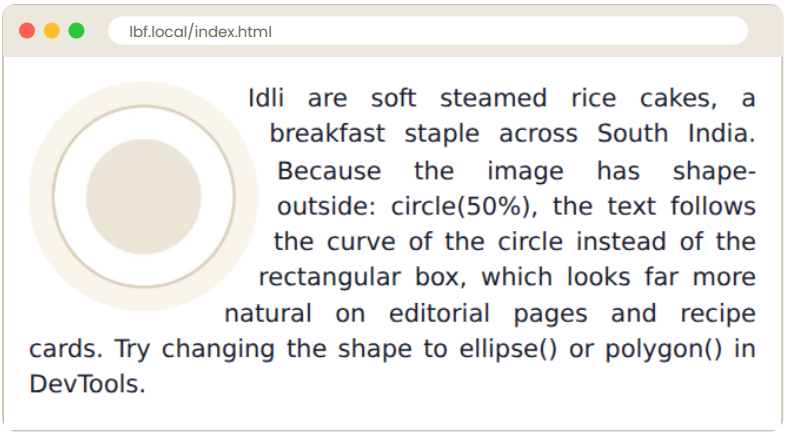
Floats combined with `shape-outside` can wrap text around non-rectangular shapes such as circles, polygons or the transparent areas of an image, a magazine effect that is impossible with Flexbox or Grid.

TEXT WRAPPING AROUND A CIRCLE WITH SHAPE-OUTSIDE CSS

```css
.round { float: left; width: 130px; height: 130px; border-radius: 50%; object-fit: cover;
         shape-outside: circle(50%); margin-right: 14px; }
p { font-size: 14px; text-align: justify; }
```

INDEX.HTML HTML

```
<img class="round" src="food-idli.jpg" alt="Idli"><p>Idli are soft steamed rice cakes, a breakfast staple across South India. Because the image has shape-outside: circle(50%), the text follows the curve of the circle instead of the rectangular box, which looks far more natural on editorial pages and recipe cards. Try changing the shape to ellipse() or polygon() in DevTools.</p>
```



HOW THE BROWSER DISPLAYS IT

7.7 Centring techniques

7.7.1 Horizontal and vertical centring methods compared

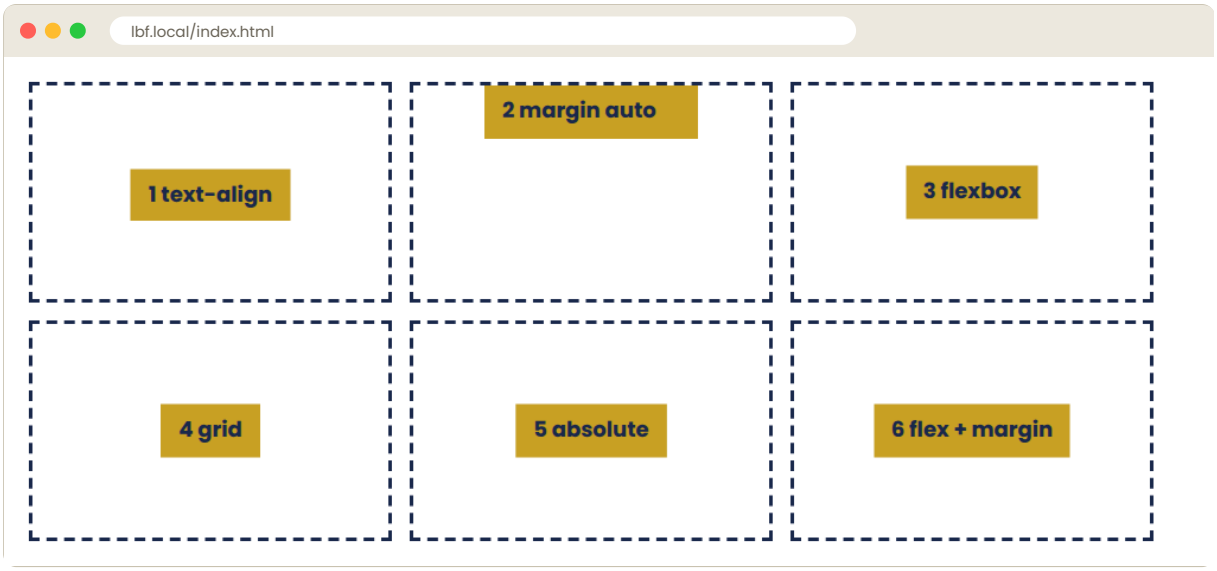
"How do you centre a div?" is the most famous CSS question. The answer depends on what you are centring (text, a block, an unknown-size element) and in which direction. All six methods below are rendered in the same 200×120 frame:

SIX WAYS TO CENTRE CSS

```
.all { display: flex; flex-wrap: wrap; gap: 10px; } .f { width: 200px; height: 120px; border: 2px dashed #1a2a4f; position: relative; } .k { background: #c9a227; color: #1a2a4f; padding: 6px 10px; font: 600 12px "Poppins", sans-serif; }
.m1 { text-align: center; line-height: 120px; }                          /* 1: inline content */
.m2 .k { width: 100px; margin: 0 auto; }                                    /* 2: block, horizontal only */
.m3 { display: flex; justify-content: center; align-items: center; }       /* 3: flexbox */
.m4 { display: grid; place-items: center; }                                 /* 4: grid */
.m5 .k { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }   /* 5: absolute */
.m6 { display: flex; } .m6 .k { margin: auto; }                             /* 6: auto margins in flex */
```

INDEX.HTML HTML

```
<div class="all">
<div class="f m1"><span class="k">1 text-align</span></div>
<div class="f m2"><div class="k">2 margin auto</div></div>
<div class="f m3"><div class="k">3 flexbox</div></div>
<div class="f m4"><div class="k">4 grid</div></div>
<div class="f m5"><div class="k">5 absolute</div></div>
<div class="f m6"><div class="k">6 flex + margin</div></div>
</div>
```



HOW THE BROWSER DISPLAYS IT

Method	Horizontal	Vertical	Unknown size	When to use
`text-align: center`	inline content	no (line-height hack only for one line)	yes	text, inline-blocks, images
`margin: 0 auto`	blocks with a width	no	needs width (or fit-content)	page containers
Flexbox `justify-content` + `align-items`	yes	yes	yes	one or several items in a row
Grid `place-items: center`	yes	yes	yes	the shortest: one item in a box
absolute + `translate(-50%, -50%)`	yes	yes	yes	overlays on top of other content
`margin: auto` in flex/grid	yes	yes	yes	pushing a single item to the centre
`position: absolute; inset: 0; margin: auto`	yes	yes	needs width and height	modal boxes

Default answer today: `display: grid; place-items: center;` on the parent. For a row of items, Flexbox. For an overlay, absolute positioning with translate or inset: 0 and margin: auto.

PRACTICE CHECKPOINT

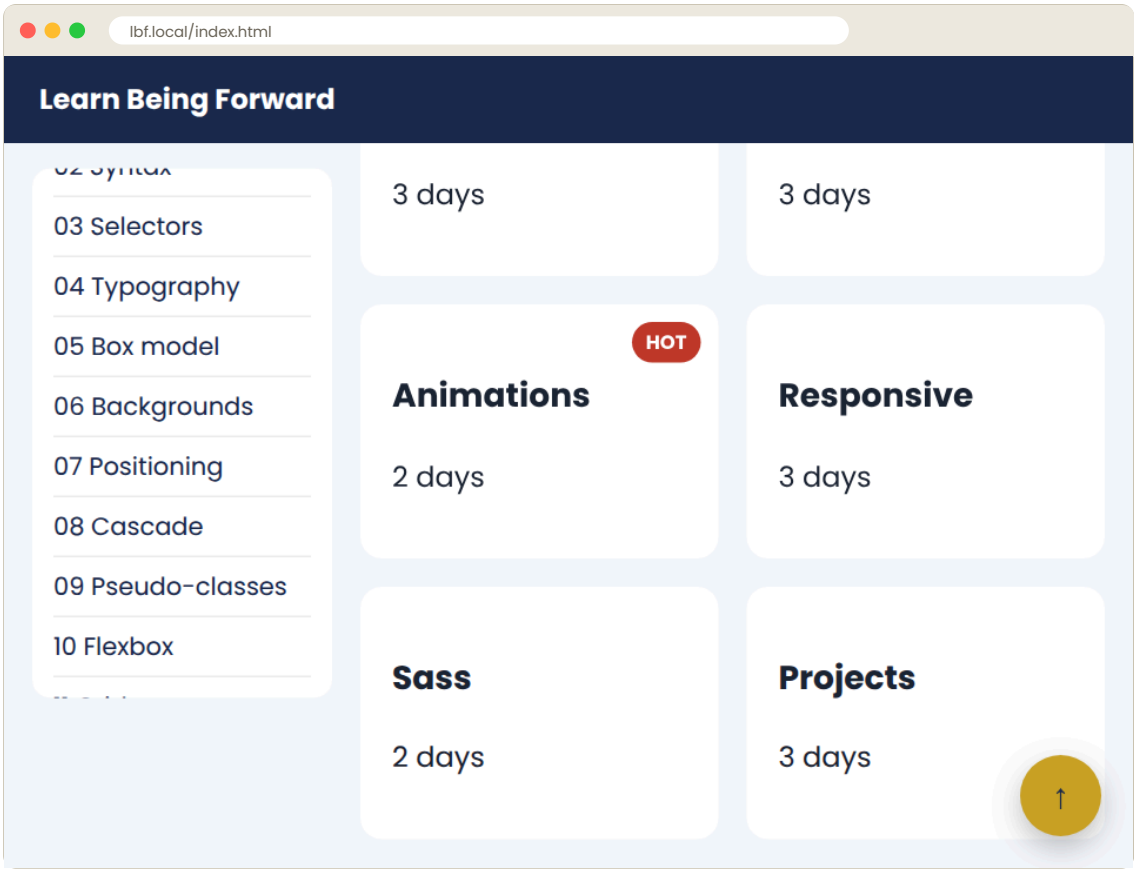
A page with a **sticky header**, a **fixed "back to top" button**, a **badge** absolutely positioned on a card, and a **scrollable sidebar**.

LAYOUT.CSS: SAMPLE SOLUTION CSS

```css
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: "Poppins", sans-serif; background: #f4f6fb; color: #1d2433; }
.site-header { position: sticky; top: 0; z-index: 10; background: #1a2a4f; color: #fff; padding: 12px
20px; font-weight: 700; }
.layout { display: flex; gap: 16px; padding: 16px; }
.sidebar { width: 170px; height: 300px; overflow-y: auto; position: sticky; top: 64px;
           background: #fff; border-radius: 10px; padding: 8px 12px; flex-shrink: 0; }
.sidebar a { display: block; padding: 6px 0; color: #1a2a4f; text-decoration: none; border-bottom: 1px
solid #eee; font-size: 14px; }
.cards { flex: 1; display: grid; grid-template-columns: repeat(2, 1fr); gap: 16px; align-content:
start; }
.card { position: relative; background: #fff; border-radius: 12px; padding: 18px; min-height: 140px; }
.badge { position: absolute; top: 10px; right: 10px; background: #c0392b; color: #fff; font-size:
11px; font-weight: 700; padding: 3px 8px; border-radius: 999px; }
.to-top { position: fixed; right: 18px; bottom: 18px; width: 46px; height: 46px; border-radius: 50%;
display: grid; place-items: center;
          background: #c9a227; color: #1a2a4f; text-decoration: none; font-size: 20px; box-shadow: 0
6px 16px rgb(0 0 0 / .25); }
```

INDEX.HTML HTML

```html
<header class="site-header" id="top">Learn Being Forward</header>
<div class="layout">
  <nav class="sidebar" aria-label="Modules">
    <a href="#">01 Introduction</a><a href="#">02 Syntax</a><a href="#">03 Selectors</a><a href="#">04
Typography</a>
    <a href="#">05 Box model</a><a href="#">06 Backgrounds</a><a href="#">07 Positioning</a><a
href="#">08 Cascade</a>
    <a href="#">09 Pseudo-classes</a><a href="#">10 Flexbox</a><a href="#">11 Grid</a><a href="#">12
Responsive</a>
  </nav>
  <main class="cards">
    <article class="card"><span class="badge">NEW</span><h3>Flexbox</h3><p>3 days</p></article>
    <article class="card"><h3>Grid</h3><p>3 days</p></article>
    <article class="card"><span class="badge">HOT</span><h3>Animations</h3><p>2 days</p></article>
    <article class="card"><h3>Responsive</h3><p>3 days</p></article>
    <article class="card"><h3>Sass</h3><p>2 days</p></article>
    <article class="card"><h3>Projects</h3><p>3 days</p></article>
  </main>
</div>
<a class="to-top" href="#top" aria-label="Back to top">↑</a>
```



SCROLLED 200PX: THE HEADER STAYS AT THE TOP, THE SIDEBAR STICKS BELOW IT AND SCROLLS ON ITS OWN, THE BUTTON STAYS FIXED

MODULE 08 · BEGINNER

The cascade, specificity and inheritance

Minimum duration **1 day** | Level **Beginner**

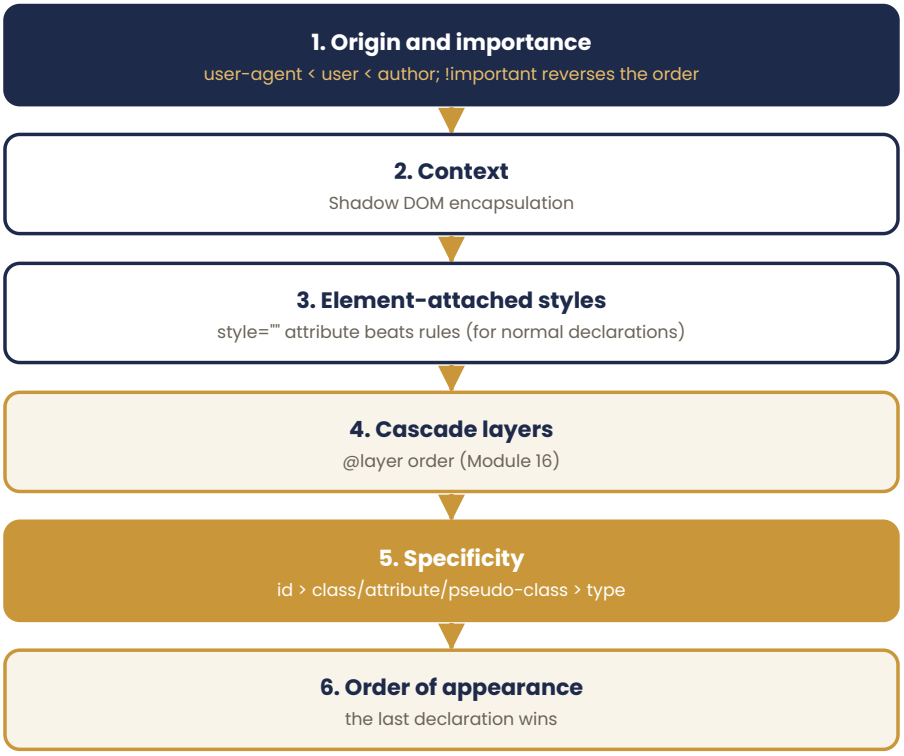
These rules decide which style wins when several rules target the same element, and they explain most "why isn't my CSS working" problems. The module covers the cascade in order (origin, importance and !important, specificity, source order), how to calculate specificity for every kind of selector and combined selectors and how to keep it low, inherited and non-inherited properties, the keywords inherit, initial, unset, revert and revert-layer, the all property, and reading crossed-out rules in DevTools. Every "which colour wins" answer was checked in Chromium.

IN THIS MODULE

8.1 The cascade
8.2 Specificity
8.3 Inheritance
8.4 Debugging conflicts

8.1 The cascade

For every property of every element, the browser gathers all the declarations that apply, from all stylesheets, and sorts them with the **cascade algorithm**. The first criterion that differs decides the winner; later criteria are only consulted on a tie.



THE CASCADE, CRITERION BY CRITERION

8.1.1 Origin: user-agent, user and author styles

Declarations come from three **origins**. **User-agent** styles are the browser defaults (Module 2). **User** styles are set by the person using the browser: minimum font size, forced colours, a user stylesheet or an extension such as a dark-mode extension. **Author** styles are the website's own CSS. For normal declarations, author beats user beats user-agent; this is why your `h1 { font-size: 24px }` overrides the browser's 2em.

8.1.2 Importance (!important) and why to avoid it

Adding `!important` after a value moves the declaration into a higher-priority group that beats every normal declaration, whatever its specificity or order. Among important declarations the origin order is **reversed** (user !important beats author !important), so that users with accessibility needs always get the last word.

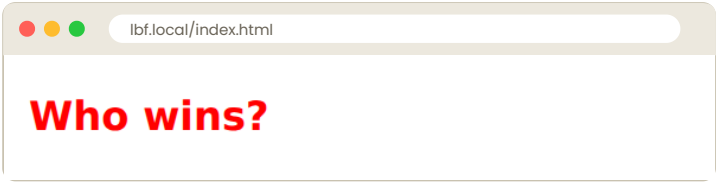
Priority (highest first)
Transitions in progress
User-agent `!important`
User `!important`
Author `!important`
Animations (@keyframes) in progress
Author normal (including inline style attributes)
User normal
User-agent normal

IMPORTANT BEATS SPECIFICITY CSS

```css
#title.main { color: blue; }      /* specificity (1,1,0) */
h2 { color: red !important; }      /* lowest specificity, but important */
```

INDEX.HTML HTML

```html
<h2 id="title" class="main">Who wins?</h2>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
winner: red   (rgb(255, 0, 0))
```

The only way to override an !important declaration is another !important one with higher specificity or later order, which starts an escalation that makes stylesheets unmaintainable. Legitimate uses are rare: utility classes that must always win (`.hidden { display: none !important; }`), overriding inline styles

from third-party widgets you cannot edit, and user stylesheets. Fix specificity problems at their source instead (8.2.3) or use cascade layers (Module 16).

8.1.3 Specificity

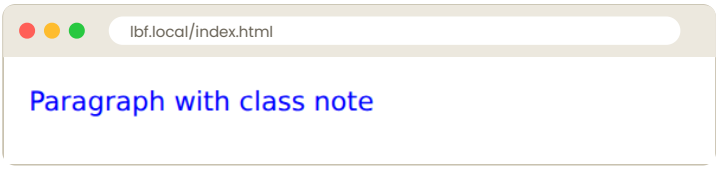
When origin, importance and layer are equal, the declaration whose **selector is more specific** wins, regardless of where it appears. Specificity is explained in detail in 8.2; in short, an id selector beats any number of classes, and a class beats any number of element types.

SPECIFICITY BEATS ORDER CSS

```css
.note { color: blue; }   /* (0,1,0): wins */
p { color: red; }        /* (0,0,1): comes later but is less specific */
```

INDEX.HTML HTML

```html
<p class="note">Paragraph with class note</p>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
winner: blue   (rgb(0, 0, 255))
```

8.1.4 Source order

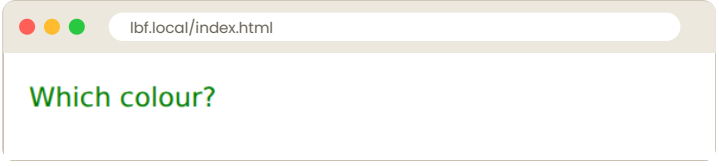
If everything else is equal, the declaration that appears **last** wins: later in the same stylesheet, in a later stylesheet, or in a later `<style>` or `<link>`. This is why the order in which you link files matters (reset first, then base styles, then components, then utilities), and why overrides go after the rules they override.

ORDER DECIDES A TIE CSS

```css
p { color: red; }
p { color: green; }   /* same specificity, later: wins */
```

INDEX.HTML HTML

```html
<p>Which colour?</p>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
winner: green   (rgb(0, 128, 0))
```

8.2 Specificity

8.2.1 Calculating specificity (inline, ID, class, type)

Specificity is a three-part score **(A, B, C)**, compared from left to right like a version number, never added up as one number:

Column	Counts	Examples
A	ID selectors	`#nav`
B	class selectors, attribute selectors, pseudo-classes	`.btn`, `[type="text"]`, `:hover`, `:nth-child(2)`
C	type selectors and pseudo-elements	`p`, `li`, `::before`
(none)	universal selector and combinators add nothing	`*`, `>`, `+`, `~`, space
special	`:is()`, `:not()`, `:has()` take the specificity of their most specific argument; `:where()` is always zero	`:is(#a, .b)` = (1,0,0)

Inline `style` attributes are not selectors; they sit above all selector specificity (sometimes written as a fourth column, (1,0,0,0)). `!important` is above that.



compare left to right: the first column that differs decides

SPECIFICITY COLUMNS

Selector	A (ids)	B (classes…)	C (types)	Specificity
`*`	0	0	0	(0,0,0)
`li`	0	0	1	(0,0,1)
`ul li`	0	0	2	(0,0,2)
`ul ol + li`	0	0	3	(0,0,3)
`.nav`	0	1	0	(0,1,0)
`a:hover`	0	1	1	(0,1,1)
`input[type="email"]`	0	1	1	(0,1,1)
`.nav .item.active`	0	3	0	(0,3,0)
`p::first-line`	0	0	2	(0,0,2)
`#header`	1	0	0	(1,0,0)
`#header .nav a:hover`	1	2	1	(1,2,1)
`:where(#header) a`	0	0	1	(0,0,1)

This script computes specificity with the same rules and confirms the comparison in the browser: eleven classes still lose to one id, because (0,11,0) is less than (1,0,0) in the first column.

COMPUTED IN CHROMIUM

```
winner: green   (rgb(0, 128, 0))
```

8.2.2 Specificity of combined selectors

For a selector list (`h1, .title`), each selector keeps its own specificity: the list is just a shortcut for separate rules. For compound and complex selectors, add up the columns of every simple selector. Ten puzzles, each answered by Chromium:

PUZZLE 1 — CSS

```css
p { color: red; }
.text { color: blue; }
```

SNIPPET — HTML

```html
<p id="t" class="text">x</p>
```

COMPUTED IN CHROMIUM

```
winner: blue   (rgb(0, 0, 255))
```

Why: .text (0,1,0) beats p (0,0,1).

PUZZLE 2 — CSS

```css
div p { color: red; }
p.text { color: blue; }
```

SNIPPET — HTML

```html
<div><p id="t" class="text">x</p></div>
```

COMPUTED IN CHROMIUM

```
winner: blue   (rgb(0, 0, 255))
```

Why: p.text (0,1,1) beats div p (0,0,2).

PUZZLE 3 — CSS

```css
#main p { color: red; }
.content .box p.text { color: blue; }
```

SNIPPET — HTML

```html
<div id="main"><div class="content"><div class="box"><p id="t" class="text">x</p></div></div></div>
```

COMPUTED IN CHROMIUM

```
winner: red   (rgb(255, 0, 0))
```

Why: #main p (1,0,1) beats (0,3,1).

PUZZLE 4 CSS

```css
.a { color: red; }
.b { color: blue; }
```

SNIPPET HTML

```html
<p id="t" class="b a">x</p>
```

COMPUTED IN CHROMIUM

```
winner: blue   (rgb(0, 0, 255))
```

Why: tie (0,1,0): the later RULE wins, not the order of the classes in the HTML.

PUZZLE 5 CSS

```css
p { color: red !important; }
#t { color: blue; }
```

SNIPPET HTML

```html
<p id="t">x</p>
```

COMPUTED IN CHROMIUM

```
winner: red   (rgb(255, 0, 0))
```

Why: !important beats the id.

PUZZLE 6 CSS

```css
#t { color: red; }
```

SNIPPET HTML

```html
<p id="t" style="color: blue">x</p>
```

COMPUTED IN CHROMIUM

```
winner: blue   (rgb(0, 0, 255))
```

Why: the inline style beats the id.

PUZZLE 7 CSS

```css
ul li:first-child { color: red; }
li.first { color: blue; }
```

SNIPPET HTML

```html
<ul><li id="t" class="first">x</li></ul>
```

COMPUTED IN CHROMIUM

```
winner: red   (rgb(255, 0, 0))
```

Why: ul li:first-child is (0,1,2) and li.first is (0,1,1): the pseudo-class counts as a class, so red wins.

PUZZLE 8 CSS

```css
:where(#nav) a { color: red; }
a { color: blue; }
```

SNIPPET HTML

```html
<nav id="nav"><a id="t" href="#">x</a></nav>
```

COMPUTED IN CHROMIUM

```
winner: blue   (rgb(0, 0, 255))
```

Why: :where() counts zero: (0,0,1) tie, later wins.

PUZZLE 9 CSS

```css
:is(#nav, .menu) a { color: red; }
.menu a.link { color: blue; }
```

SNIPPET HTML

```html
<nav id="nav" class="menu"><a id="t" class="link" href="#">x</a></nav>
```

COMPUTED IN CHROMIUM

```
winner: red   (rgb(255, 0, 0))
```

Why: :is() takes its strongest argument, #nav: (1,0,1) wins.

PUZZLE 10 CSS

```css
body { color: green; }
* { color: red; }
```

SNIPPET HTML

```html
<p id="t">x</p>
```

COMPUTED IN CHROMIUM

```
winner: red   (rgb(255, 0, 0))
```

Why: * (0,0,0) applies directly to p and beats the INHERITED green: inherited values have no specificity.

8.2.3 Keeping specificity low

A stylesheet with high specificity is hard to change: every override must be even more specific, leading to long selectors, ids and !important. Healthy CSS keeps specificity **low and flat**, so that source order and layers can do the work.

- Style with **single classes** (`.card-title`) rather than chains (`.sidebar .card div h3`).
- **Avoid ids** for styling; use them for links, labels and scripts.
- Avoid qualifying classes with types (`div.card`): it adds specificity and ties the style to one element.
- Use `:where()` for base and library styles so anything can override them; reserve `:is()` for when you want the specificity.

- Order files from generic to specific (reset, base, layout, components, utilities), as in ITCSS (Module 18), and use `@layer` to make that order explicit.
- Nesting in Sass or native CSS (Module 16) silently increases specificity: keep it to one or two levels.

HIGH VS LOW SPECIFICITY CSS

```css
/* High-specificity version: (1,3,2) - every override needs more */
#sidebar .widget ul li.active a { color: #c9a227; }

/* Low-specificity version: (0,1,0) - easy to override and reuse */
.nav-link--active { color: #c9a227; }

/* Library base styles that never fight with yours: (0,0,0)-(0,0,1) */
:where(.btn) { padding: .5rem 1rem; border-radius: 6px; }
```

8.3 Inheritance

8.3.1 Inherited vs non-inherited properties

Some properties are **inherited**: if an element has no declared value, it takes its parent's computed value. Mostly these are **text**-related, so setting a font or colour on `body` styles all the text inside. Others are **not inherited**: they fall back to their initial value; it would make no sense for every child to copy its parent's border or padding.

Inherited (typical)	Not inherited (typical)
`color`	`background` (all background properties)
`font` and all font-* properties	`border`, `margin`, `padding`
`line-height`, `letter-spacing`, `word-spacing`	`width`, `height` and sizing
`text-align`, `text-indent`, `text-transform`	`display`, `position`, `top/left`, `z-index`
`white-space`, `direction`	`overflow`, `box-shadow`, `opacity`
`visibility`, `cursor`	`transform`, `transition`, `animation`
`list-style`, `border-collapse`	`flex`, `grid` properties
custom properties (`--x`) by default	`text-decoration` (though it is drawn through children)

TEXT PROPERTIES INHERIT; BOX PROPERTIES DO NOT CSS

```css
.parent { color: #1a2a4f; font-family: Georgia, serif; font-size: 18px; border: 3px solid #c9a227;
padding: 10px; background: #fff8e1; }
```

INDEX.HTML HTML

```html
<div class="parent">Parent text<p>Child paragraph: same colour, font and size (inherited) but no
border, padding or background of its own.</p><button>A button</button></div>
```



NOTE THE BUTTON: FORM CONTROLS DO NOT INHERIT FONTS BY DEFAULT (THEIR USER-AGENT STYLES SET FONT EXPLICITLY)

COMPUTED IN CHROMIUM

```
child p:  color rgb(26, 42, 79), font Georgia, serif, border 0px, padding 0px
button:   font Arial (not inherited)
```

8.3.2 Keywords: inherit, initial, unset, revert, revert-layer

Five **CSS-wide keywords** work with every property:

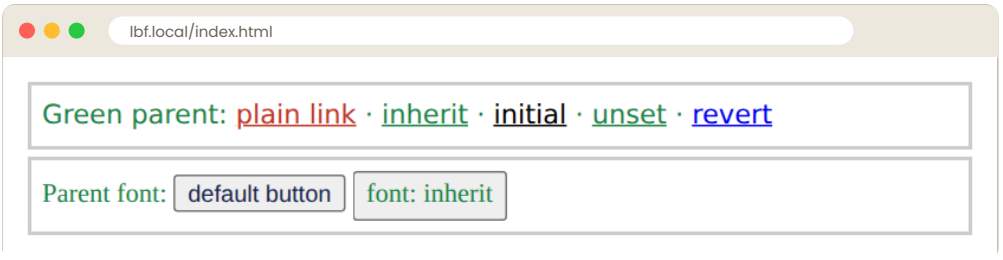
Keyword	Sets the property to
`inherit`	the parent's computed value (forces inheritance even for non-inherited properties)
`initial`	the property's initial value from the specification (e.g. `display: inline`, `color: black`)
`unset`	`inherit` for inherited properties, `initial` for the others: "as if nothing had been set"
`revert`	the value the **previous origin** would give, usually the browser default (e.g. `display: block` for a div)
`revert-layer`	the value from the previous cascade layer (Module 16)

THE FIVE KEYWORDS ON A LINK INSIDE A GREEN BOX CSS

```css
a { color: #c0392b; }
button { color: #1a2a4f; }
.box { color: #1e8449; border: 2px solid #ccc; padding: 6px; margin: 4px 0; }
.box a.inherit { color: inherit; }
.box a.initial { color: initial; }
.box a.unset { color: unset; }
.box a.revert { color: revert; }
.box button.inherit { font: inherit; color: inherit; }
```

INDEX.HTML HTML

```html
<div class="box">Green parent:
  <a href="#">plain link</a> ·
  <a class="inherit" href="#">inherit</a> ·
  <a class="initial" href="#">initial</a> ·
  <a class="unset" href="#">unset</a> ·
  <a class="revert" href="#">revert</a>
</div>
<div class="box" style="font-family: Georgia, serif">Parent font: <button>default button</button>
<button class="inherit">font: inherit</button></div>
```



INHERIT AND UNSET TAKE THE GREEN; INITIAL GIVES BLACK (THE SPEC DEFAULT); REVERT GIVES THE BROWSER'S LINK BLUE

COMPUTED IN CHROMIUM

```
color: inherit  -> rgb(0, 128, 0)
color: initial  -> rgb(0, 0, 0)
color: unset    -> rgb(0, 128, 0)
color: revert   -> rgb(0, 0, 238)
div display: revert  -> block
div display: initial -> inline
```

WATCH OUT

`initial` is not "the browser default": the initial value of `display` is `inline`, so `div { display: initial }` turns a div inline. Use `revert` when you mean "go back to the browser's default".

8.3.3 all property

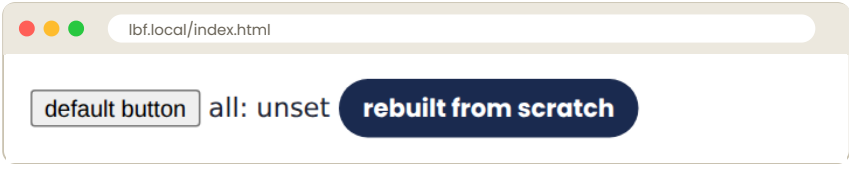
`all` resets every property (except `direction`, `unicode-bidi` and custom properties) at once, using one of the keywords. `all: unset` strips a button or link of every style, a quick start for a custom component; `all: revert` restores browser defaults, useful inside a component to protect it from a page's aggressive global styles.

RESETTING A BUTTON WITH ALL: UNSET CSS

```css
.plain { all: unset; cursor: pointer; }
.plain:focus-visible { outline: 2px solid #1a2a4f; }
.chip { all: unset; display: inline-block; padding: 6px 14px; border-radius: 999px; background:
#1a2a4f; color: #fff; font: 600 14px "Poppins", sans-serif; cursor: pointer; }
```

INDEX.HTML HTML

```html
<button>default button</button> <button class="plain">all: unset</button> <button class="chip">rebuilt
from scratch</button>
```



HOW THE BROWSER DISPLAYS IT

8.4 Debugging conflicts

8.4.1 Reading crossed-out rules in DevTools

When a style does not apply, inspect the element: the Styles pane lists every matching rule, most important first, and **strikes through** each declaration that lost. Read it from the top: the first non-crossed value of a property is the winner, and the file and line of every competing rule are linked.



CROSSED-OUT DECLARATIONS IN THE STYLES PANE

Common reasons a rule "does nothing", in the order to check them:

1. **The selector does not match**: the rule is not listed at all. Check the class name spelling, the HTML structure for combinators, and that the CSS file actually loaded (Network panel).
2. **The declaration is invalid**: a warning icon and strike-through: misspelt property or value, missing unit, missing semicolon on the previous line.
3. **It lost the cascade**: struck through because another rule has higher specificity, !important, or comes later. Click through to the winning rule.
4. **It applies but has no visible effect**: `width` on an inline element, `z-index` without position, `top` on a static element, `vertical-align` on a block, `justify-content` on a non-flex element. DevTools shows an "i" icon explaining why (Chrome's "inactive CSS" hints).
5. **It is overridden by an inherited or user style**: check the Computed tab, which shows the final value and where it came from.

PRACTICE CHECKPOINT

Solve ten **"which colour wins?"** puzzles (8.2.2 above), then refactor a stylesheet that overuses **!important** and **IDs**.

BEFORE: SPECIFICITY WAR CSS

```css
#header #nav ul li a { color: #fff !important; padding: 10px !important; }
#header #nav ul li a.active { color: #c9a227 !important; }
#content #main .post h2 { color: #1a2a4f; }
#content #main .post.featured h2 { color: #c0392b !important; }
.sidebar h2 { color: green !important; }   /* "why is every heading green?" */
```

AFTER: REFACTORED CSS

```css
/* Flat, class-based, ordered from generic to specific. No ids, no !important. */
.nav__link         { color: #fff; padding: 10px; }
.nav__link--active { color: #c9a227; }

.post__title           { color: #1a2a4f; }
.post--featured .post__title { color: #c0392b; }   /* (0,2,0): one deliberate step up */

.sidebar__title    { color: green; }
```

COMPUTED IN CHROMIUM

```
winner: orange   (rgb(255, 165, 0))
```

The refactored rules have specificities of (0,1,0) or (0,2,0), so any later component or utility can override them without escalation, and the modifier class is a single, readable switch in the HTML.

LEVEL 2

Intermediate level

Outcome: build layouts with Flexbox and Grid, make sites responsive, and add transitions, animations and theming with custom properties.

7 MODULES · 16 DAYS MINIMUM

Module 09 · Advanced selectors, pseudo-classes and pseudo-elements	**2 days**
Module 10 · Flexbox	**3 days**
Module 11 · CSS Grid	**3 days**
Module 12 · Responsive design and media queries	**3 days**
Module 13 · Transforms and transitions	**2 days**
Module 14 · Animations	**2 days**
Module 15 · Custom properties and CSS functions	**1 day**

MODULE 09 · INTERMEDIATE

Advanced selectors, pseudo-classes and pseudo-elements

Minimum duration **2 days** | Level **Intermediate**

Pseudo-classes target elements by state or position; pseudo-elements style parts of elements. Together they remove the need for many extra classes. This module covers user-action pseudo-classes (:hover, :active, :focus, :focus-visible, :focus-within), structural ones (:first-child, :last-child, :only-child, :nth-child() formulas, :nth-of-type(), :empty, :root), form states (:checked, :disabled, :required, :valid, :invalid, :user-valid, :user-invalid, :placeholder-shown), the logical :not(), :is(), :where() and :has(), and :target, :lang() and :any-link; then the pseudo-elements ::before, ::after, ::first-letter, ::first-line, ::selection, ::placeholder, ::marker, ::backdrop and ::file-selector-button, with practical recipes: decorative icons and underlines, zebra tables and CSS-only tooltips.

IN THIS MODULE

9.1 User-action pseudo-classes
9.2 Structural pseudo-classes
9.3 Form pseudo-classes
9.4 Logical pseudo-classes
9.5 Other pseudo-classes
9.6 Pseudo-elements
9.7 Practical uses

A **pseudo-class** (one colon, `:hover`) selects elements in a particular **state** or **position** that is not expressed by a class in the HTML. A **pseudo-element** (two colons, `::before`) selects a **part** of an element, or creates a box that is not in the HTML at all. Pseudo-classes count as classes in specificity (0,1,0); pseudo-elements count as types (0,0,1).

9.1 User-action pseudo-classes

9.1.1 :hover, :active, :focus

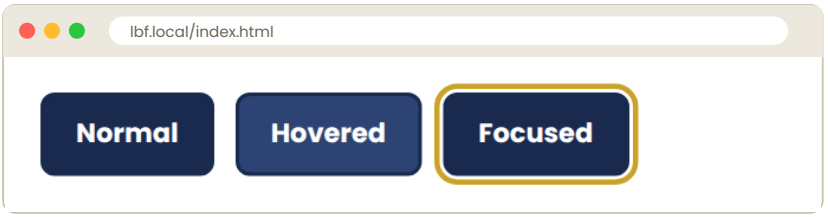
Pseudo-class	Matches while…	Notes
`:hover`	the pointer is over the element (or a descendant)	touch screens apply it on tap and it may "stick"; never hide essential content behind hover
`:active`	the element is being activated (mouse button or key held down)	pressed effect for buttons
`:focus`	the element has focus (keyboard, mouse click or script)	focusable elements: links, buttons, form controls, tabindex

BUTTON STATES CSS

```css
.btn { font: 600 15px "Poppins", sans-serif; padding: 10px 18px; margin: 6px; border-radius: 8px;
border: 2px solid #1a2a4f;
       background: #1a2a4f; color: #fff; transition: background .15s, transform .1s; }
.btn:hover { background: #2d4373; }
.btn:active { transform: translateY(2px); background: #0f1a33; }
.btn:focus { outline: 3px solid #c9a227; outline-offset: 2px; }
```

INDEX.HTML HTML

```html
<button class="btn">Normal</button><button class="btn" id="h">Hovered</button><button class="btn"
id="f">Focused</button>
```



THE MIDDLE BUTTON IS HOVERED; THE RIGHT ONE HAS FOCUS

9.1.2 :focus-visible and :focus-within

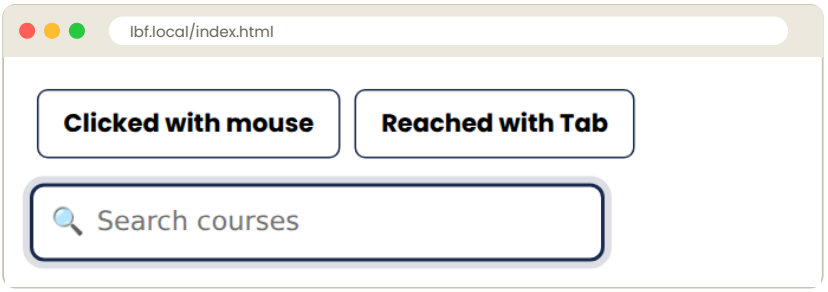
`:focus-visible` matches only when the browser judges that a focus indicator **should be shown**: after keyboard navigation, and always for text inputs, but usually not after a mouse click on a button. It lets you design a strong focus ring for keyboard users without showing it to mouse users. `:focus-within` matches an element when **it or any descendant** has focus, which is perfect for highlighting a whole form group or search box.

:FOCUS-VISIBLE AND :FOCUS-WITHIN CSS

```css
button { font: 600 14px "Poppins", sans-serif; padding: 8px 14px; margin: 4px; border-radius: 6px;
border: 1px solid #1a2a4f; background: #fff; }
button:focus { outline: none; }
button:focus-visible { outline: 3px solid #c9a227; outline-offset: 2px; }
.field { border: 2px solid #ccc; border-radius: 8px; padding: 8px 10px; margin-top: 10px; width:
300px; transition: border-color .2s, box-shadow .2s; }
.field:focus-within { border-color: #1a2a4f; box-shadow: 0 0 0 4px rgb(26 42 79 / .15); }
.field input { border: 0; outline: none; font: inherit; width: 220px; }
```

INDEX.HTML HTML

```html
<button id="m">Clicked with mouse</button><button id="k">Reached with Tab</button><div class="field">
🔍 <input id="s" placeholder="Search courses"></div>
```



FOCUS IS IN THE SEARCH INPUT, SO THE WHOLE .FIELD BOX IS HIGHLIGHTED BY :FOCUS-WITHIN

Focus can only be in one place at a time, so the :focus-visible behaviour of the buttons is shown separately: the first button was clicked with the mouse, then Tab was pressed once.



AFTER A MOUSE CLICK ON THE FIRST BUTTON AND ONE TAB: ONLY THE KEYBOARD-FOCUSED BUTTON SHOWS THE RING

9.2 Structural pseudo-classes

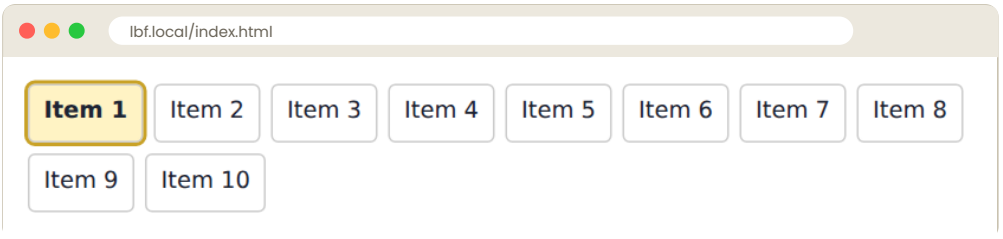
Structural pseudo-classes select elements by their **position among their siblings**, so lists, tables and grids can be styled without adding a class to every item. In the examples below, ten list items are highlighted by each selector.

9.2.1 :first-child, :last-child, :only-child

`:first-child` matches an element that is the first child of its parent; `:last-child` the last; `:only-child` one with no siblings. Note that the condition is about the element itself: `p:first-child` means "a p that is the first child", not "the first p".

:FIRST-CHILD CSS

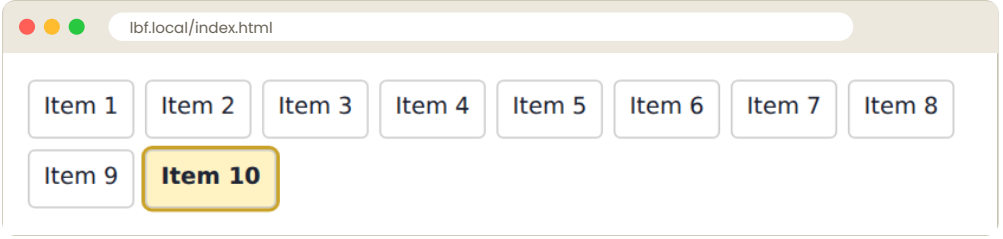
```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:first-child { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```



HOW THE BROWSER DISPLAYS IT

:LAST-CHILD CSS

```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:last-child { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```



HOW THE BROWSER DISPLAYS IT

:ONLY-CHILD CSS

```css
.chip { display: inline-block; padding: 4px 10px; border: 1px solid #1a2a4f; border-radius: 999px;
margin: 2px; }
.tags :only-child { background: #1a2a4f; color: #fff; }   /* a single tag gets emphasis */
```

INDEX.HTML HTML

```html
<p class="tags"><span class="chip">html</span><span class="chip">css</span></p><p class="tags"><span
class="chip">javascript</span></p>
```



HOW THE BROWSER DISPLAYS IT

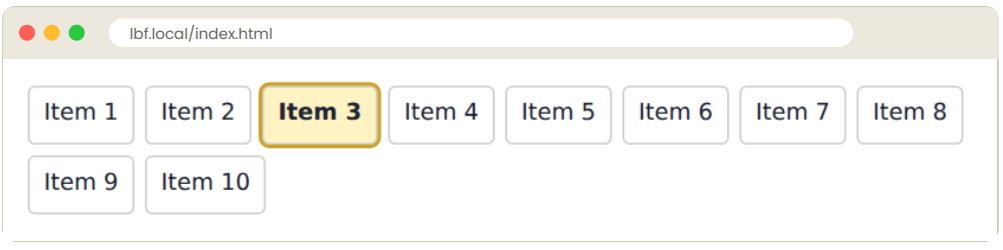
9.2.2 :nth-child(), :nth-last-child() with formulas (2n+1, odd, even)

`:nth-child(An+B)` matches the children whose position (counting from 1) equals An+B for some n = 0, 1, 2, 3… `B` is an offset and `A` a step. Keywords: `odd` = 2n+1, `even` = 2n. `:nth-last-child()` counts from the end. Since Selectors Level 4, `:nth-child(2n of .featured)` counts only the elements matching a selector.

Formula	Positions matched	Use
`3`	3	one specific item
`odd` / `2n+1`	1, 3, 5, 7…	zebra rows
`even` / `2n`	2, 4, 6…	zebra rows
`3n`	3, 6, 9…	every third item (end of a 3-column row)
`3n+1`	1, 4, 7, 10…	start of each 3-column row
`n+4`	4, 5, 6… (from the 4th on)	all but the first three
`-n+3`	3, 2, 1 (the first three)	top-3 highlight
`nth-last-child(-n+2)`	the last two	last row of a 2-column grid

:NTH-CHILD(3) CSS

```css
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:nth-child(3) { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```

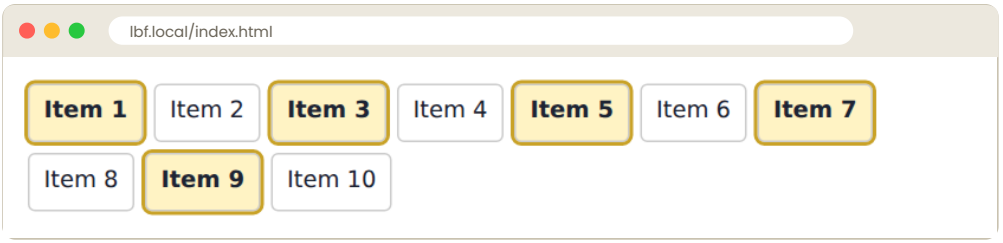


HOW THE BROWSER DISPLAYS IT

:NTH-CHILD(ODD)

CSS

```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:nth-child(odd) { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```

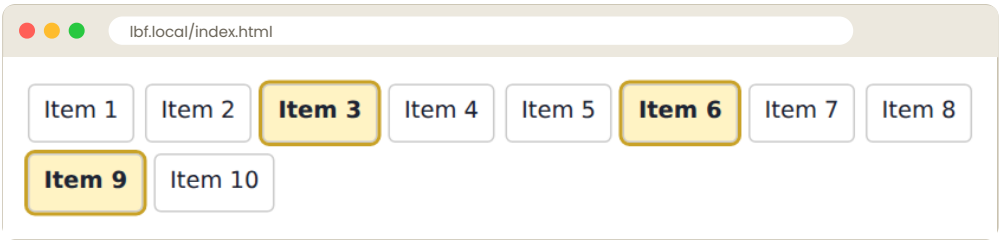


HOW THE BROWSER DISPLAYS IT

:NTH-CHILD(3N)

CSS

```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:nth-child(3n) { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```

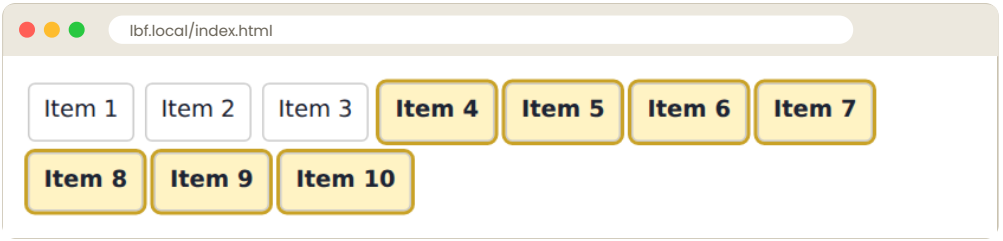


HOW THE BROWSER DISPLAYS IT

:NTH-CHILD(N+4)

CSS

```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:nth-child(n+4) { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```

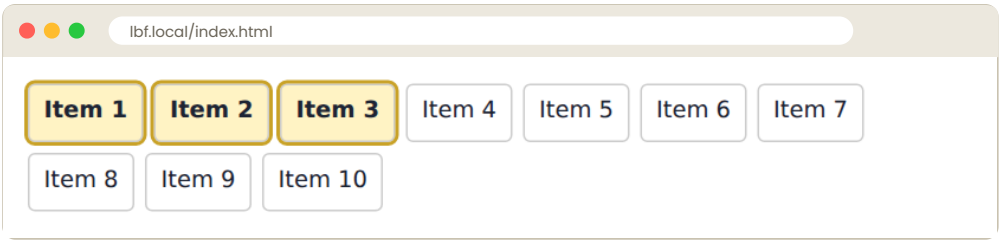


HOW THE BROWSER DISPLAYS IT

:NTH-CHILD(-N+3)

CSS

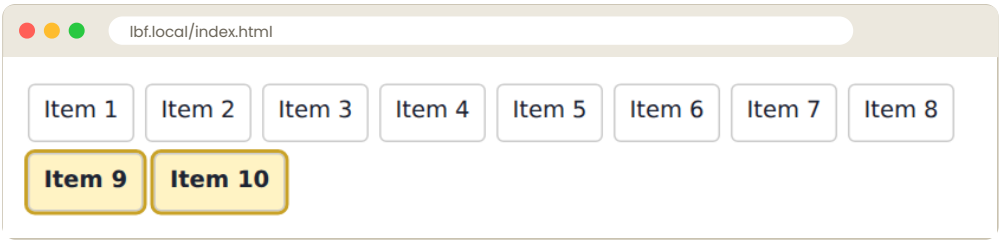
```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:nth-child(-n+3) { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```



HOW THE BROWSER DISPLAYS IT

:NTH-LAST-CHILD(-N+2) CSS

```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:nth-last-child(-n+2) { background: #fff3c4; outline: 2px solid #c9a227; font-weight: 700; }
```



HOW THE BROWSER DISPLAYS IT

How the formula `3n+1` is evaluated: substitute n = 0, 1, 2, 3 … and keep the results that are valid positions.

n	0	1	2	3	4
3n+1	1	4	7	10	13 (no 13th item)

9.2.3 :first-of-type, :nth-of-type()

The `-of-type` variants count only siblings of the **same element type**. In an article with headings, paragraphs and images mixed together, `p:first-of-type` finds the first paragraph even if a heading comes before it, where `p:first-child` would match nothing.



-CHILD VS -OF-TYPE

9.2.4 :empty, :root

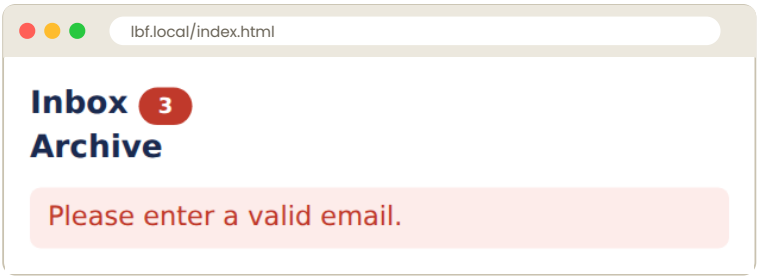
`:empty` matches elements with no children at all, not even text or a space (comments are allowed). It is handy for hiding empty containers such as an error message area or a badge with no count. `:root` matches the root element (`<html>`) with class-level specificity, and is the conventional place to declare global custom properties (Module 15).

:EMPTY HIDES EMPTY BADGES AND MESSAGES CSS

```css
.badge { display: inline-block; min-width: 18px; padding: 2px 6px; border-radius: 999px; background:
#c0392b; color: #fff; font-size: 12px; text-align: center; }
.badge:empty { display: none; }
.error:empty { display: none; }
.error { background: #fdecea; color: #c0392b; padding: 6px 10px; border-radius: 6px; margin-top: 8px;
}
:root { --brand: #1a2a4f; } h3 { color: var(--brand); margin: 0; }
```

INDEX.HTML HTML

```html
<h3>Inbox <span class="badge">3</span></h3><h3>Archive <span class="badge"></span></h3><div
class="error"></div><div class="error">Please enter a valid email.</div>
```



HOW THE BROWSER DISPLAYS IT

9.3 Form pseudo-classes

9.3.1 :checked, :disabled, :enabled, :required, :optional

`:checked` matches checked checkboxes, radio buttons and selected options. `:disabled` / `:enabled` follow the disabled attribute; `:required` / `:optional` the required attribute; `:read-only` / `:read-write` the readonly attribute. Combined with sibling combinators, `:checked` powers CSS-only toggles, tabs and custom checkboxes.

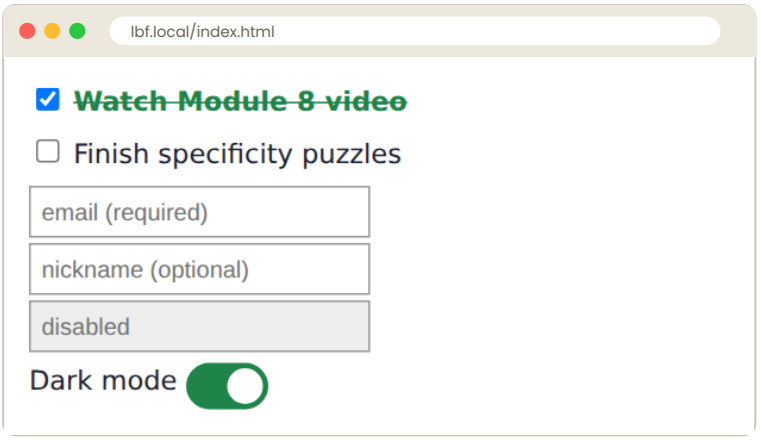
CHECKED, REQUIRED, OPTIONAL, DISABLED AND A CSS-ONLY TOGGLE SWITCH CSS

```css
label { display: block; margin: 6px 0; cursor: pointer; }
input:checked + span { font-weight: 700; color: #1e8449; text-decoration: line-through; }
input:required { border-left: 5px solid #c0392b; }
input:optional { border-left: 5px solid #ccc; }
input:disabled { background: #eee; cursor: not-allowed; }
input[type=text], input[type=email] { padding: 6px; border: 1px solid #999; margin: 3px 0; display:
block; }
.switch { position: relative; display: inline-block; width: 46px; height: 26px; }
.switch input { opacity: 0; width: 0; height: 0; }
.slider { position: absolute; inset: 0; background: #ccc; border-radius: 26px; transition: .2s; }
.slider::before { content: ""; position: absolute; width: 20px; height: 20px; left: 3px; top: 3px;
background: #fff; border-radius: 50%; transition: .2s; }
.switch input:checked + .slider { background: #1e8449; }
.switch input:checked + .slider::before { transform: translateX(20px); }
```

INDEX.HTML

HTML

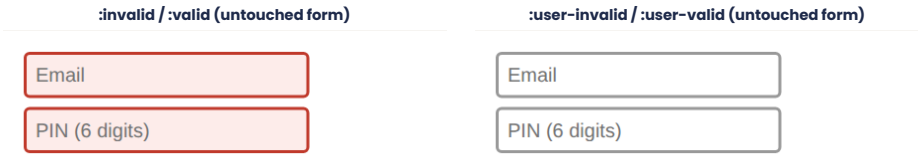
```
<label><input type="checkbox" checked> <span>Watch Module 8 video</span></label>
<label><input type="checkbox"> <span>Finish specificity puzzles</span></label>
<input type="email" placeholder="email (required)" required>
<input type="text" placeholder="nickname (optional)">
<input type="text" placeholder="disabled" disabled>
<label>Dark mode <span class="switch"><input type="checkbox" checked><span class="slider"></span>
</span></label>
```



HOW THE BROWSER DISPLAYS IT

9.3.2 :valid, :invalid, :user-valid, :user-invalid

These follow HTML constraint validation (HTML course, Module 8). `:valid` and `:invalid` apply from the start, so an empty required field is red before the user has typed anything. `:user-valid` and `:user-invalid` (Baseline 2023) apply only **after the user has interacted** with the field (changed it and moved away, or tried to submit), which is the behaviour users expect. `:in-range` / `:out-of-range` work with min and max.



:INVALID SHOUTS AT USERS BEFORE THEY START

ERRORS SHOWN ONLY AFTER INTERACTION

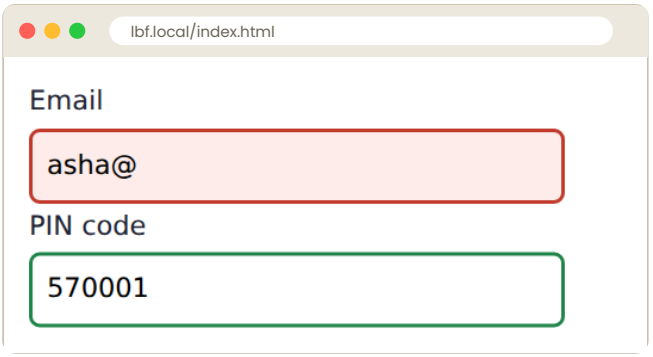
CSS

```
form { width: 300px; }
input { display: block; width: 100%; box-sizing: border-box; margin: 4px 0 2px; padding: 8px; border:
2px solid #999; border-radius: 6px; font: inherit; }
input:user-invalid { border-color: #c0392b; background: #fdecea; }
input:user-valid { border-color: #1e8449; }
.hint { font-size: 12px; color: #c0392b; display: none; margin: 0 0 8px; }
input:user-invalid + .hint { display: block; }
```

INDEX.HTML

HTML

```
<form><label>Email<input type="email" id="e" required></label><p class="hint">Enter an email like
name@example.com</p><label>PIN code<input id="p" pattern="[1-9][0-9]{5}" required></label><p
class="hint">6 digits, not starting with 0</p></form>
```



AFTER TYPING "ASHA@" AND A VALID PIN AND LEAVING EACH FIELD

9.3.3 :placeholder-shown

`:placeholder-shown` matches an input while its placeholder is visible, that is, while it is empty. The classic use is the **floating label**: the label sits inside the field like a placeholder, and moves above it once the user types. (The input needs a placeholder, even a single space.)

FLOATING LABELS WITH :PLACEHOLDER-SHOWN CSS

```css
.float { position: relative; margin: 14px 0; width: 280px; }
.float input { width: 100%; box-sizing: border-box; padding: 18px 10px 6px; border: 2px solid #999;
border-radius: 6px; font: inherit; }
.float label { position: absolute; left: 11px; top: 13px; color: #777; transition: all .15s; pointer-
events: none; }
.float input:focus + label,
.float input:not(:placeholder-shown) + label { top: 3px; font-size: 11px; color: #1a2a4f; font-weight:
700; }
```

INDEX.HTML HTML

```html
<div class="float"><input id="n" placeholder=" "><label for="n">Full name</label></div><div
class="float"><input id="c" placeholder=" " value="Mysuru"><label for="c">City</label></div>
```



EMPTY FIELD: LABEL INSIDE; FILLED FIELD: LABEL FLOATS ABOVE

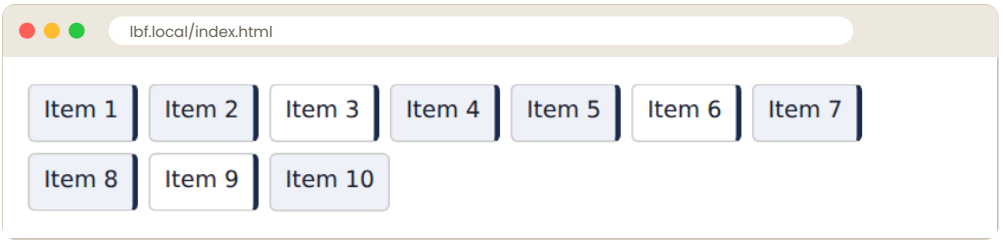
9.4 Logical pseudo-classes

9.4.1 :not()

`:not(selector list)` matches elements that do **not** match the argument. Its specificity is that of its most specific argument. Typical uses: "every item except the last" (`li:not(:last-child)`), links that are not buttons, inputs other than checkboxes.

:NOT() CSS

```
.l { list-style: none; padding: 0; display: flex; flex-wrap: wrap; gap: 6px; } .l li { padding: 6px
8px; border: 1px solid #ccc; border-radius: 4px; font-size: 13px; }
.l li:not(:last-child) { border-right: 3px solid #1a2a4f; }
.l li:not(:nth-child(3n)) { background: #eef1f8; }
```



EVERY ITEM EXCEPT THE LAST GETS A RIGHT BORDER; ITEMS NOT AT POSITIONS 3, 6, 9 GET A TINT

9.4.2 :is() and :where() (and their specificity)

`:is()` and `:where()` take a selector list and match any element that matches one of them, shortening repetitive selectors. Both are **forgiving**: an invalid selector in the list is ignored instead of killing the whole rule. They differ only in specificity: `:is()` takes the specificity of its **most specific** argument; `:where()` always has **zero**, so rules written with it are trivially easy to override, which is ideal for resets and library defaults.

:IS() FOR BREVITY, :WHERE() FOR OVERRIDABLE DEFAULTS CSS

```
/* Without :is() - six selectors */
article h1 a, article h2 a, article h3 a,
section h1 a, section h2 a, section h3 a { color: inherit; }

/* With :is() - same meaning */
:is(article, section) :is(h1, h2, h3) a { color: inherit; }

/* Zero-specificity defaults */
:where(ul, ol)[role="list"] { list-style: none; padding: 0; }
```

COMPUTED IN CHROMIUM

```
p: rgb(255, 0, 0)  (:is(#main, .content) p = (1,0,1) beats .content p.note (0,2,1))
span: rgb(0, 0, 255)  (:where(...) span = (0,0,1) ties with span; the later rule wins)
```

9.4.3 :has(), the "parent selector"

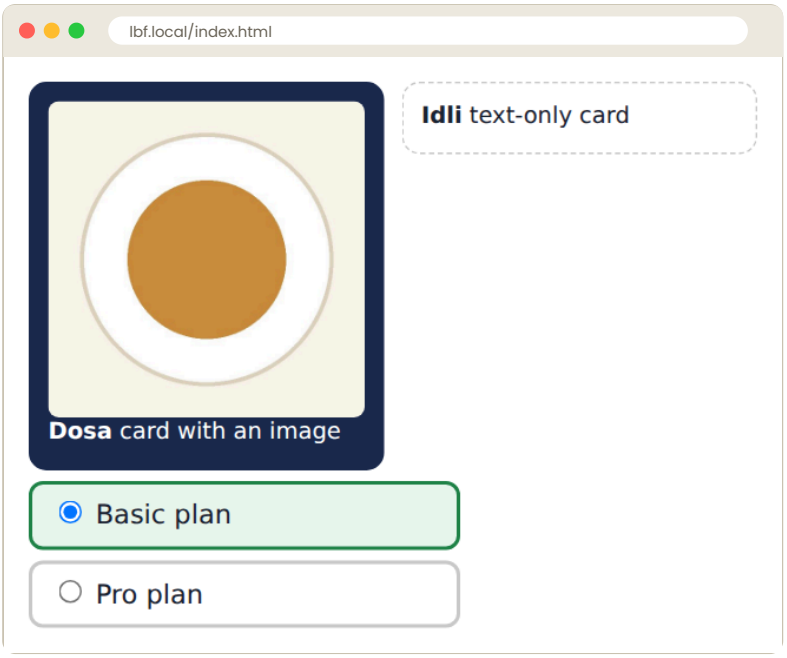
`:has(relative selector)` (Baseline December 2023) matches an element if **something relative to it** exists: a descendant, a child, or a following sibling. For the first time CSS can select a **parent** or a **previous sibling** based on what comes after. Examples: `.card:has(img)` (cards containing an image), `form:has(:invalid)` (forms with errors), `li:has(+ li.active)` (the item before the active one), `body:has(dialog[open])` (lock page scroll while a modal is open).

:HAS() STYLES PARENTS BY THEIR CONTENTS CSS

```css
.cards { display: flex; gap: 10px; align-items: start; }
.card { width: 180px; border: 1px solid #ccc; border-radius: 10px; padding: 10px; font-size: 13px; }
.card img { width: 100%; border-radius: 6px; display: block; }
.card:has(img) { background: #1a2a4f; color: #fff; border-color: #1a2a4f; }   /* cards WITH an image
*/
.card:not(:has(img)) { border-style: dashed; }                                /* cards WITHOUT one */
form:has(input:user-invalid) .submit { opacity: .4; }
label:has(input:checked) { background: #eaf5ee; border-color: #1e8449; }
.opt { display: block; border: 2px solid #ccc; border-radius: 8px; padding: 6px 10px; margin: 6px 0;
width: 220px; }
```

INDEX.HTML HTML

```html
<div class="cards">
  <div class="card"><img src="food-dosa.jpg" alt=""><b>Dosa</b> card with an image</div>
  <div class="card"><b>Idli</b> text-only card</div>
</div>
<label class="opt"><input type="radio" name="plan" checked> Basic plan</label>
<label class="opt"><input type="radio" name="plan"> Pro plan</label>
```



THE CARD CONTAINING AN IMAGE AND THE LABEL CONTAINING THE CHECKED RADIO ARE STYLED WITH NO EXTRA CLASSES

COMPUTED IN CHROMIUM

```
cards matching .card:has(img): #a
```

9.5 Other pseudo-classes

9.5.1 :target, :lang(), :any-link

`:target` matches the element whose id equals the URL fragment (`page.html#faq` targets `id="faq"`): highlight the section a link jumped to, or build CSS-only lightboxes. `:lang(hi)` matches elements in a language (from the nearest `lang` attribute), for language-specific quotes or fonts. `:any-link` matches every `a` or `area` with an href, visited or not.

:TARGET, :LANG() AND :ANY-LINK CSS

```css
section { padding: 6px 10px; border-left: 4px solid transparent; margin: 4px 0; }
section:target { border-color: #c9a227; background: #fff8e1; }
:lang(fr) { font-style: italic; quotes: "« " " »"; }
q { quotes: "“" "”"; }
:any-link { color: #1a2a4f; }
```

INDEX.HTML HTML

```html
<nav><a href="#fees">Jump to Fees</a></nav><section id="intro">Introduction</section><section
id="fees">Fees: ₹11,999 (highlighted by :target)</section><p>She said <q>bonjour</q> and <q
lang="fr">merci</q>.</p>
```



AFTER CLICKING "JUMP TO FEES", THE URL ENDS IN #FEES AND THAT SECTION IS HIGHLIGHTED

9.6 Pseudo-elements

9.6.1 ::before and ::after with content

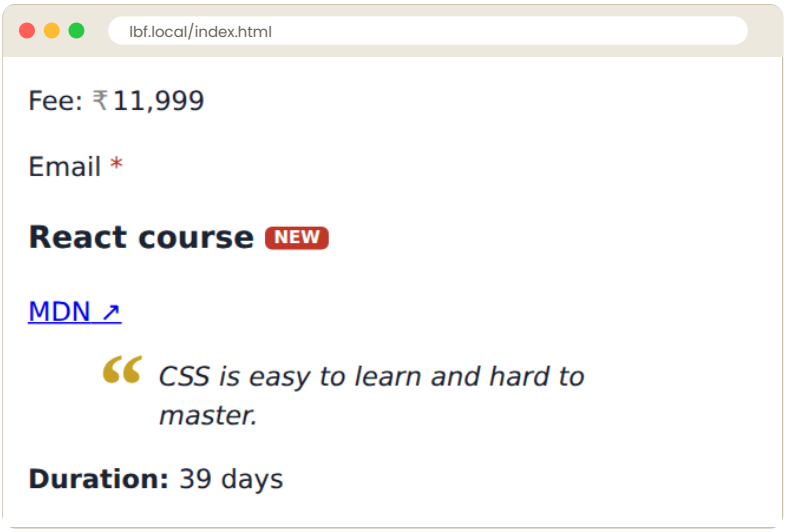
`::before` and `::after` create a child box as the **first** or **last** child of the element, but only if the `content` property is set (to a string, an empty string `""`, an image `url()`, `attr(name)` or a counter). They are inline by default and can be styled and positioned like any box. They do not work on void elements such as `img` and `input`. Use them for **decoration**: text in `content` is read by most screen readers but cannot be selected or copied, so meaningful text belongs in the HTML.

::BEFORE AND ::AFTER CSS

```css
.price::before { content: "₹"; color: #888; margin-right: 2px; }
.required::after { content: " *"; color: #c0392b; }
.new::after { content: "NEW"; font-size: 10px; background: #c0392b; color: #fff; padding: 1px 5px;
border-radius: 4px; margin-left: 6px; vertical-align: middle; }
a[href^="http"]::after { content: " ↗"; }
blockquote { position: relative; padding-left: 34px; font-style: italic; }
blockquote::before { content: "“"; position: absolute; left: 0; top: -14px; font: 700 52px Georgia,
serif; color: #c9a227; }
.dl::before { content: attr(data-label) ": "; font-weight: 700; }
```

INDEX.HTML HTML

```html
<p>Fee: <span class="price">11,999</span></p><p class="required">Email</p><h3 class="new">React
course</h3><p><a href="https://developer.mozilla.org">MDN</a></p><blockquote>CSS is easy to learn and
hard to master.</blockquote><p class="dl" data-label="Duration">39 days</p>
```



HOW THE BROWSER DISPLAYS IT

9.6.2 ::first-letter and ::first-line

`::first-letter` styles the first letter of a block (drop caps); `::first-line` styles the first rendered line, whose length depends on the width. Only font, colour, background, spacing and similar properties apply.

A DROP CAP AND A STYLED FIRST LINE CSS

```css
p { max-width: 360px; line-height: 1.6; }
p::first-letter { float: left; font: 700 3.4em/0.9 Georgia, serif; color: #1a2a4f; margin: 4px 8px 0 0; }
p::first-line { font-variant: small-caps; letter-spacing: .04em; color: #9a7a14; }
```

INDEX.HTML HTML

```html
<p>Once upon a time in Mysuru, a young developer opened a text editor and wrote her first stylesheet. The page came alive with colour and typography, and she never looked back.</p>
```



HOW THE BROWSER DISPLAYS IT

9.6.3 ::selection

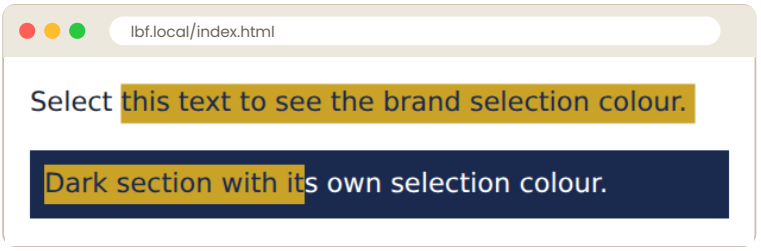
`::selection` styles the part of the document the user has highlighted (with the mouse or Shift+arrows). Only colour, background-color, text-shadow and text decoration apply. Keep good contrast.

::SELECTION CSS

```css
::selection { background: #c9a227; color: #1a2a4f; }
.dark ::selection { background: #fff; color: #1a2a4f; }
.dark { background: #1a2a4f; color: #fff; padding: 8px; }
```

INDEX.HTML HTML

```
<p id="sel">Select this text to see the brand selection colour.</p><p class="dark" id="sel2">Dark
section with its own selection colour.</p>
```



BOTH PARAGRAPHS WERE SELECTED WITH SCRIPT TO SHOW THE COLOURS

9.6.4 ::placeholder

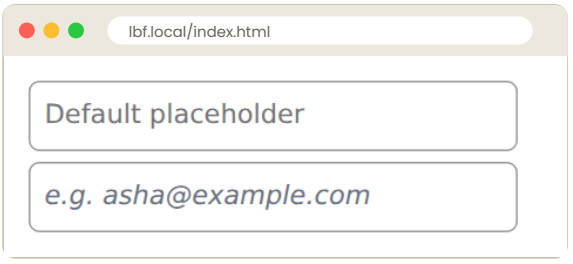
`::placeholder` styles the placeholder text of inputs and textareas. Browsers render placeholders in a light grey that often fails contrast; if you use placeholders for examples, make them dark enough (but distinguishable from typed text), and never use them instead of labels.

::PLACEHOLDER CSS

```
input { padding: 8px; font: inherit; border: 1px solid #999; border-radius: 6px; width: 260px;
display: block; margin: 6px 0; }
.styled::placeholder { color: #6b7280; font-style: italic; opacity: 1; }
```

INDEX.HTML HTML

```
<input placeholder="Default placeholder"><input class="styled" placeholder="e.g. asha@example.com">
```



HOW THE BROWSER DISPLAYS IT

9.6.5 ::marker

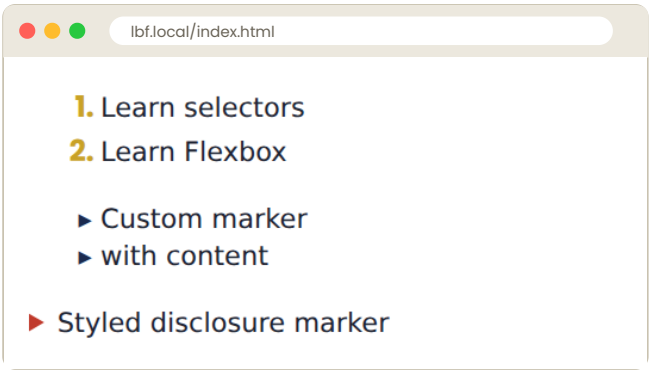
`::marker` is the bullet or number box of a list item (any element with `display: list-item`, including `summary`). It accepts colour, font properties and `content` (Baseline 2020; Safari does not support content yet).

::MARKER CSS

```
ol li::marker { color: #c9a227; font: 700 1.1em "Poppins", sans-serif; }
ul li::marker { content: "▸ "; color: #1a2a4f; }
summary::marker { color: #c0392b; }
```

INDEX.HTML HTML

```
<ol><li>Learn selectors</li><li>Learn Flexbox</li></ol><ul><li>Custom marker</li><li>with content</li>
</ul><details><summary>Styled disclosure marker</summary>Hidden text</details>
```



HOW THE BROWSER DISPLAYS IT

9.6.6 ::backdrop

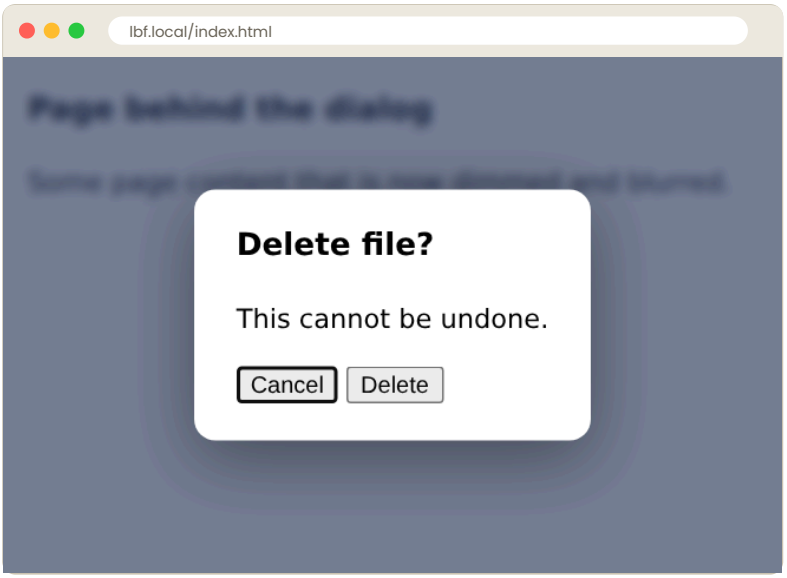
`::backdrop` is a box rendered directly beneath an element in the **top layer**: a modal `<dialog>` opened with showModal(), an open popover, or an element in full screen. It covers the viewport, so it is used to dim or blur the page behind.

::BACKDROP UNDER A MODAL DIALOG CSS

```
dialog { border: 0; border-radius: 12px; padding: 20px 24px; box-shadow: 0 10px 40px rgb(0 0 0 / .4);
}
dialog::backdrop { background: rgb(26 42 79 / .6); backdrop-filter: blur(3px); }
body { min-height: 260px; }
```

INDEX.HTML HTML

```
<h3>Page behind the dialog</h3><p>Some page content that is now dimmed and blurred.</p><dialog id="d">
<h3 style="margin-top:0">Delete file?</h3><p>This cannot be undone.</p><button>Cancel</button>
<button>Delete</button></dialog>
```



HOW THE BROWSER DISPLAYS IT

9.6.7 ::file-selector-button

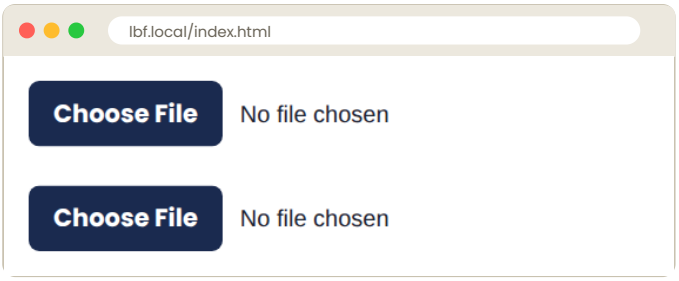
`::file-selector-button` styles the "Choose file" button of `<input type="file">`, which was impossible to style before (Baseline 2021).

::FILE-SELECTOR-BUTTON CSS

```css
input[type=file]::file-selector-button { font: 600 14px "Poppins", sans-serif; background: #1a2a4f;
color: #fff;
  border: 0; border-radius: 6px; padding: 8px 14px; margin-right: 10px; cursor: pointer; }
input[type=file]::file-selector-button:hover { background: #c9a227; color: #1a2a4f; }
```

INDEX.HTML HTML

```html
<input type="file"><br><br><input type="file" id="h">
```



THE SECOND BUTTON IS HOVERED

9.7 Practical uses

9.7.1 Decorative icons and underlines with pseudo-elements

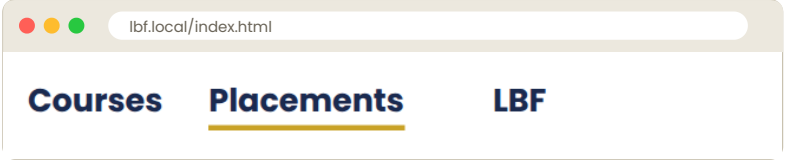
Pseudo-elements avoid extra HTML for decoration. A common effect is an **animated underline** that grows from the left on hover: an `::after` bar with `transform: scaleX(0)` that transitions to `scaleX(1)`.

ANIMATED UNDERLINE AND AN ICON CSS

```css
.link { position: relative; color: #1a2a4f; text-decoration: none; font: 600 18px "Poppins", sans-
serif; margin-right: 26px; }
.link::after { content: ""; position: absolute; left: 0; bottom: -4px; width: 100%; height: 3px;
background: #c9a227;
              transform: scaleX(0); transform-origin: left; transition: transform .3s ease; }
.link:hover::after, .link:focus-visible::after { transform: scaleX(1); }
.icon::before { content: ""; display: inline-block; width: 1em; height: 1em; margin-right: 6px;
vertical-align: -0.15em;
               background: currentColor; mask: url("logo.svg") center / contain no-repeat; }
```

INDEX.HTML HTML

```html
<a class="link" href="#">Courses</a><a class="link" href="#" id="h">Placements</a><a class="link icon"
href="#">LBF</a>
```



"PLACEMENTS" IS HOVERED: ITS UNDERLINE HAS GROWN ACROSS

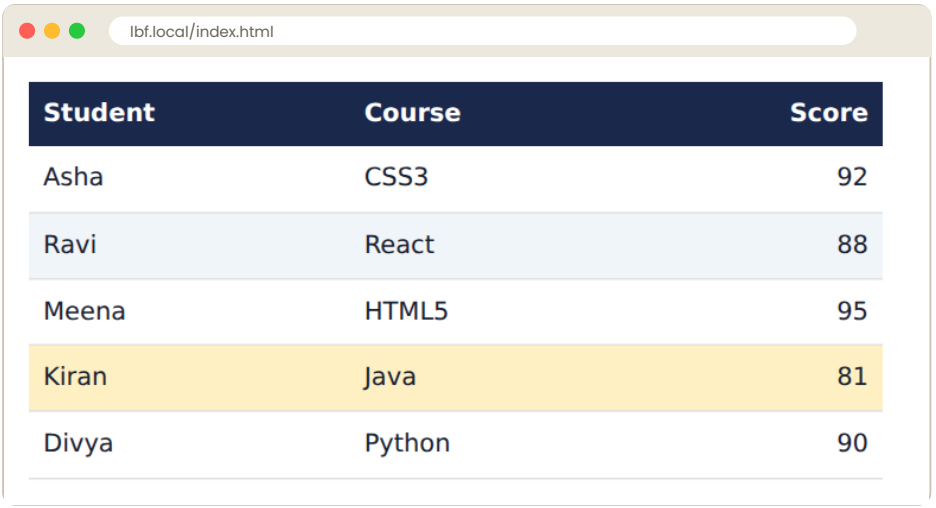
9.7.2 Zebra-striped tables

ZEBRA STRIPES AND A HOVER ROW CSS

```css
table { border-collapse: collapse; width: 100%; max-width: 480px; font-size: 14px; }
th { background: #1a2a4f; color: #fff; text-align: left; padding: 8px; }
td { padding: 8px; border-bottom: 1px solid #e5e5e5; }
tbody tr:nth-child(even) { background: #f4f6fb; }
tbody tr:hover { background: #fff3c4; }
td:last-child, th:last-child { text-align: right; }
```

INDEX.HTML HTML

```html
<table><thead><tr><th>Student</th><th>Course</th><th>Score</th></tr></thead><tbody><tr><td>Asha</td>
<td>CSS3</td><td>92</td></tr><tr><td>Ravi</td><td>React</td><td>88</td></tr><tr><td>Meena</td>
<td>HTML5</td><td>95</td></tr><tr id="h"><td>Kiran</td><td>Java</td><td>81</td></tr><tr><td>Divya</td>
<td>Python</td><td>90</td></tr></tbody></table>
```



EVEN ROWS ARE TINTED; THE HOVERED ROW IS GOLD

9.7.3 CSS-only tooltips

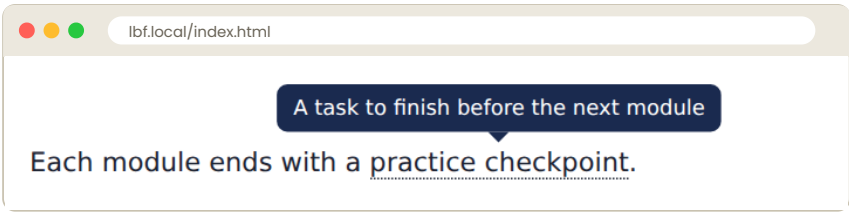
A simple tooltip can be built with `::after` reading its text from a data attribute, shown on hover **and focus**. For accessibility, the information must also be available to screen readers (an `aria-label` or visually hidden text); for complex, positioned tooltips use the Popover API and anchor positioning (Module 17).

A TOOLTIP FROM DATA-TIP AND ::AFTER CSS

```css
[data-tip] { position: relative; border-bottom: 1px dotted; cursor: help; }
[data-tip]::after { content: attr(data-tip); position: absolute; left: 50%; bottom: calc(100% + 8px);
transform: translateX(-50%);
  background: #1a2a4f; color: #fff; font-size: 12px; white-space: nowrap; padding: 5px 9px; border-
radius: 6px;
  opacity: 0; pointer-events: none; transition: opacity .15s; }
[data-tip]::before { content: ""; position: absolute; left: 50%; bottom: calc(100% + 2px); transform:
translateX(-50%);
  border: 6px solid transparent; border-top-color: #1a2a4f; border-bottom: 0; opacity: 0; transition:
opacity .15s; }
[data-tip]:hover::after, [data-tip]:hover::before, [data-tip]:focus::after, [data-tip]:focus::before {
opacity: 1; }
p { margin-top: 50px; }
```

INDEX.HTML HTML

```
<p>Each module ends with a <span tabindex="0" data-tip="A task to finish before the next module" aria-label="practice checkpoint: a task to finish before the next module" id="t">practice checkpoint</span>.</p>
```



HOVERED: THE TOOLTIP AND ITS ARROW FADE IN

PRACTICE CHECKPOINT

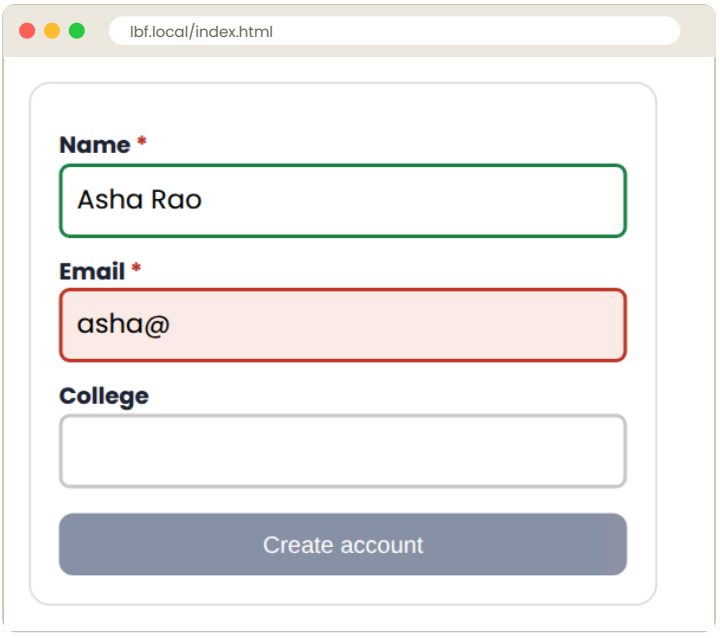
A styled **sign-up form** with valid/invalid states, a **zebra-striped table**, **animated link underlines** with ::after, and a **card that changes when it contains an image** using :has().

SIGNUP.CSS: SAMPLE SOLUTION (FORM PART) CSS

```
body { font-family: "Poppins", sans-serif; }
.signup { max-width: 320px; padding: 16px; border: 1px solid #ddd; border-radius: 12px; }
.signup:focus-within { border-color: #1a2a4f; }
.signup label { display: block; font-size: 13px; font-weight: 600; margin-top: 10px; }
.signup label:has(+ input:required)::after { content: " *"; color: #c0392b; }
.signup input { width: 100%; box-sizing: border-box; padding: 8px; border: 2px solid #ccc; border-radius: 6px; font: inherit; }
.signup input:focus-visible { outline: 3px solid #c9a227; outline-offset: 1px; }
.signup input:user-invalid { border-color: #c0392b; background: #fdecea; }
.signup input:user-valid { border-color: #1e8449; }
.signup button { margin-top: 14px; width: 100%; padding: 10px; border: 0; border-radius: 8px; background: #1a2a4f; color: #fff; font: 600 15px inherit; }
.signup:has(input:user-invalid) button { opacity: .5; }
```

INDEX.HTML HTML

```
<form class="signup">
  <label for="n">Name</label><input id="n" required value="Asha Rao">
  <label for="e">Email</label><input id="e" type="email" required value="asha@">
  <label for="c">College</label><input id="c">
  <button>Create account</button>
</form>
```



AFTER EDITING BOTH FIELDS: NAME VALID (GREEN), EMAIL INVALID (RED), BUTTON DIMMED BY :HAS()

The table, underline and :has() card parts reuse the recipes from 9.7.2, 9.7.1 and 9.4.3 directly; combine them on one practice page and check every state with the DevTools **:hov** panel.

MODULE 10 · INTERMEDIATE

Flexbox

Minimum duration **3 days** | Level **Intermediate**

Flexbox is a one-dimensional layout system for arranging items in a row or a column, and it is used in almost every modern interface. This module covers flex containers and items, the main and cross axes, every container property (display: flex and inline-flex, flex-direction, flex-wrap and flex-flow, justify-content, align-items, align-content, gap) and every item property (flex-grow, flex-shrink, flex-basis, the flex shorthand, align-self, order), with the exact arithmetic of growing and shrinking verified in the browser. It then builds the common patterns: navigation bars, perfect centring, equal-height cards, the sticky footer, the media object and pushing items apart with auto margins, and points to Flexbox Froggy for practice.

IN THIS MODULE

10.1 Flexbox basics
10.2 Container properties
10.3 Item properties
10.4 Common flexbox patterns
10.5 Practice with Flexbox Froggy

10.1 Flexbox basics

10.1.1 Flex container and flex items

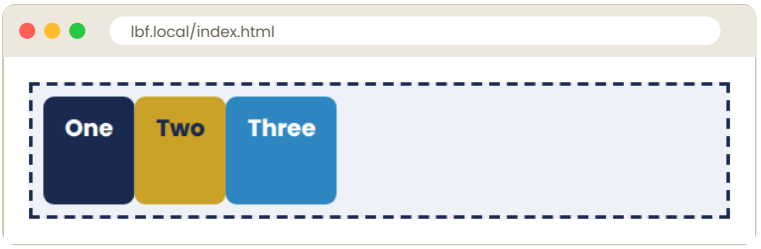
Setting `display: flex` on an element makes it a **flex container** and its **direct children** become **flex items** (grandchildren are not affected). Items line up in a row by default, shrink if needed to fit, stretch to the container's height, and can be aligned, distributed, reordered and resized with a handful of properties. Margins of flex items never collapse, `float` has no effect on them, and even text directly inside the container becomes an anonymous item.

THE SMALLEST FLEX LAYOUT — CSS

```
.c { display: flex; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; min-height: 60px; }
.c > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 8px 12px;
border-radius: 6px; }
.c > div:nth-child(2) { background: #c9a227; color: #1a2a4f; } .c > div:nth-child(3) { background:
#2e86c1; }
.c > div:nth-child(4) { background: #1e8449; } .c > div:nth-child(5) { background: #c0392b; }
```

INDEX.HTML — HTML

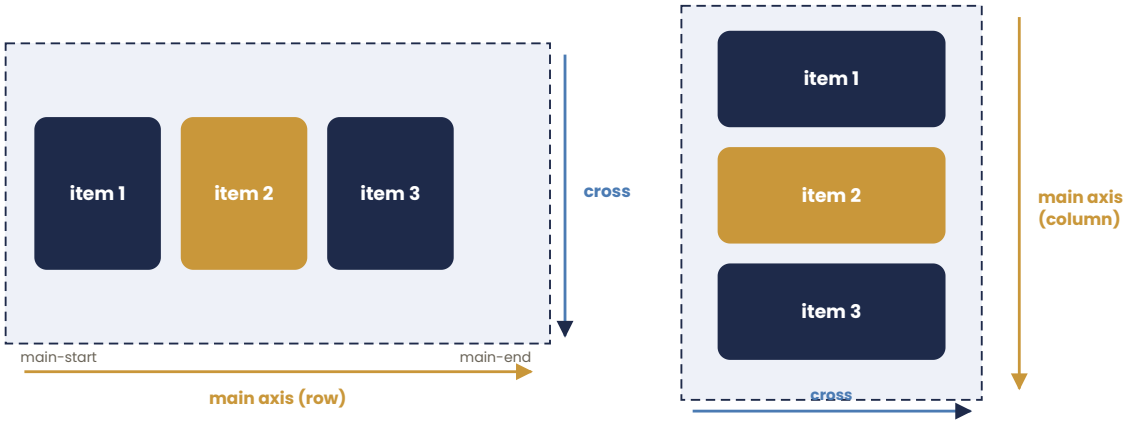
```
<div class="c"><div>One</div><div>Two</div><div>Three</div></div>
```



THREE ITEMS IN A ROW: NO WIDTHS, FLOATS OR INLINE-BLOCK HACKS

10.1.2 Main axis and cross axis

Flexbox lays items out along the **main axis**, set by `flex-direction` (horizontal for `row`, vertical for `column`). The **cross axis** is perpendicular to it. Every flexbox property refers to one of these axes, not to "horizontal" or "vertical": `justify-content` works on the main axis and `align-items` on the cross axis, so they swap visually when the direction becomes column.



MAIN AND CROSS AXES FOR FLEX-DIRECTION: ROW (LEFT) AND COLUMN (RIGHT)

Property	Works along	In a row	In a column
`justify-content`	main axis	horizontal	vertical
`align-items`, `align-self`	cross axis	vertical	horizontal
`align-content`	cross axis (between lines)	vertical	horizontal
`flex-basis`, grow, shrink	main axis	width	height

10.2 Container properties

10.2.1 display: flex and inline-flex

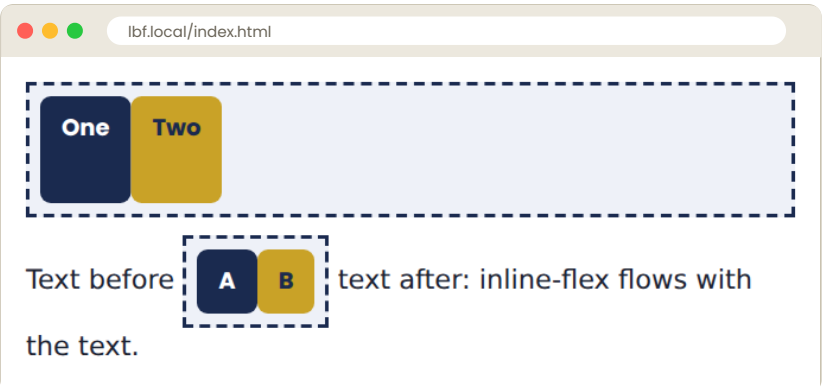
`display: flex` makes a **block-level** container (it starts on a new line and fills the available width). `display: inline-flex` makes an **inline-level** container that sits within text and is only as wide as its items, useful for buttons containing an icon and a label, or tags.

FLEX VS INLINE-FLEX CSS

```css
.c { display: flex; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; min-height: 60px; }
.c > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 8px 12px;
border-radius: 6px; }
.c > div:nth-child(2) { background: #c9a227; color: #1a2a4f; } .c > div:nth-child(3) { background:
#2e86c1; }
.c > div:nth-child(4) { background: #1e8449; } .c > div:nth-child(5) { background: #c0392b; }
.c.inline { display: inline-flex; vertical-align: middle; min-height: 0; }
```

INDEX.HTML HTML

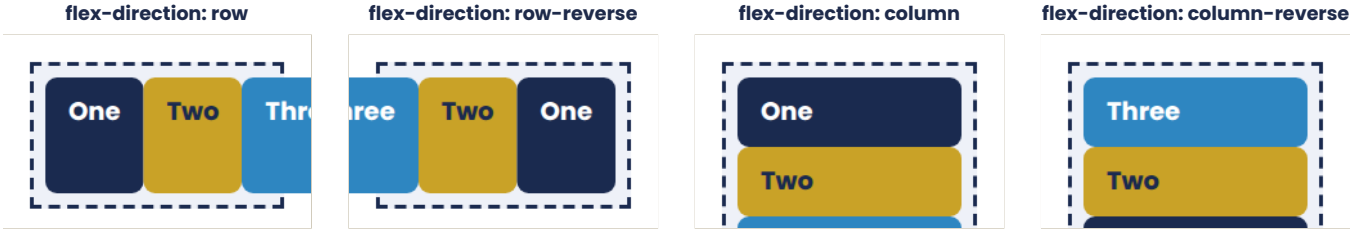
```html
<div class="c"><div>One</div><div>Two</div></div><div style="margin-top:10px">Text before <div
class="c inline"><div>A</div><div>B</div></div> text after: inline-flex flows with the text.</div>
```



HOW THE BROWSER DISPLAYS IT

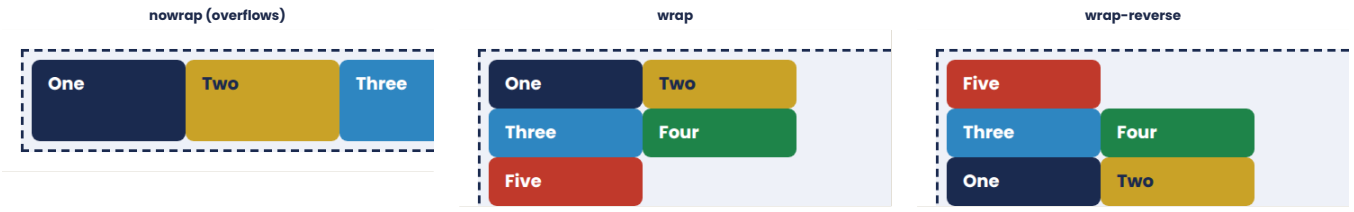
10.2.2 flex-direction

`flex-direction` sets the main axis and its direction: `row` (default; follows the writing direction, so right-to-left in Arabic), `row-reverse`, `column`, `column-reverse`. The reverse values change the **visual** order only: screen readers and the Tab key still follow the HTML order, so use them for visual adjustments, not to reorder meaningful content.



10.2.3 flex-wrap and flex-flow

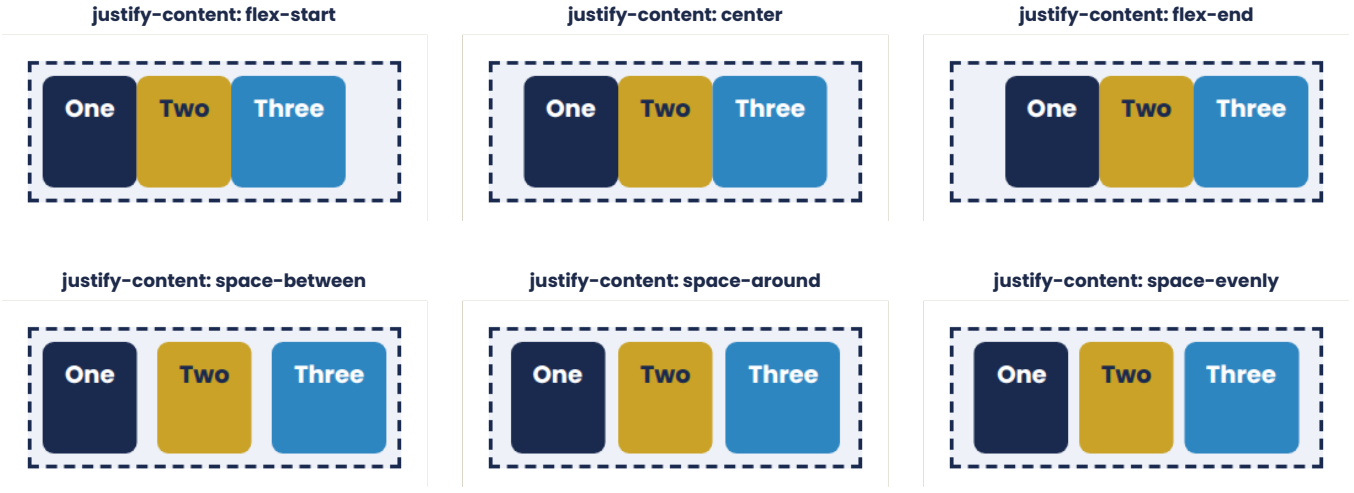
By default (`flex-wrap: nowrap`), all items stay on one line and **shrink** to fit, overflowing if they cannot shrink further. `flex-wrap: wrap` lets items move onto new lines (flex lines) when they run out of space; `wrap-reverse` stacks new lines above. `flex-flow` is the shorthand for direction and wrap: `flex-flow: row wrap`.

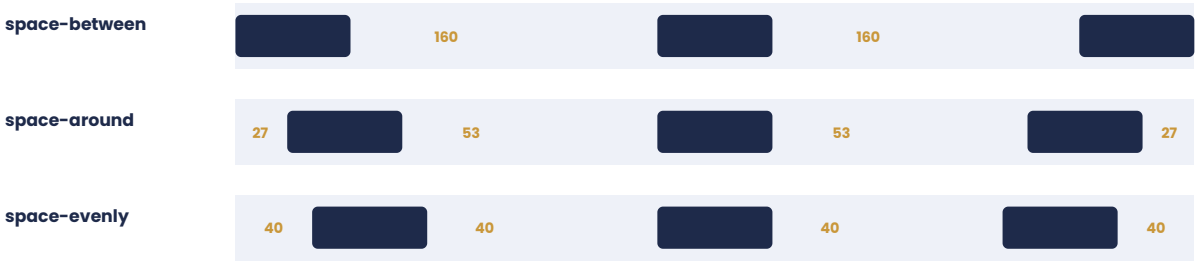


FIVE 90PX ITEMS IN A 300PX CONTAINER

10.2.4 justify-content

`justify-content` distributes **free space on the main axis** (it has an effect only when the items do not fill the line, that is, when nothing grows):



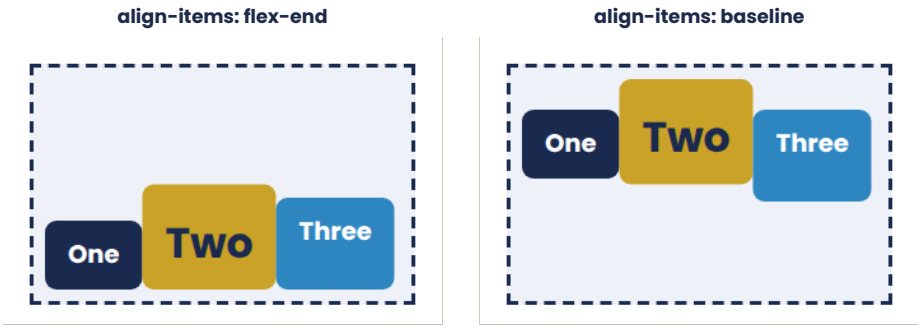


HOW THE 320PX OF FREE SPACE IS SHARED BETWEEN THREE 60PX ITEMS IN A 500PX LINE (GAPS IN PX)

10.2.5 align-items

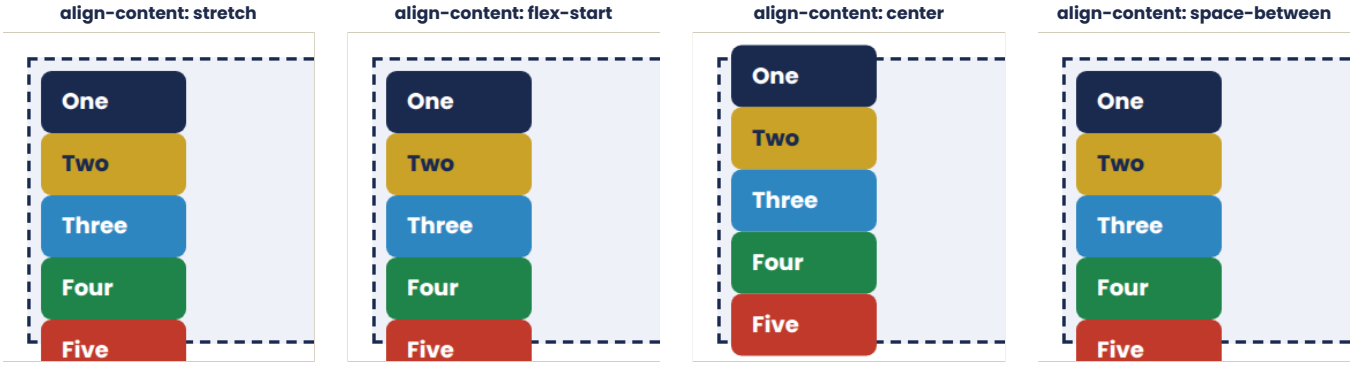
`align-items` aligns items on the **cross axis** within their line. The default, `stretch`, makes items as tall as the line (in a row), which is why flex columns have equal heights. Other values: `flex-start`, `center`, `flex-end`, `baseline` (align the first lines of text, useful when items have different font sizes).





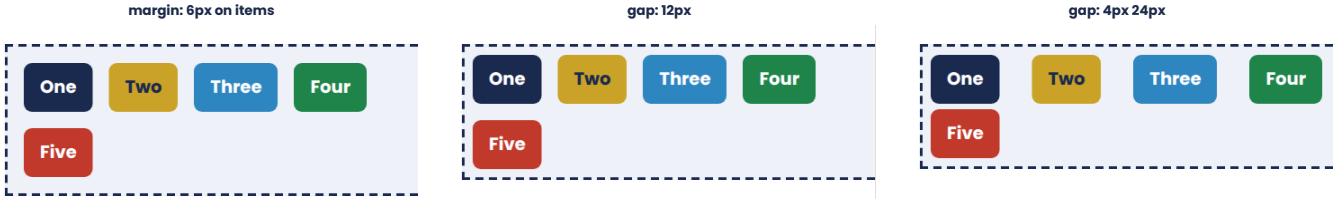
10.2.6 align-content

`align-content` distributes **flex lines** on the cross axis, so it only matters when items **wrap** onto several lines and the container is taller than they need. It accepts the same values as justify-content plus `stretch` (default).



10.2.7 gap, row-gap, column-gap

`gap` sets the space **between** items (not around the outside), replacing margins and the negative-margin tricks of the past. `gap: 20px` sets both; `gap: 10px 20px` sets row-gap then column-gap. Unlike margins, gaps never appear at the start or end of a line.



GAP ONLY PUTS SPACE BETWEEN ITEMS

10.3 Item properties

10.3.1 flex-grow

When the line has **free space**, `flex-grow` (a number, default `0`) decides how it is shared. Each item gets *free space × (its grow ÷ total grow)* added to its base size. With `flex-grow: 1` on every item, the free space is shared equally (but items with longer content stay wider, because growth is added to the content-based size).



SHARING FREE SPACE

The arithmetic, checked in the browser: in a 600px container with three items whose basis is 100px each, there are 300px of free space. With grow 1, 2, 1 the shares are 300 × 1/4 = 75, 300 × 2/4 = 150 and 75:

COMPUTED IN CHROMIUM

```
item 1 (grow 1): 175px
item 2 (grow 2): 250px
item 3 (grow 1): 175px
```

10.3.2 flex-shrink

When items are **too big** for the line (and wrapping is off), `flex-shrink` (default `1`) decides how much each one gives up. The overflow is removed in proportion to *shrink × base size*, so bigger items shrink more. `flex-shrink: 0` means "never shrink below my basis", essential for icons, avatars and fixed sidebars. Items never shrink below their **min-content** size (the longest word) unless you set `min-width: 0`, a common cause of overflowing flex items with long text.

COMPUTED IN CHROMIUM

```
three 200px items in 400px: 200px of overflow
item 1 (shrink 0): 200.00px
item 2 (shrink 1): 133.33px
item 3 (shrink 2): 66.67px
```

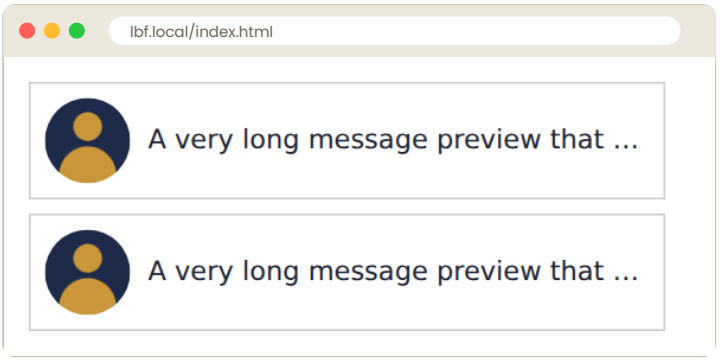
Item 1 (shrink 0) keeps 200px. Items 2 and 3 share the 200px overflow in the ratio 1×200 : 2×200, so item 2 loses 66.67px and item 3 loses 133.33px.

FLEX-SHRINK: 0 AND MIN-WIDTH: 0 — CSS

```css
.row { display: flex; gap: 10px; align-items: center; width: 340px; border: 1px solid #ccc; padding: 8px; margin-bottom: 8px; }
.row img { width: 48px; height: 48px; border-radius: 50%; }
.text { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.fixed img { flex-shrink: 0; }
.fixed .text { min-width: 0; }
```

INDEX.HTML — HTML

```html
<div class="row"><img src="avatar.png" alt=""><span class="text">A very long message preview that does not fit in the row at all</span></div>
<div class="row fixed"><img src="avatar.png" alt=""><span class="text">A very long message preview that does not fit in the row at all</span></div>
```



TOP: THE AVATAR IS SQUASHED BY DEFAULT AND NOTHING TRUNCATES; BOTTOM: THE AVATAR KEEPS ITS SIZE AND THE TEXT GETS AN ELLIPSIS

10.3.3 flex-basis

`flex-basis` is the item's starting size on the main axis **before** growing or shrinking: a length (`200px`, `30%`), `auto` (default: use width/height, or else the content size) or `0` (ignore the content, so growth alone decides the size). In a row it plays the role of width, in a column the role of height, and it takes priority over width.



BASIS AUTO: FREE SPACE ADDED TO CONTENT WIDTHS (UNEQUAL); BASIS 0: ALL SPACE SHARED EQUALLY (EQUAL WIDTHS)

10.3.4 flex shorthand (flex: 1, flex: 0 0 200px)

`flex` sets grow, shrink and basis together, and its single-value forms are what you will use most. Always prefer the shorthand: it sets sensible defaults for the parts you omit.

Shorthand	Expands to	Meaning
`flex: initial` (default)	`0 1 auto`	size from content/width, can shrink, does not grow
`flex: 1`	`1 1 0%`	take an equal share of all space: equal columns
`flex: auto`	`1 1 auto`	grow and shrink from the content size
`flex: none`	`0 0 auto`	rigid: exactly the content/width size
`flex: 2`	`2 1 0%`	twice the share of a flex: 1 sibling
`flex: 0 0 200px`	grow 0, shrink 0, basis 200px	a fixed 200px column (sidebar)
`flex: 1 1 250px`		"at least about 250px, then share": responsive cards with wrap

COMPUTED IN CHROMIUM

```
.a flex: 1          -> grow 1, shrink 1, basis 0%
.b flex: auto       -> grow 1, shrink 1, basis auto
.n flex: none       -> grow 0, shrink 0, basis auto
.f flex: 0 0 200px  -> grow 0, shrink 0, basis 200px
```

A THREE-COLUMN LAYOUT WITH FLEX CSS

```css
.layout { display: flex; gap: 10px; min-height: 150px; font: 600 13px "Poppins", sans-serif; }
.layout > * { padding: 10px; border-radius: 8px; color: #fff; }
.side { flex: 0 0 140px; background: #1a2a4f; }       /* fixed sidebar        */
.main { flex: 1; background: #2e86c1; }               /* takes the rest       */
.aside { flex: 0 1 120px; background: #c9a227; color: #1a2a4f; }  /* can shrink */
```

INDEX.HTML HTML

```html
<div class="layout"><nav class="side">Sidebar 140px</nav><main class="main">Main: flex: 1</main><aside class="aside">Aside up to 120px</aside></div>
```



HOW THE BROWSER DISPLAYS IT

10.3.5 align-self

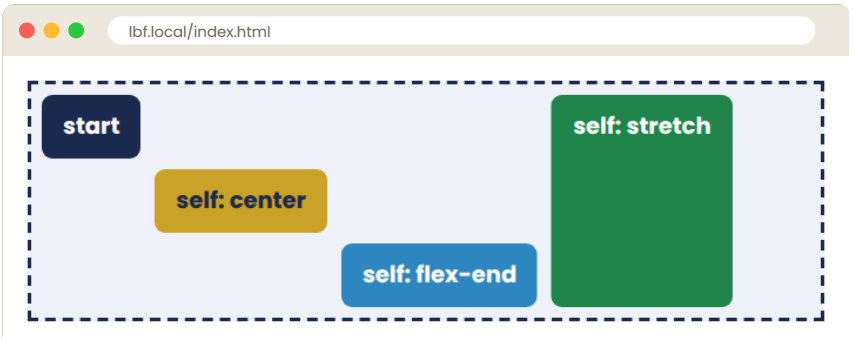
`align-self` overrides the container's `align-items` for one item: `auto` (default, inherit the container's), `flex-start`, `center`, `flex-end`, `stretch`, `baseline`.

ALIGN-SELF ON INDIVIDUAL ITEMS CSS

```css
.c { display: flex; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; min-height: 60px; }
.c > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 8px 12px; border-radius: 6px; }
.c > div:nth-child(2) { background: #c9a227; color: #1a2a4f; } .c > div:nth-child(3) { background: #2e86c1; }
.c > div:nth-child(4) { background: #1e8449; } .c > div:nth-child(5) { background: #c0392b; }
.c { height: 120px; align-items: flex-start; gap: 8px; }
.c > div:nth-child(2) { align-self: center; }
.c > div:nth-child(3) { align-self: flex-end; }
.c > div:nth-child(4) { align-self: stretch; }
```

INDEX.HTML HTML

```html
<div class="c"><div>start</div><div>self: center</div><div>self: flex-end</div><div>self: stretch</div></div>
```



HOW THE BROWSER DISPLAYS IT

10.3.6 order

`order` (an integer, default 0) changes the **visual** position of an item: items are laid out in ascending order, with HTML order breaking ties. Negative values move an item to the front. As with row-reverse, keyboard focus and screen readers still follow the DOM, so only use order for visual rearrangement that does not change meaning (for example, moving an image above the text on phones).

ORDER — CSS

```
.c { display: flex; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; min-height: 60px; }
.c > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 8px 12px;
border-radius: 6px; }
.c > div:nth-child(2) { background: #c9a227; color: #1a2a4f; } .c > div:nth-child(3) { background:
#2e86c1; }
.c > div:nth-child(4) { background: #1e8449; } .c > div:nth-child(5) { background: #c0392b; }
.c { gap: 8px; }
.c > div:nth-child(1) { order: 2; }
.c > div:nth-child(3) { order: -1; }
```

INDEX.HTML — HTML

```
<div class="c"><div>1 (order 2)</div><div>2 (order 0)</div><div>3 (order -1)</div><div>4 (order 0)
</div></div>
```



VISUAL ORDER: 3, 2, 4, 1

10.4 Common flexbox patterns

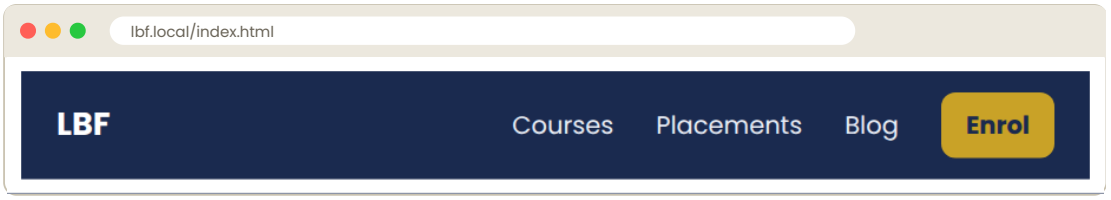
10.4.1 Navigation bars

A NAVBAR: LOGO LEFT, LINKS RIGHT CSS

```css
.nav { display: flex; align-items: center; gap: 24px; padding: 12px 20px; background: #1a2a4f; font-family: "Poppins", sans-serif; }
.nav .logo { color: #fff; font-weight: 700; font-size: 18px; margin-right: auto; }   /* pushes the rest right */
.nav a { color: #e8ecf5; text-decoration: none; font-size: 14px; }
.nav a:hover { color: #c9a227; }
.nav .cta { background: #c9a227; color: #1a2a4f; padding: 8px 14px; border-radius: 8px; font-weight: 700; }
```

INDEX.HTML HTML

```html
<nav class="nav"><span class="logo">LBF</span><a href="#">Courses</a><a href="#">Placements</a><a href="#">Blog</a><a class="cta" href="#">Enrol</a></nav>
```



HOW THE BROWSER DISPLAYS IT

10.4.2 Perfect centring

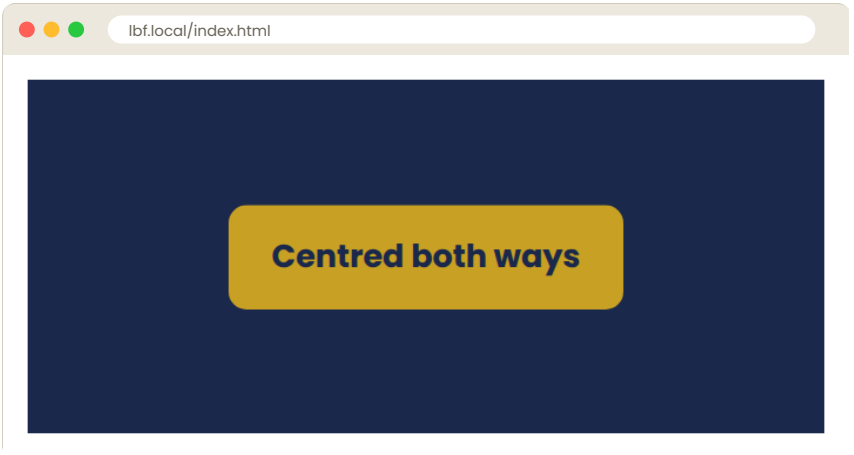
Two lines centre anything, of any size, in both directions:

CENTRING WITH FLEXBOX CSS

```css
.hero { display: flex; justify-content: center; align-items: center; height: 200px; background: #1a2a4f; }
.box { background: #c9a227; padding: 16px 24px; border-radius: 10px; font: 700 18px "Poppins", sans-serif; color: #1a2a4f; }
```

INDEX.HTML HTML

```html
<div class="hero"><div class="box">Centred both ways</div></div>
```



HOW THE BROWSER DISPLAYS IT

10.4.3 Equal-height cards

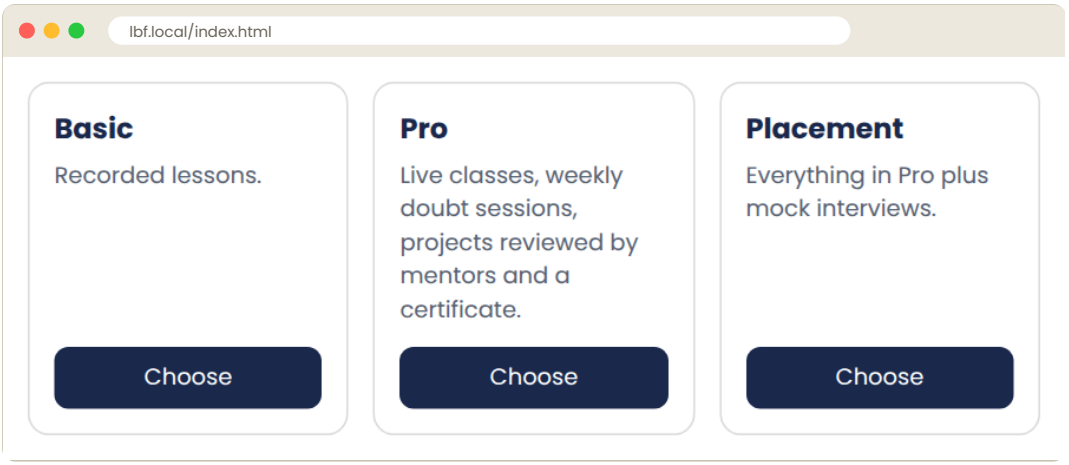
Because flex items stretch to the tallest item in their line, cards in a row automatically have equal heights. Make each card a **column** flex container too, and give the button `margin-top: auto`, so all buttons line up at the bottom regardless of text length.

EQUAL-HEIGHT CARDS WITH ALIGNED BUTTONS CSS

```
.cards { display: flex; gap: 14px; }
.card { flex: 1; display: flex; flex-direction: column; background: #fff; border: 1px solid #ddd;
border-radius: 12px; padding: 14px; font-family: "Poppins", sans-serif; }
.card h3 { margin: 0 0 6px; color: #1a2a4f; font-size: 16px; }
.card p { margin: 0 0 12px; font-size: 13px; color: #5c6475; }
.card a { margin-top: auto; background: #1a2a4f; color: #fff; text-align: center; padding: 8px;
border-radius: 8px; text-decoration: none; font-size: 13px; }
```

INDEX.HTML HTML

```
<div class="cards">
  <article class="card"><h3>Basic</h3><p>Recorded lessons.</p><a href="#">Choose</a></article>
  <article class="card"><h3>Pro</h3><p>Live classes, weekly doubt sessions, projects reviewed by
mentors and a certificate.</p><a href="#">Choose</a></article>
  <article class="card"><h3>Placement</h3><p>Everything in Pro plus mock interviews.</p><a
href="#">Choose</a></article>
</div>
```



HOW THE BROWSER DISPLAYS IT

10.4.4 Sticky footer layout

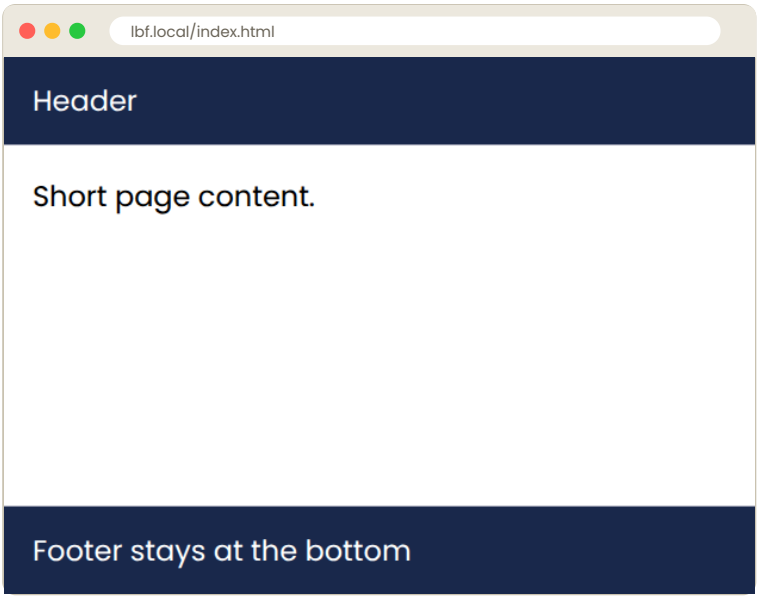
On short pages the footer should sit at the bottom of the window, not in the middle. Make the body a full-height column and let the main content grow:

STICKY FOOTER CSS

```
body { margin: 0; min-height: 100vh; display: flex; flex-direction: column; font-family: "Poppins",
sans-serif; }
header, footer { background: #1a2a4f; color: #fff; padding: 12px 16px; }
main { flex: 1; padding: 16px; }      /* takes all spare height, pushing the footer down */
```

INDEX.HTML HTML

```
<header>Header</header><main>Short page content.</main><footer>Footer stays at the bottom</footer>
```



THE PAGE IS ONLY A FEW LINES LONG, YET THE FOOTER IS AT THE BOTTOM OF THE 300PX-TALL WINDOW

10.4.5 Media object (image beside text)

THE MEDIA OBJECT CSS

```
.media { display: flex; gap: 14px; align-items: flex-start; max-width: 460px; padding: 12px; border:
1px solid #ddd; border-radius: 10px; }
.media img { flex-shrink: 0; width: 64px; height: 64px; border-radius: 50%; object-fit: cover; }
.media h4 { margin: 0 0 4px; color: #1a2a4f; } .media p { margin: 0; font-size: 14px; }
.media.reverse { flex-direction: row-reverse; text-align: right; }
```

INDEX.HTML HTML

```
<div class="media"><img src="mysuru-palace.jpg" alt=""><div><h4>Asha Rao</h4><p>The Flexbox module
finally made layout click for me. The patterns section is gold.</p></div></div><br><div class="media
reverse"><img src="food-coffee.jpg" alt=""><div><h4>Ravi K.</h4><p>Reversed media object with row-
reverse.</p></div></div>
```



HOW THE BROWSER DISPLAYS IT

10.4.6 Pushing items apart with margin-left: auto

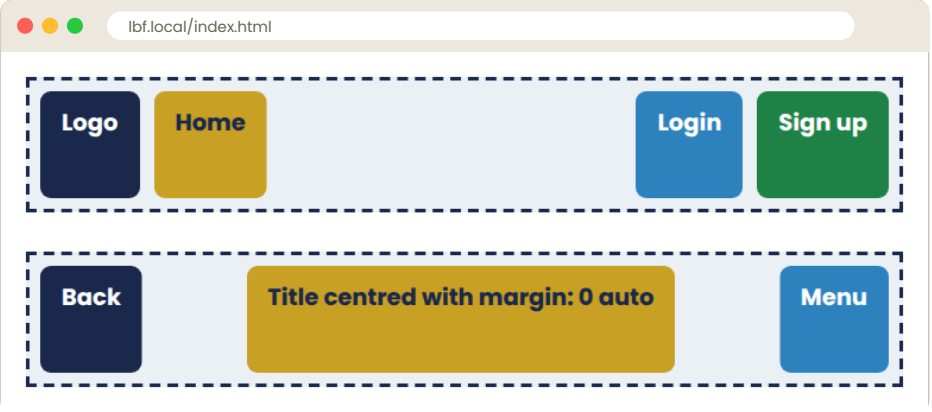
In a flex container, an **auto margin** absorbs all the free space on its side. `margin-left: auto` on an item pushes it, and everything after it, to the right end; `margin: auto` on a single item centres it. It is often clearer than wrapping groups in extra divs with space-between.

AUTO MARGINS IN FLEXBOX CSS

```
.c { display: flex; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; min-height: 60px; }
.c > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 8px 12px;
border-radius: 6px; }
.c > div:nth-child(2) { background: #c9a227; color: #1a2a4f; } .c > div:nth-child(3) { background:
#2e86c1; }
.c > div:nth-child(4) { background: #1e8449; } .c > div:nth-child(5) { background: #c0392b; }
.c { gap: 8px; }
.push { margin-left: auto; }
```

INDEX.HTML HTML

```
<div class="c"><div>Logo</div><div>Home</div><div class="push">Login</div><div>Sign up</div></div><br>
<div class="c"><div>Back</div><div style="margin: 0 auto">Title centred with margin: 0 auto</div>
<div>Menu</div></div>
```



HOW THE BROWSER DISPLAYS IT

10.5 Practice with Flexbox Froggy

Flexbox Froggy (flexboxfroggy.com) is a free game with 24 levels: you write flexbox CSS to move frogs onto lily pads of the right colour. It drills justify-content, align-items, flex-direction, order, align-self, flex-wrap, flex-flow and align-content until they become automatic. Complete all levels, then try **Flexbox Defense** (flexboxdefense.com) and the "Flexbox Zombies" course for more practice.

A FLEXBOX FROGGY ANSWER CSS

```
/* Froggy level 21 (two frogs per row, aligned to the end of the pond) - a typical answer */
#pond {
  display: flex;
  flex-flow: column-reverse wrap-reverse;
  justify-content: center;
  align-content: space-between;
}
```

TIP

Chrome and Firefox DevTools show a **flex** badge next to every flex container in the Elements panel; click it to overlay the container, items, free space and gaps on the page. The Layout tab lists all flex containers.

PRACTICE CHECKPOINT

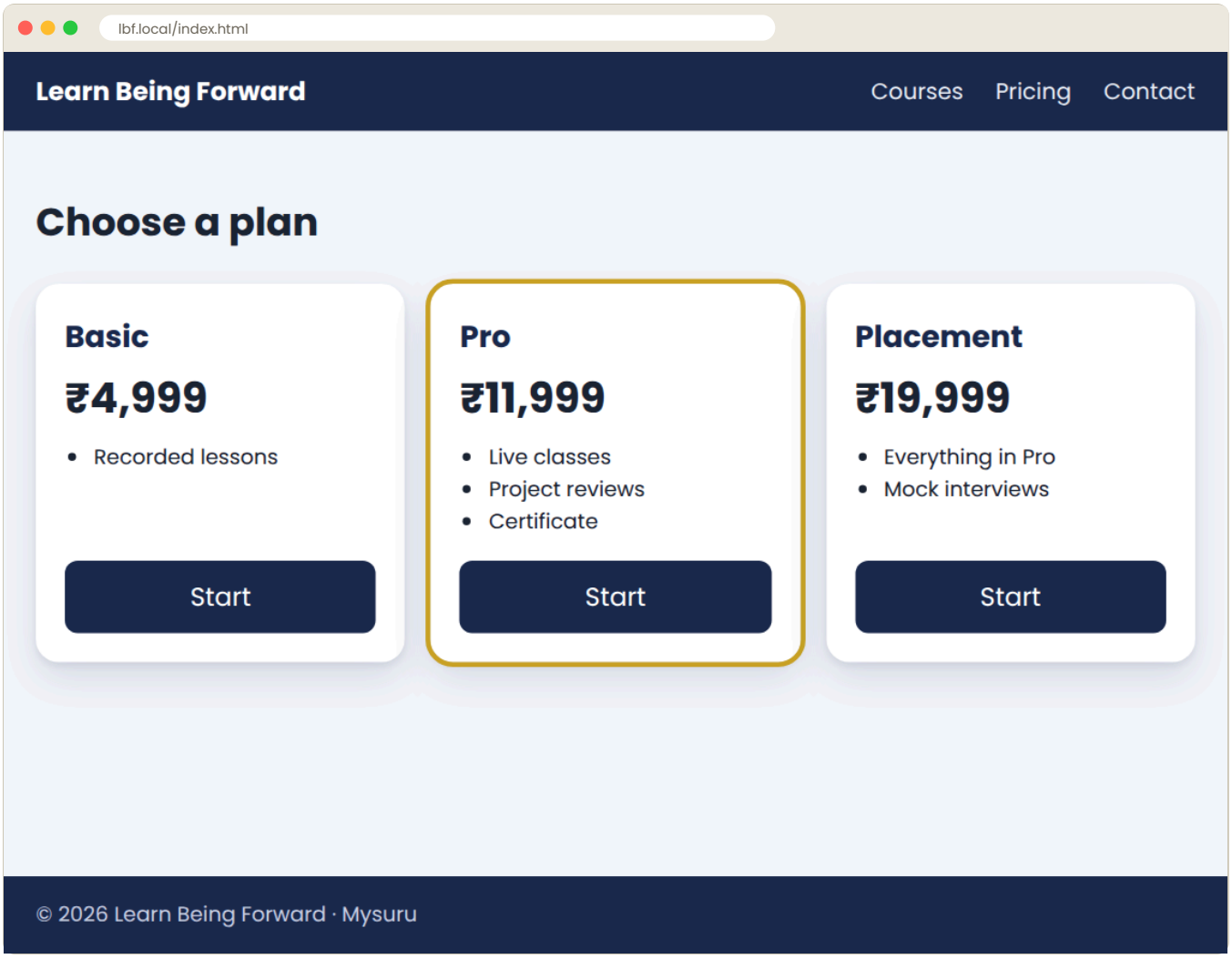
A **responsive navbar**, a row of **equal-height pricing cards that wrap**, and a **full-height page layout with a sticky footer**, all with Flexbox.

FLEX-LAYOUT.CSS: SAMPLE SOLUTION CSS

```
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; min-height: 100vh; display: flex; flex-direction: column; font-family: "Poppins",
sans-serif; background: #f4f6fb; color: #1d2433; }
.nav { display: flex; flex-wrap: wrap; align-items: center; gap: 8px 20px; padding: 12px 20px;
background: #1a2a4f; }
.nav .logo { color: #fff; font-weight: 700; margin-right: auto; }
.nav a { color: #e8ecf5; text-decoration: none; font-size: 14px; }
main { flex: 1; padding: 20px; }
.plans { display: flex; flex-wrap: wrap; gap: 16px; }
.plan { flex: 1 1 200px; display: flex; flex-direction: column; background: #fff; border-radius: 14px;
padding: 18px;
        box-shadow: 0 2px 4px rgb(26 42 79 / .08), 0 10px 24px rgb(26 42 79 / .12); }
.plan h3 { margin: 0; color: #1a2a4f; } .plan .price { font-size: 26px; font-weight: 700; margin: 6px
0; }
.plan ul { padding-left: 18px; font-size: 13px; margin: 0 0 14px; }
.plan a { margin-top: auto; text-align: center; padding: 10px; border-radius: 8px; background:
#1a2a4f; color: #fff; text-decoration: none; }
.plan.featured { outline: 3px solid #c9a227; }
footer { background: #1a2a4f; color: #c9d0e4; padding: 14px 20px; font-size: 13px; }
```

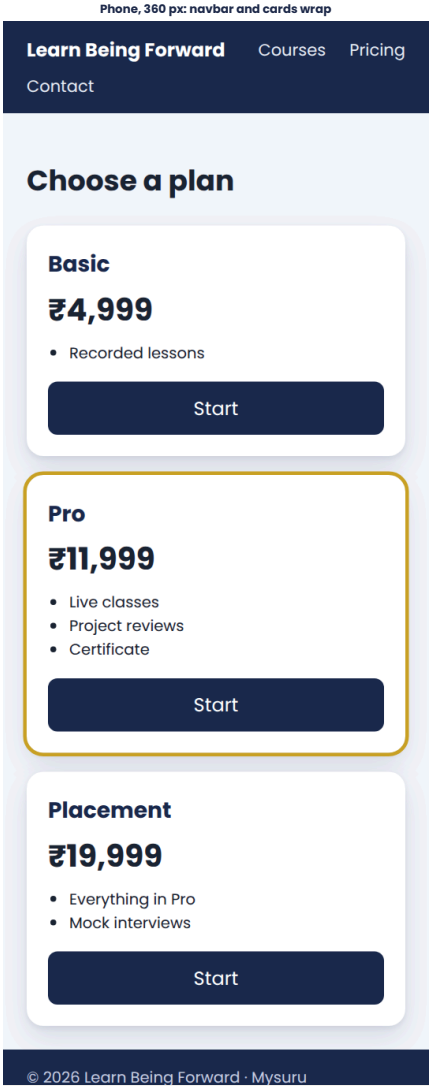
INDEX.HTML HTML

```
<nav class="nav"><span class="logo">Learn Being Forward</span><a href="#">Courses</a><a
href="#">Pricing</a><a href="#">Contact</a></nav>
<main>
  <h2>Choose a plan</h2>
  <div class="plans">
    <article class="plan"><h3>Basic</h3><p class="price">₹4,999</p><ul><li>Recorded lessons</li></ul>
<a href="#">Start</a></article>
    <article class="plan featured"><h3>Pro</h3><p class="price">₹11,999</p><ul><li>Live classes</li>
<li>Project reviews</li><li>Certificate</li></ul><a href="#">Start</a></article>
    <article class="plan"><h3>Placement</h3><p class="price">₹19,999</p><ul><li>Everything in Pro</li>
<li>Mock interviews</li></ul><a href="#">Start</a></article>
  </div>
</main>
<footer>© 2026 Learn Being Forward · Mysuru</footer>
```

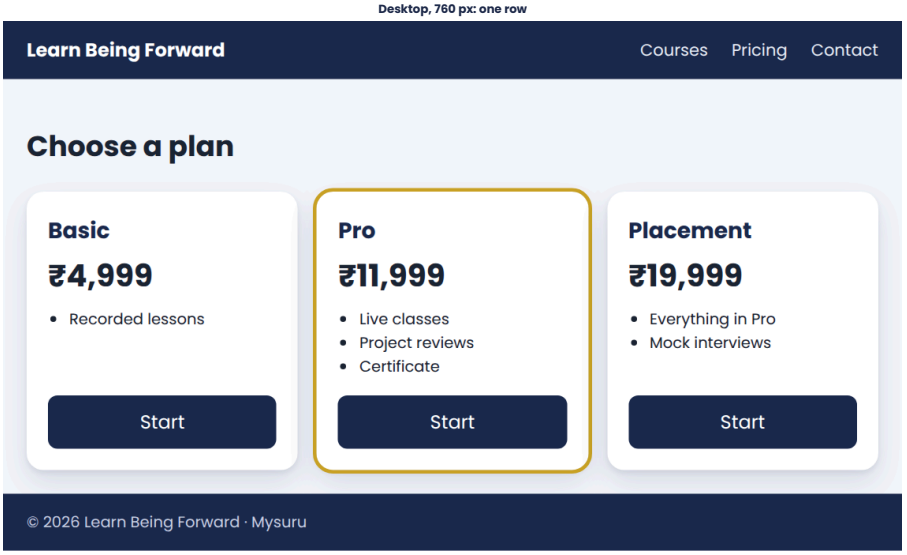


HOW THE BROWSER DISPLAYS IT

Phone, 360 px: navbar and cards wrap



Desktop, 760 px: one row



THE SAME FLEXBOX LAYOUT AT TWO WIDTHS, WITH NO MEDIA QUERIES

MODULE 11 · INTERMEDIATE

CSS Grid

Minimum duration **3 days** | Level **Intermediate**

Grid is a two-dimensional layout system for rows and columns together, ideal for page layouts and galleries. This module covers the grid vocabulary (container, items, lines, tracks, cells, areas), when to choose Grid or Flexbox, defining tracks with grid-template-columns and rows, the fr unit, repeat(), minmax(), auto-fill versus auto-fit and gap; placing items by line numbers, spans, named lines and named template areas; the implicit grid with grid-auto-rows, grid-auto-columns, grid-auto-flow and dense packing; every alignment property; and four complete patterns: the holy grail layout, a responsive card grid without media queries, an image gallery with spanning items and a dashboard. Track sizes quoted in the text were read back from Chromium.

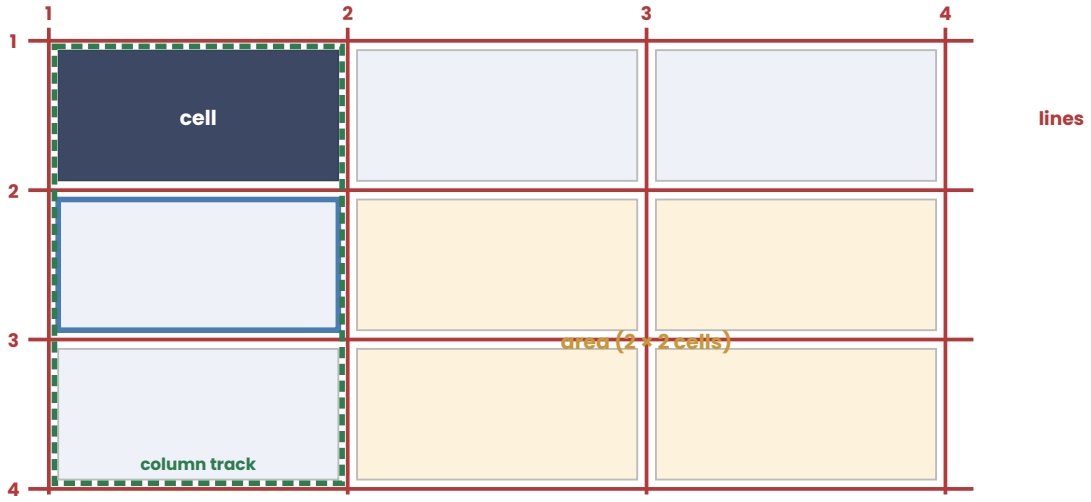
IN THIS MODULE

11.1 Grid basics
11.2 Defining the grid
11.3 Placing items
11.4 Implicit grid
11.5 Alignment in grid
11.6 Common grid patterns
11.7 Practice with Grid Garden

11.1 Grid basics

11.1.1 Grid container, items, lines, tracks, cells and areas

`display: grid` turns an element into a **grid container**; its direct children are **grid items**. You define **tracks** (columns and rows); the browser places items into the resulting cells automatically, or exactly where you say.



GRID VOCABULARY: LINES (NUMBERED FROM 1), TRACKS (A COLUMN OR ROW), CELLS AND AREAS

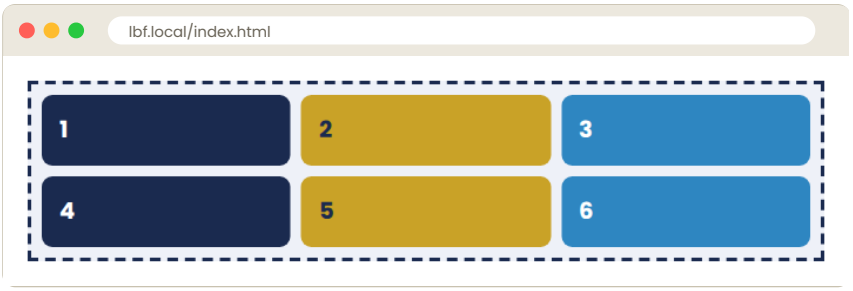
Term	Meaning
Grid line	the dividing lines, numbered from 1 at the start edge (and from -1 at the end edge); can be named
Track	the space between two adjacent lines: a column or a row
Cell	the intersection of one row and one column
Area	any rectangle of cells, often named with grid-template-areas
Gutter	the gap between tracks (`gap`)
Explicit / implicit grid	tracks you define / tracks the browser creates for extra items (11.4)

THREE EQUAL COLUMNS CSS

```css
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: 1fr 1fr 1fr; }
```

INDEX.HTML HTML

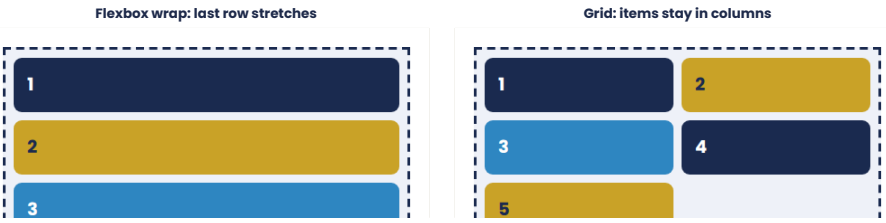
```html
<div class="g"><div>1</div><div>2</div><div>3</div><div>4</div><div>5</div><div>6</div></div>
```



SIX ITEMS FLOWED INTO A 3 × 2 GRID AUTOMATICALLY

11.1.2 Grid vs Flexbox: when to use which

	Flexbox	Grid
Dimensions	one: a row **or** a column	two: rows **and** columns at once
Driven by	content: items decide their size, the layout adapts	layout: you define tracks, content fills them
Alignment across rows	each wrapped line is independent (cards do not line up in columns)	items line up in both directions
Overlapping items	no	yes (place items in the same cells)
Best for	navbars, toolbars, button groups, media objects, centring, small components	page layouts, card grids, galleries, dashboards, forms with aligned labels



FIVE ITEMS, ABOUT 110PX MINIMUM EACH

They are complementary, and real pages nest them: Grid for the overall page and card grid, Flexbox inside each card and in the header.

11.2 Defining the grid

11.2.1 grid-template-columns and grid-template-rows

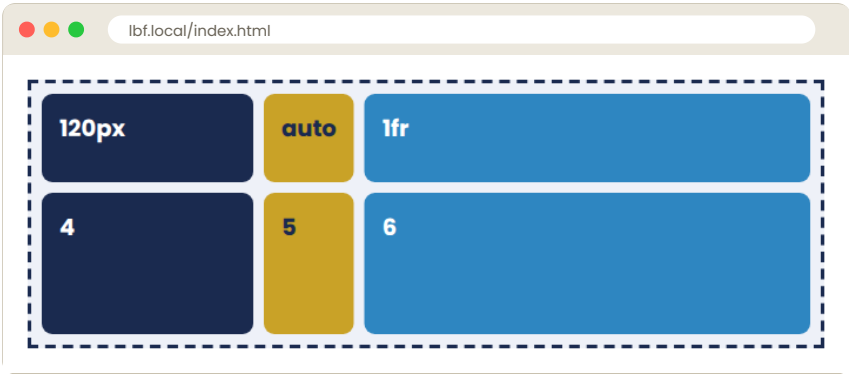
These two properties define the **explicit grid**: a space-separated list of track sizes. Any length, percentage, `fr`, `auto` (fit the content, then share leftover space), `min-content`, `max-content` or `fit-content(size)` may be used.

FIXED, AUTO AND FR COLUMNS; FIXED ROWS CSS

```css
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: 120px auto 1fr; grid-template-rows: 50px 80px; }
```

INDEX.HTML HTML

```html
<div class="g"><div>120px</div><div>auto</div><div>1fr</div><div>4</div><div>5</div><div>6</div></div>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
grid-template-columns (resolved): 120px 33.7656px 314.234px
grid-template-rows (resolved):    50px 80px
```

11.2.2 The fr unit

`fr` (fraction) represents a share of the **free space** left after fixed tracks, gaps and content-sized tracks have been placed. `1fr 2fr 1fr` splits the remaining space in the ratio 1 : 2 : 1. Strictly, `1fr` means

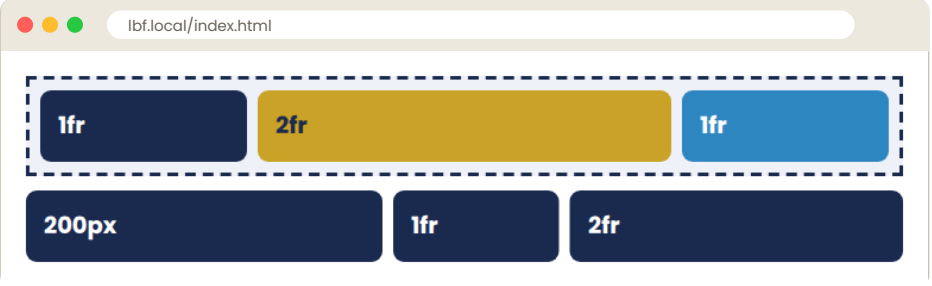
`minmax(auto, 1fr)`, so a track never shrinks below its content's minimum size; use `minmax(0, 1fr)` to allow truly equal columns when content is long.

FR SHARES THE FREE SPACE CSS

```
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: 1fr 2fr 1fr; }
.h { display: grid; grid-template-columns: 200px 1fr 2fr; gap: 6px; margin-top: 8px; }
.h > div { background: #1a2a4f; color: #fff; padding: 10px; border-radius: 6px; font: 600 13px
"Poppins", sans-serif; }
```

INDEX.HTML HTML

```
<div class="g"><div>1fr</div><div>2fr</div><div>1fr</div></div><div class="h"><div>200px</div>
<div>1fr</div><div>2fr</div></div>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
grid-template-columns (resolved): 200px 133.328px 266.672px
grid-template-rows (resolved):    21.75px
```

Here the container is 620px: minus the 200px column and two 10px gaps leaves 400px of free space, divided 1 : 2 into 133.33px and 266.67px.

11.2.3 repeat()

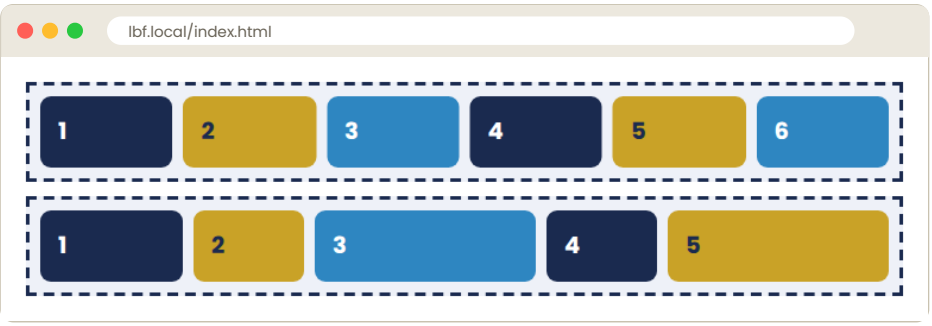
`repeat(count, tracks)` avoids writing the same size many times: `repeat(12, 1fr)` is a 12-column grid, and the pattern can contain several tracks: `repeat(3, 1fr 2fr)` gives six columns alternating 1fr and 2fr. It can be mixed with other sizes: `200px repeat(3, 1fr)`.

REPEAT() CSS

```
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: repeat(6, 1fr); }
.p { grid-template-columns: 80px repeat(2, 1fr 2fr); margin-top: 8px; }
```

INDEX.HTML HTML

```
<div class="g"><div>1</div><div>2</div><div>3</div><div>4</div><div>5</div><div>6</div></div><div
class="g p"><div>1</div><div>2</div><div>3</div><div>4</div><div>5</div></div>
```



HOW THE BROWSER DISPLAYS IT

11.2.4 minmax()

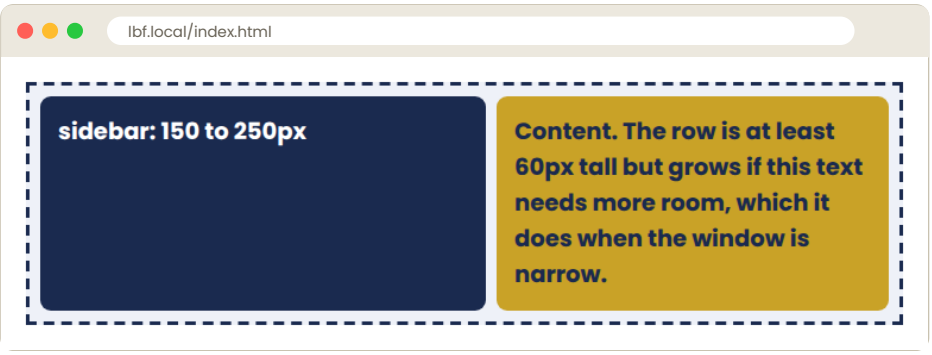
`minmax(min, max)` sets a size range for a track: it is at least *min* and at most *max*. `minmax(200px, 1fr)` means "never narrower than 200px, otherwise share the space"; `minmax(100px, auto)` for rows means "at least 100px, taller if the content needs it".

MINMAX() FOR COLUMNS AND ROWS CSS

```
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: minmax(150px, 250px) 1fr; grid-template-rows: minmax(60px, auto); }
```

INDEX.HTML HTML

```
<div class="g"><div>sidebar: 150 to 250px</div><div>Content. The row is at least 60px tall but grows
if this text needs more room, which it does when the window is narrow.</div></div>
```



HOW THE BROWSER DISPLAYS IT

11.2.5 auto-fill vs auto-fit

Instead of a number, `repeat()` can take `auto-fill` or `auto-fit`: the browser creates **as many tracks as fit** in the container. Combined with `minmax()`, this produces responsive grids with **no media queries**. The two keywords differ only when there are fewer items than tracks: `auto-fill` keeps the empty tracks (items keep their size), `auto-fit` collapses empty tracks to zero so the items stretch to fill the row.



THREE ITEMS IN A CONTAINER WIDE ENOUGH FOR FIVE 100PX COLUMNS

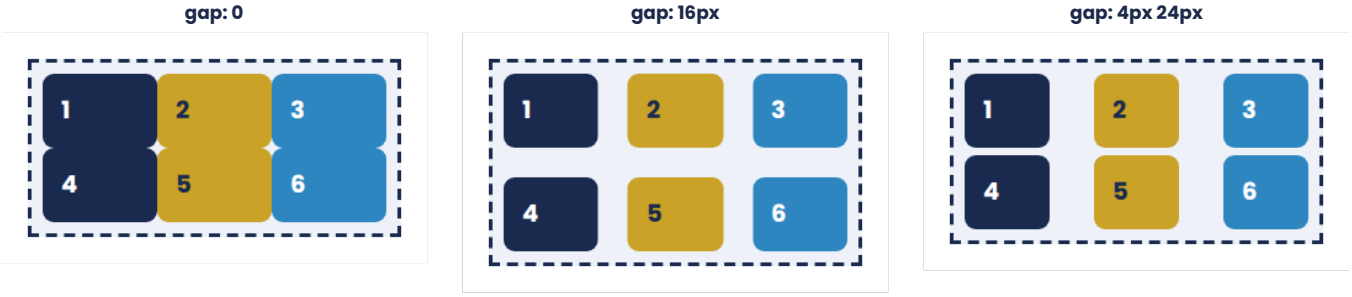
COMPUTED IN CHROMIUM

```
auto-fill columns: 104px 104px 104px 104px 104px
auto-fit columns:  180px 180px 180px 0px 0px
```

The most useful line in CSS Grid: `grid-template-columns: repeat(auto-fit, minmax(min(250px, 100%), 1fr));` A responsive card grid: as many columns of at least 250px as fit, one column on small phones (the inner min() prevents overflow below 250px).

11.2.6 gap

`gap` (shorthand for `row-gap` and `column-gap`) sets the gutters between tracks, never at the outer edges. It is the reason grid layouts need no margin arithmetic. (Old code may use `grid-gap`, the original name, still supported.)



11.3 Placing items

11.3.1 Line numbers: grid-column and grid-row

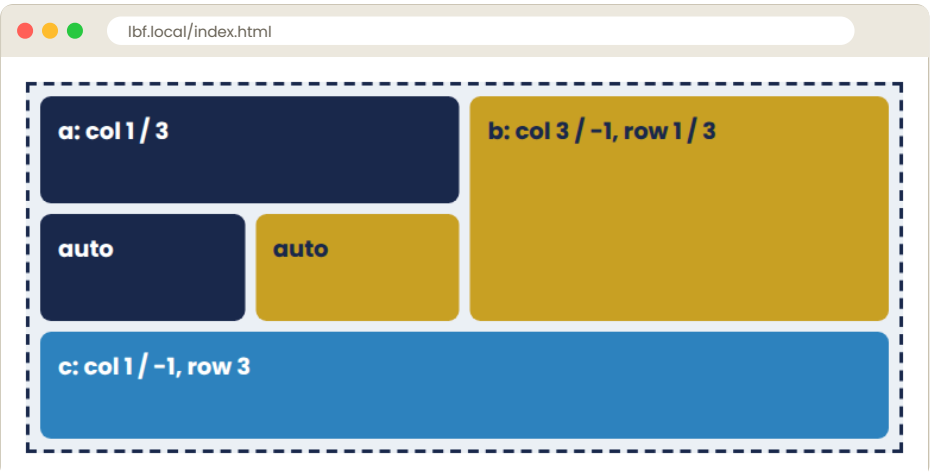
By default items fill cells in order (auto-placement). To place an item explicitly, give its start and end **lines**: `grid-column: 1 / 3` means from column line 1 to column line 3 (two columns wide); `grid-row: 2 / 4` spans rows 2 and 3. Negative numbers count from the end: `grid-column: 1 / -1` spans the full width of the explicit grid. The longhands are `grid-column-start`, `grid-column-end`, `grid-row-start`, `grid-row-end`.

PLACING BY LINE NUMBERS — CSS

```css
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: repeat(4, 1fr); grid-template-rows: repeat(3, 60px); }
.a { grid-column: 1 / 3; grid-row: 1 / 2; }
.b { grid-column: 3 / -1; grid-row: 1 / 3; }
.c { grid-column: 1 / -1; grid-row: 3; }
```

INDEX.HTML — HTML

```html
<div class="g"><div class="a">a: col 1 / 3</div><div class="b">b: col 3 / -1, row 1 / 3</div><div
class="c">c: col 1 / -1, row 3</div><div>auto</div><div>auto</div></div>
```



EXPLICITLY PLACED ITEMS FIRST; THE TWO "AUTO" ITEMS FILL THE REMAINING CELLS

11.3.2 Spanning with span

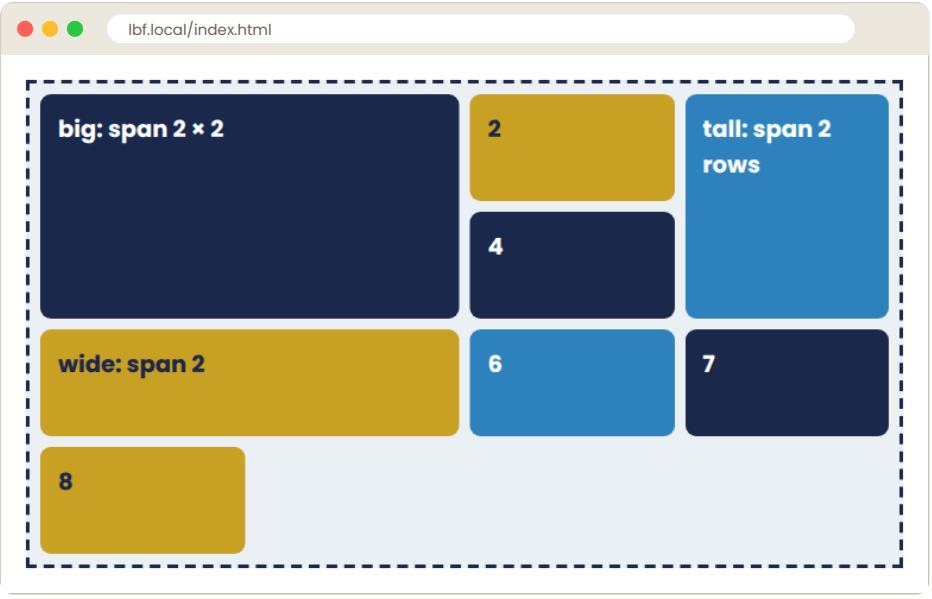
`span n` says "cover n tracks" without naming the end line: `grid-column: span 2` (start wherever auto-placement puts it), `grid-column: 2 / span 3` (start at line 2, cover three columns). It is the usual way to make some cards in an auto-placed grid bigger.

SPAN CSS

```
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: repeat(4, 1fr); grid-auto-rows: 60px; }
.wide { grid-column: span 2; }
.tall { grid-row: span 2; }
.big { grid-column: span 2; grid-row: span 2; }
```

INDEX.HTML HTML

```
<div class="g"><div class="big">big: span 2 × 2</div><div>2</div><div class="tall">tall: span 2
rows</div><div>4</div><div class="wide">wide: span 2</div><div>6</div><div>7</div><div>8</div></div>
```



HOW THE BROWSER DISPLAYS IT

11.3.3 Named lines

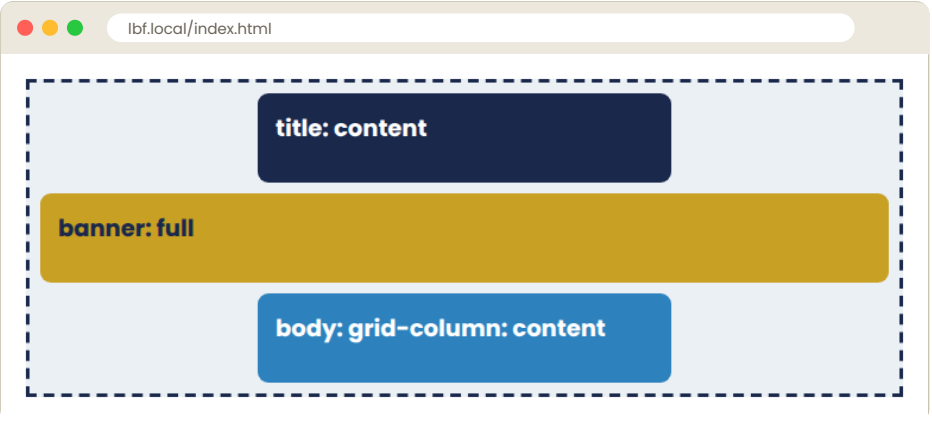
Lines can be named in square brackets inside the track list, and items can then be placed by name, which reads better and survives changes in the number of columns. A line may have several names. Lines named `xxx-start` and `xxx-end` automatically define an area `xxx`.

NAMED LINES — CSS

```
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: [full-start] 1fr [content-start] 2fr [content-end] 1fr [full-end]; grid-
auto-rows: 50px; }
.title { grid-column: content-start / content-end; }
.banner { grid-column: full-start / full-end; }
.body { grid-column: content; }        /* uses content-start / content-end */
```

INDEX.HTML — HTML

```
<div class="g"><div class="title">title: content</div><div class="banner">banner: full</div><div
class="body">body: grid-column: content</div></div>
```



HOW THE BROWSER DISPLAYS IT

This is the basis of a popular article layout: text in a readable central column, with images "breaking out" to the full width.

11.3.4 grid-template-areas and grid-area

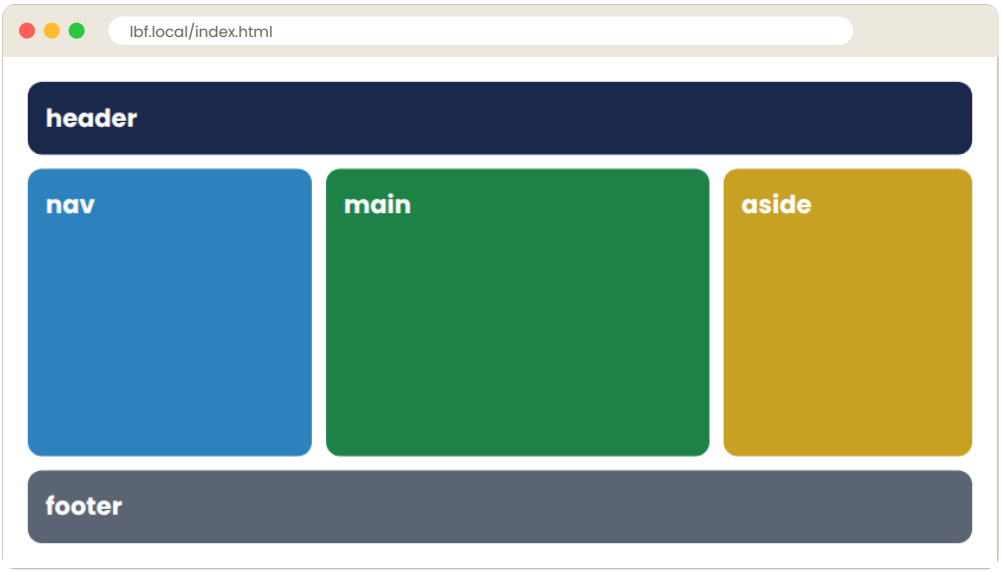
`grid-template-areas` draws the layout as ASCII art: each string is a row and each word a cell; repeating a name across cells makes an area span them (it must form a rectangle); a dot `.` is an empty cell. Items are assigned with `grid-area: name`. Changing the layout at a breakpoint is then a matter of rewriting the strings.

A PAGE LAYOUT DRAWN WITH GRID-TEMPLATE-AREAS CSS

```css
.page { display: grid; gap: 8px; min-height: 260px;
  grid-template-columns: 160px 1fr 140px;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header  header"
    "nav    main    aside"
    "footer footer  footer";
}
.page > * { padding: 10px; border-radius: 8px; color: #fff; font: 600 14px "Poppins", sans-serif; }
header { grid-area: header; background: #1a2a4f; }
nav    { grid-area: nav;    background: #2e86c1; }
main   { grid-area: main;   background: #1e8449; }
aside  { grid-area: aside;  background: #c9a227; color: #1a2a4f; }
footer { grid-area: footer; background: #5c6475; }
```

INDEX.HTML HTML

```html
<div class="page"><header>header</header><nav>nav</nav><main>main</main><aside>aside</aside>
<footer>footer</footer></div>
```

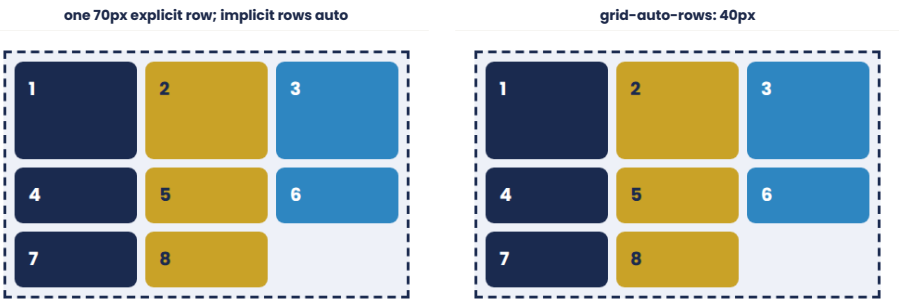


HOW THE BROWSER DISPLAYS IT

11.4 Implicit grid

11.4.1 grid-auto-rows, grid-auto-columns

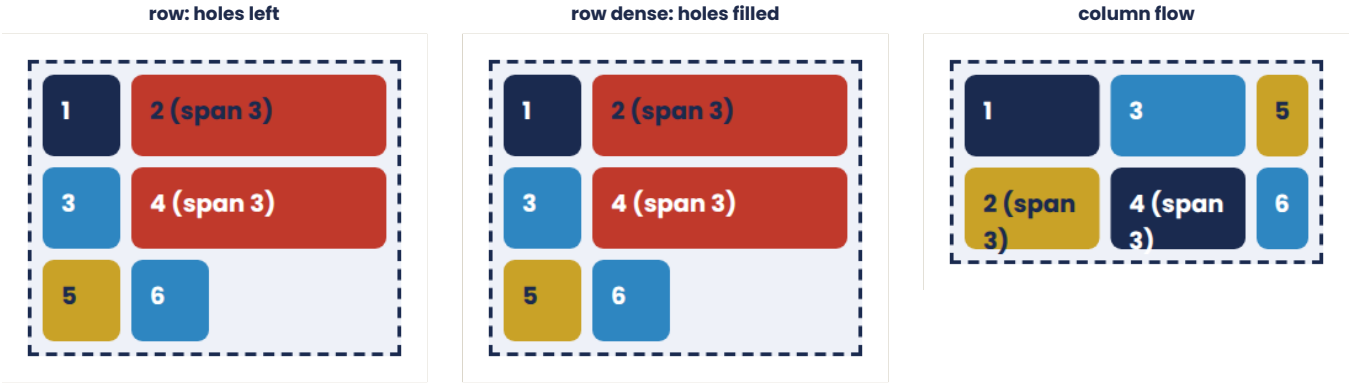
When there are more items than explicit cells, or an item is placed outside the explicit grid, the browser creates **implicit tracks**. By default they are `auto` sized (as tall as their content). `grid-auto-rows` and `grid-auto-columns` size them, often with minmax: `grid-auto-rows: minmax(120px, auto)`. In practice many grids define only columns and let rows be implicit.



ROWS 2 AND 3 ARE IMPLICIT

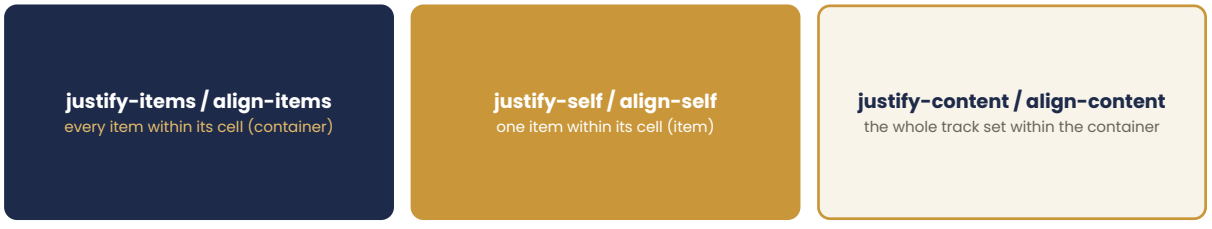
11.4.2 grid-auto-flow and dense

`grid-auto-flow` controls auto-placement: `row` (default, fill each row then start a new one), `column` (fill columns, adding columns as needed) and the `dense` keyword, which back-fills earlier holes with later items that fit. Dense packing is good for galleries where order does not matter; avoid it when order is meaningful, because the visual order no longer matches the reading and tab order.



11.5 Alignment in grid

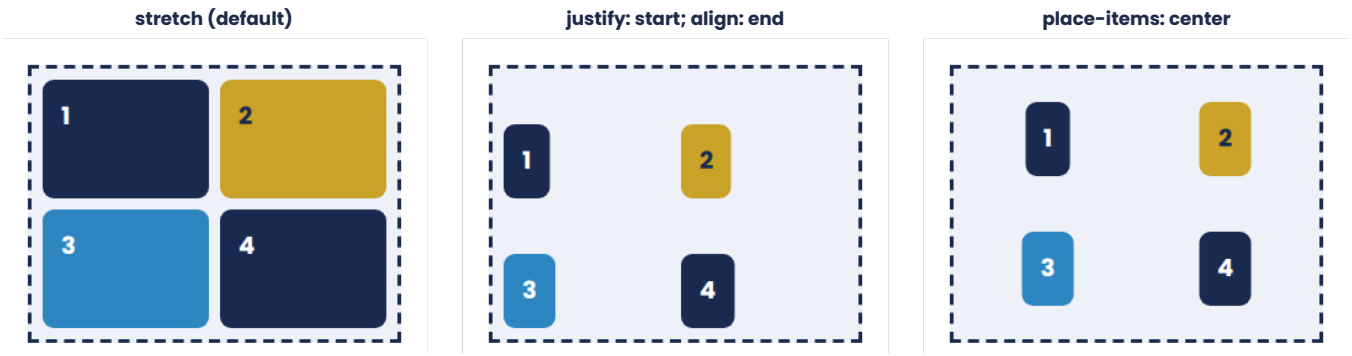
Grid uses the same Box Alignment properties as Flexbox, on two axes: **justify-*** works along the **inline** (row) axis and **align-*** along the **block** (column) axis. `*-items` aligns every item inside its cell (set on the container), `*-self` one item (set on the item), and `*-content` moves the whole grid of tracks inside a larger container.



THREE FAMILIES OF ALIGNMENT PROPERTIES

11.5.1 justify-items, align-items, place-items

Default `stretch` makes items fill their cells. Other values: `start`, `end`, `center`, and for align-items `baseline`. `place-items: center` is the shorthand for `align-items` then `justify-items`, the famous one-line centring.



ITEMS ALIGNED INSIDE THEIR CELLS

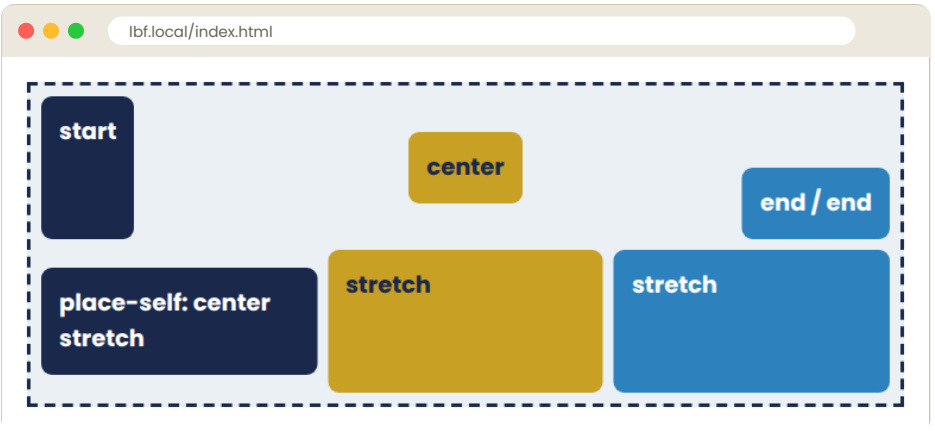
11.5.2 justify-self, align-self, place-self

ALIGNING INDIVIDUAL ITEMS CSS

```css
.g { display: grid; background: #eef1f8; border: 2px dashed #1a2a4f; padding: 6px; gap: 6px; }
.g > div { background: #1a2a4f; color: #fff; font: 600 13px "Poppins", sans-serif; padding: 10px;
border-radius: 6px; }
.g > div:nth-child(3n+2) { background: #c9a227; color: #1a2a4f; } .g > div:nth-child(3n) { background:
#2e86c1; }
.g { grid-template-columns: repeat(3, 1fr); grid-auto-rows: 80px; }
.s1 { justify-self: start; } .s2 { justify-self: center; align-self: center; } .s3 { justify-self:
end; align-self: end; }
.s4 { place-self: center stretch; }
```

INDEX.HTML HTML

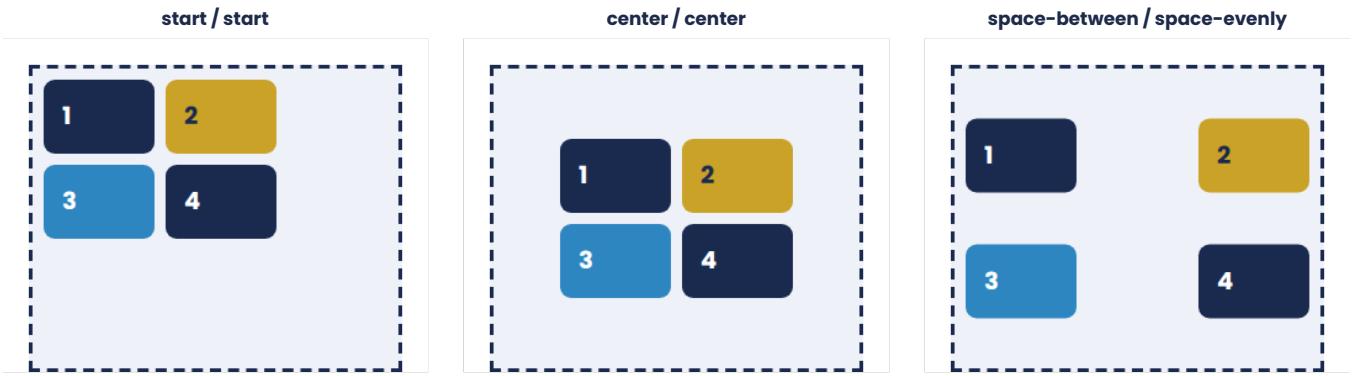
```html
<div class="g"><div class="s1">start</div><div class="s2">center</div><div class="s3">end / end</div>
<div class="s4">place-self: center stretch</div><div>stretch</div><div>stretch</div></div>
```



HOW THE BROWSER DISPLAYS IT

11.5.3 justify-content, align-content

When the tracks are smaller than the container (fixed-size tracks in a large box), `justify-content` and `align-content` position the whole grid: `start`, `center`, `end`, `space-between`, `space-around`, `space-evenly`, `stretch` (for auto tracks).



MOVING THE WHOLE 2 × 2 GRID OF FIXED TRACKS

11.6 Common grid patterns

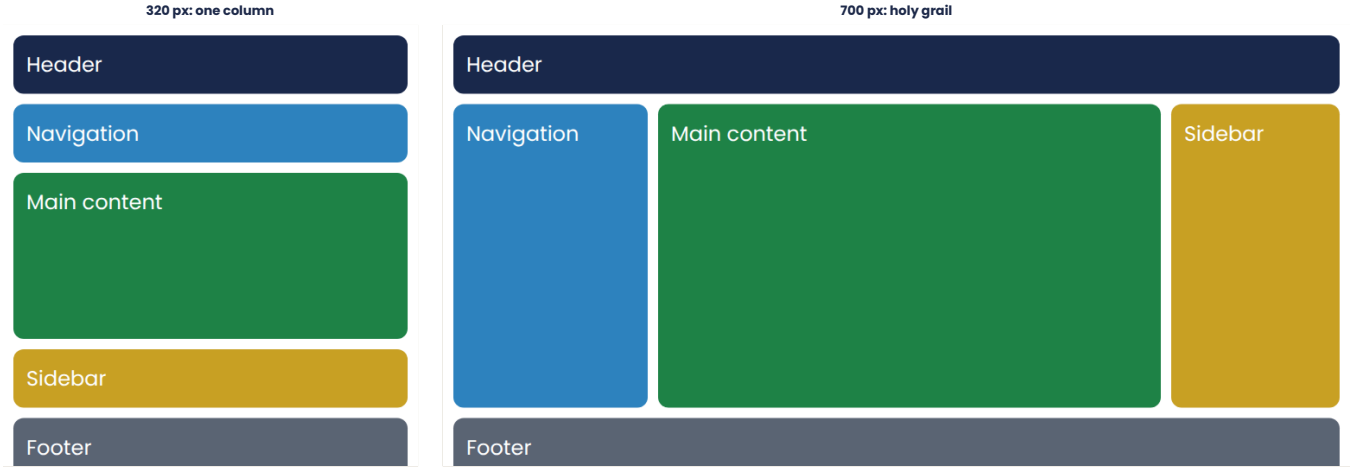
11.6.1 Holy grail layout (header, sidebar, content, footer)

The "holy grail" is a header, a footer and three columns (navigation, content, sidebar) with the footer at the bottom of short pages: a nightmare with floats, a few lines with Grid. On small screens the areas are redrawn as a single column.

HOLY-GRAIL.CSS

CSS

```
body { margin: 0; font-family: "Poppins", sans-serif; }
.app {
  min-height: 100vh; display: grid; gap: 8px; padding: 8px;
  grid-template: "header" auto "nav" auto "main" 1fr "aside" auto "footer" auto / 1fr;
}
@media (min-width: 640px) {
  .app {
    grid-template-columns: 150px 1fr 130px;
    grid-template-rows: auto 1fr auto;
    grid-template-areas: "header header header" "nav main aside" "footer footer footer";
  }
}
.app > * { padding: 10px; border-radius: 8px; color: #fff; }
header { grid-area: header; background: #1a2a4f; } nav { grid-area: nav; background: #2e86c1; }
main { grid-area: main; background: #1e8449; } aside { grid-area: aside; background: #c9a227; color:
#1a2a4f; }
footer { grid-area: footer; background: #5c6475; }
```



HOLY GRAIL LAYOUT, MOBILE-FIRST WITH ONE MEDIA QUERY

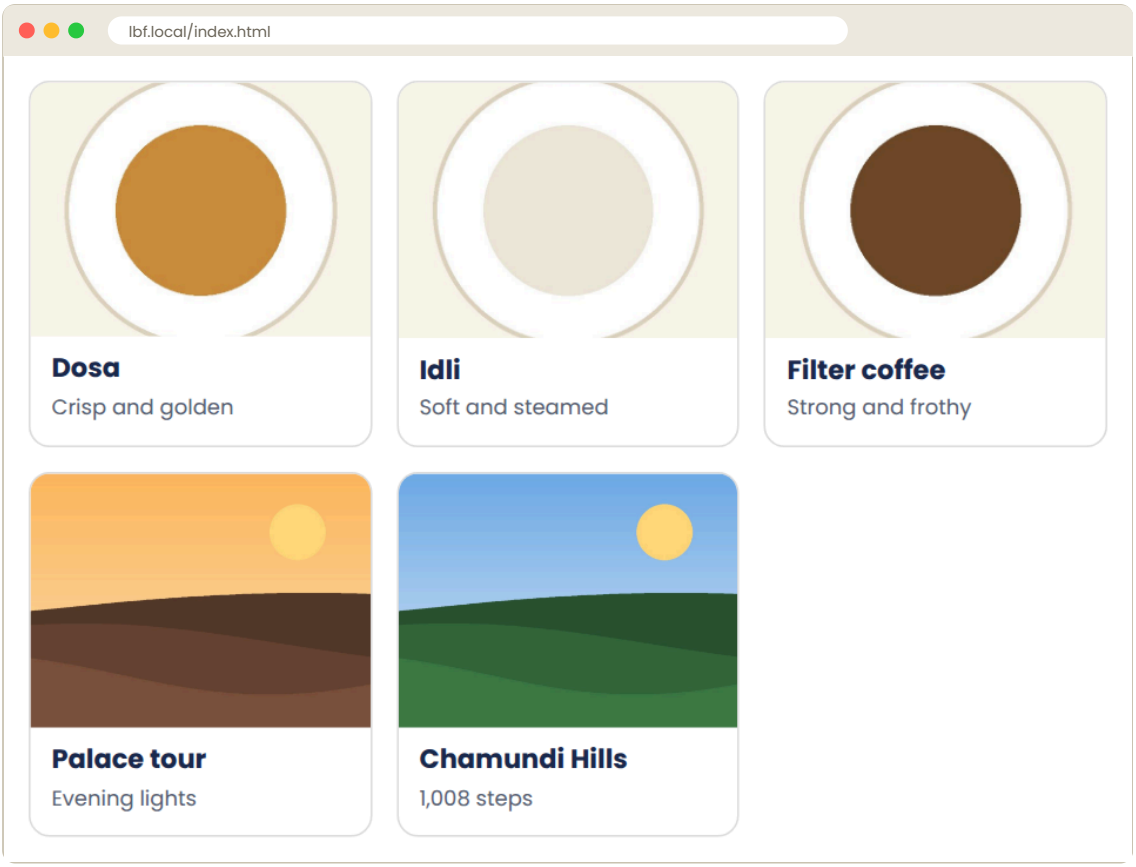
11.6.2 Responsive card grid without media queries

AUTO-FIT + MINMAX: A CARD GRID CSS

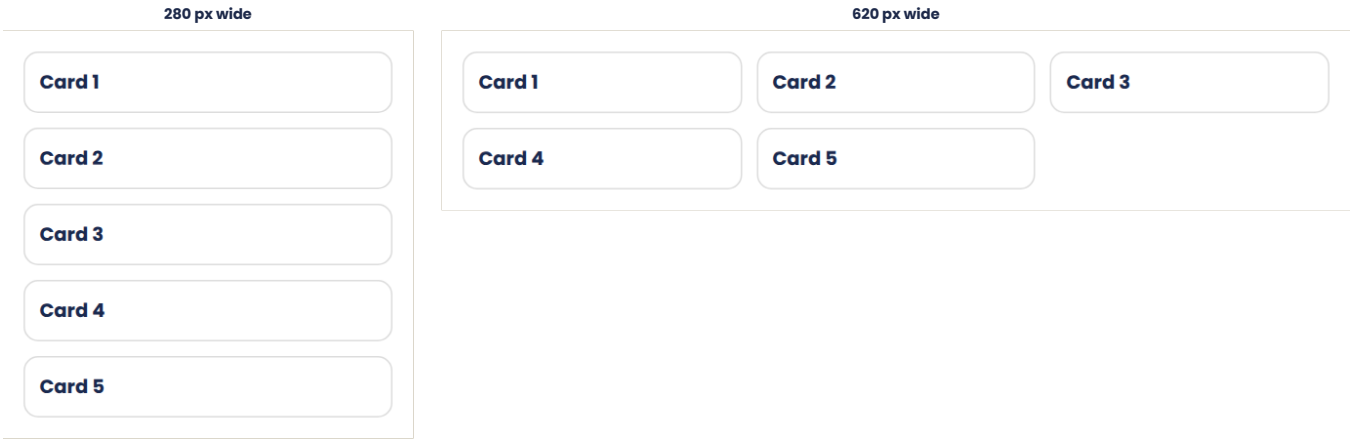
```
.cards { display: grid; gap: 14px; grid-template-columns: repeat(auto-fit, minmax(min(180px, 100%), 1fr)); }
.card { background: #fff; border: 1px solid #ddd; border-radius: 12px; overflow: hidden; font-family: "Poppins", sans-serif; }
.card img { width: 100%; aspect-ratio: 4 / 3; object-fit: cover; display: block; }
.card h3 { margin: 8px 12px 2px; font-size: 15px; color: #1a2a4f; } .card p { margin: 0 12px 12px; font-size: 12px; color: #5c6475; }
```

INDEX.HTML HTML

```
<div class="cards"><article class="card"><img src="food-dosa.jpg" alt=""><h3>Dosa</h3><p>Crisp and golden</p></article><article class="card"><img src="food-idli.jpg" alt=""><h3>Idli</h3><p>Soft and steamed</p></article><article class="card"><img src="food-coffee.jpg" alt=""><h3>Filter coffee</h3><p>Strong and frothy</p></article><article class="card"><img src="mysuru-palace.jpg" alt=""><h3>Palace tour</h3><p>Evening lights</p></article><article class="card"><img src="hills.jpg" alt=""><h3>Chamundi Hills</h3><p>1,008 steps</p></article></div>
```



HOW THE BROWSER DISPLAYS IT



THE SAME RULE GIVES ONE COLUMN ON A NARROW SCREEN AND THREE ON A WIDER ONE

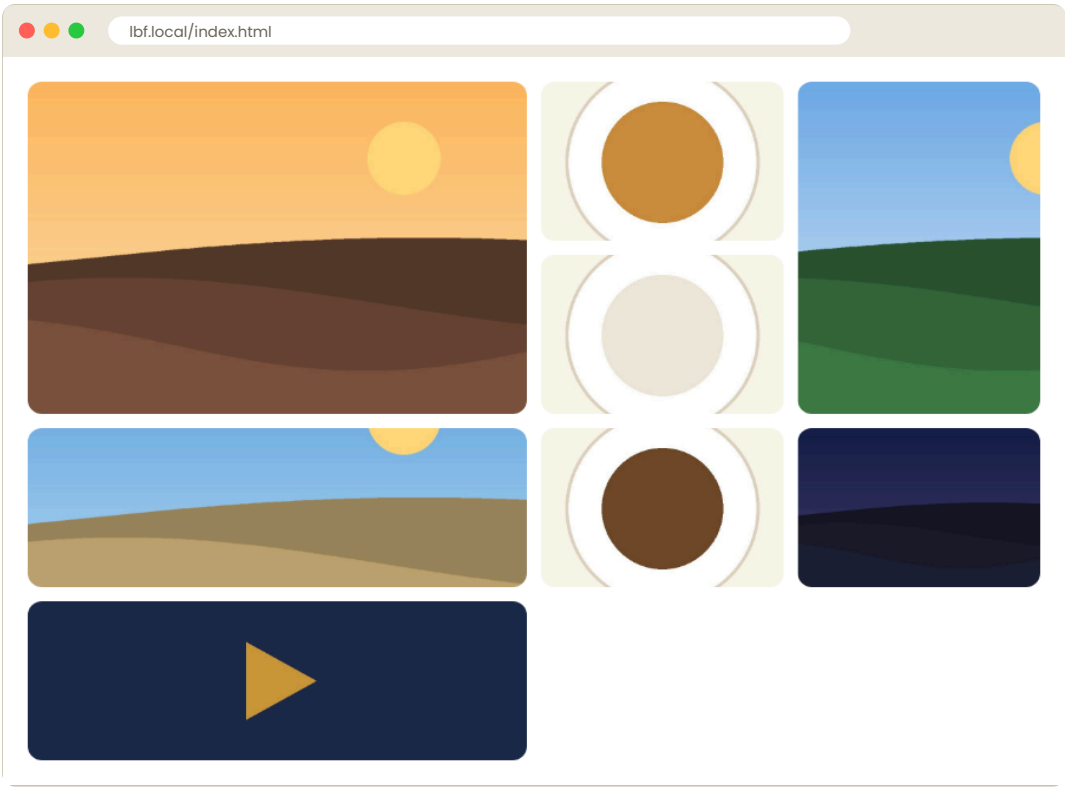
11.6.3 Image gallery with spanning items

A MOSAIC GALLERY CSS

```css
.gallery { display: grid; gap: 8px; grid-template-columns: repeat(4, 1fr); grid-auto-rows: 90px; grid-auto-flow: dense; }
.gallery img { width: 100%; height: 100%; object-fit: cover; border-radius: 8px; display: block; }
.gallery .wide { grid-column: span 2; }
.gallery .tall { grid-row: span 2; }
.gallery .big { grid-column: span 2; grid-row: span 2; }
```

INDEX.HTML HTML

```html
<div class="gallery">
  <img class="big" src="mysuru-palace.jpg" alt="Mysuru Palace">
  <img src="food-dosa.jpg" alt="Dosa"><img class="tall" src="hills.jpg" alt="Hills">
  <img src="food-idli.jpg" alt="Idli"><img class="wide" src="beach.jpg" alt="Beach">
  <img src="food-coffee.jpg" alt="Coffee"><img src="night.jpg" alt="Night">
  <img class="wide" src="poster.jpg" alt="Poster">
</div>
```



HOW THE BROWSER DISPLAYS IT

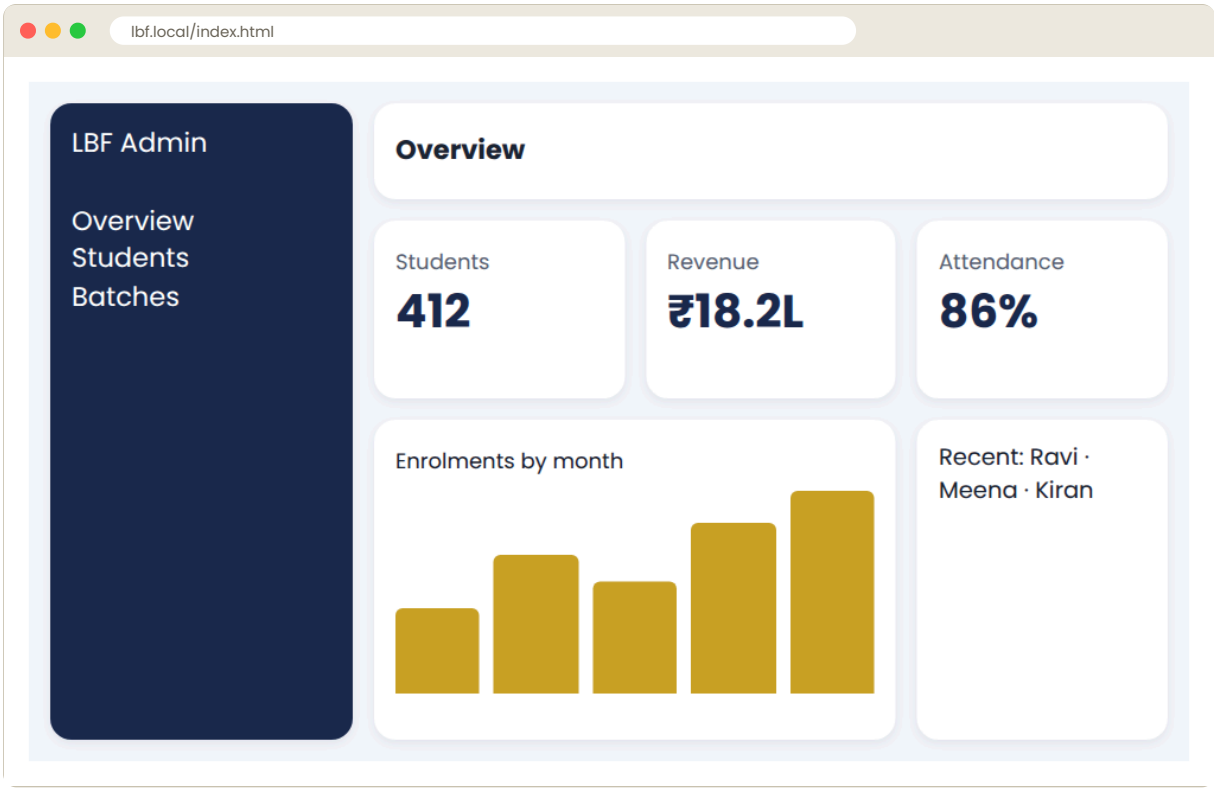
11.6.4 Dashboard layout

A DASHBOARD WITH NAMED AREAS CSS

```
.dash { display: grid; gap: 12px; padding: 12px; background: #f4f6fb; font-family: "Poppins", sans-serif;
  grid-template-columns: 170px repeat(3, 1fr);
  grid-template-rows: 54px 100px 180px;
  grid-template-areas: "side top top top" "side k1 k2 k3" "side chart chart list"; }
.dash > * { background: #fff; border-radius: 12px; padding: 12px; box-shadow: 0 2px 6px rgb(26 42 79 / .08); }
.side { grid-area: side; background: #1a2a4f; color: #fff; } .top { grid-area: top; display: flex; align-items: center; }
.k1 { grid-area: k1; } .k2 { grid-area: k2; } .k3 { grid-area: k3; } .chart { grid-area: chart; }
.list { grid-area: list; }
.kpi b { display: block; font-size: 26px; color: #1a2a4f; } .kpi span { font-size: 12px; color: #5c6475; }
.bars { display: flex; align-items: end; gap: 8px; height: 120px; } .bars i { flex: 1; background: #c9a227; border-radius: 4px 4px 0 0; }
```

INDEX.HTML HTML

```
<div class="dash">
  <nav class="side">LBF Admin<br><br>Overview<br>Students<br>Batches</nav>
  <header class="top"><b>Overview</b></header>
  <div class="k1 kpi"><span>Students</span><b>412</b></div>
  <div class="k2 kpi"><span>Revenue</span><b>₹18.2L</b></div>
  <div class="k3 kpi"><span>Attendance</span><b>86%</b></div>
  <div class="chart"><span style="font-size:12px">Enrolments by month</span>
    <div class="bars"><i style="height:40%"></i><i style="height:65%"></i><i style="height:52%"></i><i style="height:80%"></i><i style="height:95%"></i></div></div>
  <div class="list" style="font-size:13px">Recent: Ravi · Meena · Kiran</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

11.7 Practice with Grid Garden

Grid Garden (cssgridgarden.com), by the authors of Flexbox Froggy, has 28 levels in which you water carrots and poison weeds with grid-column, grid-row, grid-area, order and grid-template-columns. Then try the free "Grid Critters" levels or Wes Bos's free CSS Grid course (cssgrid.io).

A GRID GARDEN ANSWER CSS

```css
/* Grid Garden level 28 (final) - one possible answer */
#garden {
  display: grid;
  grid-template-columns: 75px 3fr 2fr;
  grid-template-rows: 100% 200px;
}
```

TIP

In DevTools, click the **grid** badge beside a grid container (or use the Layout tab) to overlay line numbers, line names, track sizes and area names directly on the page. It is the fastest way to understand any grid.

PRACTICE CHECKPOINT

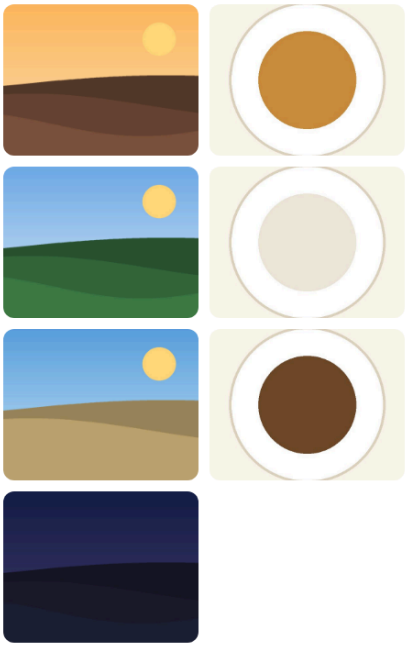
A **dashboard** using grid-template-areas, and a **responsive photo gallery** using repeat(auto-fit, minmax()) with some images **spanning two cells**.

GALLERY.CSS: SAMPLE SOLUTION (THE DASHBOARD IS 11.6.4)

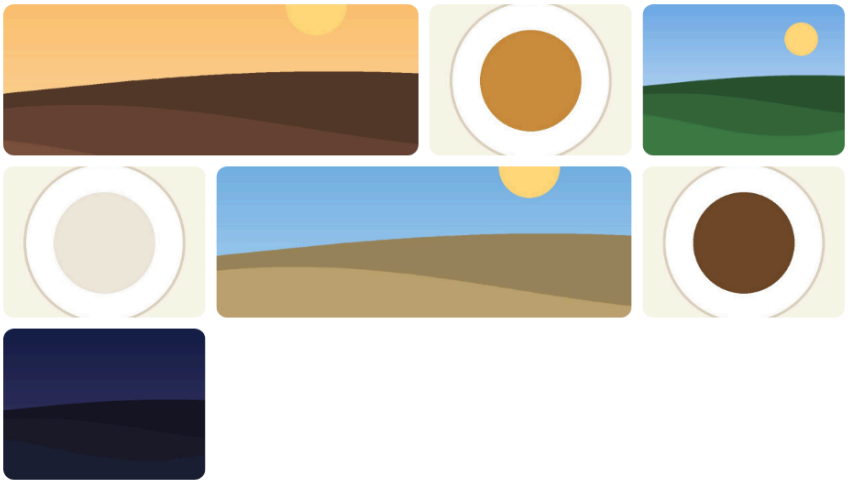
CSS

```
.gallery { display: grid; gap: 8px; grid-template-columns: repeat(auto-fit, minmax(min(140px, 100%), 1fr)); grid-auto-rows: 110px; grid-auto-flow: dense; }
.gallery img { width: 100%; height: 100%; object-fit: cover; border-radius: 8px; display: block; }
.gallery .span-2 { grid-column: span 2; }
@media (max-width: 340px) { .gallery .span-2 { grid-column: auto; } }   /* one column: no spanning */
```

320 px



640 px



THE GALLERY ADAPTS ITS COLUMN COUNT; WIDE IMAGES SPAN TWO COLUMNS WHEN THERE IS ROOM

MODULE 12 · INTERMEDIATE

Responsive design and media queries

Minimum duration **3 days** Level **Intermediate**

Responsive design makes one website work well on phones, tablets and desktops. This module covers the three ingredients (fluid layouts, flexible images and media queries), the viewport meta tag, mobile-first versus desktop-first, media query syntax and the newer range syntax, choosing breakpoints from content, orientation, hover and pointer queries, print stylesheets, the user preference queries prefers-color-scheme, prefers-reduced-motion and prefers-contrast, responsive images with max-width and object-fit, fluid typography, a responsive hamburger navigation, responsive tables, intrinsic layouts with Flexbox and Grid, and testing in DevTools device mode and on real devices. Each layout is shown at several real viewport widths, and preference queries were tested by emulating the setting in Chromium.

IN THIS MODULE

12.1 Responsive design principles
12.2 Media queries
12.3 User preference queries
12.4 Responsive techniques
12.5 Testing

12.1 Responsive design principles

Mobile devices produce more than half of all web traffic worldwide, and in India the large majority of internet users are primarily on phones. A site that only works on a laptop loses most of its visitors, and Google indexes the **mobile** version of pages first. **Responsive web design**, named by Ethan Marcotte in 2010, means building one site whose layout adapts to any screen, instead of separate desktop and "m." sites.

12.1.1 Fluid layouts, flexible images and media queries

Fluid layouts

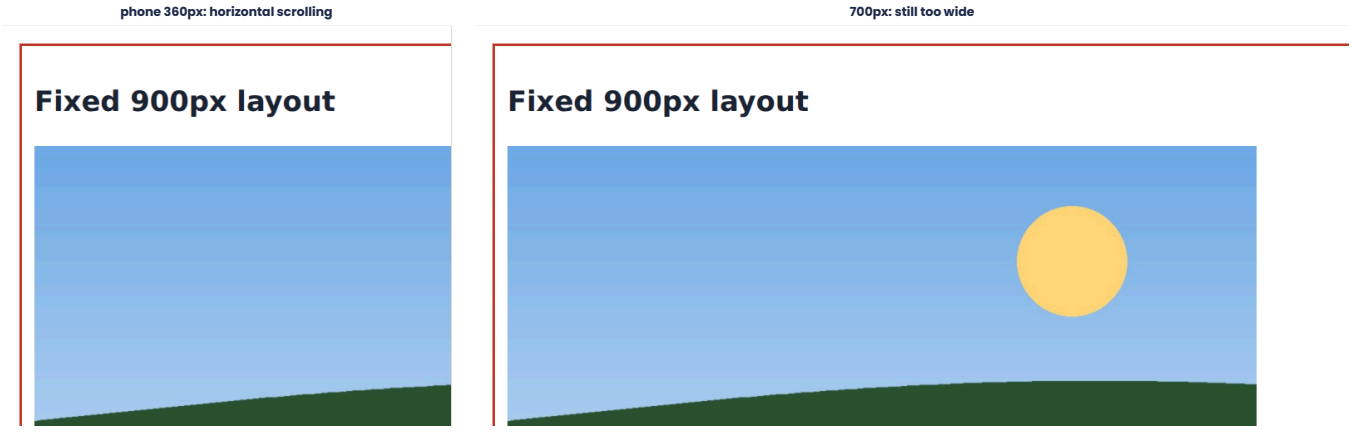
Widths in %, fr, flex and max-width instead of fixed pixels, so the layout stretches and shrinks with the screen.

Flexible images

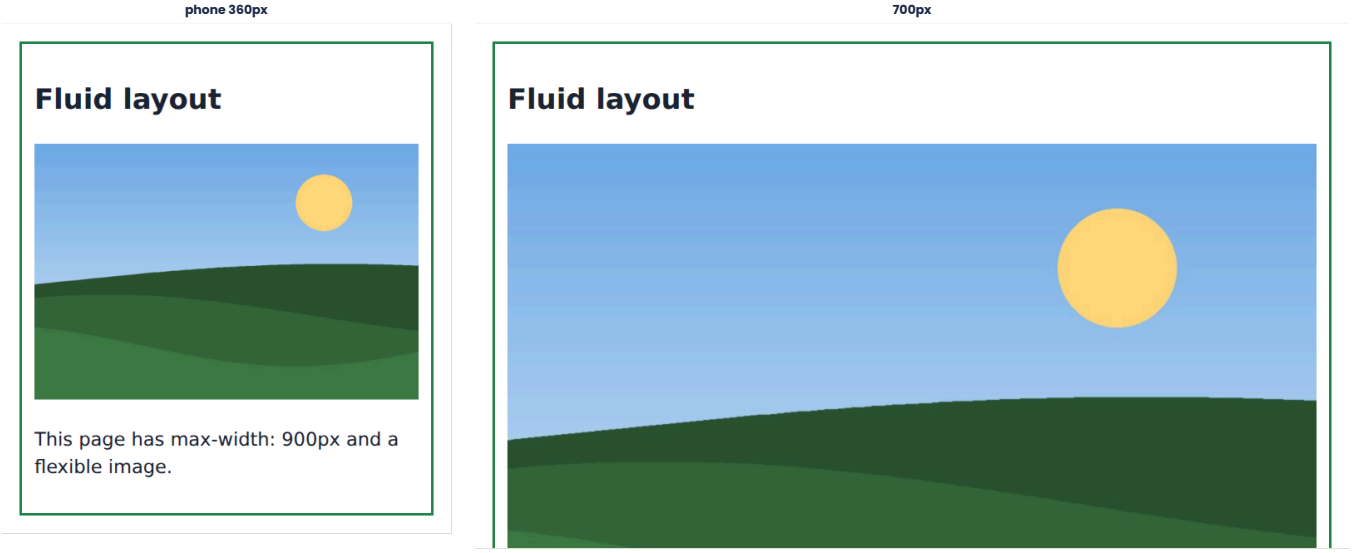
img { max-width: 100%; height: auto; } so media never overflows its container; srcset delivers the right size (HTML course).

Media queries

@media rules that change the layout at chosen widths (breakpoints): one column on phones, several on desktops.



A FIXED-WIDTH PAGE OVERFLOWS SMALL SCREENS



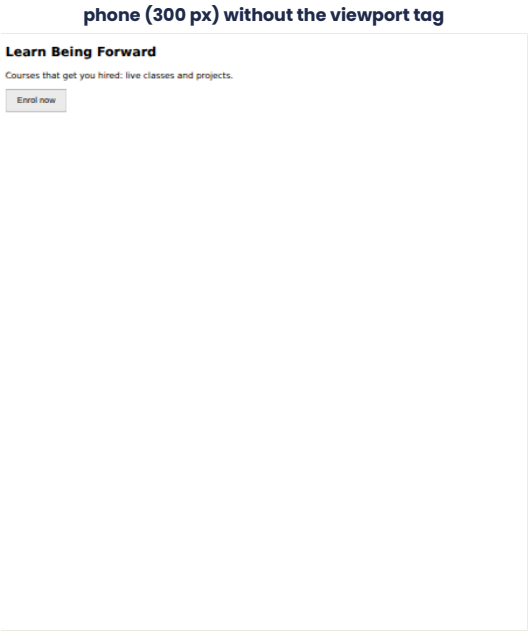
THE SAME PAGE WITH MAX-WIDTH AND A FLEXIBLE IMAGE FITS EVERY WIDTH

12.1.2 The viewport meta tag

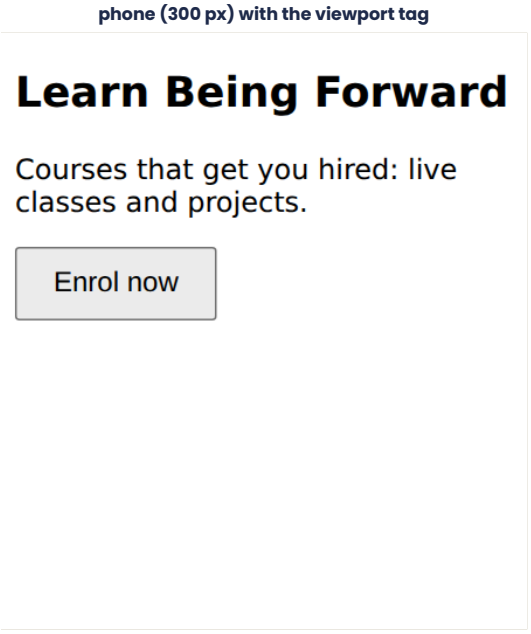
Mobile browsers assume, unless told otherwise, that pages were designed for desktops: they lay the page out about **980px** wide and zoom out to fit, so text is tiny and media queries for small screens never apply. One line in the head fixes this; without it, no responsive CSS works on phones:

REQUIRED IN EVERY PAGE HEAD HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```



WITHOUT THE TAG THE PHONE RENDERS A 980PX-WIDE PAGE AND SHRINKS IT: THE TEXT IS UNREADABLE



WITH THE TAG, THE LAYOUT WIDTH EQUALS THE DEVICE WIDTH

WATCH OUT

Do not add `maximum-scale=1` or `user-scalable=no`: they stop people from pinch-zooming, an accessibility failure (WCAG 1.4.4). Modern browsers ignore them in most cases anyway.

12.1.3 Mobile-first vs desktop-first

Mobile-first CSS writes the base styles for the smallest screen (usually a simple single column) and then **adds** complexity with `min-width` queries as space grows. **Desktop-first** starts with the full desktop layout and **undoes** it with `max-width` queries. Mobile-first is the recommended approach: small screens download and apply less CSS, the base layout is the simplest one, and it forces you to prioritise content.

THE SAME GRID BOTH WAYS CSS

```css
/* MOBILE-FIRST (recommended) */
.grid { display: grid; gap: 1rem; }                                    /* phones: 1 column */
@media (min-width: 640px)  { .grid { grid-template-columns: 1fr 1fr; } }  /* tablets: 2       */
@media (min-width: 1024px) { .grid { grid-template-columns: repeat(4, 1fr); } }  /* desktop: 4 */

/* DESKTOP-FIRST (the same result, written the other way round) */
.grid { display: grid; gap: 1rem; grid-template-columns: repeat(4, 1fr); }
@media (max-width: 1023px) { .grid { grid-template-columns: 1fr 1fr; } }
@media (max-width: 639px)  { .grid { grid-template-columns: 1fr; } }
```



mobile-first: each query ADDS to the previous styles

MOBILE-FIRST LAYERS STYLES UPWARDS

12.2 Media queries

12.2.1 Syntax @media (min-width: 768px)

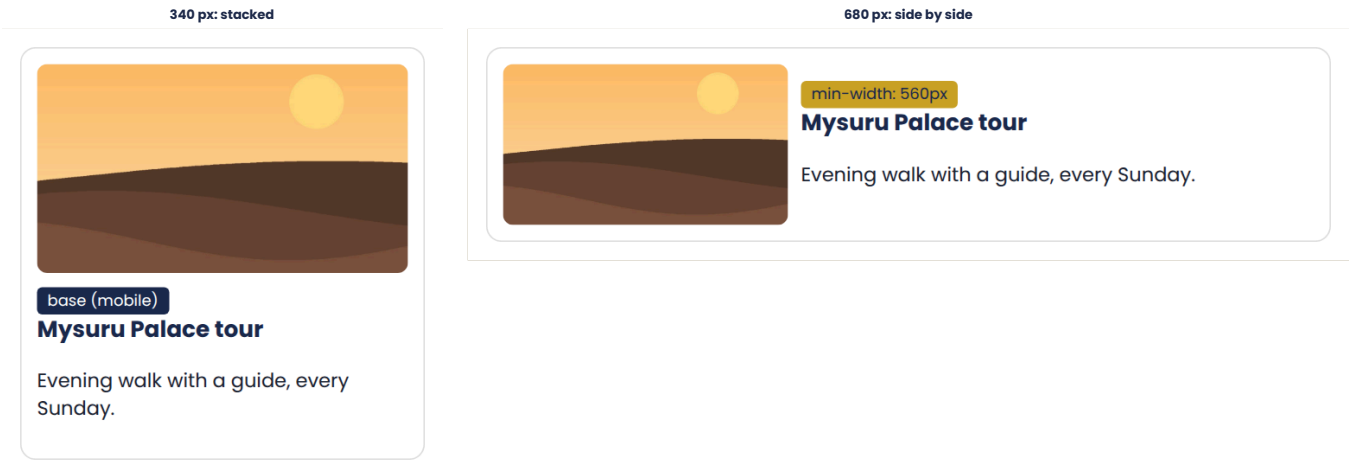
A media query is an at-rule containing normal CSS rules that apply only when its condition is true. It can test the **media type** (`screen`, `print`, `all`) and **media features** in parentheses, combined with `and`, commas (or) and `not`. Media queries do not add specificity: their rules compete normally, so place them **after** the base rules they modify.

MEDIA QUERY SYNTAX CSS

```css
@media (min-width: 768px) { ... }                    /* viewport at least 768px wide       */
@media (max-width: 767.98px) { ... }                 /* below 768px                         */
@media screen and (min-width: 768px) and (max-width: 1023px) { ... }   /* a range (tablets) */
@media (orientation: landscape), (min-width: 1200px) { ... }           /* comma = OR        */
@media not print { ... }                             /* everything except print             */
<link rel="stylesheet" href="wide.css" media="(min-width: 1200px)">   /* in HTML too       */
```

A CARD THAT CHANGES LAYOUT AT 560PX CSS

```css
.card { display: flex; flex-direction: column; gap: 10px; border: 1px solid #ddd; border-radius: 12px;
padding: 12px; font-family: "Poppins", sans-serif; }
.card img { width: 100%; aspect-ratio: 16 / 9; object-fit: cover; border-radius: 8px; }
.card h3 { margin: 0; color: #1a2a4f; }
.label { background: #1a2a4f; color: #fff; display: inline-block; padding: 2px 8px; border-radius:
4px; font-size: 12px; }
.label::after { content: "base (mobile)"; }
@media (min-width: 560px) {
  .card { flex-direction: row; align-items: center; }
  .card img { width: 220px; }
  .label { background: #c9a227; color: #1a2a4f; }
  .label::after { content: "min-width: 560px"; }
}
```



BELOW AND ABOVE THE BREAKPOINT

Which rules apply at a given width can be checked from JavaScript with `matchMedia()`, the same engine CSS uses:

COMPUTED IN CHROMIUM

```
(min-width: 560px)          true
(max-width: 559.98px)       false
(orientation: landscape)    true
(400px <= width <= 800px)   true
(viewport 700 x 600)
```

12.2.2 Range syntax @media (400px <= width <= 800px)

Media Queries Level 4 added a **range syntax** with comparison operators, supported in all modern browsers since 2023. It is easier to read and avoids the "off by one pixel" problem of min/max (where 767px and 768px leave a gap at 767.5px on high-density screens).

Old syntax	Range syntax
`(min-width: 768px)`	`(width >= 768px)`
`(max-width: 767.98px)`	`(width < 768px)`
`(min-width: 400px) and (max-width: 800px)`	`(400px <= width <= 800px)`
`(min-aspect-ratio: 16/9)`	`(aspect-ratio >= 16/9)`

12.2.3 Choosing breakpoints based on content

Device widths change every year, so do not target specific phones. Start with the narrowest layout, widen the browser slowly, and add a breakpoint **where the design starts to look wrong**: lines become too long, cards too wide, or there is room for a second column. Most sites need only two or three. Common starting points (also used by frameworks) are shown below; Tailwind CSS uses 640, 768, 1024, 1280 and 1536px.

Range	Typical devices	Typical layout
below 640px	phones (portrait)	one column, stacked navigation
640px to 1023px	large phones in landscape, tablets	two columns, compact navigation
1024px to 1279px	small laptops, tablets in landscape	full navigation, sidebars
1280px and up	desktops	max-width container, more columns

TIP

Consider em-based breakpoints (`@media (min-width: 40em)`): they move with the user's default font size, so a person who enlarges text gets the more spacious layout sooner.

12.2.4 Orientation, hover and pointer queries

Feature	Values	Use
`orientation`	portrait / landscape	full-screen layouts, video players, galleries
`hover`	hover / none (primary input can or cannot hover)	show hover-only effects only where hovering exists
`pointer`	fine (mouse) / coarse (finger) / none	larger touch targets on touch screens
`any-hover` , `any-pointer`	as above, for any available input	tablets with a keyboard and trackpad
`aspect-ratio` , `height` , `resolution`	ratios, lengths, dppx	short landscape phones; high-density images

INTERACTION MEDIA FEATURES CSS

```css
/* Hover effects only for devices that can hover, so they do not "stick" on phones */
@media (hover: hover) {
  .card:hover { transform: translateY(-4px); box-shadow: 0 10px 24px rgb(0 0 0 / .15); }
}
/* Bigger tap targets for fingers */
@media (pointer: coarse) {
  .btn, .nav a { min-height: 44px; padding-block: 12px; }
}
/* Short landscape screens: hide the large hero image */
@media (orientation: landscape) and (max-height: 500px) {
  .hero { min-height: auto; }
}
```

COMPUTED IN CHROMIUM

```
(hover: hover)      true
(pointer: fine)     true
(pointer: coarse)   false
(any-hover: hover)  true
(desktop Chromium with a mouse)
```

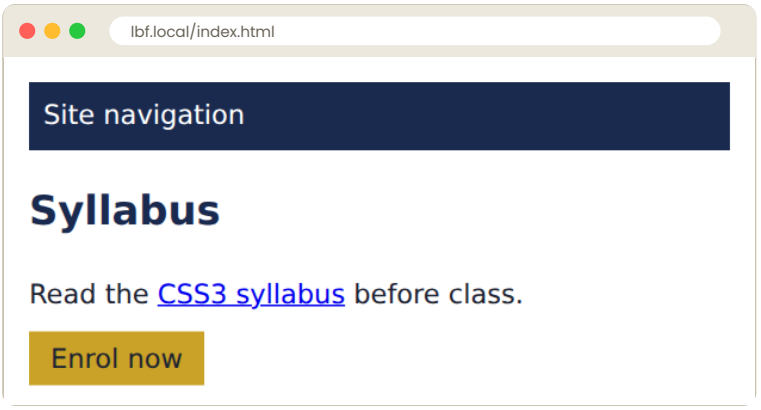
12.2.5 Print styles @media print

When a page is printed or saved as PDF, the browser applies `@media print` rules. Good print styles hide navigation, ads and buttons, use black text on white, show link URLs after the link text, avoid breaking cards or images across pages, and set page margins with `@page` .

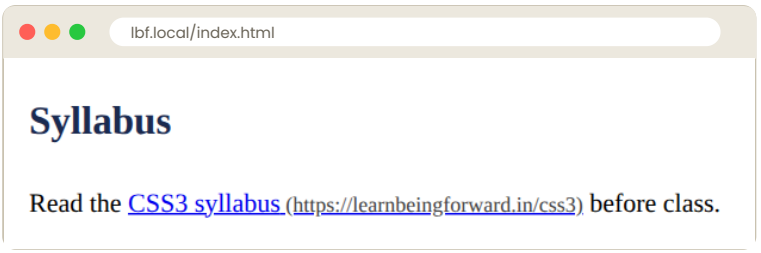
PRINT.CSS

CSS

```css
@media print {
  nav, .no-print, .btn { display: none !important; }
  body { color: #000; background: #fff; font: 11pt/1.5 Georgia, serif; }
  a[href^="http"]::after { content: " (" attr(href) ")"; font-size: 9pt; color: #444; }
  h2, h3 { break-after: avoid; }          /* keep headings with the text that follows */
  .card, img, table { break-inside: avoid; }
  @page { margin: 2cm; }
}
```



ON SCREEN



EMULATING PRINT MEDIA IN CHROMIUM: NAVIGATION AND BUTTON HIDDEN, URL PRINTED AFTER THE LINK, SERIF BLACK TEXT

12.3 User preference queries

Operating systems let people choose dark mode, reduced motion and higher contrast. CSS can read these choices with **user preference media features**; respecting them is both an accessibility requirement and a mark of quality.

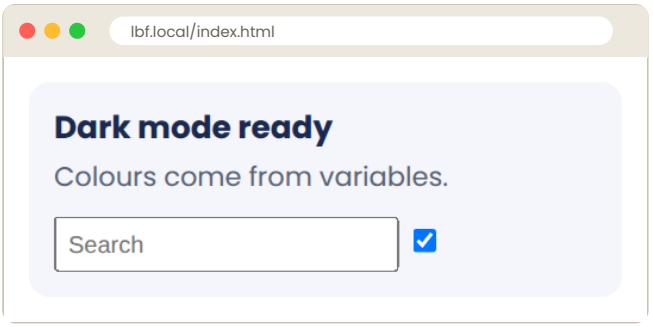
12.3.1 prefers-color-scheme (dark mode)

`prefers-color-scheme: dark` (or `light`) matches the user's system setting. The cleanest implementation defines colours as custom properties (Module 15) and only swaps their values inside the query. Also declare `color-scheme: light dark` so browser-drawn parts (form controls, scrollbars) follow the theme.

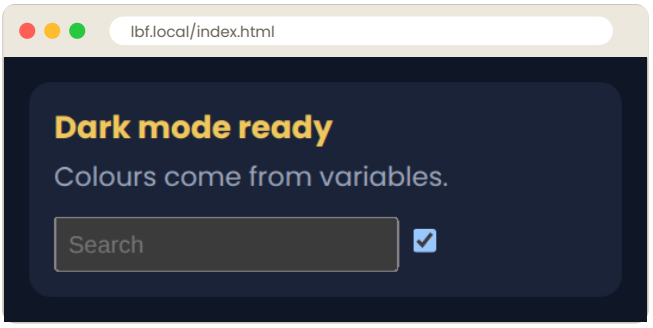
THEME.CSS

CSS

```css
:root { color-scheme: light dark; --bg: #ffffff; --text: #1d2433; --card: #f4f6fb; --accent: #1a2a4f;
--muted: #5c6475; }
@media (prefers-color-scheme: dark) {
  :root { --bg: #0f1626; --text: #e8ecf5; --card: #1a2338; --accent: #f0c75e; --muted: #9aa3b8; }
}
body { background: var(--bg); color: var(--text); font-family: "Poppins", sans-serif; }
.card { background: var(--card); border-radius: 12px; padding: 14px; }
h3 { color: var(--accent); margin: 0 0 4px; } p { color: var(--muted); margin: 0 0 10px; }
input { padding: 6px; }
```



CHROMIUM EMULATING PREFERS-COLOR-SCHEME: LIGHT



CHROMIUM EMULATING PREFERS-COLOR-SCHEME: DARK (NOTE THE DARK FORM CONTROLS FROM COLOR-SCHEME)

12.3.2 prefers-reduced-motion

People with vestibular disorders, migraines or attention difficulties can feel ill from parallax, large zooms and constant animation; they enable "Reduce motion" (macOS, iOS), "Remove animations" (Android) or "Show animations: off" (Windows). `@media (prefers-reduced-motion: reduce)` matches that setting: remove or soften non-essential motion there. Module 14 shows full examples.

TWO WAYS TO RESPECT REDUCED MOTION CSS

```css
/* Opt-in approach: animate only when the user has not asked for less motion */
@media (prefers-reduced-motion: no-preference) {
  .hero-title { animation: slide-in 0.8s ease-out both; }
  html { scroll-behavior: smooth; }
}

/* Or a global safety net */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

COMPUTED IN CHROMIUM

```
(prefers-reduced-motion: reduce) matches: false   (default desktop Chromium, setting off)
```

12.3.3 prefers-contrast

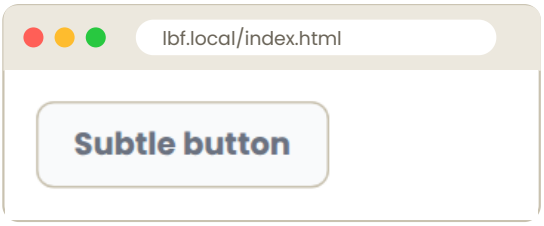
`prefers-contrast: more` matches users who asked the system for higher contrast (macOS "Increase contrast", for example): strengthen borders, remove translucent backgrounds and subtle greys. A related feature, `forced-colors: active`, matches Windows High Contrast / Contrast themes, where the browser

replaces your colours with the user's palette; use system colour keywords such as `CanvasText` and `ButtonText` to keep custom widgets visible there.

HIGHER CONTRAST ON REQUEST

CSS

```
.btn { border: 1px solid #c9c2b0; color: #6b7280; background: #f9fafb; padding: 8px 16px; border-radius: 8px; font: 600 14px "Poppins", sans-serif; }
@media (prefers-contrast: more) {
  .btn { border: 2px solid #000; color: #000; background: #fff; }
}
```



DEFAULT

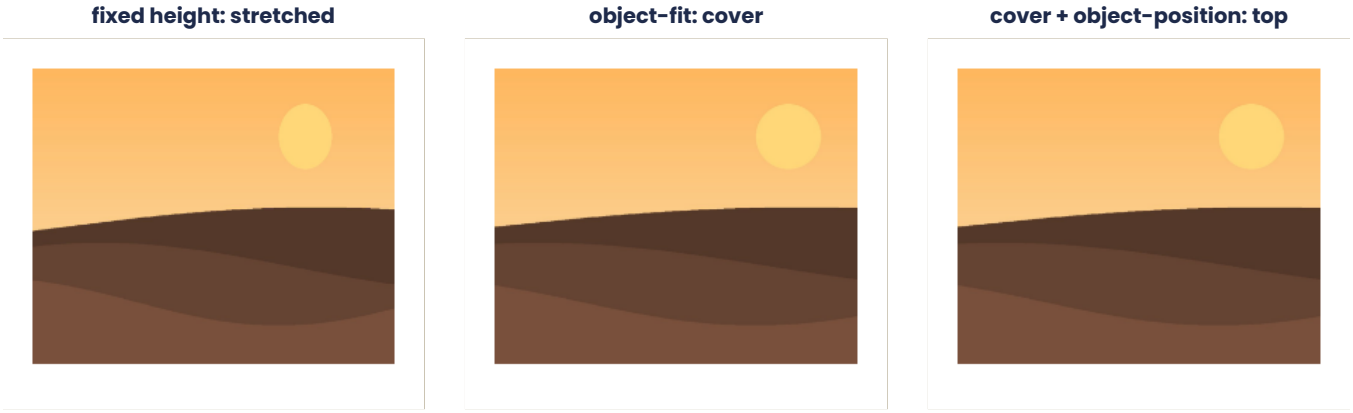


CHROMIUM EMULATING PREFERS-CONTRAST: MORE

12.4 Responsive techniques

12.4.1 Responsive images with max-width: 100% and object-fit

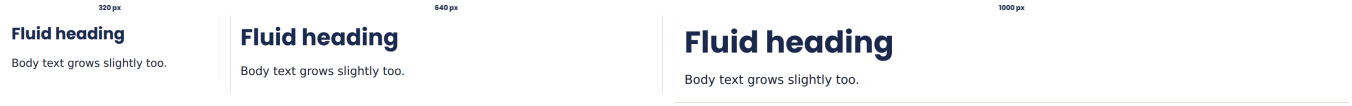
`img { max-width: 100%; height: auto; }` lets images shrink with their container but never grow beyond their natural size. When an image must fill a box of a fixed shape (card thumbnails, avatars, heroes), give the box the size or aspect-ratio and use `object-fit: cover`, which crops instead of distorting; `object-position` chooses which part stays visible. Use `srcset` / `sizes` or `<picture>` (HTML course, Module 13) so phones download smaller files.



FILLING A BOX WITHOUT DISTORTION

12.4.2 Fluid typography

Instead of jumping between font sizes at breakpoints, text can scale smoothly with the viewport using `clamp(min, preferred, max)` (Module 15): `font-size: clamp(1.75rem, 1rem + 3vw, 3rem)` is never smaller than 1.75rem, never larger than 3rem, and grows with the window in between. Including a rem part in the preferred value keeps zoom working.



THE HEADING SIZE CHANGES CONTINUOUSLY WITH THE VIEWPORT

COMPUTED IN CHROMIUM

at 640px wide, font-size = 33.6px

12.4.3 Responsive navigation (hamburger menu)

On narrow screens the menu collapses behind a **hamburger button**. A robust version uses a real `<button>` with `aria-expanded` and `aria-controls` and three lines of JavaScript to toggle it (a "checkbox hack" works without JavaScript but is less accessible). The menu is always visible from the breakpoint upwards.

NAV.CSS CSS

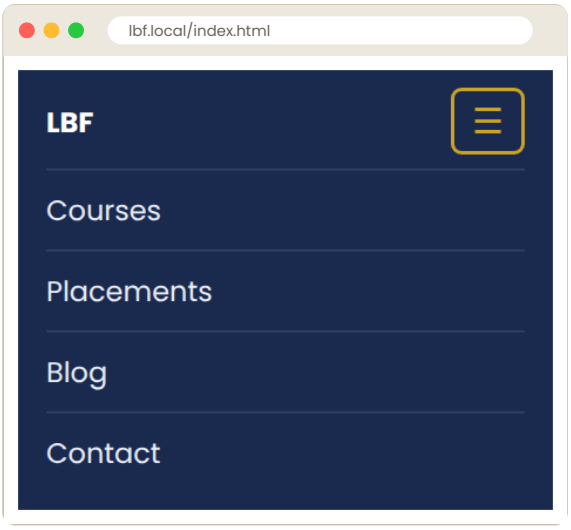
```css
.nav { display: flex; flex-wrap: wrap; align-items: center; justify-content: space-between;
background: #1a2a4f; padding: 10px 16px; font-family: "Poppins", sans-serif; }
.logo { color: #fff; font-weight: 700; }
.toggle { background: none; border: 2px solid #c9a227; color: #c9a227; border-radius: 6px; font-size:
20px; width: 42px; height: 38px; }
.menu { display: none; flex-basis: 100%; list-style: none; margin: 8px 0 0; padding: 0; }
.menu a { display: block; color: #e8ecf5; padding: 10px 0; border-top: 1px solid rgb(255 255 255 /
.12); text-decoration: none; }
.toggle[aria-expanded="true"] + .menu { display: block; }
@media (min-width: 640px) {
  .toggle { display: none; }
  .menu, .toggle[aria-expanded] + .menu { display: flex; flex-basis: auto; gap: 22px; margin: 0; }
  .menu a { border: 0; padding: 0; }
}
```

NAV.HTML HTML

```html
<nav class="nav">
  <span class="logo">LBF</span>
  <button class="toggle" aria-expanded="false" aria-controls="menu" aria-label="Menu">☰</button>
  <ul class="menu" id="menu"><li><a href="#">Courses</a></li><li><a href="#">Placements</a></li><li><a
href="#">Blog</a></li><li><a href="#">Contact</a></li></ul>
</nav>
<script>
  const t = document.querySelector(".toggle");
  t.addEventListener("click", () => t.setAttribute("aria-expanded", t.getAttribute("aria-expanded")
=== "false"));
</script>
```



HAMBURGER BELOW 640PX, INLINE MENU ABOVE



320 PX AFTER TAPPING THE HAMBURGER: ARIA-EXPANDED="TRUE" REVEALS THE MENU

12.4.4 Responsive tables

Wide tables do not fit phones. Three techniques: wrap the table in a container with `overflow-x: auto` (simplest, keeps the table semantics; Module 7); hide less important columns at small widths; or turn each row into a "card" below a breakpoint, using `data-label` attributes and `::before` to show the column headers.

ROWS BECOME CARDS ON PHONES CSS

```
table { border-collapse: collapse; width: 100%; font: 14px "Poppins", sans-serif; }
th, td { padding: 8px; border-bottom: 1px solid #e5e5e5; text-align: left; }
th { background: #1a2a4f; color: #fff; }
@media (max-width: 520px) {
  thead { position: absolute; width: 1px; height: 1px; overflow: hidden; clip-path: inset(50%); }  /*
visually hidden */
  tr { display: block; border: 1px solid #ddd; border-radius: 10px; margin-bottom: 10px; padding: 4px
10px; }
  td { display: flex; justify-content: space-between; border: 0; padding: 4px 0; }
  td::before { content: attr(data-label); font-weight: 700; color: #1a2a4f; }
}
```

340 px: cards

Course	HTML5
Duration	28 days
Mode	Online
Fee	₹9,999

Course	CSS3
Duration	39 days
Mode	Classroom
Fee	₹11,999

640 px: table

Course	Duration	Mode	Fee
HTML5	28 days	Online	₹9,999
CSS3	39 days	Classroom	₹11,999

A RESPONSIVE TABLE

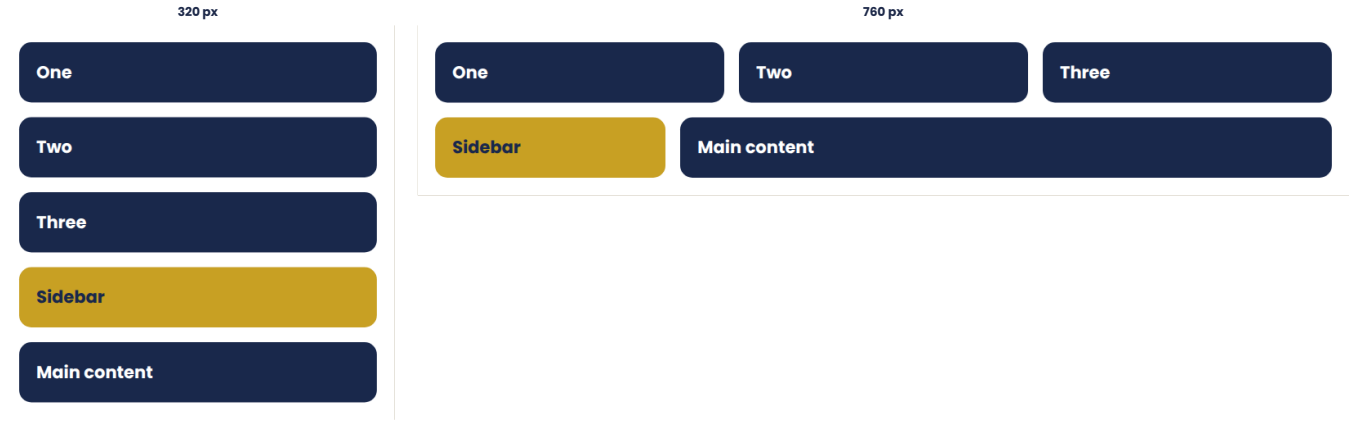
12.4.5 Intrinsic layouts with Flexbox and Grid

Intrinsic web design (a term coined by Jen Simmons) lets content and the layout algorithms decide, so many components need **no media queries at all**: `flex-wrap` with a flex-basis, Grid's `repeat(auto-fit,`

`minmax())`, `clamp()` for sizes and container queries (Module 16) for components. Media queries remain for page-level changes such as the navigation.

RESPONSIVE WITHOUT MEDIA QUERIES CSS

```
.wrap { display: flex; flex-wrap: wrap; gap: 12px; }
.wrap > * { flex: 1 1 200px; }   /* "at least about 200px, otherwise share" */
.box { background: #1a2a4f; color: #fff; padding: 14px; border-radius: 10px; font: 600 14px "Poppins",
sans-serif; }
.sidebar-layout { display: flex; flex-wrap: wrap; gap: 12px; margin-top: 12px; }
.sidebar-layout > :first-child { flex: 1 1 160px; background: #c9a227; color: #1a2a4f; }
.sidebar-layout > :last-child { flex: 999 1 280px; }   /* "the sidebar pattern": main grows, wraps
under when narrow */
```

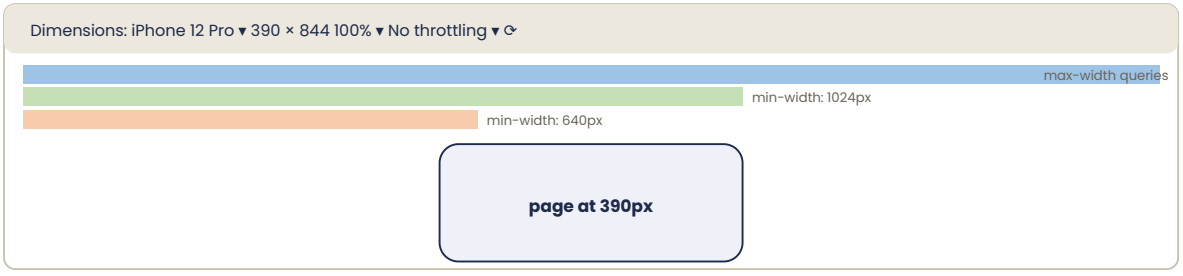


BOTH COMPONENTS REARRANGE PURELY FROM FLEX-BASIS AND WRAPPING

12.5 Testing

12.5.1 DevTools device mode

Press **Ctrl+Shift+M** (Cmd+Shift+M) in Chrome DevTools, or click the phone/tablet icon, to enter **device mode**: choose a device preset or drag the viewport edges; switch orientation; simulate touch, device pixel ratio and slow networks (Network throttling). The "Show media queries" option (in the device toolbar menu) draws your breakpoints as coloured bars you can click. The **Rendering** panel (Ctrl+Shift+P, "Show Rendering") emulates prefers-color-scheme, prefers-reduced-motion, prefers-contrast, forced colours and print media, exactly what was used for the screenshots in this module.



DEVICE MODE WITH THE MEDIA-QUERY BARS

12.5.2 Testing on real devices

Emulation is not the real thing: touch behaviour, real performance on a budget Android phone, mobile browser toolbars (the 100vh problem), on-screen keyboards covering inputs, Safari-specific bugs and fonts can only be checked on devices. Practical options:

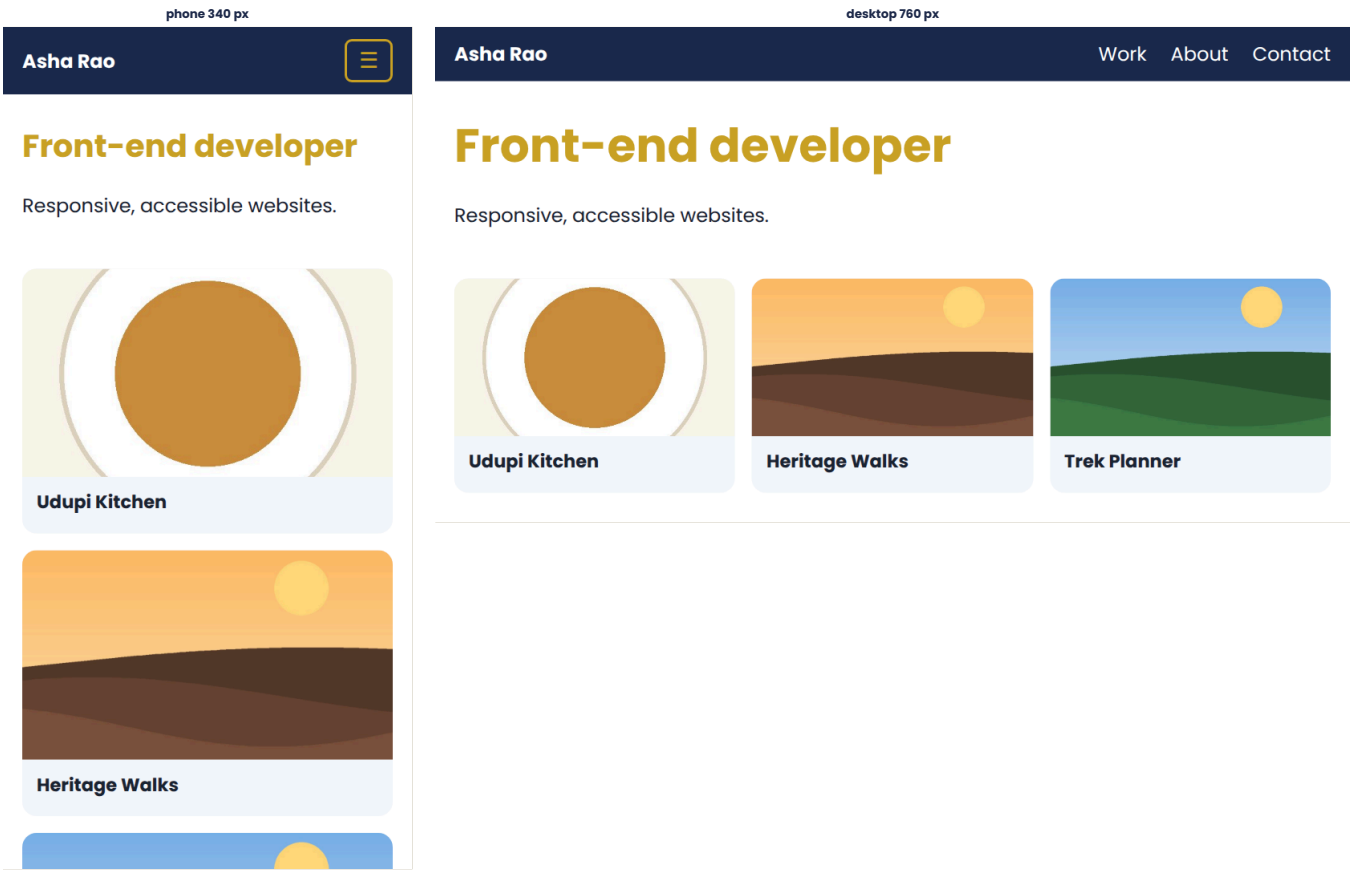
- Open your local site on your own phone: run Live Server or `npx serve`, then visit `http://<your-computer's-IP>:5500` on the same Wi-Fi.
- Remote debugging: connect an Android phone by USB and open `chrome://inspect` on the computer to use full DevTools on the phone's page; Safari offers the same for iPhones via the Develop menu on a Mac.
- Cloud device labs such as BrowserStack or LambdaTest provide real iOS and Android devices in the browser.
- Test at least: a small Android phone, an iPhone, a tablet, a laptop, 200% browser zoom and landscape orientation.

PRACTICE CHECKPOINT

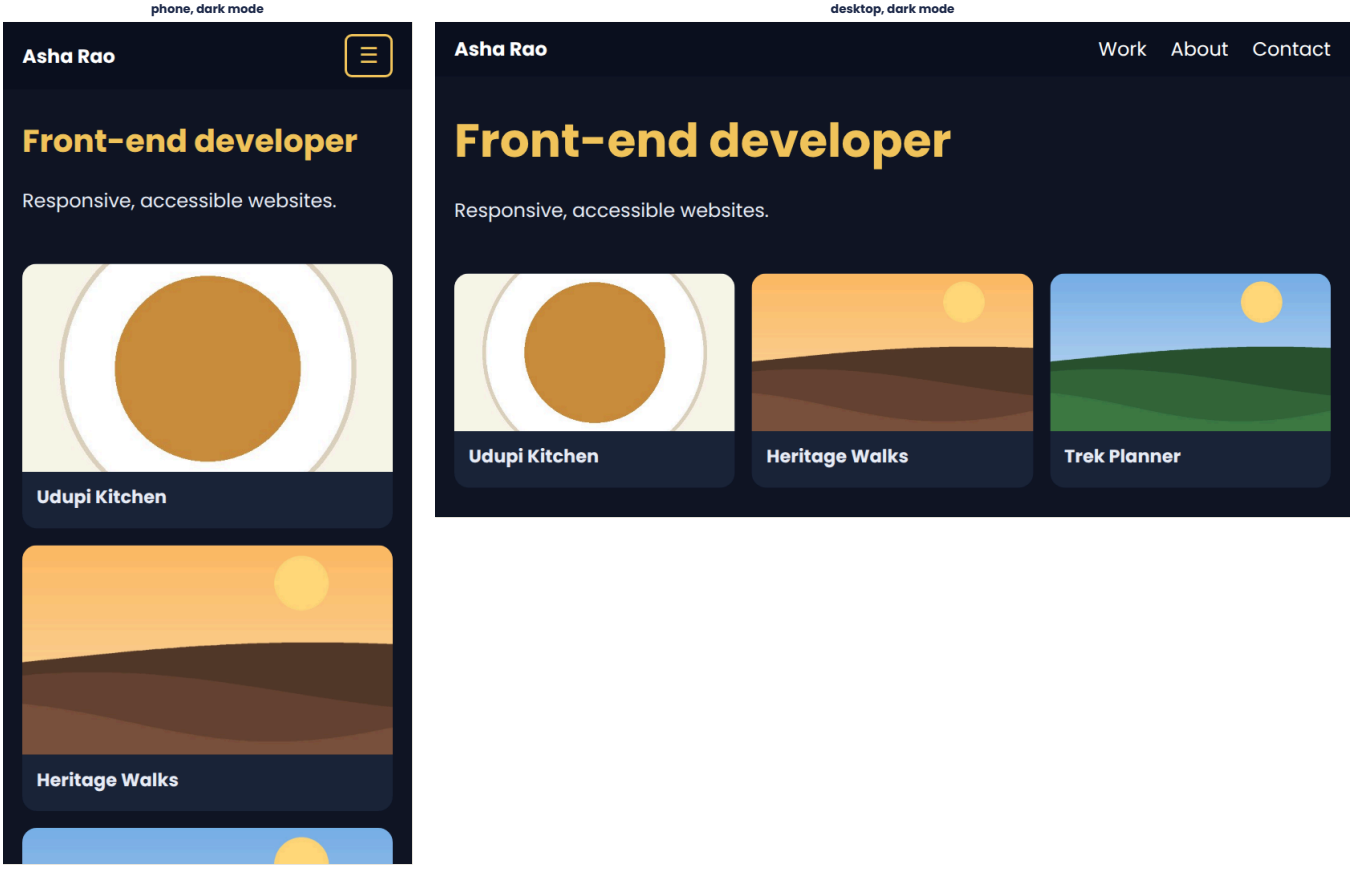
Make the **portfolio site fully responsive** (mobile-first) with a **hamburger menu**, a **responsive grid**, a **dark mode** and a **print stylesheet**.

PORTFOLIO.CSS: SAMPLE SOLUTION CSS

```
:root { color-scheme: light dark; --bg: #fff; --text: #1d2433; --card: #f4f6fb; --brand: #1a2a4f; --
accent: #c9a227; }
@media (prefers-color-scheme: dark) { :root { --bg: #0f1626; --text: #e8ecf5; --card: #1a2338; --
brand: #0b1120; --accent: #f0c75e; } }
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; background: var(--bg); color: var(--text); font-family: "Poppins", sans-serif; }
img { max-width: 100%; display: block; }
.nav { display: flex; flex-wrap: wrap; justify-content: space-between; align-items: center; padding:
10px 16px; background: var(--brand); color: #fff; }
.toggle { background: none; border: 2px solid var(--accent); color: var(--accent); border-radius: 6px;
width: 40px; height: 36px; font-size: 18px; }
.menu { display: none; flex-basis: 100%; list-style: none; padding: 0; margin: 6px 0 0; }
.menu a { display: block; color: #fff; padding: 8px 0; text-decoration: none; }
.toggle[aria-expanded="true"] + .menu { display: block; }
.hero { padding: 24px 16px; } .hero h1 { font-size: clamp(1.6rem, 1rem + 3vw, 2.6rem); margin: 0;
color: var(--accent); }
.projects { display: grid; gap: 14px; padding: 0 16px 24px; grid-template-columns: repeat(auto-fit,
minmax(min(220px, 100%), 1fr)); }
.project { background: var(--card); border-radius: 12px; overflow: hidden; }
.project img { aspect-ratio: 16 / 9; object-fit: cover; width: 100%; } .project h3 { margin: 10px 12px
14px; font-size: 15px; }
@media (min-width: 640px) {
  .toggle { display: none; }
  .menu, .toggle[aria-expanded] + .menu { display: flex; gap: 20px; flex-basis: auto; margin: 0; }
  .menu a { padding: 0; }
}
@media print {
  .nav { display: none; }
  body { background: #fff; color: #000; } .project { break-inside: avoid; border: 1px solid #999; }
}
```



LIGHT THEME AT TWO WIDTHS



THE SAME PAGE WITH PREFERS-COLOR-SCHEME: DARK EMULATED

MODULE 13 · INTERMEDIATE

Transforms and transitions

Minimum duration **2 days** | Level **Intermediate**

Transforms move, rotate, scale and skew elements; transitions animate changes between states smoothly. This module covers the 2D transform functions translate(), rotate(), scale() and skew(), combining them and why their order matters, transform-origin and the individual translate, rotate and scale properties; 3D transforms with perspective, rotateX(), rotateY(), translateZ(), transform-style: preserve-3d and backface-visibility; and transitions: property, duration, delay, timing functions (ease, linear, ease-in-out, cubic-bezier(), steps()), the shorthand, which properties can animate, and transitioning to and from display: none with @starting-style. It ends with practical button, card, image-zoom and 3D flip-card effects. Transitions are shown as frames frozen at exact times in Chromium.

IN THIS MODULE

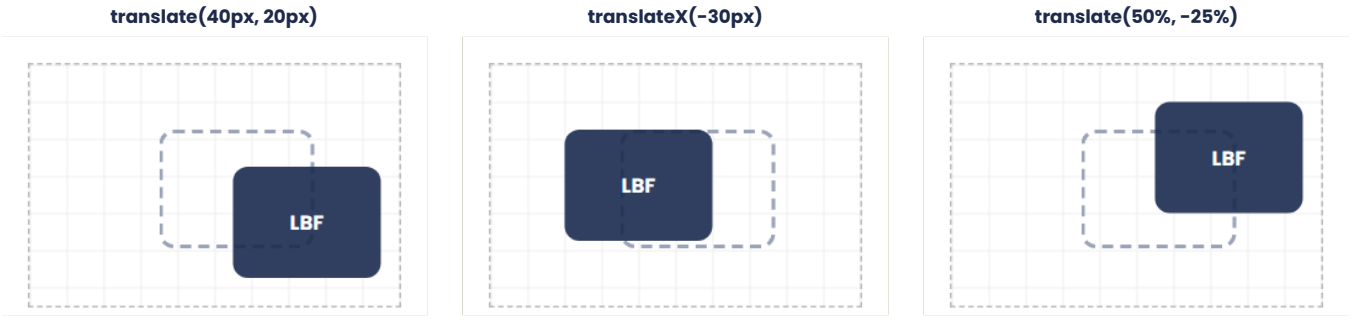
13.1 2D transforms
13.2 3D transforms
13.3 Transitions
13.4 Practical effects

13.1 2D transforms

The `transform` property changes how an element is **drawn** without changing its place in the layout: other elements do not move, as if the element still occupied its original box. Transforms are performed by the GPU in the compositing stage, so animating them is smooth and cheap (Module 20). In the figures below, the dashed outline is the element's original position on a 20px grid.

13.1.1 translate()

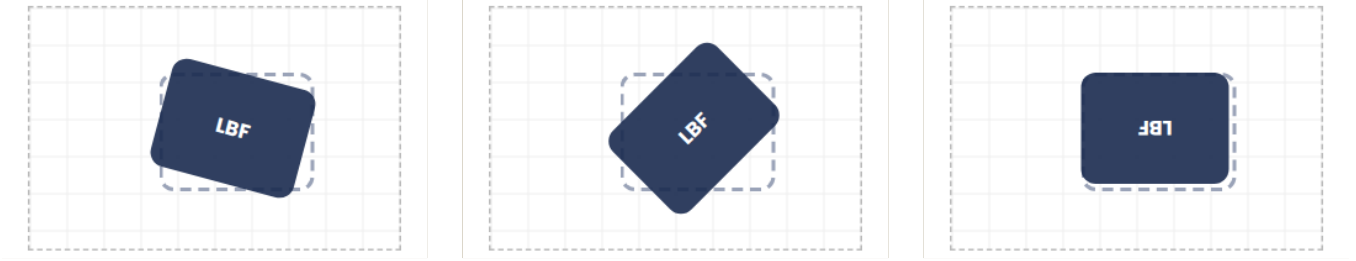
`translate(x, y)` moves the element. `translateX()` and `translateY()` move along one axis. Percentages refer to the **element's own size** (not the parent), which is why `translate(-50%, -50%)` centres an absolutely positioned element of unknown size (Module 7).



13.1.2 rotate()

`rotate(angle)` turns the element clockwise for positive angles. Units: `deg` (360 per turn), `turn` (1turn = 360deg), `rad`, `grad`.

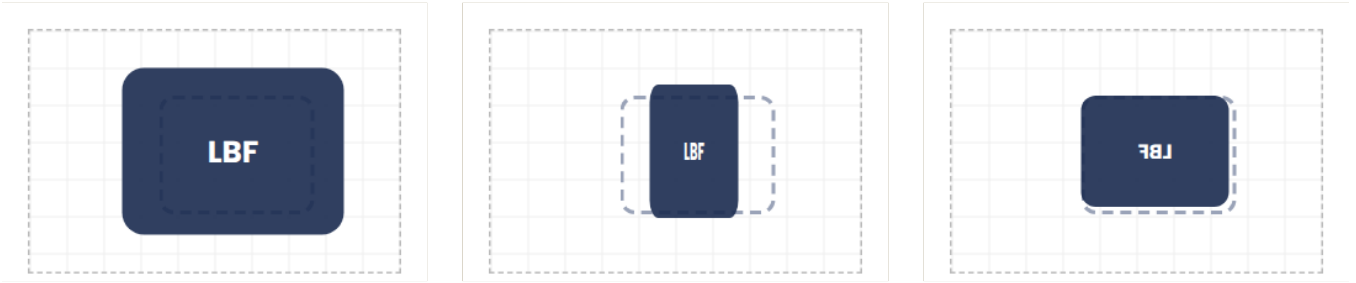
rotate(15deg) rotate(-45deg) rotate(0.5turn)



13.1.3 scale()

`scale(sx, sy)` multiplies the size: 1 is unchanged, 2 doubles, 0.5 halves, and a negative value mirrors (`scaleX(-1)` flips horizontally). One value scales both axes. Text and borders scale too.

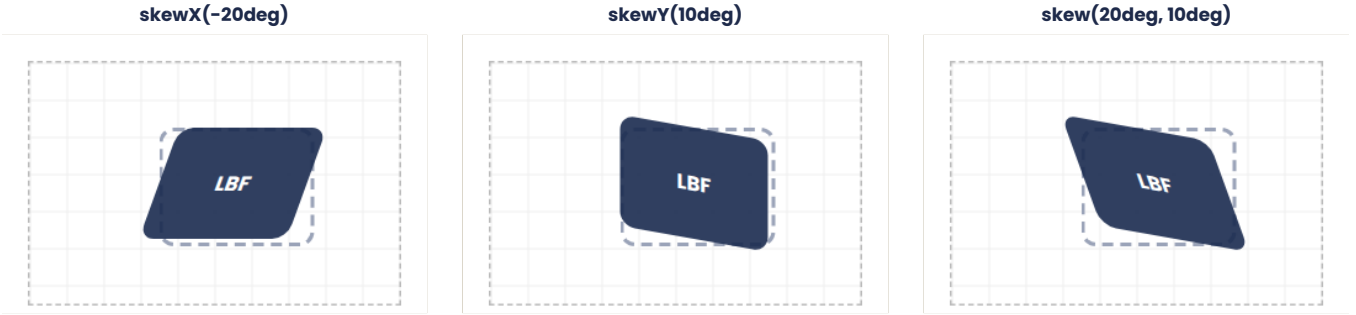
scale(1.5) scale(0.6, 1.2) scaleX(-1)



13.1.4 skew()

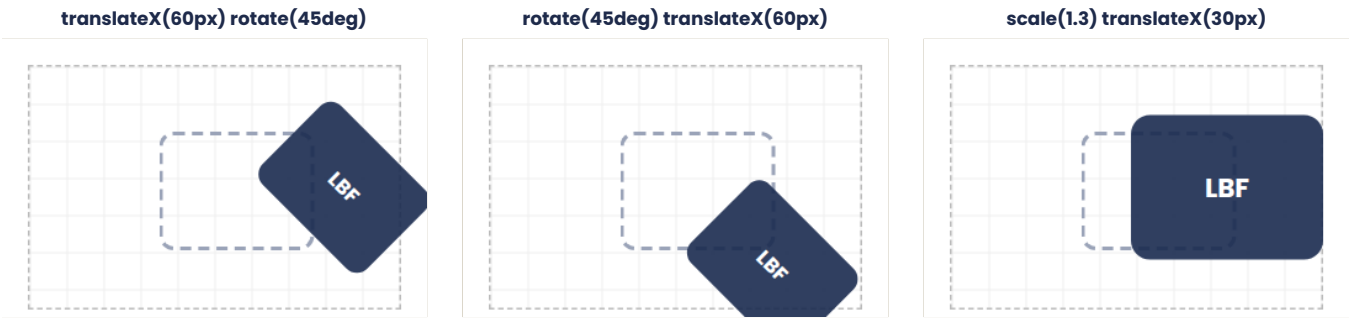
`skew(ax, ay)` slants the element along the X and/or Y axis by the given angles, producing parallelograms: used for dynamic banners and "speed" effects. Skew the container and counter-skew its text to keep the text upright.

skewX(-20deg) skewY(10deg) skew(20deg, 10deg)



13.1.5 Combining transforms and order

Several functions can be listed in one `transform`, separated by spaces. They are applied **from right to left** in terms of the coordinate system: each function transforms the coordinate system for the ones written after it. Therefore **order matters**: rotating then translating moves the element along the rotated axis.



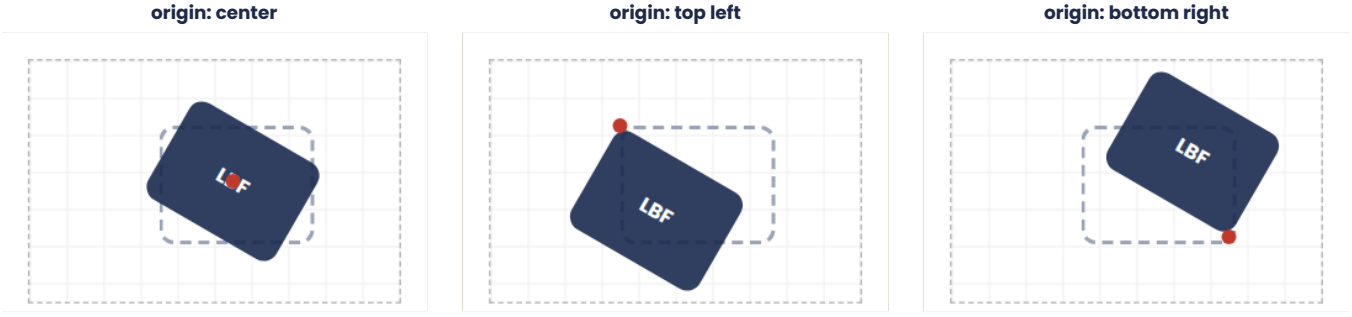
In the second case, the element was rotated first (in reading order), so "move right 60px" happened along the 45-degree axis, down and to the right. Note also that writing a new `transform` (for example on `:hover`) **replaces** the whole list: `.card:hover { transform: scale(1.1); }` removes a `rotate()` that was set on `.card`.

COMPUTED IN CHROMIUM

```
.a  matrix(0.707107, 0.707107, -0.707107, 0.707107, 60, 0)
    centre moved to (165.0, 105.0) from (105, 105)
.b  matrix(0.707107, 0.707107, -0.707107, 0.707107, 42.4264, 42.4264)
    centre moved to (147.4, 147.4) from (105, 105)
```

13.1.6 transform-origin

Rotation, scaling and skewing happen around the **transform origin**, by default the centre (`50% 50%`). `transform-origin` moves it: keywords (`top left`, `bottom center`), lengths or percentages. A door swings around its hinge (`left`), a needle around its base, a dropdown grows from its top.



ROTATE(30DEG) AROUND THREE DIFFERENT ORIGINS (RED DOT)

13.1.7 Individual transform properties: translate, rotate, scale

Since 2022 every browser also supports separate `translate`, `rotate` and `scale` **properties**. They can be set, overridden and animated independently (a hover can change only `scale` without wiping out a `rotate`). They are always applied in the fixed order translate, then rotate, then scale, then the `transform` property.

INDEPENDENT TRANSFORM PROPERTIES

CSS

```css
.badge { rotate: -8deg; }                /* keeps its tilt...                */
.badge:hover { scale: 1.15; }            /* ...while scaling on hover        */

/* With the transform property you would have to repeat everything: */
.badge2 { transform: rotate(-8deg); }
.badge2:hover { transform: rotate(-8deg) scale(1.15); }
```

ROTATE AND SCALE AS SEPARATE PROPERTIES CSS

```
.badge { display: inline-block; background: #c0392b; color: #fff; font: 700 14px "Poppins", sans-serif; padding: 8px 14px; border-radius: 8px; margin: 20px; rotate: -8deg; transition: scale .2s; }
.badge:hover { scale: 1.25; }
```

INDEX.HTML HTML

```
<span class="badge">SALE</span><span class="badge" id="h">SALE (hovered)</span>
```

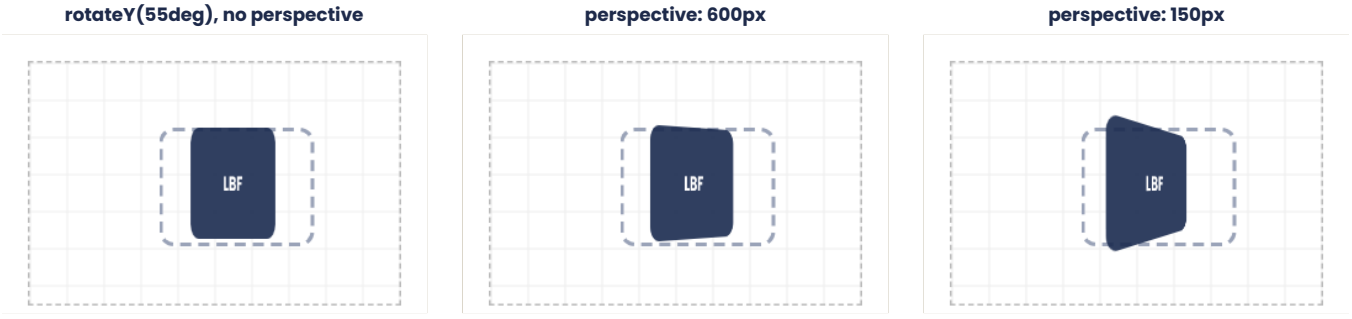


HOVERING SCALED THE BADGE WHILE KEEPING ITS ROTATION

13.2 3D transforms

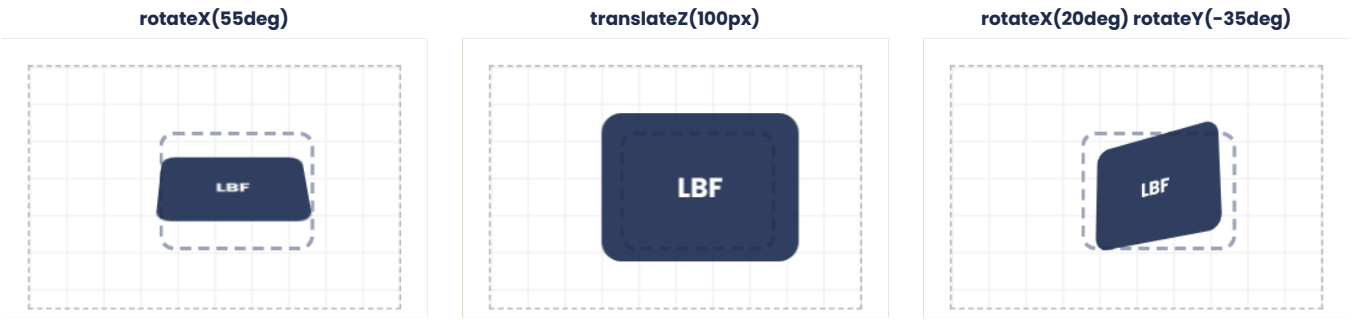
13.2.1 perspective

3D transforms rotate and move elements in depth (the Z axis, pointing towards the viewer). Without **perspective**, 3D rotations look flat (like a squashed 2D shape). Perspective is the distance between the viewer and the z = 0 plane: small values (200px) give dramatic depth, large values (1500px) subtle depth. It can be set on the **parent** with the `perspective` property (shared vanishing point for all children) or per element with the `perspective()` function inside transform.



13.2.2 rotateX(), rotateY(), translateZ()

Function	Effect
`rotateX(a)`	tilt around the horizontal axis (top edge goes away from or towards you)
`rotateY(a)`	turn around the vertical axis, like a door or a page
`rotateZ(a)`	same as 2D rotate()
`translateZ(z)`	move towards (positive) or away from (negative) the viewer: looks bigger or smaller with perspective
`rotate3d(x, y, z, a)`, `translate3d()`, `scale3d()`, `matrix3d()`	general forms



ALL WITH PERSPECTIVE: 400PX ON THE STAGE

13.2.3 transform-style: preserve-3d and backface-visibility

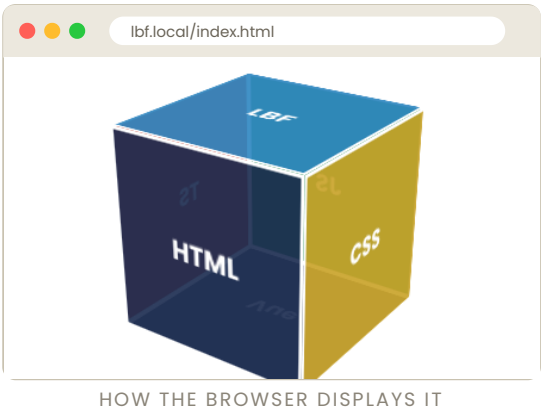
By default, the children of a 3D-transformed element are flattened into its plane. `transform-style: preserve-3d` on the parent keeps them in a shared 3D space, so you can build real 3D objects such as cubes and flip cards. `backface-visibility: hidden` hides an element when it is turned away from the viewer (rotated more than 90 degrees), which is how the two faces of a flip card take turns.

A 3D CUBE FROM SIX FACES WITH PRESERVE-3D CSS

```
.scene { width: 120px; height: 120px; perspective: 500px; margin: 30px auto; }
.cube { width: 100%; height: 100%; position: relative; transform-style: preserve-3d; transform:
rotateX(-25deg) rotateY(-35deg); }
.face { position: absolute; inset: 0; display: grid; place-items: center; font: 700 16px "Poppins",
sans-serif; color: #fff; border: 2px solid #fff; }
.front  { background: rgb(26 42 79 / .9);  transform: translateZ(60px); }
.right  { background: rgb(201 162 39 / .9); transform: rotateY(90deg) translateZ(60px); }
.top    { background: rgb(46 134 193 / .9); transform: rotateX(90deg) translateZ(60px); }
.back   { background: rgb(30 132 73 / .9);  transform: rotateY(180deg) translateZ(60px); }
.left   { background: rgb(192 57 43 / .9);  transform: rotateY(-90deg) translateZ(60px); }
.bottom { background: rgb(92 100 117 / .9); transform: rotateX(-90deg) translateZ(60px); }
```

INDEX.HTML HTML

```
<div class="scene"><div class="cube"><div class="face front">HTML</div><div class="face back">JS</div>
<div class="face right">CSS</div><div class="face left">TS</div><div class="face top">LBF</div><div
class="face bottom">Vue</div></div></div>
```



HOW THE BROWSER DISPLAYS IT

13.3 Transitions

Without transitions, a property change (on hover, on focus, when a class is added by JavaScript) happens instantly. A **transition** tells the browser to **interpolate** from the old value to the new one over

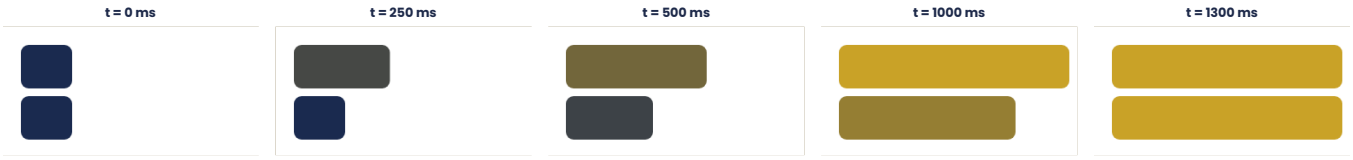
time. The transition is declared on the element's **base** state so it runs both ways (in and out).

13.3.1 transition-property, transition-duration, transition-delay

Property	Value	Default
`transition-property`	which properties to animate: names, comma-separated, or `all`	`all`
`transition-duration`	how long: `0.3s` or `300ms`	`0s` (no transition)
`transition-delay`	wait before starting; negative starts part-way through	`0s`
`transition-timing-function`	the speed curve (13.3.2)	`ease`
`transition-behavior`	`allow-discrete` for display and similar (13.3.5)	`normal`

A 1-SECOND LINEAR TRANSITION OF WIDTH AND COLOUR CSS

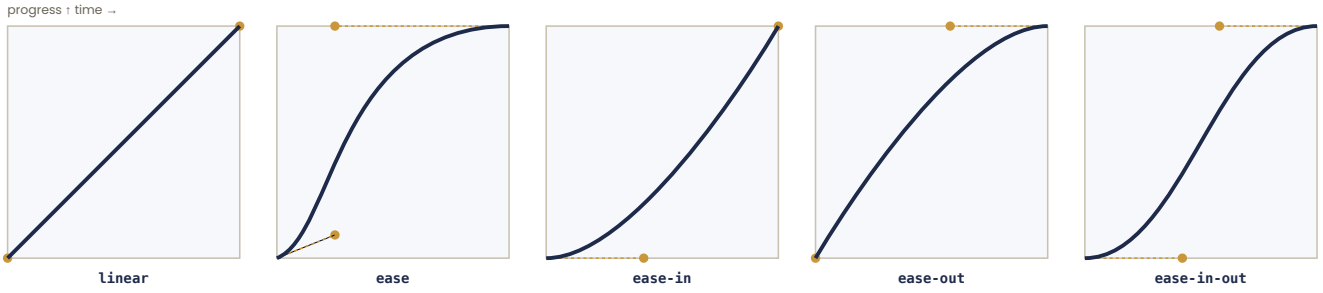
```
.bar { width: 40px; height: 34px; background: #1a2a4f; border-radius: 6px; margin: 6px 0;
       transition-property: width, background-color; transition-duration: 1000ms; transition-timing-
function: linear; }
.bar.go { width: 180px; background-color: #c9a227; }
.d { transition-delay: 300ms; }
```



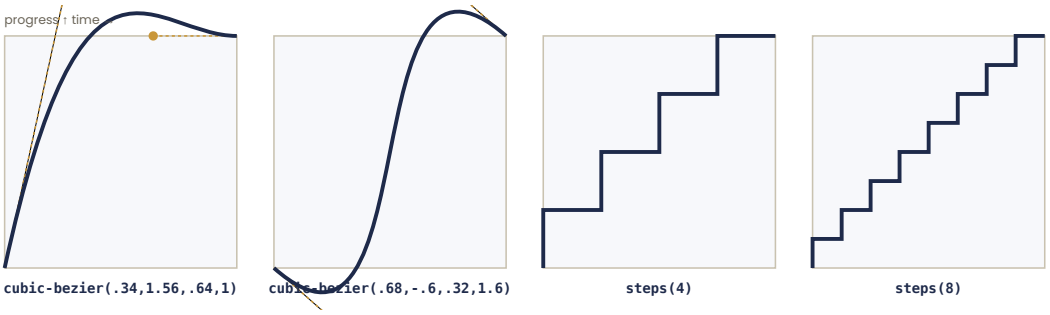
FRAMES OF THE TRANSITION FROZEN IN CHROMIUM: TOP BAR WITHOUT DELAY, BOTTOM BAR WITH TRANSITION-DELAY: 300MS

13.3.2 Timing functions: ease, linear, ease-in-out, cubic-bezier(), steps()

The **timing function** (easing) maps elapsed time to progress. Natural motion rarely moves at constant speed: things accelerate and decelerate. The keywords are predefined cubic Bézier curves; `cubic-bezier(x1, y1, x2, y2)` defines your own (y values outside 0–1 create overshoot and "bounce back" effects); `steps(n)` jumps in n discrete steps (sprite animations, typewriter effects); `linear(...)` (Baseline 2023) approximates any curve, such as a spring or bounce, with points.

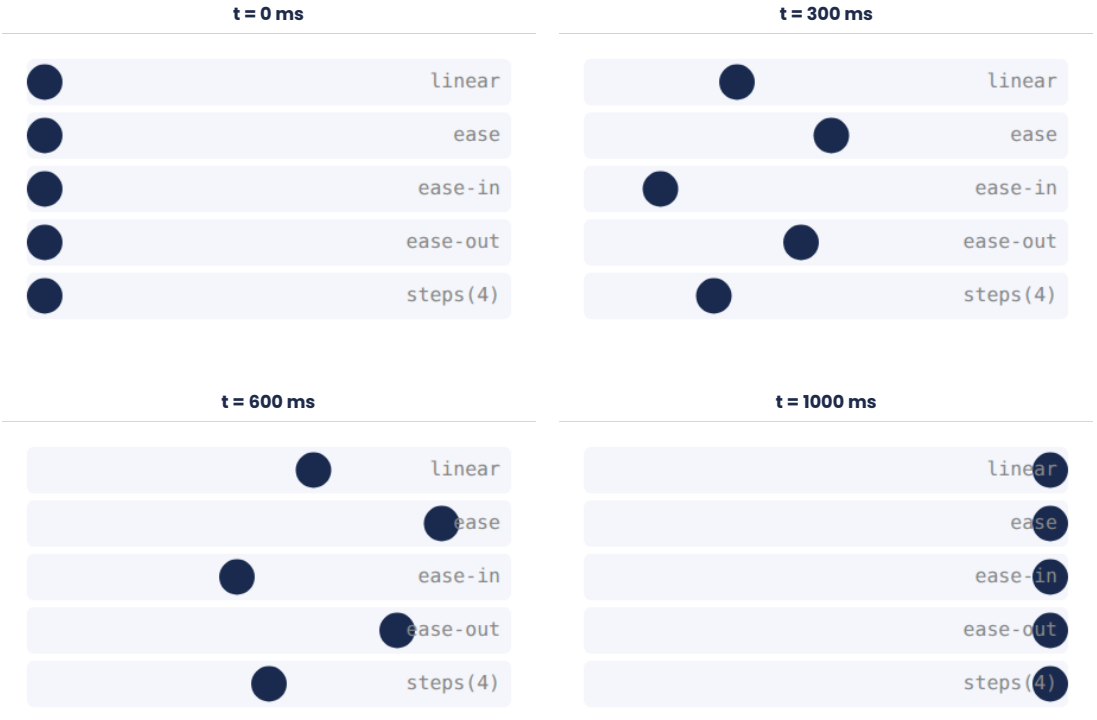


THE KEYWORD TIMING FUNCTIONS AS CUBIC BÉZIER CURVES (CONTROL POINTS IN GOLD)



CUSTOM CURVES WITH OVERSHOOT, AND STEPPED TIMING

Timing	Feels like	Use for
`ease` (default)	quick start, gentle stop	general UI
`ease-out`	fast then slowing: arrives softly	elements entering the screen, hover feedback
`ease-in`	slow then fast: leaves quickly	elements leaving the screen
`ease-in-out`	slow-fast-slow	moving between two on-screen positions
`linear`	mechanical	colour fades, progress bars, rotations of spinners
`cubic-bezier()` with overshoot	springy, playful	pop-ins, toggles



FIVE TIMING FUNCTIONS, SAME 1-SECOND DURATION, FROZEN AT FOUR MOMENTS

13.3.3 transition shorthand

`transition: property duration timing-function delay;` The first time value is always the duration and the second the delay. Separate multiple transitions with commas. Prefer listing specific properties to `all`: `all` can animate things you did not intend (layout properties), and costs more.

TRANSITION SHORTHAND

CSS

```
.btn { transition: background-color 200ms ease-out, transform 150ms ease-out; }
.menu { transition: opacity .25s, visibility .25s; }
.panel { transition: transform .4s cubic-bezier(.2, .8, .2, 1) .1s; }   /* with a 0.1s delay */
```

13.3.4 Which properties can be animated

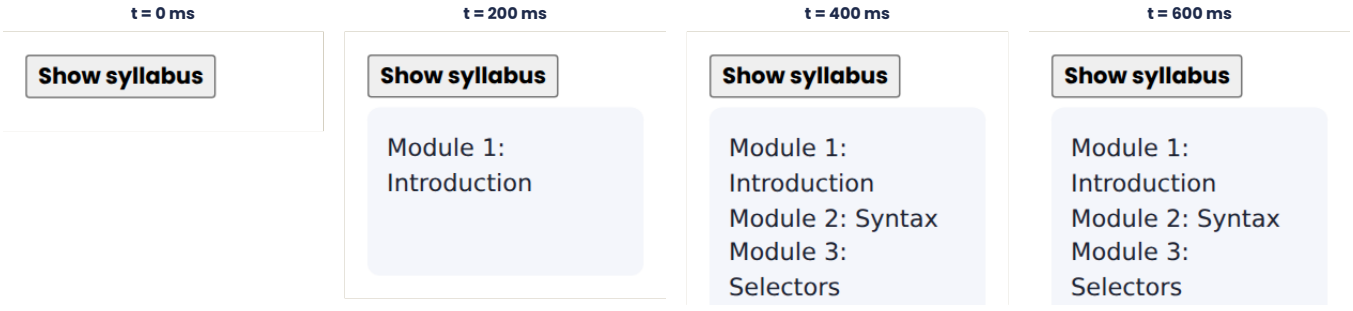
A property can transition if the browser can compute intermediate values: numbers, lengths, percentages, colours, transforms, shadows, filters, opacity, many grid track sizes and custom properties registered with `@property` (Module 15). Discrete properties such as `display`, `visibility` or `font-family` flip from one value to the other (at 50 percent by default). `height: auto` could not be transitioned until recently; `interpolate-size: allow-keywords` (Chromium 129+) and `calc-size()` now allow it, with grid `0fr → 1fr` as a cross-browser trick.

Animate cheaply (compositor only)	Animatable but costly (layout or paint)	Not interpolated (discrete)
`transform`, `translate`, `rotate`, `scale`	`width`, `height`, `margin`, `padding`, `top/left`	`display`, `visibility`
`opacity`	`box-shadow`, `filter`, `background`	`font-family`, `position`
	`font-size`, `border-width`, `color`	unregistered custom properties

ANIMATING TO "AUTO" HEIGHT WITH GRID 0FR → 1FR

CSS

```
.acc { display: grid; grid-template-rows: 0fr; transition: grid-template-rows 600ms ease; background:
#f4f6fb; border-radius: 8px; }
.acc.open { grid-template-rows: 1fr; }
.acc > div { overflow: hidden; padding: 0 12px; }
button { font: 600 14px "Poppins", sans-serif; margin-bottom: 6px; }
```



THE PANEL EXPANDS SMOOTHLY TO ITS NATURAL HEIGHT

13.3.5 Transitioning to and from display: none with @starting-style (overview)

Two problems used to stop "fade in when shown" transitions: an element appearing with `display: block` has no previous style to transition **from**, and `display: none` removes it instantly so a fade-out cannot run. Two features released in 2023-2024 (Baseline newly available in 2024) solve this:

- `@starting-style { ... }` defines the styles an element starts from the first time it is rendered (or goes from display: none to shown), so it can transition into its normal styles.
- `transition-behavior: allow-discrete` (or `allow-discrete` in the shorthand) lets `display` and `overlay` take part, flipping at the right moment so the fade-out finishes before the element disappears.

FADING ELEMENTS IN AND OUT CSS

```css
.toast {
  opacity: 1; translate: 0 0;
  transition: opacity .4s, translate .4s, display .4s allow-discrete;
}
.toast[hidden] { display: none; opacity: 0; translate: 0 20px; }   /* exit state */
@starting-style {
  .toast { opacity: 0; translate: 0 20px; }                        /* entry state */
}

/* Popovers and dialogs (top layer) also need overlay: */
[popover] { transition: opacity .3s, display .3s allow-discrete, overlay .3s allow-discrete; opacity:
0; }
[popover]:popover-open { opacity: 1; }
@starting-style { [popover]:popover-open { opacity: 0; } }
```



REMOVING THE HIDDEN ATTRIBUTE: THE TOAST TRANSITIONS IN FROM ITS @STARTING-STYLE

13.4 Practical effects

13.4.1 Button hover effects

FOUR BUTTON EFFECTS CSS

```css
.btn { font: 600 14px "Poppins", sans-serif; padding: 10px 18px; margin: 8px; border-radius: 8px;
border: 2px solid #1a2a4f; cursor: pointer; transition: all .25s ease-out; }
.b1 { background: #1a2a4f; color: #fff; } .b1:hover { background: #c9a227; color: #1a2a4f; border-
color: #c9a227; }
.b2 { background: #fff; color: #1a2a4f; box-shadow: 0 0 0 0 #1a2a4f; } .b2:hover { box-shadow: 4px 4px
0 0 #1a2a4f; translate: -2px -2px; }
.b3 { background: linear-gradient(90deg, #1a2a4f 50%, #fff 50%) right / 200% 100%; color: #1a2a4f; }
.b3:hover { background-position: left; color: #fff; }
.b4 { background: #1a2a4f; color: #fff; } .b4:hover { scale: 1.08; box-shadow: 0 8px 18px rgb(26 42 79
/ .35); }
.b4:active { scale: .97; }
```



HOVERING "COLOUR"



HOVERING "OFFSET"



HOVERING "SLIDE FILL"

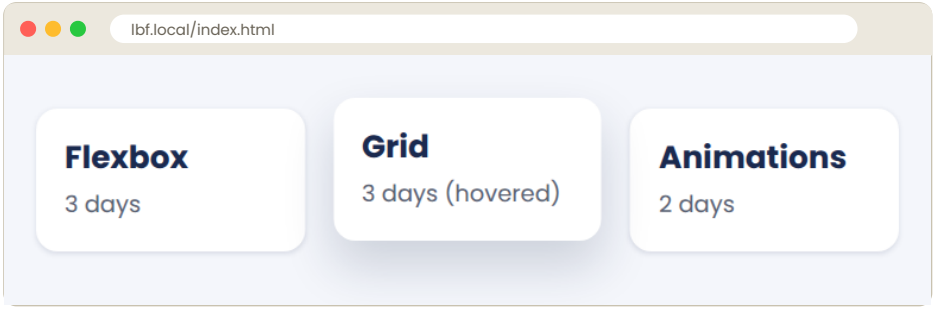
13.4.2 Card lift on hover

CARDS THAT LIFT CSS

```
.cards { display: flex; gap: 16px; padding: 16px 4px; }
.card { flex: 1; background: #fff; border-radius: 12px; padding: 16px; font-family: "Poppins", sans-
serif;
        box-shadow: 0 1px 3px rgb(26 42 79 / .12); transition: translate .25s ease-out, box-shadow
.25s ease-out; }
.card:hover { translate: 0 -6px; box-shadow: 0 14px 28px rgb(26 42 79 / .18); }
.card h3 { margin: 0 0 4px; color: #1a2a4f; } .card p { margin: 0; font-size: 13px; color: #5c6475; }
body { background: #f4f6fb; }
```

INDEX.HTML HTML

```
<div class="cards"><article class="card"><h3>Flexbox</h3><p>3 days</p></article><article class="card"
id="h"><h3>Grid</h3><p>3 days (hovered)</p></article><article class="card"><h3>Animations</h3><p>2
days</p></article></div>
```



HOW THE BROWSER DISPLAYS IT

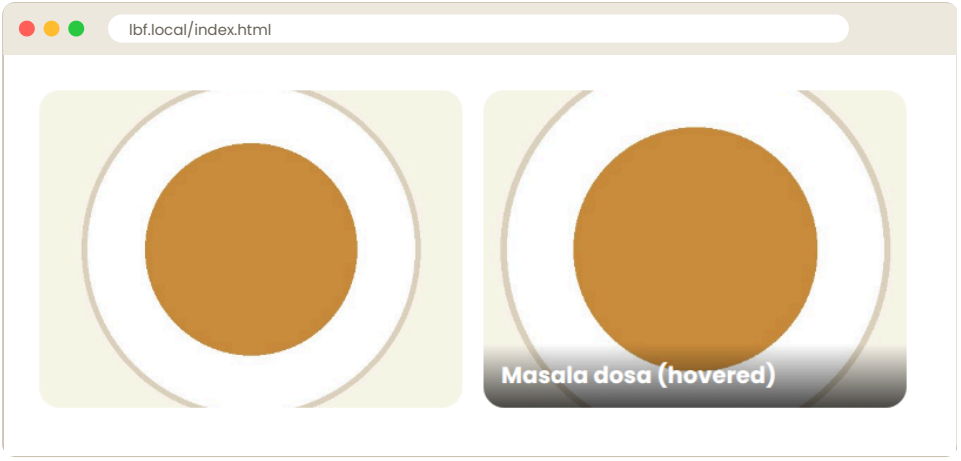
13.4.3 Image zoom on hover

ZOOM INSIDE A CLIPPING FRAME, WITH A SLIDING CAPTION CSS

```css
.thumb { width: 240px; aspect-ratio: 4 / 3; overflow: hidden; border-radius: 12px; display: inline-block; margin: 6px; position: relative; }
.thumb img { width: 100%; height: 100%; object-fit: cover; display: block; transition: scale .5s ease; }
.thumb:hover img { scale: 1.15; }
.thumb figcaption { position: absolute; inset: auto 0 0; padding: 8px 10px; color: #fff; font: 600 13px "Poppins", sans-serif;
  background: linear-gradient(transparent, rgb(0 0 0 / .7)); translate: 0 100%; transition: translate .3s; }
.thumb:hover figcaption { translate: 0 0; }
```

INDEX.HTML HTML

```html
<figure class="thumb"><img src="food-dosa.jpg" alt=""><figcaption>Masala dosa</figcaption></figure>
<figure class="thumb" id="h"><img src="food-dosa.jpg" alt=""><figcaption>Masala dosa (hovered)</figcaption></figure>
```



HOW THE BROWSER DISPLAYS IT

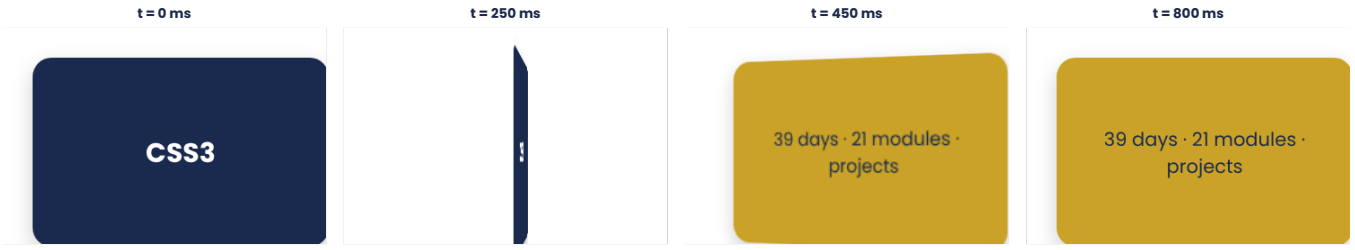
13.4.4 3D flip card

FLIP-CARD.CSS CSS

```css
.flip { width: 220px; height: 140px; perspective: 800px; display: inline-block; margin: 8px; font-family: "Poppins", sans-serif; }
.inner { position: relative; width: 100%; height: 100%; transform-style: preserve-3d; transition: transform .8s cubic-bezier(.4, .2, .2, 1); }
.flip:hover .inner, .flip:focus-within .inner { transform: rotateY(180deg); }
.face { position: absolute; inset: 0; border-radius: 14px; display: grid; place-items: center; text-align: center; padding: 14px;
        backface-visibility: hidden; box-shadow: 0 6px 16px rgb(0 0 0 / .18); }
.front { background: #1a2a4f; color: #fff; font-size: 20px; font-weight: 700; }
.back { background: #c9a227; color: #1a2a4f; transform: rotateY(180deg); font-size: 14px; }
```

FLIP-CARD.HTML HTML

```html
<div class="flip" tabindex="0"><div class="inner"><div class="face front">CSS3</div><div class="face back">39 days · 21 modules · projects</div></div></div>
```



THE FLIP FROZEN AT FOUR MOMENTS: THE FRONT HIDES ITSELF PAST 90 DEGREES AND THE BACK APPEARS

PRACTICE CHECKPOINT

A set of **interactive buttons** with smooth hover transitions, **cards that lift** on hover, and a **3D flip card** showing front and back.

All three parts are complete in this module: the button set in 13.4.1, the lifting cards in 13.4.2 and the flip card in 13.4.4. Combine them on one page, then extend: add `:focus-visible` states equal to the hover states so keyboard users get the same feedback, wrap the hover effects in `@media (hover: hover)`, and turn off movement under `prefers-reduced-motion: reduce`:

ACCESSIBILITY ADDITIONS CSS

```css
@media (prefers-reduced-motion: reduce) {
  .btn, .card, .inner { transition: none; }
  .card:hover { translate: none; }
  .flip:hover .inner { transform: none; }          /* show the back by fading instead: */
  .flip:hover .front { opacity: 0; }
}
```

MODULE 14 · INTERMEDIATE

Animations

Minimum duration **2 days** | Level **Intermediate**

Keyframe animations run multi-step motion without JavaScript. This module covers @keyframes with from/to and percentages, every animation property (name, duration, timing function, delay, iteration count, direction, fill mode, play state), the animation shorthand and multiple animations on one element; common animations (loading spinners, fade-in and slide-in effects with staggering, pulsing and bouncing, the typewriter effect, marquees and infinite carousels); and performance and accessibility: animating only transform and opacity, will-change, and respecting prefers-reduced-motion. Animated examples are shown as frames frozen at exact times in Chromium.

IN THIS MODULE

14.1 Keyframes
14.2 Animation properties
14.3 Common animations
14.4 Performance and accessibility

14.1 Keyframes

14.1.1 @keyframes with from/to and percentages

Transitions go from one state to another when something changes. **Animations** run on their own (on page load, or whenever the animation property is applied), can have **many steps**, can repeat, alternate and pause. They have two parts: a named `@keyframes` rule that describes the styles at points along the timeline, and the `animation` properties that attach it to an element and control its timing.

THREE @KEYFRAMES RULES

CSS

```css
@keyframes slide-right {
  from { transform: translateX(0); }          /* same as 0%   */
  to   { transform: translateX(240px); }      /* same as 100% */
}

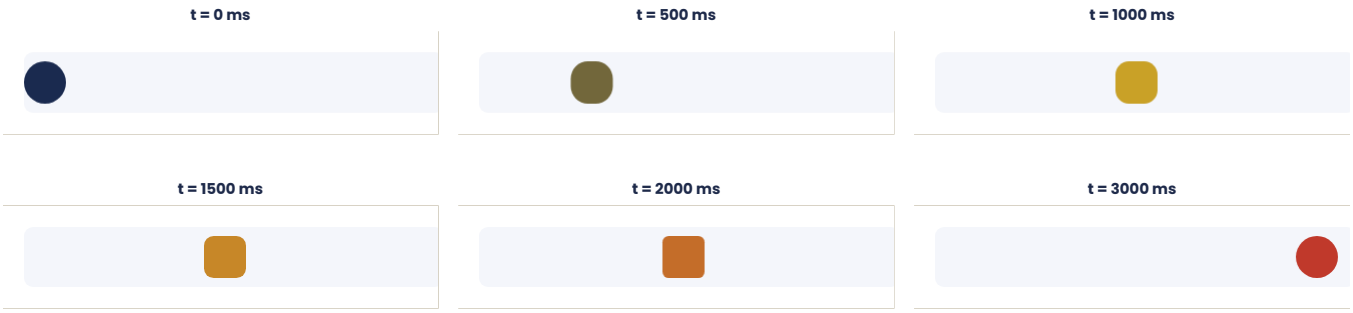
@keyframes traffic-light {
  0%, 30%   { background: #1e8449; }          /* several percentages can share a block */
  40%, 60%  { background: #c9a227; }
  70%, 100% { background: #c0392b; }
}

@keyframes pop {
  0%   { transform: scale(0);   opacity: 0; }
  60%  { transform: scale(1.15); opacity: 1; }  /* overshoot */
  100% { transform: scale(1); }                 /* properties missing here keep interpolating */
}
```

Each keyframe lists the property values at that percentage of the duration; the browser interpolates between them using the timing function, which applies **per segment** (between each pair of keyframes), not over the whole animation. If 0% or 100% is missing, the element's normal styles are used for it.

A FOUR-KEYFRAME ANIMATION CSS

```
@keyframes path { 0% { left: 0; top: 6px; } 33% { left: 120px; top: 6px; background: #c9a227; } 66% {
left: 120px; top: 6px; border-radius: 4px; } 100% { left: 240px; top: 6px; background: #c0392b;
border-radius: 50%; } }
.ball { animation: path 3s linear forwards; }
```



FROZEN AT 0, 0.5, 1, 1.5, 2 AND 3 SECONDS: MOVING, THEN PAUSING WHILE THE SHAPE CHANGES, THEN MOVING AND CHANGING COLOUR

14.2 Animation properties

14.2.1 animation-name, animation-duration

`animation-name` refers to a `@keyframes` name (case-sensitive); `animation-duration` sets one cycle's length (`2s`, `400ms`). Both are required for anything to happen: the default duration is 0. Keyframe names should describe the motion: `fade-in`, `spin`, `slide-up`.

14.2.2 animation-timing-function, animation-delay

`animation-timing-function` accepts the same values as transitions (Module 13) and is applied between each pair of keyframes. A keyframe can also set its own `animation-timing-function` for the segment that starts there. `animation-delay` waits before starting; a **negative** delay starts the animation part-way through its cycle, which is the trick for staggering looping animations so they do not all start together.

14.2.3 animation-iteration-count

How many cycles to run: a number (`1` default, `3`, even `2.5`) or `infinite`.

14.2.4 animation-direction

Value	Each cycle plays…
`normal`	forwards (0% → 100%)
`reverse`	backwards (100% → 0%)
`alternate`	forwards, then backwards, then forwards… (smooth back-and-forth: pulsing, floating)
`alternate-reverse`	backwards first, then forwards…



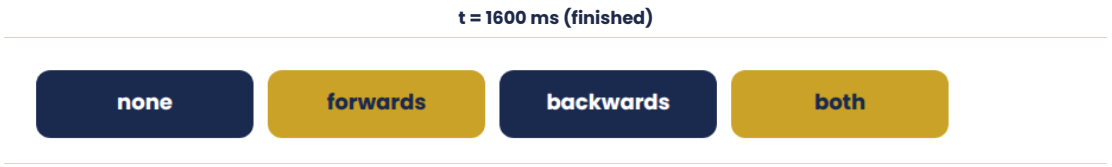
TWO 1-SECOND CYCLES WITH EACH DIRECTION, FROZEN AT 0.25S, 0.9S (FIRST CYCLE) AND 1.25S, 1.9S (SECOND CYCLE)

14.2.5 animation-fill-mode

By default, an animation affects the element **only while it runs**: before a delayed start and after the last cycle, the element shows its normal styles, often causing a jump. `animation-fill-mode` fixes this:

Value	Before the animation (during the delay)	After it ends
`none` (default)	normal styles	normal styles (jumps back)
`forwards`	normal styles	**keeps the last keyframe**
`backwards`	**applies the first keyframe**	normal styles
`both`	first keyframe	last keyframe: the usual choice for entrance effects





FILL-MODE WITH A 500 MS DELAY AND 1 S DURATION: "BACKWARDS" AND "BOTH" HIDE THE ELEMENT DURING THE DELAY; "FORWARDS" AND "BOTH" KEEP THE GOLD END STATE

14.2.6 animation-play-state

`animation-play-state: paused` freezes an animation at its current point; `running` resumes it. A common pattern pauses a marquee or carousel on hover so people can read it, or pauses decorative animations when the user clicks a "pause animations" toggle (needed for WCAG 2.2.2 if motion lasts more than five seconds).

PAUSE ON HOVER AND FOCUS CSS

```css
.marquee__track { animation: scroll 20s linear infinite; }
.marquee:hover .marquee__track,
.marquee:focus-within .marquee__track { animation-play-state: paused; }
```

14.2.7 animation shorthand

`animation: name duration timing-function delay iteration-count direction fill-mode play-state;` Order is flexible except that the first time is the duration and the second is the delay. The name should not clash with keywords such as `ease` or `infinite`.

SHORTHAND EXAMPLES CSS

```css
.spinner { animation: spin 1s linear infinite; }
.toast   { animation: slide-in .4s ease-out .2s both; }
.heart   { animation: pulse 1.2s ease-in-out infinite alternate; }
```

COMPUTED IN CHROMIUM

```
name: x
duration: 0.4s
timing-function: ease-out
delay: 0.2s
iteration-count: 3
direction: alternate
fill-mode: both
play-state: running
```

14.2.8 Multiple animations

Several animations can run on one element, separated by commas, each with its own timing. If two animate the same property, the later one in the list wins for that property. Combining different properties (one changes `translate`, one `rotate`, one `opacity`) is easiest with the individual transform properties, because they do not overwrite each other.

THREE ANIMATIONS ON ONE ELEMENT CSS

```css
@keyframes move { to { translate: 220px 0; } }
@keyframes spin { to { rotate: 360deg; } }
@keyframes colour { 50% { background: #c9a227; } 100% { background: #c0392b; } }
.sq { width: 40px; height: 40px; margin: 10px; background: #1a2a4f; border-radius: 6px;
      animation: move 2s ease-in-out, spin 1s linear 2, colour 2s linear; }
```



TRANSLATE, ROTATE AND COLOUR ANIMATED INDEPENDENTLY

14.3 Common animations

14.3.1 Loading spinners

FOUR LOADERS CSS

```
.loaders { display: flex; gap: 36px; align-items: center; padding: 16px; }
.ring { width: 44px; height: 44px; border: 5px solid #e5e7eb; border-top-color: #1a2a4f; border-
radius: 50%; animation: spin .9s linear infinite; }
@keyframes spin { to { rotate: 360deg; } }
.dots { display: flex; gap: 6px; } .dots i { width: 12px; height: 12px; border-radius: 50%;
background: #c9a227; animation: bounce .6s ease-in-out infinite alternate; }
.dots i:nth-child(2) { animation-delay: .2s; } .dots i:nth-child(3) { animation-delay: .4s; }
@keyframes bounce { to { translate: 0 -14px; opacity: .4; } }
.bar { width: 120px; height: 8px; background: #e5e7eb; border-radius: 4px; overflow: hidden; }
.bar::before { content: ""; display: block; width: 40%; height: 100%; background: #1a2a4f; border-
radius: 4px; animation: slide 1.2s ease-in-out infinite; }
@keyframes slide { from { translate: -100%; } to { translate: 250%; } }
.skeleton { width: 150px; height: 46px; border-radius: 8px; background: linear-gradient(90deg, #eee
25%, #f8f8f8 50%, #eee 75%) 0 0 / 300% 100%; animation: shimmer 1.4s linear infinite; }
@keyframes shimmer { to { background-position: -150% 0; } }
```

SNIPPET HTML

```
<div class="loaders" role="status" aria-label="Loading"><div class="ring"></div><div class="dots"><i>
</i><i></i><i></i></div><div class="bar"></div><div class="skeleton"></div></div>
```



RING SPINNER, BOUNCING DOTS (STAGGERED WITH DELAYS), INDETERMINATE BAR AND SKELETON SHIMMER AT THREE MOMENTS

NOTE

Give loaders an accessible name: `role="status"` with an `aria-label` (or visually hidden text such as "Loading courses…") so screen-reader users know something is happening.

14.3.2 Fade-in and slide-in effects

Entrance animations make content appear smoothly. Use `both` fill mode so elements are invisible before their delay, and **stagger** siblings with increasing delays; a custom property per item keeps it tidy.

STAGGERED FADE-UP CSS

```
@keyframes fade-up { from { opacity: 0; translate: 0 24px; } }
.item { animation: fade-up .6s ease-out both; animation-delay: calc(var(--i) * 150ms);
        background: #fff; border-left: 4px solid #c9a227; padding: 10px 12px; margin: 6px 0; border-
radius: 6px; font: 600 14px "Poppins", sans-serif;
        box-shadow: 0 1px 3px rgb(0 0 0 / .1); }
```

SNIPPET HTML

```
<div class="item" style="--i: 0">Module 1</div><div class="item" style="--i: 1">Module 2</div><div
class="item" style="--i: 2">Module 3</div><div class="item" style="--i: 3">Module 4</div>
```

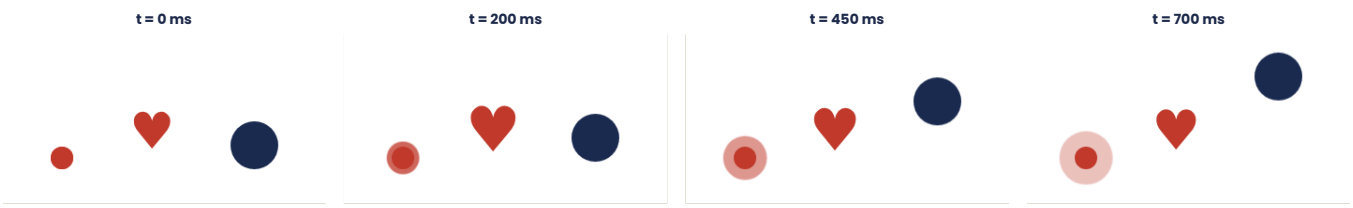


EACH ITEM STARTS 150 MS AFTER THE PREVIOUS ONE

14.3.3 Pulsing and bouncing elements

A "LIVE" PING, A HEARTBEAT AND A BOUNCING BALL CSS

```
.row { display: flex; gap: 40px; align-items: end; padding: 30px 20px 10px; }
.live { position: relative; width: 16px; height: 16px; border-radius: 50%; background: #c0392b; }
.live::after { content: ""; position: absolute; inset: 0; border-radius: 50%; background: #c0392b;
animation: ping 1.4s ease-out infinite; }
@keyframes ping { to { scale: 3; opacity: 0; } }
.heart { font-size: 36px; color: #c0392b; animation: beat 1s ease-in-out infinite; }
@keyframes beat { 0%, 100% { scale: 1; } 15% { scale: 1.3; } 30% { scale: 1; } 45% { scale: 1.15; } }
.ball { width: 34px; height: 34px; border-radius: 50%; background: #1a2a4f; animation: jump .8s
infinite alternate cubic-bezier(.5, 0, .5, 1); }
@keyframes jump { from { translate: 0 0; } to { translate: 0 -50px; } }
```



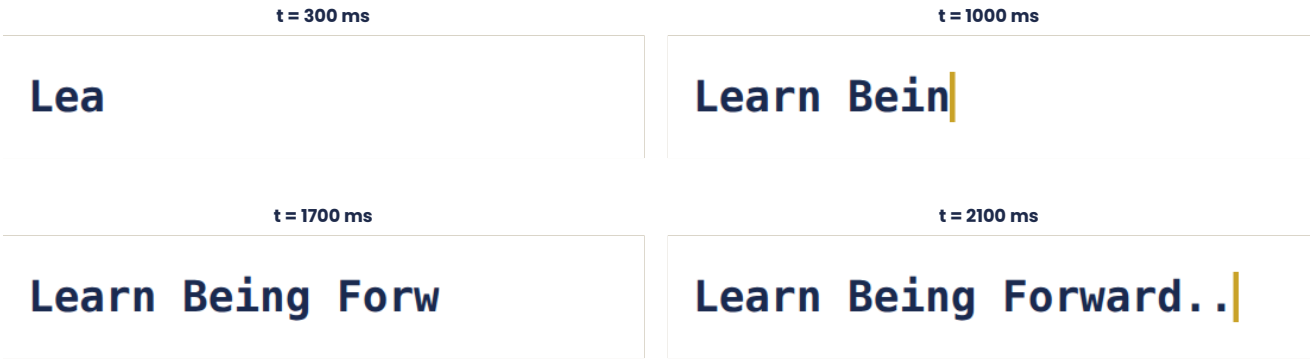
FRAMES AT 0, 0.2, 0.45 AND 0.7 S

14.3.4 Typewriter text effect

A monospace heading revealed character by character: animate `width` from 0 to the text length in `ch` with `steps()` equal to the number of characters, and blink a border as the caret. (For variable-width fonts or wrapping text, a JavaScript solution is more accurate.)

TYPEWRITER.CSS CSS

```
.type { font: 700 24px ui-monospace, "Cascadia Code", monospace; color: #1a2a4f; width: 21ch; white-space: nowrap; overflow: hidden;
        border-right: 3px solid #c9a227;
        animation: typing 2.1s steps(21) both, caret .6s step-end infinite alternate; }
@keyframes typing { from { width: 0; } }
@keyframes caret { 50% { border-color: transparent; } }
```



THE TEXT "TYPES" ITSELF IN 21 STEPS

14.3.5 Marquee and infinite carousels

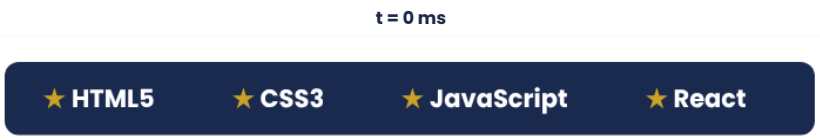
An infinitely scrolling strip (client logos, announcements) duplicates its content once and moves the track by **-50%**, so the second copy lands exactly where the first began and the loop is seamless. Pause it on hover and focus, and mark the duplicate copy `aria-hidden="true"` so screen readers do not read it twice.

MARQUEE.CSS CSS

```
.marquee { overflow: hidden; width: 460px; background: #1a2a4f; border-radius: 8px; padding: 10px 0; }
.track { display: flex; width: max-content; animation: scroll 8s linear infinite; }
.track span { color: #fff; font: 600 14px "Poppins", sans-serif; padding: 0 22px; white-space: nowrap; }
.track span::before { content: "★ "; color: #c9a227; }
@keyframes scroll { to { translate: -50% 0; } }
.marquee:hover .track { animation-play-state: paused; }
```

SNIPPET HTML

```
<div class="marquee"><div class="track"><span>HTML5</span><span>CSS3</span><span>JavaScript</span><span>React</span><span>Angular</span><span>Vue</span><span aria-hidden="true">HTML5</span><span aria-hidden="true">CSS3</span><span aria-hidden="true">JavaScript</span><span aria-hidden="true">React</span><span aria-hidden="true">Angular</span><span aria-hidden="true">Vue</span></div></div>
```



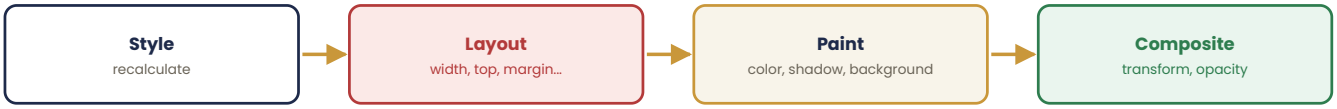


THE TRACK HAS MOVED A QUARTER, THEN HALF OF ITS CONTENT WIDTH

14.4 Performance and accessibility

14.4.1 Animating transform and opacity only

Each frame, the browser may need to recompute **style**, **layout** (sizes and positions of everything affected), **paint** (pixels) and **composite** (layers on the GPU). Animating `width`, `top` or `margin` triggers layout every frame, which can drop frames on phones. `transform` (and translate/rotate/scale) and `opacity` can be handled by the compositor alone, usually on the GPU, and stay smooth even when the main thread is busy.



THE RENDERING PIPELINE: ANIMATE PROPERTIES THAT SKIP TO THE LAST STAGE

Instead of animating	Animate
`left` / `top` / `margin`	`translate`
`width` / `height` (to grow)	`scale` (or the grid 0fr trick for real size)
`visibility` / `display` to fade	`opacity` (+ @starting-style)
`box-shadow` on hover	the `opacity` of a pseudo-element that already has the bigger shadow

14.4.2 will-change

`will-change: transform` (or `opacity`) tells the browser in advance that a property is about to animate, so it can promote the element to its own compositor layer before the animation starts, avoiding a stutter on the first frame. It costs memory, so use it sparingly: on elements that really animate often, ideally added just before the animation (for example on hover of the parent) and removed afterwards; never `* { will-change: transform; }`.

TARGETED WILL-CHANGE CSS

```css
.drawer { will-change: transform; }                /* opens and closes frequently */
.card:hover .card__image { will-change: transform; } /* about to zoom */
```

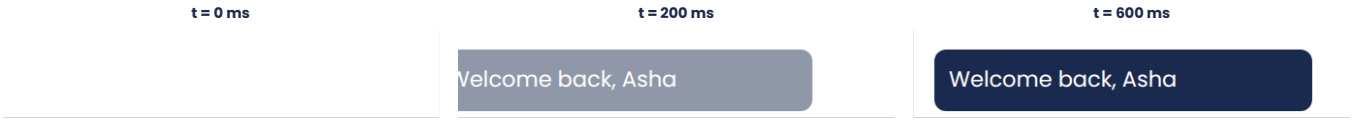
14.4.3 Respecting prefers-reduced-motion

Motion can cause dizziness and nausea for people with vestibular disorders. WCAG 2.3.3 asks that motion triggered by interaction can be disabled, and 2.2.2 that moving content lasting more than five seconds

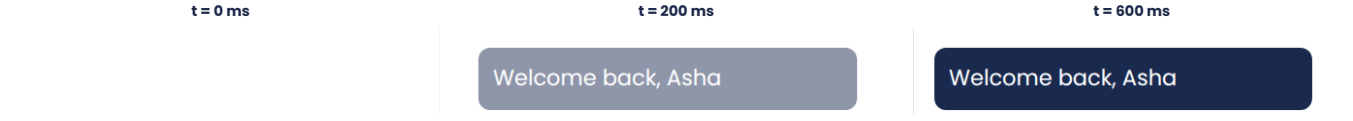
can be paused. Honour the operating system's reduced-motion setting: remove large movements (slides, zooms, parallax, spins, marquees) and keep only gentle opacity changes.

A REDUCED-MOTION ALTERNATIVE CSS

```css
@keyframes slide-in { from { opacity: 0; translate: -60px 0; } }
@keyframes fade-in { from { opacity: 0; } }
.panel { animation: slide-in .6s ease-out both; padding: 10px; background: #1a2a4f; color: #fff;
border-radius: 8px; width: 240px; font-family: "Poppins", sans-serif; }
@media (prefers-reduced-motion: reduce) {
  .panel { animation-name: fade-in; }   /* same timing, no movement */
}
```



DEFAULT: THE PANEL SLIDES IN FROM THE LEFT



PREFERS-REDUCED-MOTION: REDUCE EMULATED IN CHROMIUM: THE PANEL ONLY FADES IN PLACE

PRACTICE CHECKPOINT

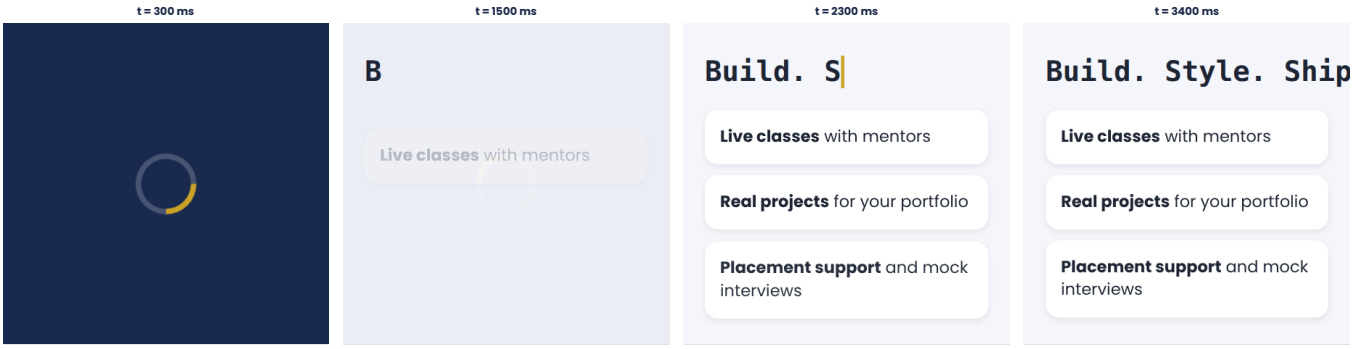
A **landing page** with a **loading spinner**, **staggered fade-in sections**, a **typewriter heading** and **reduced-motion** support.

LANDING.CSS: SAMPLE SOLUTION CSS

```css
body { margin: 0; font-family: "Poppins", sans-serif; background: #f4f6fb; color: #1d2433; }
.loader { position: fixed; inset: 0; display: grid; place-items: center; background: #1a2a4f;
animation: hide .4s 1.2s both; z-index: 10; }
.loader::after { content: ""; width: 46px; height: 46px; border: 5px solid rgb(255 255 255 / .2);
border-top-color: #c9a227; border-radius: 50%; animation: spin .8s linear infinite; }
@keyframes spin { to { rotate: 360deg; } }
@keyframes hide { to { opacity: 0; visibility: hidden; } }
.hero { padding: 30px 20px 10px; }
.hero h1 { font: 700 26px ui-monospace, monospace; width: 18ch; white-space: nowrap; overflow: hidden;
border-right: 3px solid #c9a227; margin: 0;
           animation: typing 1.8s steps(18) 1.4s both, caret .6s step-end infinite alternate; }
@keyframes typing { from { width: 0; } }
@keyframes caret { 50% { border-color: transparent; } }
.section { margin: 10px 20px; padding: 14px; background: #fff; border-radius: 12px; box-shadow: 0 2px
6px rgb(0 0 0 / .08);
           animation: fade-up .6s ease-out both; animation-delay: calc(1.4s + var(--i) * .2s); }
@keyframes fade-up { from { opacity: 0; translate: 0 24px; } }
@media (prefers-reduced-motion: reduce) {
  .loader::after { animation-duration: 2.4s; }                    /* slower, still shows progress */
  .hero h1 { animation: none; width: auto; border: 0; }           /* show the heading at once */
  .section { animation-name: fade-in; }
  @keyframes fade-in { from { opacity: 0; } }
}
```

LANDING.HTML HTML

```
<div class="loader" role="status" aria-label="Loading"></div>
<header class="hero"><h1>Build. Style. Ship.</h1></header>
<section class="section" style="--i: 0"><b>Live classes</b> with mentors</section>
<section class="section" style="--i: 1"><b>Real projects</b> for your portfolio</section>
<section class="section" style="--i: 2"><b>Placement support</b> and mock interviews</section>
```



0.3 S: LOADER; 1.5 S: LOADER FADING, TYPING BEGINS; 2.3 S: SECTIONS APPEARING; 3.4 S: FINAL STATE

MODULE 15 · INTERMEDIATE

Custom properties and CSS functions

Minimum duration **1 day** | Level **Intermediate**

Custom properties (CSS variables) and maths functions make stylesheets dynamic, consistent and easy to theme. This module covers declaring and using custom properties with var(), fallbacks, scope and inheritance, changing variables in media queries and states, updating them from JavaScript, building a theme with dark mode, and design tokens; the maths functions calc(), min(), max() and clamp() for fluid typography and spacing, with an overview of round(), mod() and abs(); attr() with content and env() for safe areas; and @property for typed, animatable variables. Computed values were read back from Chromium.

IN THIS MODULE

15.1 Custom properties
15.2 Maths functions
15.3 Other useful functions
15.4 Registered properties

15.1 Custom properties

15.1.1 Declaring --brand-color and using var()

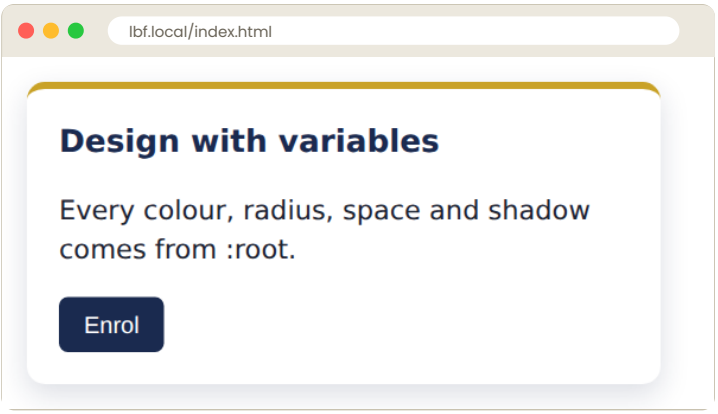
A **custom property** is any property whose name starts with two hyphens. It holds any value, and `var(--name)` inserts that value wherever a value is allowed. Names are **case-sensitive** (`--Brand` and `--brand` differ). Unlike Sass variables (Module 19), custom properties are real properties that live in the browser: they cascade, inherit, can be changed at run time and can be different in different parts of the page.

VARIABLES ON :ROOT — CSS

```
:root {
  --brand-color: #1a2a4f;
  --accent-color: #c9a227;
  --radius: 10px;
  --space: 12px;
  --shadow: 0 6px 16px rgb(26 42 79 / .15);
}
.card { border-radius: var(--radius); padding: calc(var(--space) * 1.5); box-shadow: var(--shadow);
border-top: 4px solid var(--accent-color); max-width: 320px; }
.card h3 { color: var(--brand-color); margin: 0 0 var(--space); }
.btn { background: var(--brand-color); color: #fff; border-radius: calc(var(--radius) / 2); padding:
8px 14px; border: 0; }
```

INDEX.HTML — HTML

```
<div class="card"><h3>Design with variables</h3><p>Every colour, radius, space and shadow comes from
:root.</p><button class="btn">Enrol</button></div>
```



HOW THE BROWSER DISPLAYS IT

Changing `--brand-color` once updates every place that uses it. Variables can hold any token sequence: colours, lengths, whole shadows, font stacks, numbers to multiply with calc(), even a list of grid tracks.

15.1.2 Fallback values

`var(--name, fallback)` uses the fallback when the variable is not defined (or is the initial "guaranteed-invalid" value). Fallbacks can nest: `var(--btn-bg, var(--brand-color, navy))`. Everything after the first comma is the fallback, including further commas: `var(--font, "Poppins", sans-serif)`.

COMPUTED IN CHROMIUM

```
.a color: rgb(26, 42, 79)
.b color: rgb(201, 162, 39)
.c color: rgb(0, 128, 0)
```

WATCH OUT

A defined but **invalid** variable does not use the fallback. If `--size: red` and you write `width: var(--size, 100px)`, the declaration becomes "invalid at computed-value time" and `width` falls back to its initial or inherited value (`auto`), not to 100px. @property (15.4) can prevent this by giving variables a type.

15.1.3 Scope and inheritance (:root vs local)

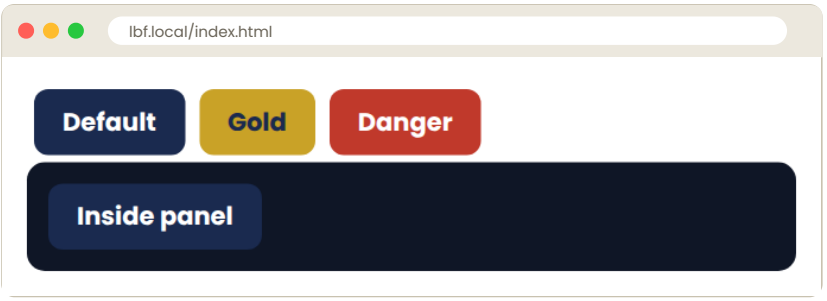
Custom properties follow the cascade and are **inherited** by default. Declared on `:root` they are global; declared on a component they apply only inside it and can be overridden by any descendant. This makes variants trivial: a component reads `var(--x)`, and each variant only sets `--x`.

VARIANTS BY OVERRIDING LOCAL VARIABLES CSS

```css
.btn { --bg: #1a2a4f; --fg: #fff;                  /* defaults, local to .btn */
       background: var(--bg); color: var(--fg); border: 0; padding: 8px 16px; border-radius: 8px;
margin: 4px; font: 600 14px "Poppins", sans-serif; }
.btn--gold   { --bg: #c9a227; --fg: #1a2a4f; }
.btn--danger { --bg: #c0392b; }
.dark-panel  { --fg: #c9a227; background: #0f1626; padding: 8px; border-radius: 10px; }   /* does NOT
affect .btn: .btn redefines --fg itself */
```

INDEX.HTML HTML

```html
<button class="btn">Default</button><button class="btn btn--gold">Gold</button><button class="btn btn-
-danger">Danger</button><div class="dark-panel"><button class="btn">Inside panel</button></div>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
#a (outside): rgb(255, 0, 0)
#b (inside .box): rgb(0, 128, 0)
#c (deeper): rgb(0, 128, 0)
```

15.1.4 Changing variables in media queries and states

Because variables are ordinary properties, media queries, pseudo-classes and classes can change them, and every use updates. Responsive spacing, hover colours and themes become one-line changes:

VARIABLES CHANGED BY QUERIES AND STATES CSS

```css
:root { --gutter: 16px; --columns: 1; }
@media (min-width: 640px)  { :root { --gutter: 24px; --columns: 2; } }
@media (min-width: 1024px) { :root { --gutter: 32px; --columns: 4; } }
.grid { display: grid; gap: var(--gutter); grid-template-columns: repeat(var(--columns), 1fr); }

.link { --line: 0%; background: linear-gradient(currentColor 0 0) left bottom / var(--line) 2px no-repeat; transition: background-size .3s; }
.link:hover { --line: 100%; }   /* the hover only changes a variable */
```



ONE GRID RULE, DRIVEN BY VARIABLES SET IN A MEDIA QUERY

15.1.5 Updating variables with JavaScript

JavaScript reads variables with `getComputedStyle(el).getPropertyValue("--name")` and sets them with `el.style.setProperty("--name", value)`. This is the cleanest bridge between scripts and styles: the script only provides data (a mouse position, a progress value, a chosen colour), and CSS decides how to use it.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<style>
  :root { --brand: #1a2a4f; }
  button { background: var(--brand); color: #fff; }
  .bar { width: 300px; height: 10px; background: #eee; }
  .bar span { display: block; height: 100%; width: calc(var(--progress, 0) * 100%); background: var(--brand); }
</style>
<button>Enrol</button><div class="bar"><span></span></div>
```

READING AND WRITING VARIABLES FROM JAVASCRIPT

main.js

```
const root = document.documentElement;
console.log("before:", getComputedStyle(root).getPropertyValue("--brand").trim());
root.style.setProperty("--brand", "#c0392b");
console.log("after: ", getComputedStyle(root).getPropertyValue("--brand").trim());
console.log("button background now:",
getComputedStyle(document.querySelector("button")).backgroundColor);
document.querySelector(".bar").style.setProperty("--progress", 0.72);
console.log("bar width:", document.querySelector(".bar span").getBoundingClientRect().width + "px of
300px");
```

BROWSER CONSOLE

```
before: #1a2a4f
after:  #c0392b
button background now: rgb(192, 57, 43)
bar width: 216px of 300px
```

A SPOTLIGHT THAT FOLLOWS THE POINTER

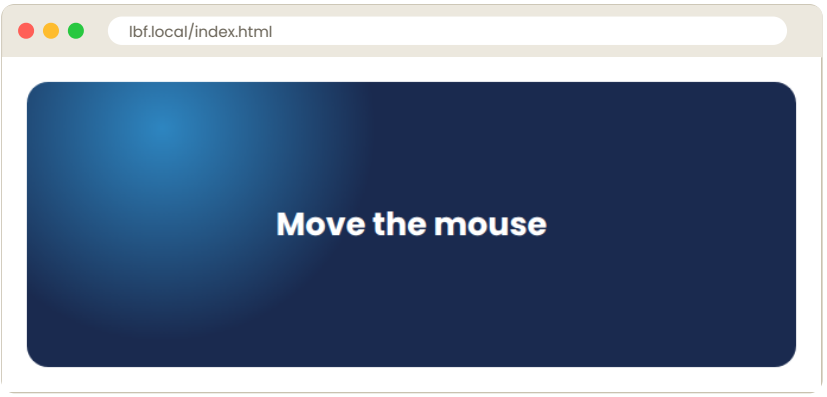
CSS

```
.spot { height: 160px; border-radius: 12px; color: #fff; display: grid; place-items: center; font: 700
18px "Poppins", sans-serif;
        background: radial-gradient(circle 120px at var(--x, 50%) var(--y, 50%), #2e86c1, #1a2a4f); }
```

INDEX.HTML

HTML

```
<div class="spot">Move the mouse</div>
<script>
  const s = document.querySelector(".spot");
  s.addEventListener("pointermove", e => {
    const r = s.getBoundingClientRect();
    s.style.setProperty("--x", e.clientX - r.left + "px");
    s.style.setProperty("--y", e.clientY - r.top + "px");
  });
</script>
```



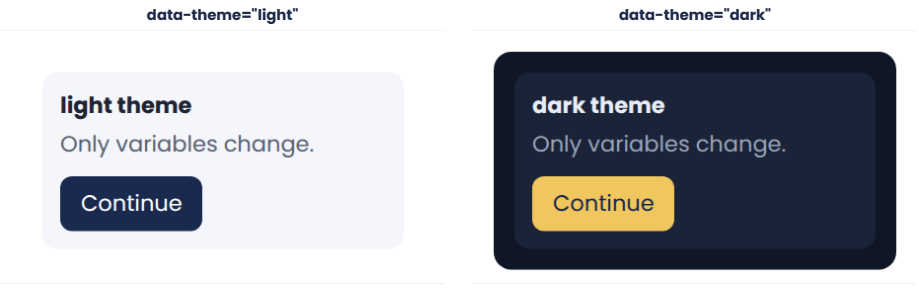
THE POINTER WAS MOVED TO THE TOP-LEFT AREA: THE SCRIPT ONLY SETS --X AND --Y

15.1.6 Building a theme and dark mode with variables

Define **semantic** variables (what a colour is for: `--color-bg`, `--color-text`, `--color-surface`, `--color-primary`), use only those in components, and switch their values per theme. A theme can be chosen automatically (`prefers-color-scheme`, Module 12), by a class or `data-theme` attribute that the user toggles, or both.

THEME.CSS CSS

```css
:root, [data-theme="light"] {
  --color-bg: #ffffff; --color-surface: #f4f6fb; --color-text: #1d2433; --color-muted: #5c6475;
  --color-primary: #1a2a4f; --color-on-primary: #ffffff; --color-accent: #c9a227;
  color-scheme: light;
}
[data-theme="dark"] {
  --color-bg: #0f1626; --color-surface: #1a2338; --color-text: #e8ecf5; --color-muted: #9aa3b8;
  --color-primary: #f0c75e; --color-on-primary: #1a2a4f; --color-accent: #f0c75e;
  color-scheme: dark;
}
.app { background: var(--color-bg); color: var(--color-text); padding: 14px; border-radius: 12px;
font-family: "Poppins", sans-serif; }
.card { background: var(--color-surface); border-radius: 10px; padding: 12px; }
.card p { color: var(--color-muted); margin: 4px 0 10px; }
.card button { background: var(--color-primary); color: var(--color-on-primary); border: 0; padding:
8px 14px; border-radius: 8px; font: inherit; }
```



ONE COMPONENT, TWO THEMES

A THEME SWITCH THAT REMEMBERS THE CHOICE JavaScript

```javascript
// theme-toggle.js
const btn = document.querySelector("#theme-toggle");
const saved = localStorage.getItem("theme");
const prefersDark = matchMedia("(prefers-color-scheme: dark)").matches;
document.documentElement.dataset.theme = saved ?? (prefersDark ? "dark" : "light");

btn.addEventListener("click", () => {
  const next = document.documentElement.dataset.theme === "dark" ? "light" : "dark";
  document.documentElement.dataset.theme = next;
  localStorage.setItem("theme", next);
  btn.setAttribute("aria-pressed", next === "dark");
});
```

15.1.7 Design tokens

Design tokens are the named decisions of a design system: colours, font sizes, spacing steps, radii, shadows, durations. Designers define them (often in Figma variables), and they are exported to code (CSS custom properties, JSON, iOS and Android formats) with tools such as Style Dictionary. A common structure has two tiers: **primitive** tokens that name raw values, and **semantic** tokens that say how they are used; components use only semantic tokens.

TOKENS.CSS

CSS

```css
/* Tier 1: primitive tokens (the palette and scales) */
:root {
  --navy-900: #0f1626;  --navy-700: #1a2a4f;  --navy-100: #eef1f8;
  --gold-500: #c9a227;  --gold-200: #f0d9a0;
  --space-1: 4px; --space-2: 8px; --space-3: 12px; --space-4: 16px; --space-6: 24px; --space-8: 32px;
  --radius-sm: 4px; --radius-md: 8px; --radius-lg: 16px; --radius-full: 999px;
  --font-sans: "Poppins", system-ui, sans-serif;
  --text-sm: .875rem; --text-md: 1rem; --text-lg: 1.25rem; --text-xl: 1.75rem;
  --duration-fast: 150ms; --duration-base: 250ms;
}
/* Tier 2: semantic tokens (what they mean) */
:root {
  --color-primary: var(--navy-700);
  --color-accent: var(--gold-500);
  --color-surface: var(--navy-100);
  --button-padding: var(--space-2) var(--space-4);
  --card-radius: var(--radius-lg);
}
```

15.2 Maths functions

15.2.1 calc()

`calc()` computes a value from an expression with `+ - * /`, and can **mix units**, which nothing else can: `calc(100% - 2rem)`, `calc(100vh - 64px)`. `+` and `-` must have spaces around them (`calc(100%-2rem)` is invalid); `*` and `/` need one side to be a plain number. It works inside other functions and with variables: `calc(var(--space) * 3)`.

COMPUTED IN CHROMIUM

```
calc(100% - 2 * 40px) in a 772px body -> 692px
calc(100vh - 64px) with a 600px viewport -> 536px
calc(var(--n) * 1px) with --n: 37 -> 37px
calc(1rem + 2px) -> 18px
```

TYPICAL USES OF CALC()

CSS

```css
.sidebar-layout main { width: calc(100% - 260px - 2rem); }       /* fill what the sidebar leaves */
.hero { min-height: calc(100dvh - var(--header-height)); }          /* screen minus a fixed header */
.col { flex-basis: calc((100% - 2 * var(--gap)) / 3); }              /* three columns with gaps    */
.icon { width: calc(1em + 4px); }
```

15.2.2 min() and max()

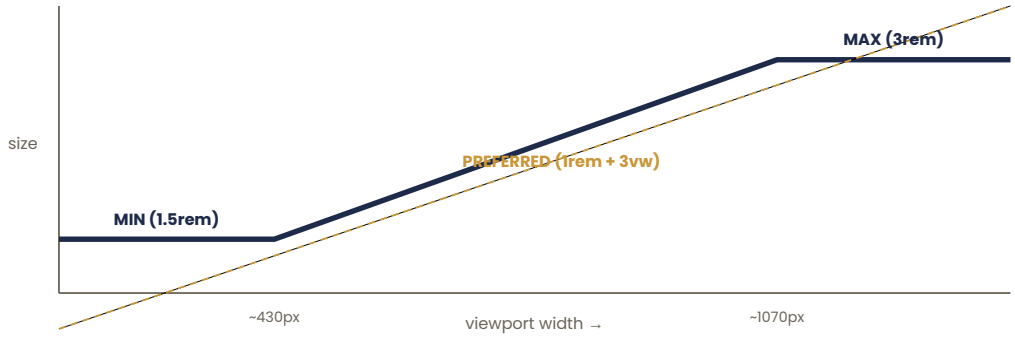
`min(a, b, ...)` returns the **smallest** value at the moment of use and `max()` the **largest**; arguments can mix units. They express responsive limits in one declaration: `width: min(100%, 600px)` is "full width, but at most 600px" (the same as width plus max-width). Remember the inversion: min() sets a **maximum**, max() sets a **minimum**.

COMPUTED IN CHROMIUM

```
viewport 360px
width: min(100%, 600px) -> 332px
padding: max(16px, 5vw)  -> 18px
width: min(90vw, 400px)  -> 324px
```

15.2.3 clamp() for fluid typography and spacing

`clamp(MIN, PREFERRED, MAX)` returns the preferred value but never less than MIN or more than MAX: exactly `max(MIN, min(PREFERRED, MAX))`. With a viewport-relative preferred value it makes **fluid** type and spacing that scale smoothly between two screen sizes, with no breakpoints.



CLAMP(1.5REM, 1REM + 3VW, 3REM): FLAT, THEN FLUID, THEN FLAT

COMPUTED IN CHROMIUM
```
viewport 360px: h1 font-size = 26.8px, padding = 18px
```

COMPUTED IN CHROMIUM
```
viewport 800px: h1 font-size = 40px, padding = 40px
```

COMPUTED IN CHROMIUM
```
viewport 1400px: h1 font-size = 48px, padding = 48px
```

With a 16px root size: at 360px the preferred value is 16 + 10.8 = 26.8px, above the 24px minimum, so it is used; at 800px it is 16 + 24 = 40px; at 1400px it would be 58px, so the 48px maximum applies. To design a clamp, choose the sizes you want at a small and a large viewport and compute the slope between them; online fluid type calculators (such as utopia.fyi) generate whole fluid type and space scales.

TIP
Always include a `rem` term in the preferred value (`1rem + 3vw`, not just `5vw`). Pure vw text does not grow when users zoom, which fails WCAG 1.4.4 (resize text).

15.2.4 Other functions: round(), mod(), abs() (overview)

CSS Values Level 4 added more maths, supported in all major browsers since 2024: stepped values with `round(strategy, value, step)` (nearest, up, down, to-zero), `mod()` and `rem()` (remainders), `abs()` and `sign()`, trigonometry `sin()`, `cos()`, `tan()`, `atan2()` for circular layouts, and `pow()`, `sqrt()`, `hypot()`, `log()`, `exp()` for type scales.

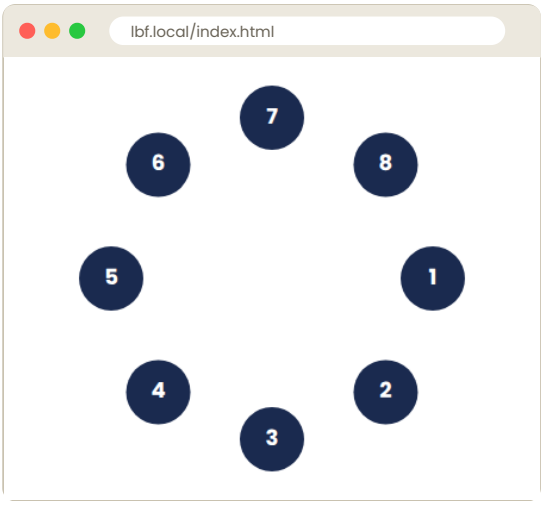
COMPUTED IN CHROMIUM
```
round(up, 117px, 20px)    = 120px
round(down, 117px, 20px)  = 100px
mod(117px, 20px)          = 17px
100px * sin(30deg)        = 50px
16px * pow(1.25, 3)       = 31.25px
```

PLACING ITEMS ON A CIRCLE WITH SIN() AND COS() CSS

```css
.circle { position: relative; width: 220px; height: 220px; margin: 10px auto; }
.circle span { --angle: calc(var(--i) * 360deg / 8); position: absolute; left: 50%; top: 50%; width: 36px; height: 36px; margin: -18px;
  border-radius: 50%; background: #1a2a4f; color: #fff; display: grid; place-items: center; font: 600 12px "Poppins", sans-serif;
  translate: calc(cos(var(--angle)) * 90px) calc(sin(var(--angle)) * 90px); }
```

INDEX.HTML HTML

```html
<div class="circle"><span style="--i: 0">1</span><span style="--i: 1">2</span><span style="--i: 2">3</span><span style="--i: 3">4</span><span style="--i: 4">5</span><span style="--i: 5">6</span><span style="--i: 6">7</span><span style="--i: 7">8</span></div>
```



HOW THE BROWSER DISPLAYS IT

15.3 Other useful functions

15.3.1 attr() with content

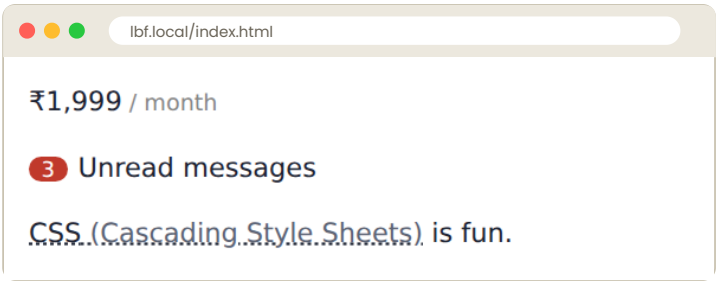
`attr(name)` returns the value of an HTML attribute of the element as a string. In all browsers it works inside the `content` property of pseudo-elements: data-driven tooltips, labels for responsive tables (Module 12), printing link URLs. Chromium 133+ also supports typed `attr(data-size type(<length>))` in any property, but that is not yet in other browsers.

ATTR() IN CONTENT CSS

```css
.price::after { content: " / " attr(data-unit); color: #888; font-size: .85em; }
.badge::before { content: attr(data-count); background: #c0392b; color: #fff; border-radius: 999px; padding: 0 7px; margin-right: 6px; font-size: 12px; }
abbr[title]::after { content: " (" attr(title) ")"; color: #5c6475; }
```

INDEX.HTML HTML

```html
<p class="price" data-unit="month">₹1,999</p><p class="badge" data-count="3">Unread messages</p><p>
<abbr title="Cascading Style Sheets">CSS</abbr> is fun.</p>
```



HOW THE BROWSER DISPLAYS IT

15.3.2 env() for safe areas on mobile

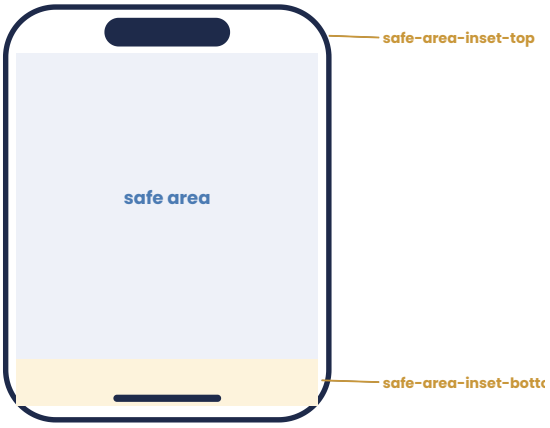
Phones with notches, rounded corners and home indicators define **safe areas**. With `viewport-fit=cover` in the viewport meta tag the page extends edge to edge, and `env(safe-area-inset-top / right / bottom / left)` provides the insets so that content (a fixed bottom bar, a header) is not hidden under the notch or the home indicator. The second argument is a fallback for devices without safe areas.

OPT IN TO EDGE-TO-EDGE HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
```

RESPECTING SAFE AREAS CSS

```
.bottom-nav {
  position: fixed; inset: auto 0 0 0;
  padding-bottom: calc(10px + env(safe-area-inset-bottom, 0px));   /* clear the iPhone home indicator
*/
}
.header {
  padding-top: max(12px, env(safe-area-inset-top));
  padding-inline: max(16px, env(safe-area-inset-left)) max(16px, env(safe-area-inset-right));
}
```



SAFE-AREA INSETS ON A PHONE WITH A NOTCH AND HOME INDICATOR

15.4 Registered properties

15.4.1 @property for typed, animatable variables

Ordinary custom properties are untyped strings to the browser: it cannot interpolate between `--angle: 0deg` and `--angle: 360deg`, so transitions and animations of them just jump. `@property` (Baseline 2024) **registers** a custom property with a `syntax` (type), whether it `inherits`, and an `initial-value`. A

registered property can be animated smoothly, is validated (an invalid value falls back to the initial value), and can be non-inherited for performance.

REGISTERING TYPED PROPERTIES CSS

```css
@property --angle {
  syntax: "<angle>";
  inherits: false;
  initial-value: 0deg;
}
@property --stop {
  syntax: "<percentage>";
  inherits: false;
  initial-value: 0%;
}
```

syntax	Accepts
`"<length>"`	10px, 2rem
`"<percentage>"`	50%
`"<length-percentage>"`	either
`"<number>"` / `"<integer>"`	1.5 / 3
`"<color>"`	any colour
`"<angle>"`	45deg, 0.25turn
`"<time>"`	200ms
`"*"`	anything (not animatable)

AN ANIMATED GRADIENT BORDER CSS

```css
@property --angle { syntax: "<angle>"; inherits: false; initial-value: 0deg; }
.border-card { width: 240px; padding: 3px; border-radius: 14px; margin: 10px;
  background: conic-gradient(from var(--angle), #1a2a4f, #c9a227, #2e86c1, #1a2a4f);
  animation: rotate-border 4s linear infinite; }
.border-card > div { background: #fff; border-radius: 11px; padding: 16px; font: 600 14px "Poppins",
sans-serif; color: #1a2a4f; }
@keyframes rotate-border { to { --angle: 360deg; } }
```



--ANGLE ANIMATES FROM 0 TO 360DEG, ROTATING THE GRADIENT (FRAMES AT 0, 1, 2, 3 S)

COMPUTED IN CHROMIUM

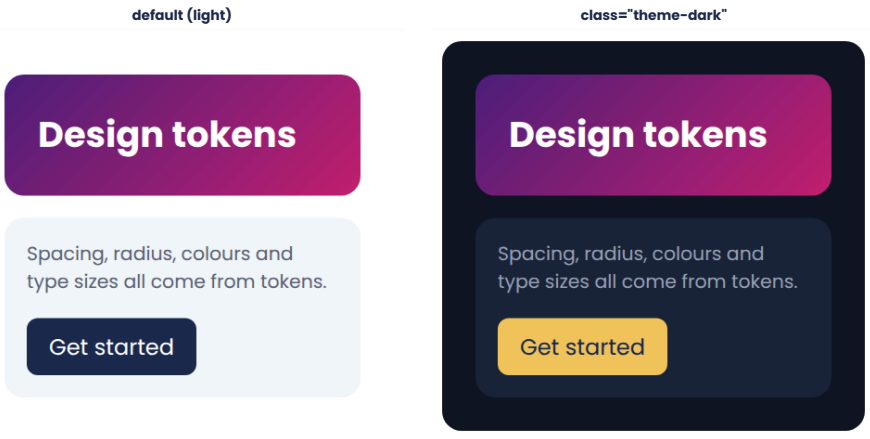
```
registered --n: 25px  -> 25px
registered --n: red   -> 10px   (invalid, initial value used)
unregistered --m: red -> red
```

PRACTICE CHECKPOINT

A **design-token-based theme** with light and dark modes switched by a **single class**, **fluid type** using clamp(), and an **animated gradient** using @property.

TOKENS-THEME.CSS: SAMPLE SOLUTION CSS

```css
@property --hue { syntax: "<number>"; inherits: true; initial-value: 220; }
:root {
  --space-2: .5rem; --space-4: 1rem; --space-6: 1.5rem; --radius: 14px;
  --text-sm: clamp(.85rem, .8rem + .2vw, .95rem);
  --text-xl: clamp(1.6rem, 1rem + 2.5vw, 2.6rem);
  --bg: #ffffff; --surface: #f4f6fb; --text: #1d2433; --muted: #5c6475; --primary: #1a2a4f; --on-primary: #fff;
}
.theme-dark {
  --bg: #0f1626; --surface: #1a2338; --text: #e8ecf5; --muted: #9aa3b8; --primary: #f0c75e; --on-primary: #1a2a4f;
}
.page { background: var(--bg); color: var(--text); padding: var(--space-6); font-family: "Poppins", sans-serif; border-radius: var(--radius); }
.hero { border-radius: var(--radius); padding: var(--space-6); color: #fff;
        background: linear-gradient(135deg, hsl(var(--hue) 60% 30%), hsl(calc(var(--hue) + 60) 70% 45%));
        animation: hue 6s linear infinite; }
@keyframes hue { to { --hue: 580; } }
.hero h1 { font-size: var(--text-xl); margin: 0; }
.card { background: var(--surface); border-radius: var(--radius); padding: var(--space-4); margin-top: var(--space-4); }
.card p { color: var(--muted); font-size: var(--text-sm); margin: 0 0 var(--space-4); }
.card button { background: var(--primary); color: var(--on-primary); border: 0; border-radius: 8px; padding: var(--space-2) var(--space-4); font: inherit; }
```



ONE CLASS SWITCHES EVERY TOKEN; THE HERO HUE ANIMATES THROUGH THE REGISTERED --HUE PROPERTY

LEVEL 3

Advanced level

Outcome: use container queries, nesting, layers and modern visual features, organise CSS at scale, use Sass and tooling, and ship fast, accessible projects.

6 MODULES · 12 DAYS MINIMUM

Module 16 · Modern CSS layout and architecture features	**3 days**
Module 17 · Modern colour, visual effects and scroll features	**2 days**
Module 18 · CSS architecture and methodologies	**1 day**
Module 19 · Preprocessors, frameworks and tooling	**2 days**
Module 20 · Performance, accessibility and debugging	**1 day**
Module 21 · Projects and interview preparation	**3 days**

MODULE 16 · ADVANCED

Modern CSS layout and architecture features

Minimum duration **3 days** | Level **Advanced**

Recent CSS features change how components are built: they respond to their containers, nest rules and control the cascade explicitly. This module covers container queries (container-type, container-name, @container, container query units, the comparison with media queries and style queries), native CSS nesting with & and nested media queries and how to avoid over-nesting, cascade layers (@layer order, a reset-base-components-utilities layer stack and how layers interact with specificity), @scope with upper and lower bounds, subgrid for aligning nested card content, logical properties in depth with right-to-left support, and feature queries with @supports. All of these are Baseline features; every example was rendered in Chromium 141.

IN THIS MODULE

16.1 Container queries
16.2 CSS nesting
16.3 Cascade layers
16.4 Scoping
16.5 Subgrid
16.6 Logical properties in depth
16.7 Feature queries

16.1 Container queries

Media queries ask about the **viewport**. But a card component can be placed in a wide main column, a narrow sidebar or a three-column grid on the same screen, and only its **own available width** tells it which layout fits. **Container queries** (Baseline 2023) let elements respond to the size of a chosen ancestor, the **container**, which makes components truly reusable.

16.1.1 container-type and container-name

An element becomes a size container with `container-type: inline-size` (query its width; the usual choice) or `size` (width and height; the element then needs an explicit height, because its size can no longer depend on its children). `container-name` gives it a name so queries can target a specific ancestor; the shorthand is `container: card / inline-size`. Containment means the container's inline size cannot depend on its content, so a container usually gets its width from its parent (block or grid layout).

DECLARING CONTAINERS

CSS

```css
.sidebar, .main { container-type: inline-size; }        /* anonymous containers */
.product-list { container: products / inline-size; }       /* named: name / type   */
```

16.1.2 @container rules

`@container (min-width: 400px) { ... }` applies its rules to elements inside the **nearest ancestor container** when that container matches. `@container products (width > 600px)` targets the nearest

container named products. The range syntax and `and` / `or` / `not` work as in media queries. A container cannot style itself with a query on its own size; the rules apply to its descendants.

CARD.CSS: ONE COMPONENT, THREE LAYOUTS CSS

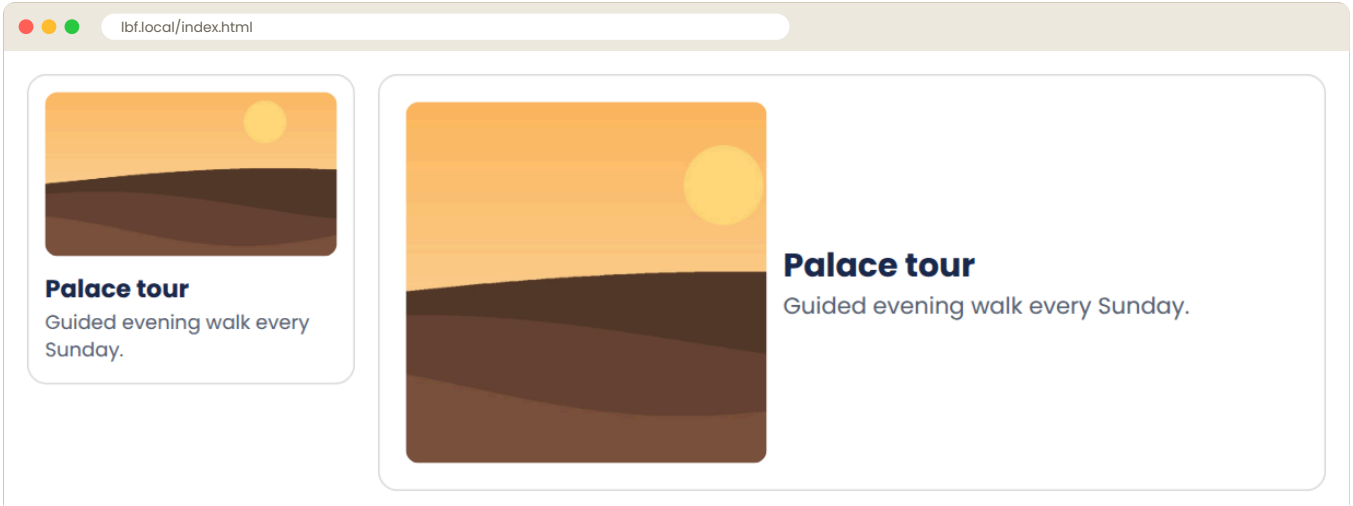
```
.wrap { container-type: inline-size; }
.card { display: grid; gap: 10px; background: #fff; border: 1px solid #ddd; border-radius: 12px;
padding: 10px; font-family: "Poppins", sans-serif; }
.card img { width: 100%; aspect-ratio: 16 / 9; object-fit: cover; border-radius: 8px; }
.card h3 { margin: 0; color: #1a2a4f; font-size: 15px; } .card p { margin: 0; font-size: 12px; color:
#5c6475; }
@container (min-width: 380px) {
  .card { grid-template-columns: 140px 1fr; align-items: center; }
  .card img { aspect-ratio: 1; }
}
@container (min-width: 560px) {
  .card { grid-template-columns: 220px 1fr; padding: 16px; }
  .card h3 { font-size: 20px; } .card p { font-size: 14px; }
}
```

SNIPPET HTML

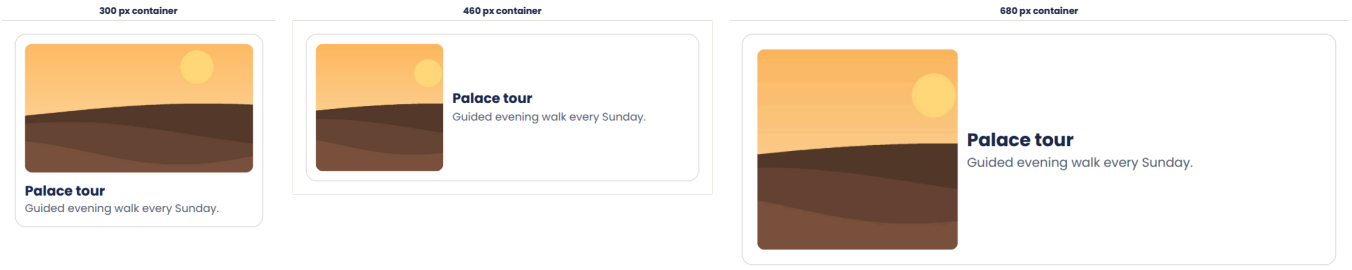
```
<div class="wrap"><article class="card"><img src="mysuru-palace.jpg" alt=""><div><h3>Palace tour</h3>
<p>Guided evening walk every Sunday.</p></div></article></div>
```

THE SAME CARD IN A SIDEBAR AND IN THE MAIN COLUMN CSS

```
.wrap { container-type: inline-size; }
.card { display: grid; gap: 10px; background: #fff; border: 1px solid #ddd; border-radius: 12px;
padding: 10px; font-family: "Poppins", sans-serif; }
.card img { width: 100%; aspect-ratio: 16 / 9; object-fit: cover; border-radius: 8px; }
.card h3 { margin: 0; color: #1a2a4f; font-size: 15px; } .card p { margin: 0; font-size: 12px; color:
#5c6475; }
@container (min-width: 380px) {
  .card { grid-template-columns: 140px 1fr; align-items: center; }
  .card img { aspect-ratio: 1; }
}
@container (min-width: 560px) {
  .card { grid-template-columns: 220px 1fr; padding: 16px; }
  .card h3 { font-size: 20px; } .card p { font-size: 14px; }
}
.layout { display: grid; grid-template-columns: 200px 1fr; gap: 14px; } aside, main { container-type:
inline-size; }
```



ONE VIEWPORT, TWO LAYOUTS: THE SIDEBAR IS A NARROW CONTAINER, THE MAIN COLUMN A WIDE ONE



THE CARD AT THREE CONTAINER WIDTHS

COMPUTED IN CHROMIUM

```
card in a 460px container: rgb(0, 128, 0) (query matched)
card in a 300px container: rgb(29, 36, 51) (no match)
```

16.1.3 Container query units: cqw, cqi

Inside a container, `cqw` / `cqh` are 1% of the container's width / height, and the logical `cqi` / `cqb` 1% of its inline / block size; `cqmin` and `cqmax` the smaller and larger. They size text and spacing relative to the component instead of the viewport, for example `font-size: clamp(1rem, 5cqi, 2rem)` for a heading that grows with its card.



FONT-SIZE: CLAMP(14PX, 8CQI, 40PX) FOLLOWS THE CONTAINER

16.1.4 Container queries vs media queries

	Media queries	Container queries
Ask about	the viewport and device (and user preferences)	an ancestor element's size (or styles)
Best for	page-level layout, navigation, user preferences (dark mode, motion), print	reusable components that appear in different places
Setup	none	`container-type` on an ancestor
Units	vw, vh, dvh	cqi, cqw, cqh, cqb
Support	universal	Baseline 2023 (size); style queries partial

Use media queries for the page and container queries for the components inside it. A component that adapts to its container works in any layout, any sidebar and any framework.

16.1.5 Style queries (overview)

Container **style queries** test the computed value of a custom property on the container: `@container style(--variant: featured) { ... }`. They let a parent switch a theme or variant for everything inside it without classes on every child. Every element is a style container by default (no container-type needed). Supported in Chromium and Safari for custom properties; Firefox support is in progress, so treat them as an enhancement.

A STYLE QUERY ON A CUSTOM PROPERTY CSS

```css
.theme-box { --tone: normal; padding: 8px; border-radius: 10px; margin: 6px 0; }
.theme-box.promo { --tone: promo; }
@container style(--tone: promo) {
  .note { background: #c9a227; color: #1a2a4f; font-weight: 700; }
}
.note { padding: 8px 12px; border-radius: 8px; background: #eef1f8; }
```

INDEX.HTML HTML

```html
<div class="theme-box"><p class="note">Normal tone</p></div><div class="theme-box promo"><p
class="note">Promo tone, from a style query</p></div>
```



HOW THE BROWSER DISPLAYS IT

16.2 CSS nesting

Native CSS nesting (Baseline 2023) lets you write rules inside other rules, as Sass has done for years, so related styles stay together and selectors are not repeated.

16.2.1 Native nesting syntax and &

Inside a rule, a nested rule is relative to its parent. The `&` **nesting selector** stands for the parent selector: `&:hover` (the parent itself hovered), `&.is-active` (the parent with a class), `.dark &` (the parent inside .dark). A nested selector without & is treated as a descendant (`h3` inside `.card` means `.card h3`). Unlike Sass, `&` cannot be concatenated into a new class name (`&__title` does not mean `.card__title`).

NATIVE NESTING CSS

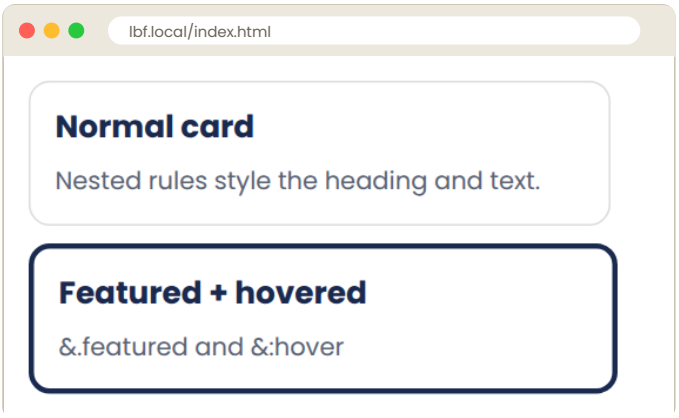
```css
.card {
  background: #fff;
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 14px;
  font-family: "Poppins", sans-serif;
  max-width: 300px;

  h3 { margin: 0 0 6px; color: #1a2a4f; }            /* = .card h3            */
  p  { margin: 0; color: #5c6475; font-size: 14px; }   /* = .card p             */

  &:hover { border-color: #c9a227; }                  /* = .card:hover         */
  &.featured { border-width: 3px; border-color: #1a2a4f; }   /* = .card.featured */
  & + & { margin-top: 10px; }                         /* = .card + .card       */
  .dark & { background: #1a2338; }                    /* = .dark .card         */
}
```

INDEX.HTML HTML

```html
<article class="card"><h3>Normal card</h3><p>Nested rules style the heading and text.</p></article>
<article class="card featured" id="h"><h3>Featured + hovered</h3><p>&amp;.featured and &amp;:hover</p>
</article>
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
h3 inside .card: rgb(0, 128, 0)
.card.featured:   rgb(0, 0, 255)
cssRules[0].cssRules.length = 2 nested rules
```

16.2.2 Nesting media queries

At-rules such as `@media`, `@container`, `@supports` and `@layer` can be nested directly inside a rule; their declarations apply to the parent selector. Keeping a component's responsive variations next to its base styles is a big readability gain.

NESTED AT-RULES

CSS

```
.hero {
  padding: 2rem 1rem;
  font-size: 1.5rem;

  @media (min-width: 768px) {
    padding: 4rem 2rem;          /* applies to .hero */
    font-size: 2.5rem;
  }
  @media (prefers-reduced-motion: no-preference) {
    animation: fade-in .6s both;
  }
  @container (min-width: 500px) {
    display: grid; grid-template-columns: 1fr 1fr;
  }
}
```

16.2.3 Avoiding over-nesting

Every level of nesting adds a selector to the chain, raising specificity and coupling CSS to the HTML structure. A useful rule: **nest at most two or three levels**, nest mainly for pseudo-classes, modifiers and media queries, and give components flat class names instead of deep descendant chains. Also note that nested selectors are wrapped in `:is()`, so `#main, .card { p { } }` gives `:is(#main, .card) p` the specificity of the id for every match.

KEEP NESTING SHALLOW

CSS

```
/* Over-nested: (0,5,3) and breaks if the markup changes */
.page { .content { .sidebar { ul { li { a { color: red; } } } } } }

/* Better: a flat class with its states nested */
.sidebar-link {
  color: var(--link);
  &:hover { color: var(--link-hover); }
  &[aria-current="page"] { font-weight: 700; }
}
```

16.3 Cascade layers

Specificity fights (Module 8) happen because unrelated rules compete. **Cascade layers** (`@layer`, Baseline 2022) add a level to the cascade **above specificity**: you declare the order of layers once, and any rule in a later layer beats any rule in an earlier layer, whatever their selectors. Unlayered styles come last and beat all layers.

16.3.1 @layer declaration and order

DECLARING AND FILLING LAYERS CSS

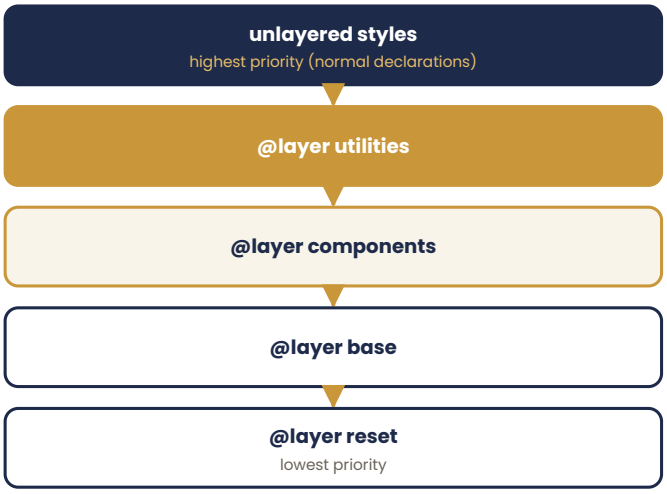
```css
/* 1. Declare the order first (earliest = lowest priority) */
@layer reset, base, components, utilities;

/* 2. Add rules to layers anywhere, in any order */
@layer base {
  a { color: #1a2a4f; }
}
@layer utilities {
  .text-red { color: #c0392b; }
}
@layer components {
  .nav a#home.link { color: #1e8449; }   /* high specificity, but in an earlier layer */
}

/* 3. Imports can go straight into a layer */
@import url("bootstrap.css") layer(framework);
```

COMPUTED IN CHROMIUM

```
winner: rgb(192, 57, 43)
  utilities .text-red (0,1,0) beats components .nav a#home.link (1,2,1): the later layer wins before
specificity is compared
```



LAYER ORDER FOR NORMAL DECLARATIONS: LATER LAYERS WIN

16.3.2 Layering resets, base, components and utilities

A recommended stack: **reset** (normalise browsers), **base** (element defaults: typography, links, forms), **components** (cards, buttons, navbars), **utilities** (single-purpose classes like .visually-hidden or .mt-4 that must always win). Third-party CSS (a framework, a date-picker library) goes into its own early layer, so your component styles override it without !important. Layers can be nested (`@layer components.buttons`).

MAIN.CSS WITH A LAYER STACK CSS

```css
@layer reset, vendor, base, components, utilities;

@import url("reset.css") layer(reset);
@import url("datepicker.css") layer(vendor);

@layer base {
  body { font-family: var(--font-sans); color: var(--color-text); }
  a { color: var(--color-link); }
}
@layer components {
  .btn { padding: .5rem 1rem; border-radius: 8px; background: var(--color-primary); }
}
@layer utilities {
  .hidden { display: none; }
  .visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden; clip-path:
inset(50%); }
}
```

16.3.3 How layers interact with specificity

- **Between layers**: layer order decides; specificity is not even compared.
- **Within one layer**: normal cascade (specificity, then order).
- **Unlayered** author styles beat all layered styles (for normal declarations).
- **!important reverses layer order**: an important declaration in an *earlier* layer (such as reset) beats an important one in a later layer, so resets can protect essentials.
- The `revert-layer` keyword rolls a property back to the value from the previous layers.

COMPUTED IN CHROMIUM

```
!important in base vs !important in utilities -> rgb(255, 0, 0)  (the EARLIER layer wins for
important)
layered .y vs unlayered .y -> rgb(128, 0, 128)  (unlayered wins)
```

16.4 Scoping

16.4.1 @scope with upper and lower bounds

`@scope (.card) { ... }` limits rules to elements inside `.card` (the **scope root**, an upper bound) without adding the root's specificity to the selectors. A **lower bound** with `to (...)` stops the scope at nested parts, creating a "donut" scope: `@scope (.card) to (.card__content)` styles the card's own frame but not whatever content is placed inside it (such as another component). When two scopes conflict at equal specificity, the **nearer** scope root wins (scoping proximity). Baseline since late 2025 (Firefox 146); Chromium and Safari earlier.

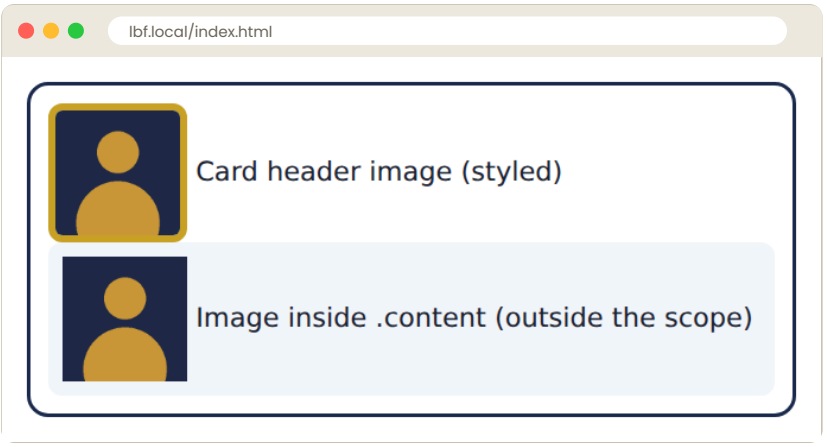
A DONUT SCOPE CSS

```css
@scope (.card) to (.content) {
  img { border: 4px solid #c9a227; border-radius: 8px; }   /* only images in the card frame */
  :scope { border: 2px solid #1a2a4f; border-radius: 12px; padding: 10px; display: block; }
}
.content { background: #f4f6fb; padding: 8px; border-radius: 8px; }
img { width: 70px; vertical-align: middle; }
```

INDEX.HTML

HTML

```
<article class="card"><img src="avatar.png" alt=""> Card header image (styled)<div class="content">
<img src="avatar.png" alt=""> Image inside .content (outside the scope)</div></article>
```



HOW THE BROWSER DISPLAYS IT

SCOPE PROXIMITY

CSS

```
/* Proximity: the closer theme wins, regardless of source order */
@scope (.light) { a { color: #1a2a4f; } }
@scope (.dark)  { a { color: #f0c75e; } }
/* <div class="dark"> <div class="light"> <a>  -> navy (light is nearer) */
```

16.5 Subgrid

16.5.1 grid-template-columns: subgrid

A grid item can itself be a grid, but normally its tracks are independent of the parent's. With `grid-template-columns: subgrid` and/or `grid-template-rows: subgrid` (Baseline 2023), a nested grid **reuses the parent's tracks** for the area it spans, so content in different children lines up perfectly. The subgrid inherits the parent's gaps (it may override them).

ALIGNED FORM LABELS WITH SUBGRID

CSS

```
.form { display: grid; grid-template-columns: max-content 1fr; gap: 8px 12px; }
.field { grid-column: 1 / -1; display: grid; grid-template-columns: subgrid; }   /* label | input */
```

LABELS IN DIFFERENT WRAPPERS SHARE ONE COLUMN WIDTH

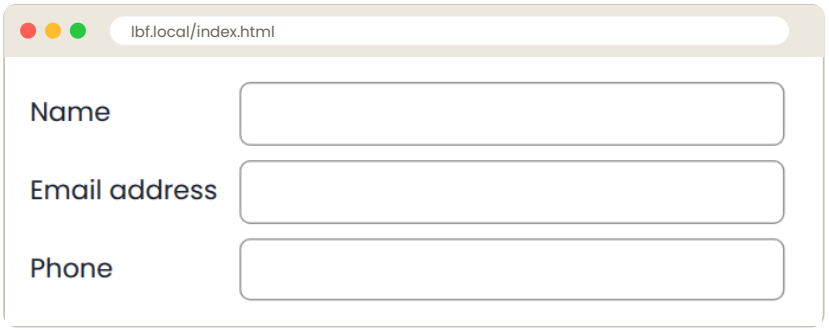
CSS

```
.form { display: grid; grid-template-columns: max-content 1fr; gap: 8px 12px; max-width: 420px; font-
family: "Poppins", sans-serif; }
.field { grid-column: 1 / -1; display: grid; grid-template-columns: subgrid; align-items: center; }
input { padding: 6px; border: 1px solid #999; border-radius: 6px; font: inherit; }
```

INDEX.HTML

HTML

```
<form class="form"><div class="field"><label>Name</label><input></div><div class="field"><label>Email
address</label><input></div><div class="field"><label>Phone</label><input></div></form>
```



HOW THE BROWSER DISPLAYS IT

16.5.2 Aligning nested card content

The classic problem: cards in a row have titles and descriptions of different lengths, so their buttons and footers do not line up. With row subgrid, each card spans several rows of the parent grid and its parts sit in shared rows:



WITH SUBGRID, TITLES, DESCRIPTIONS AND FOOTERS LINE UP ACROSS CARDS

THE SUBGRID FIX CSS

```css
.cards { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; }
.card  { display: grid; grid-row: span 3;              /* title, text, footer = 3 parent rows */
         grid-template-rows: subgrid; }
```

16.6 Logical properties in depth

16.6.1 Block and inline directions

Logical properties (introduced in Module 5) describe layout relative to the **writing mode** instead of the physical screen. The **inline** axis is the direction of a line of text; the **block** axis is the direction in which blocks (paragraphs) stack. For English these are horizontal and vertical; for vertical Japanese or Mongolian text (`writing-mode: vertical-rl`) they swap.

Physical	Logical
`width` / `height`	`inline-size` / `block-size`
`min-width`, `max-height`	`min-inline-size`, `max-block-size`
`margin-left` / `margin-right`	`margin-inline-start` / `margin-inline-end`
`margin-top` / `margin-bottom`	`margin-block-start` / `margin-block-end`
`padding-left` + `padding-right`	`padding-inline`
`border-top-left-radius`	`border-start-start-radius`

Physical	Logical
`left`, `right`, `top`, `bottom`	`inset-inline-start`, `inset-inline-end`, `inset-block-start`, `inset-block-end`
`text-align: left`	`text-align: start`
`float: left`	`float: inline-start`



THE SAME LOGICAL RULE IN TWO WRITING MODES: THE ACCENT BORDER IS ALWAYS AT THE START OF THE LINES

16.6.2 Supporting right-to-left languages

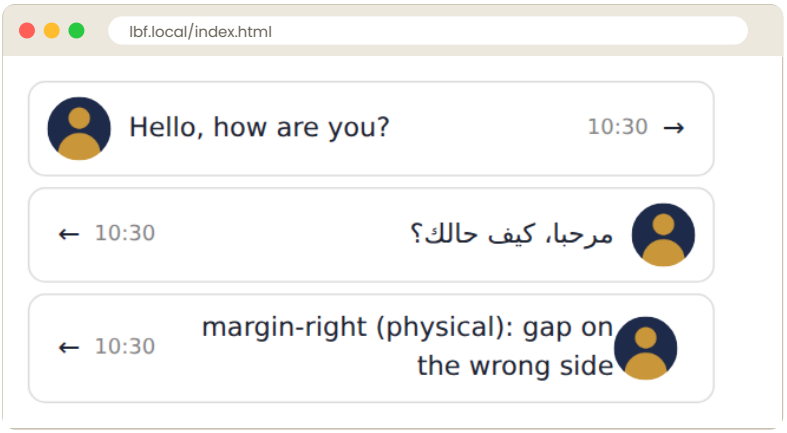
Arabic, Hebrew, Persian and Urdu are written right to left. With `dir="rtl"` on the html element (or any element), the browser mirrors text alignment, list markers, tables, Flexbox and Grid row order automatically. Physical properties do not mirror, so an icon with `margin-right` ends up on the wrong side. Writing styles with logical properties makes a single stylesheet work for both directions; the `:dir(rtl)` pseudo-class handles the rare remaining cases (such as flipping an arrow icon).

ONE STYLESHEET FOR LEFT-TO-RIGHT AND RIGHT-TO-LEFT CSS

```
.msg { display: flex; align-items: center; gap: 0; max-inline-size: 360px; border: 1px solid #ddd;
border-radius: 10px; padding-block: 8px; padding-inline: 10px 16px; margin-block: 6px; font-family:
system-ui, sans-serif; }
.msg img { inline-size: 36px; border-radius: 50%; margin-inline-end: 10px; }
.msg .time { margin-inline-start: auto; color: #888; font-size: 12px; }
.msg::after { content: "→"; margin-inline-start: 8px; }
.msg:dir(rtl)::after { content: "←"; }
.physical img { margin-right: 10px; margin-inline-end: 0; }
```

INDEX.HTML HTML

```
<div class="msg" dir="ltr"><img src="avatar.png" alt="">Hello, how are you?<span
class="time">10:30</span></div>
<div class="msg" dir="rtl" lang="ar"><img src="avatar.png" alt="">مرحبا ، كيف حالك؟<span
class="time">10:30</span></div>
<div class="msg physical" dir="rtl" lang="ar"><img src="avatar.png" alt="">margin-right (physical):
gap on the wrong side<span class="time">10:30</span></div>
```



LOGICAL MARGINS MIRROR AUTOMATICALLY; THE PHYSICAL MARGIN-RIGHT IN THE LAST ROW LEAVES THE GAP ON THE WRONG SIDE OF THE AVATAR

16.7 Feature queries

16.7.1 @supports

`@supports (property: value) { ... }` applies rules only if the browser supports that declaration; `not`, `and` and `or` combine conditions, and `selector(...)` tests selector support. Feature queries let you write a solid baseline for every browser and enhance it where a newer feature exists (progressive enhancement, Module 20). JavaScript has the same test: `CSS.supports()`.

FEATURE QUERIES CSS

```css
.gallery { display: flex; flex-wrap: wrap; gap: 1rem; }              /* baseline for everyone */

@supports (display: grid) and (grid-template-rows: masonry) {          /* future enhancement    */
  .gallery { display: grid; grid-template-rows: masonry; }
}
@supports not (container-type: inline-size) {
  .card { flex-direction: column; }                                    /* fallback layout       */
}
@supports selector(:has(a, b)) {
  .form:has(:user-invalid) .submit { opacity: .5; }
}
@supports (color: oklch(0% 0 0)) {
  :root { --brand: oklch(32% 0.07 265); }
}
```

CHROMIUM 141

```
CSS.supports("display: grid")                 true
CSS.supports("container-type: inline-size")   true
CSS.supports("color: oklch(50% 0.1 200)")     true
CSS.supports("anchor-name: --a")              true
CSS.supports("animation-timeline: scroll()")  true
CSS.supports("grid-template-rows: masonry")   false
CSS.supports("selector(:has(a))")             true
CSS.supports("selector(:focus-visible)")      true
```

PRACTICE CHECKPOINT

A **card component** that changes layout based on its **container width**, written with **native nesting**, inside a **cascade-layered** stylesheet, with **subgrid-aligned card footers**.

COMPONENTS.CSS: SAMPLE SOLUTION

CSS

```css
@layer reset, base, components, utilities;

@layer reset {
  *, *::before, *::after { box-sizing: border-box; }
  img { display: block; max-inline-size: 100%; }
}
@layer base {
  body { font-family: "Poppins", sans-serif; color: #1d2433; margin: 0; padding: 14px; background:
#f4f6fb; }
}
@layer components {
  .course-grid {
    display: grid; gap: 14px;
    grid-template-columns: repeat(auto-fit, minmax(min(220px, 100%), 1fr));
  }
  .course {
    container-type: inline-size;
    display: grid; grid-row: span 3; grid-template-rows: subgrid;   /* footers align */
    background: #fff; border-radius: 14px; overflow: hidden;
    box-shadow: 0 2px 6px rgb(26 42 79 / .1);

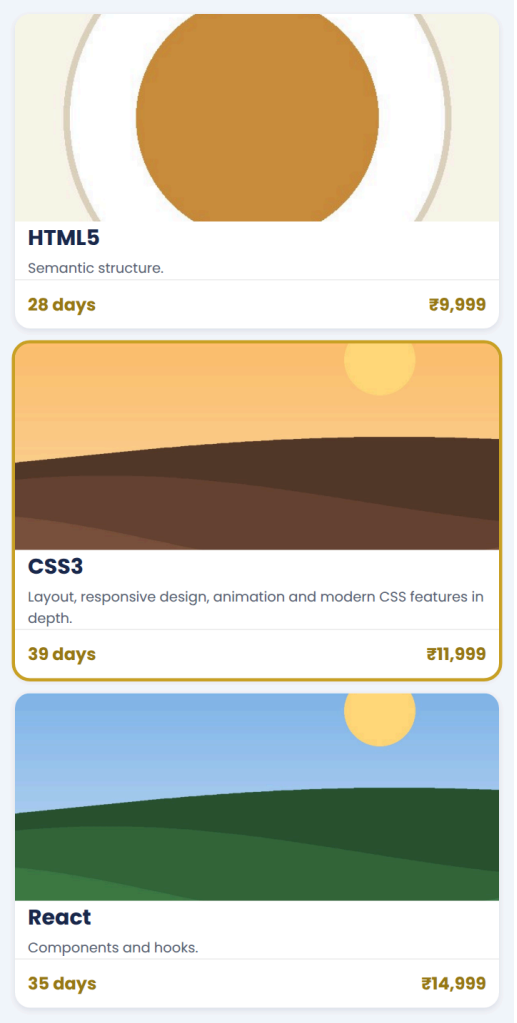
    .course__media img { inline-size: 100%; aspect-ratio: 16 / 9; object-fit: cover; }
    .course__body { padding-inline: 12px; }
    h3 { margin: 0 0 4px; color: #1a2a4f; font-size: clamp(15px, 6cqi, 20px); }
    p { margin: 0; font-size: 13px; color: #5c6475; }
    .course__footer { padding: 10px 12px; border-block-start: 1px solid #eee; font-weight: 700; color:
#9a7a14; }

    /* .course is the container; its descendants respond to its width */
    @container (min-width: 300px) {
      .course__media img { aspect-ratio: 21 / 9; }
      .course__footer { display: flex; justify-content: space-between; }
    }
  }
}
@layer utilities {
  .featured { outline: 3px solid #c9a227; }
}
```

760 px: three columns



480 px: one column



AT 760PX THE FOOTERS LINE UP THROUGH SUBGRID; AT 480PX EACH CARD IS WIDER THAN 300PX, SO ITS CONTAINER QUERY APPLIES (21:9 IMAGE, FOOTER IN A ROW)

MODULE 17 · ADVANCED

Modern colour, visual effects and scroll features

Minimum duration **2 days** | Level **Advanced**

This module covers the newest visual capabilities of CSS, many of which replace JavaScript libraries: the oklch() and oklab() colour spaces, color-mix(), relative colour syntax, light-dark() with color-scheme and accent-color; filters, backdrop-filter glassmorphism, blend modes, clip-path shapes, masks, object-fit and object-position; smooth scrolling, scroll snapping, scroll-margin and scroll-padding, overscroll-behavior, scrollbar styling and scroll-driven animations; and an overview of view transitions, anchor positioning, styling dialogs and popovers, and checking support with Baseline and Can I Use. Colour conversions and support checks were evaluated in Chromium 141.

IN THIS MODULE

17.1 Modern colour
17.2 Visual effects
17.3 Scrolling
17.4 Other modern features

17.1 Modern colour

17.1.1 oklch() and oklab() colour spaces

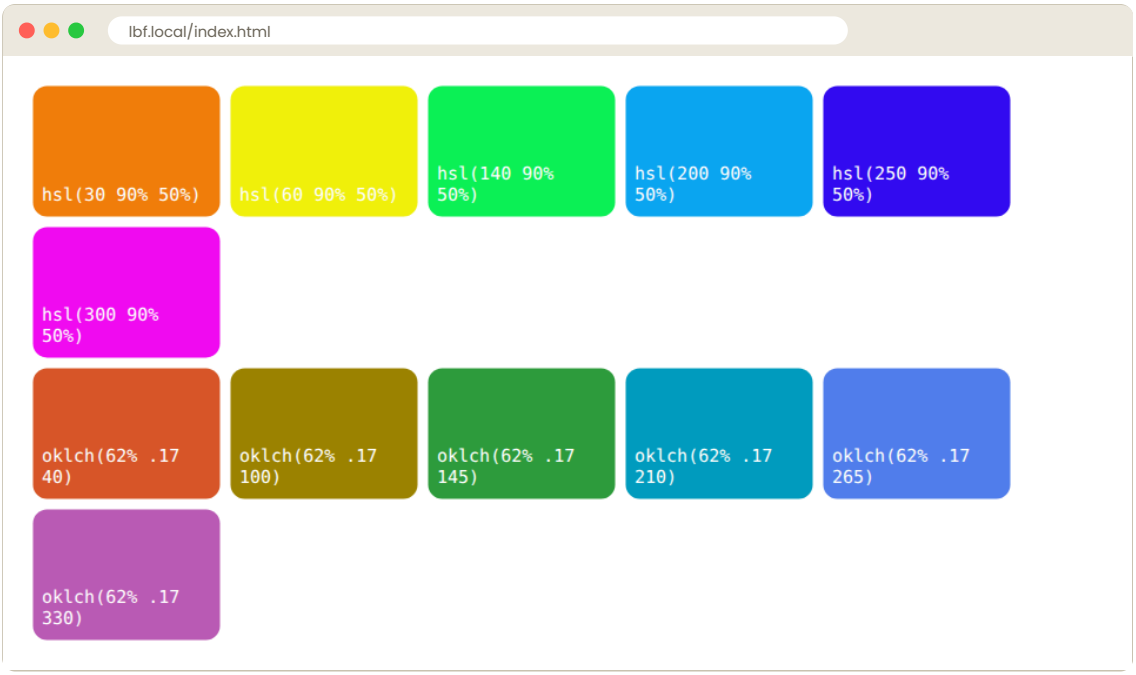
`rgb()` and `hsl()` describe colours in the sRGB space, and hsl's "lightness" is not how bright a colour **looks**: `hsl(60 100% 50%)` (yellow) looks far brighter than `hsl(240 100% 50%)` (blue) at the same lightness. **OKLCH** (and its rectangular form OKLab), Baseline 2023, is **perceptually uniform**: equal changes look like equal changes, and colours with the same L really look equally light. It also reaches the wider **P3** gamut of modern screens.

Channel	Range	Meaning
L (lightness)	0% to 100% (or 0 to 1)	perceived lightness: 0 black, 100% white
C (chroma)	0 to about 0.37	colourfulness: 0 is grey
H (hue)	0 to 360 (degrees)	hue angle: about 25 red, 70 orange, 110 yellow, 145 green, 230 cyan-blue, 265 blue, 330 pink
/ alpha	0 to 1	opacity

SAME LIGHTNESS IN HSL (TOP) AND OKLCH (BOTTOM)

CSS

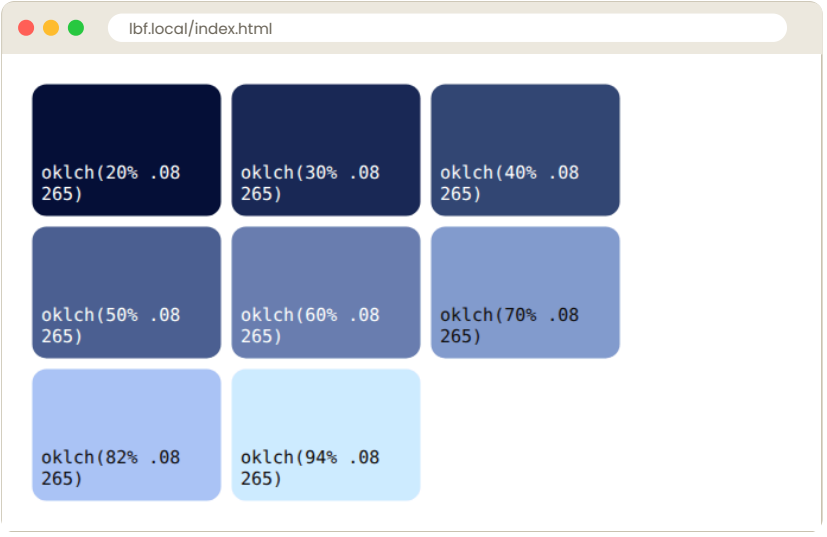
```
.sw { display: inline-flex; align-items: end; width: 96px; height: 64px; margin: 3px; padding: 5px; border-radius: 8px; font: 10px/1.2 ui-monospace, monospace; color: #fff; vertical-align: top; } .dk { color: #111; }
```



THE HSL ROW VARIES WILDLY IN BRIGHTNESS; THE OKLCH ROW LOOKS EVENLY BRIGHT

A NAVY PALETTE: ONLY L CHANGES CSS

```
.sw { display: inline-flex; align-items: end; width: 96px; height: 64px; margin: 3px; padding: 5px; border-radius: 8px; font: 10px/1.2 ui-monospace, monospace; color: #fff; vertical-align: top; } .dk { color: #111; }
```



HOW THE BROWSER DISPLAYS IT

COMPUTED IN CHROMIUM

```
oklch(32% 0.07 265)      -> oklch(0.32 0.07 265)
oklch(72% 0.14 85)       -> oklch(0.72 0.14 85)
oklab(0.5 0.1 -0.1)      -> oklab(0.5 0.1 -0.1)
```

Chromium keeps modern colours in their own space for computed values; DevTools can show and convert them in the colour picker.

17.1.2 color-mix()

`color-mix(in <space>, colour1 p1%, colour2 p2%)` (Baseline 2023) blends two colours in a chosen colour space. It creates tints, shades and translucent versions of brand colours without new variables: `color-`

`mix(in oklch, var(--brand), white 20%)` for a hover, `color-mix(in srgb, var(--brand) 15%, transparent)` for a subtle background. The space matters: `oklch` keeps blends vivid; `srgb` can pass through greyish mid-tones.

TINTS WITH COLOR-MIX(), AND 50/50 GREEN-RED MIXES IN FOUR COLOUR SPACES CSS

```
.sw { display: inline-flex; align-items: end; width: 96px; height: 64px; margin: 3px; padding: 5px; border-radius: 8px; font: 10px/1.2 ui-monospace, monospace; color: #fff; vertical-align: top; } .dk { color: #111; } :root { --brand: #1a2a4f; --accent: #c9a227; }
```



HOW THE BROWSER DISPLAYS IT

STATES DERIVED FROM ONE COLOUR CSS

```
:root { --brand: #1a2a4f; }
.btn { background: var(--brand); }
.btn:hover { background: color-mix(in oklch, var(--brand), white 15%); }
.btn:active { background: color-mix(in oklch, var(--brand), black 20%); }
.badge { background: color-mix(in srgb, var(--brand) 12%, transparent); color: var(--brand); }
```

COMPUTED IN CHROMIUM

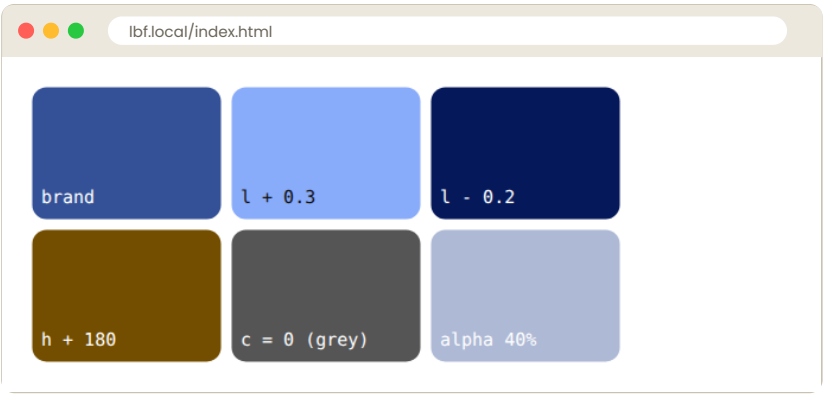
```
color-mix(in srgb, red 25%, blue) -> color(srgb 0.25 0 0.75)
```

17.1.3 Relative colour syntax

Relative colour syntax (Baseline 2024) derives a new colour from an existing one by naming its channels: `oklch(from var(--brand) calc(l + 0.2) c h)` is the brand colour 20 points lighter; `rgb(from var(--brand) r g b / 50%)` is the brand at half opacity; `hsl(from var(--accent) calc(h + 180) s l)` is the complementary hue. Inside, `l c h` (or `r g b`, `h s l`) are numbers you can use in calc().

COLOURS DERIVED FROM --BRAND WITH RELATIVE COLOUR SYNTAX CSS

```
.sw { display: inline-flex; align-items: end; width: 96px; height: 64px; margin: 3px; padding: 5px; border-radius: 8px; font: 10px/1.2 ui-monospace, monospace; color: #fff; vertical-align: top; } .dk { color: #111; } :root { --brand: oklch(45% 0.12 265); }
```



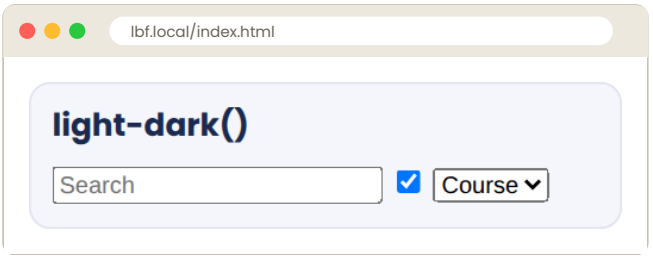
HOW THE BROWSER DISPLAYS IT

17.1.4 light-dark() and color-scheme

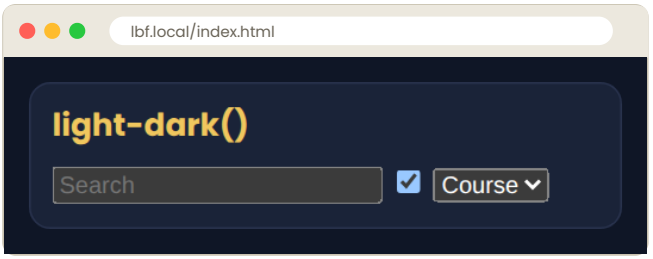
`color-scheme: light dark` on `:root` declares that the page supports both schemes; the browser then uses the user's preference for its own UI (form controls, scrollbars, default canvas) and enables `light-dark(lightValue, darkValue)` (Baseline 2024), which picks one of two colours according to the scheme in effect. Theming with only a few lines and no media queries:

THEMING WITH LIGHT-DARK() CSS

```
:root { color-scheme: light dark; }
body { background: light-dark(#ffffff, #0f1626); color: light-dark(#1d2433, #e8ecf5); }
.card { background: light-dark(#f4f6fb, #1a2338); border: 1px solid light-dark(#dde2ee, #2b3550);
border-radius: 12px; padding: 12px; font-family: "Poppins", sans-serif; }
h3 { color: light-dark(#1a2a4f, #f0c75e); margin: 0 0 8px; }
.force-dark { color-scheme: dark; }    /* a section can force one scheme */
```



USER PREFERENCE: LIGHT



USER PREFERENCE: DARK (EMULATED): COLOURS AND NATIVE CONTROLS BOTH SWITCH

17.1.5 accent-color for form controls

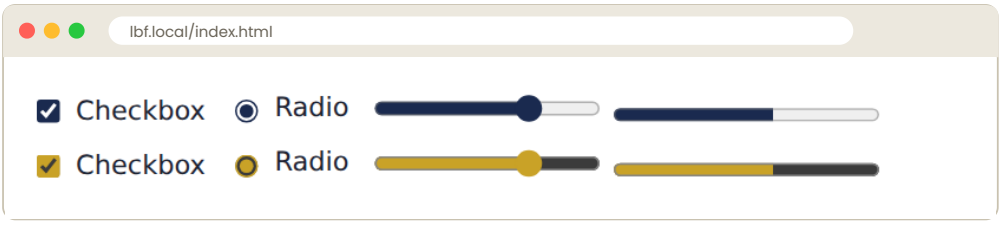
`accent-color` sets the brand colour of native checkboxes, radio buttons, range sliders and progress bars in one line, keeping their accessibility and behaviour. The browser automatically picks a contrasting tick colour.

ACCENT-COLOR: NAVY AND GOLD CSS

```css
.brand { accent-color: #c9a227; }
.navy { accent-color: #1a2a4f; }
label { display: inline-flex; gap: 6px; align-items: center; margin: 4px 12px 4px 0; }
```

INDEX.HTML HTML

```html
<div class="navy"><label><input type="checkbox" checked> Checkbox</label><label><input type="radio"
checked> Radio</label><input type="range" value="70"> <progress value="60" max="100"></progress></div>
<div class="brand"><label><input type="checkbox" checked> Checkbox</label><label><input type="radio"
checked> Radio</label><input type="range" value="70"> <progress value="60" max="100"></progress></div>
```



HOW THE BROWSER DISPLAYS IT

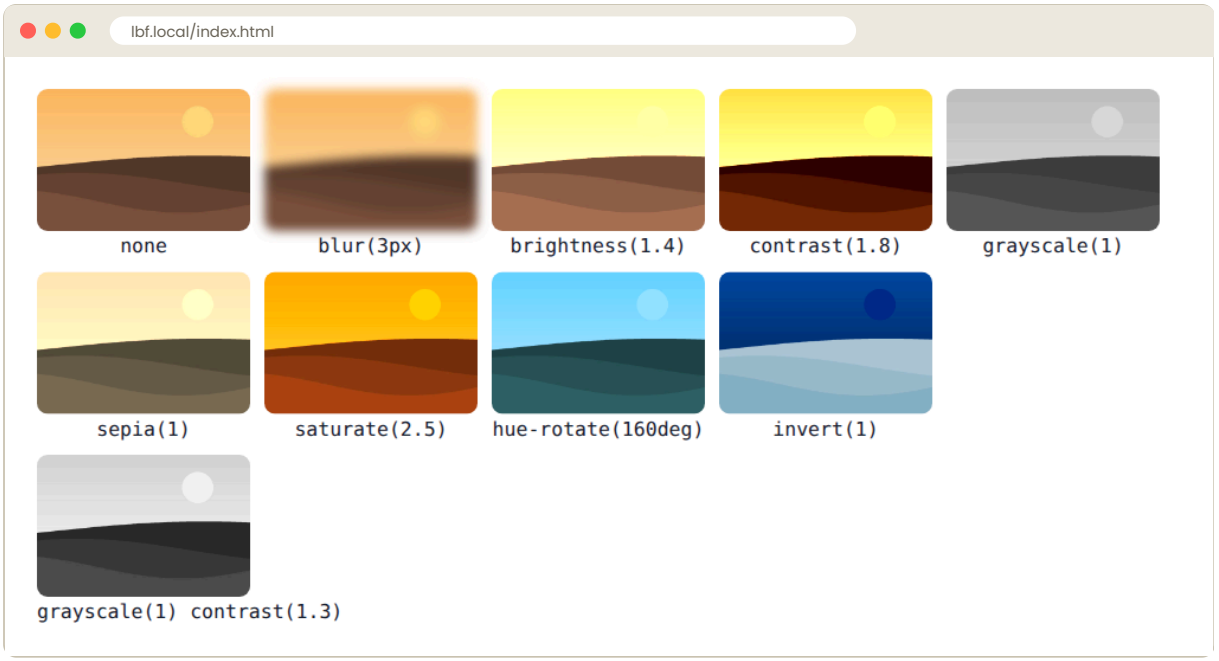
17.2 Visual effects

17.2.1 filter: blur, brightness, contrast, grayscale, drop-shadow

`filter` applies graphical effects to an element and everything inside it, like photo-editing adjustments: `blur()`, `brightness()`, `contrast()`, `grayscale()`, `sepia()`, `saturate()`, `hue-rotate()`, `invert()`, `opacity()` and `drop-shadow()`. Several can be chained and they animate smoothly. `drop-shadow()` follows the actual shape (transparent PNGs, SVG icons), unlike box-shadow.

FILTER FUNCTIONS CSS

```css
.f { display: inline-block; text-align: center; font: 11px ui-monospace, monospace; margin: 4px; } .f
img { width: 120px; height: 80px; object-fit: cover; display: block; border-radius: 6px; margin-
bottom: 2px; }
```



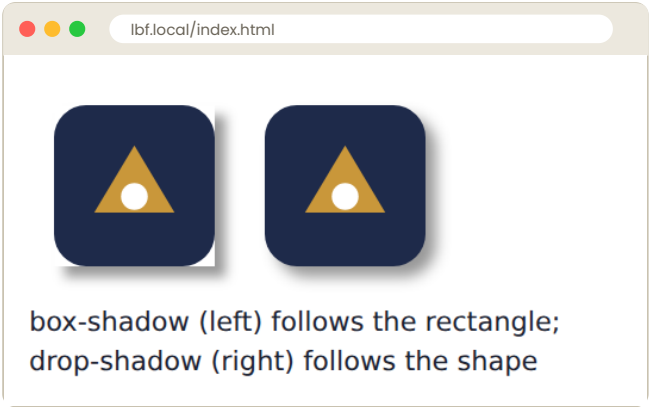
HOW THE BROWSER DISPLAYS IT

BOX-SHADOW VS FILTER: DROP-SHADOW() CSS

```css
img { width: 90px; margin: 14px; }
.box { box-shadow: 6px 6px 6px rgb(0 0 0 / .4); }
.drop { filter: drop-shadow(6px 6px 4px rgb(0 0 0 / .4)); }
```

INDEX.HTML HTML

```html
<img class="box" src="logo.svg" alt=""><img class="drop" src="logo.svg" alt=""><p
style="margin:0">box-shadow (left) follows the rectangle; drop-shadow (right) follows the shape</p>
```



HOW THE BROWSER DISPLAYS IT

17.2.2 backdrop-filter (glassmorphism)

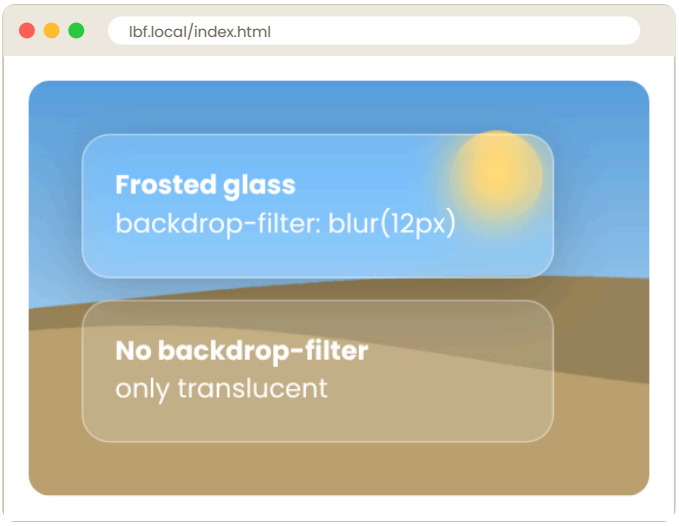
`backdrop-filter` applies filters to whatever is **behind** the element (the backdrop), seen through a translucent background. Blur plus a semi-transparent white or dark fill produces the frosted-glass look ("glassmorphism") used in macOS, iOS and Windows 11. Keep enough contrast for text, and provide a more opaque fallback with `@supports not (backdrop-filter: blur(1px))`.

GLASSMORPHISM CSS

```
.bg { background: url("beach.jpg") center / cover; padding: 30px; border-radius: 12px; }
.glass { background: rgb(255 255 255 / .18); backdrop-filter: blur(12px) saturate(1.4); -webkit-
backdrop-filter: blur(12px) saturate(1.4);
         border: 1px solid rgb(255 255 255 / .35); border-radius: 16px; padding: 18px; color: #fff;
width: 230px; font-family: "Poppins", sans-serif;
         box-shadow: 0 8px 32px rgb(0 0 0 / .2); }
.plain { background: rgb(255 255 255 / .18); border: 1px solid rgb(255 255 255 / .35); border-radius:
16px; padding: 18px; color: #fff; width: 230px; font-family: "Poppins", sans-serif; margin-top: 12px;
}
```

INDEX.HTML HTML

```
<div class="bg"><div class="glass"><b>Frosted glass</b><br>backdrop-filter: blur(12px)</div><div
class="plain"><b>No backdrop-filter</b><br>only translucent</div></div>
```



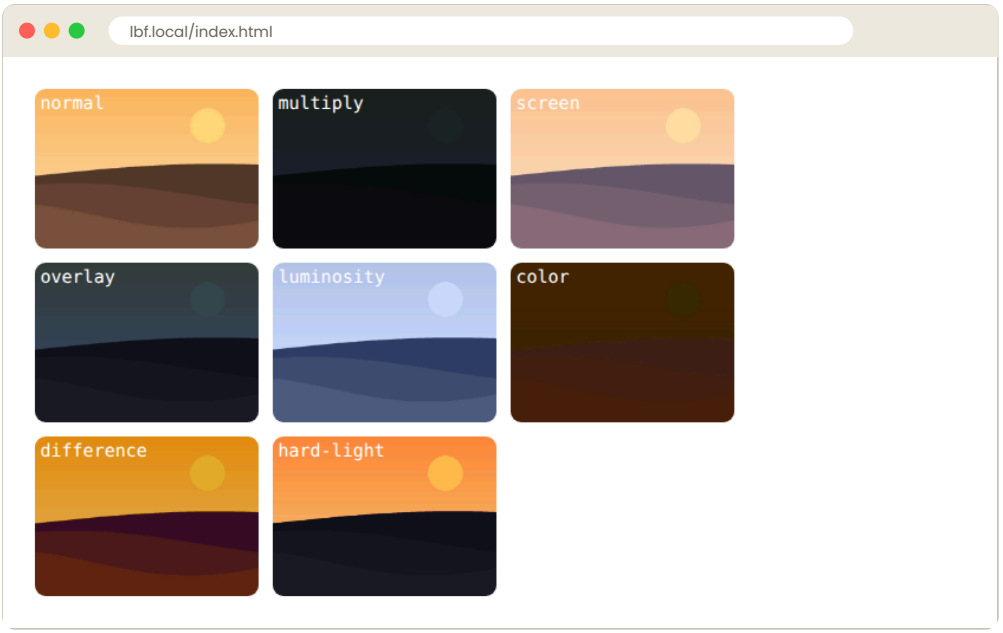
HOW THE BROWSER DISPLAYS IT

17.2.3 mix-blend-mode and background-blend-mode

Blend modes combine colours like layers in Photoshop: `multiply` (darken), `screen` (lighten), `overlay`, `darken`, `lighten`, `color-dodge`, `difference`, `luminosity`, `color` and more. `background-blend-mode` blends an element's own background layers (a colour tint on a photo); `mix-blend-mode` blends the element with what is behind it (text that inverts over images). `isolation: isolate` limits blending to a group.

BACKGROUND-BLEND-MODE: A NAVY LAYER BLENDED WITH A PHOTO CSS

```
.b { display: inline-block; width: 120px; height: 84px; margin: 4px; border-radius: 6px; background:
url("mysuru-palace.jpg") center / cover, #1a2a4f; font: 10px ui-monospace, monospace; color: #fff;
padding: 3px; }
```



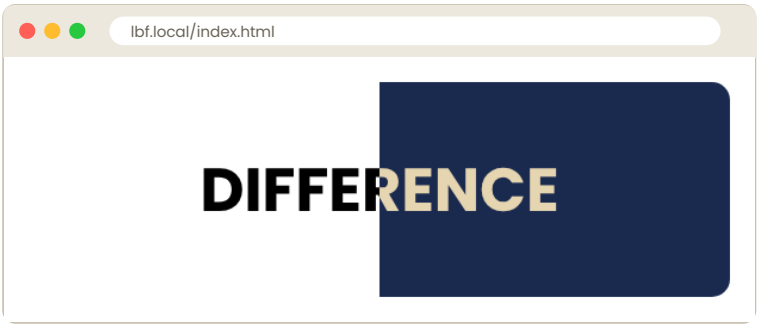
HOW THE BROWSER DISPLAYS IT

TEXT THAT INVERTS WITH MIX-BLEND-MODE: DIFFERENCE CSS

```css
.hero { position: relative; height: 120px; background: linear-gradient(90deg, #fff 50%, #1a2a4f 50%);
border-radius: 10px; display: grid; place-items: center; }
.hero h2 { color: #fff; mix-blend-mode: difference; font: 800 34px "Poppins", sans-serif; margin: 0; }
```

INDEX.HTML HTML

```html
<div class="hero"><h2>DIFFERENCE</h2></div>
```



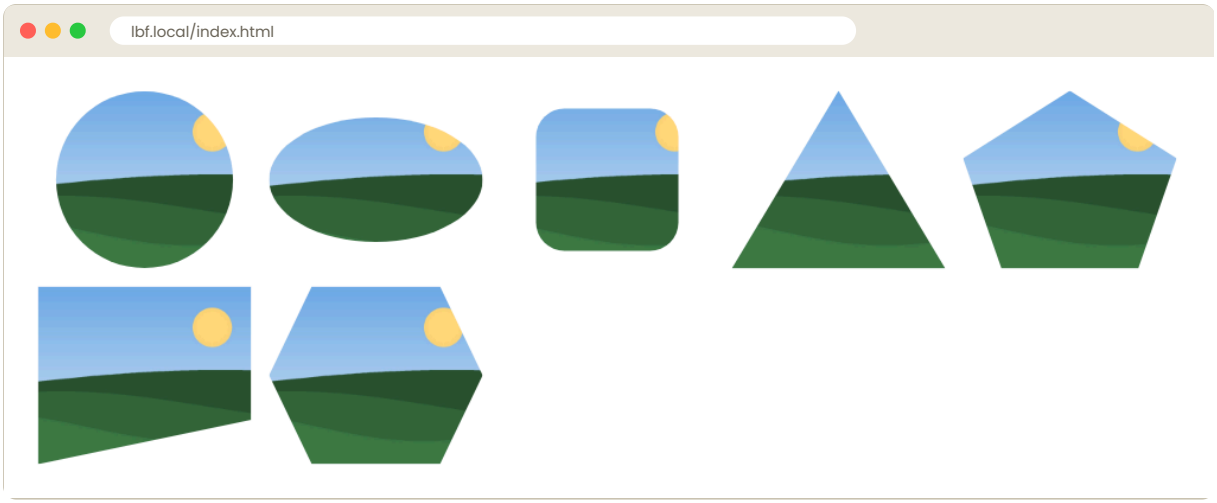
HOW THE BROWSER DISPLAYS IT

17.2.4 clip-path shapes

`clip-path` shows only part of an element, cut to a shape: `circle()`, `ellipse()`, `inset()` (rounded rectangles), `polygon()` (any shape from points), `path()` (SVG path data), and the newer `shape()`. Shapes with the same number of points can be animated, enabling reveal effects. Clipped-away areas do not receive clicks.

CLIP-PATH SHAPES CSS

```css
.c { display: inline-block; width: 120px; height: 100px; margin: 5px; background: url("hills.jpg")
center / cover; font: 10px ui-monospace, monospace; color: #fff; vertical-align: top; }
```



CIRCLE, ELLIPSE, ROUNDED INSET, TRIANGLE, PENTAGON, SLANTED SECTION EDGE, HEXAGON

COMMON CLIP-PATH USES CSS

```css
.avatar  { clip-path: circle(50%); }
.section { clip-path: polygon(0 0, 100% 0, 100% 88%, 0 100%); }       /* slanted bottom edge */
.reveal  { clip-path: inset(0 100% 0 0); transition: clip-path .6s; }  /* hidden to the right */
.reveal.is-visible { clip-path: inset(0 0 0 0); }                       /* wipes in */
```

17.2.5 mask-image

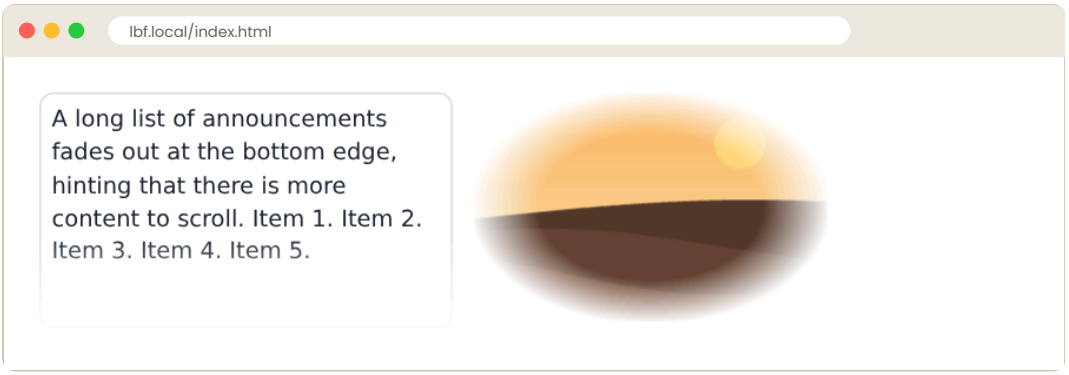
A **mask** uses an image's alpha channel (or luminance) to decide visibility: opaque parts show the element, transparent parts hide it, and semi-transparent parts fade it. Unlike clip-path, masks can be soft. Gradients make the most common masks: fading the edge of a scrolling list, a vignette, or colouring an SVG icon with `background-color` while the SVG is the mask (Baseline 2023, with a `-webkit-` prefix still useful).

FADING TEXT, A VIGNETTE AND AN SVG ICON PAINTED WITH A GRADIENT CSS

```css
.fade { width: 220px; height: 120px; overflow: hidden; display: inline-block; margin: 6px; vertical-
align: top; border: 1px solid #ddd; border-radius: 8px; padding: 6px; font-size: 13px;
        mask-image: linear-gradient(to bottom, #000 60%, transparent); }
.vig { width: 200px; height: 130px; display: inline-block; margin: 6px; background: url("mysuru-
palace.jpg") center / cover; mask-image: radial-gradient(ellipse, #000 45%, transparent 72%); }
.icon { display: inline-block; width: 60px; height: 60px; margin: 6px; background: linear-
gradient(135deg, #1a2a4f, #c9a227); mask: url("logo.svg") center / contain no-repeat; }
```

INDEX.HTML HTML

```html
<div class="fade">A long list of announcements fades out at the bottom edge, hinting that there is
more content to scroll. Item 1. Item 2. Item 3. Item 4. Item 5.</div><div class="vig"></div><span
class="icon"></span>
```



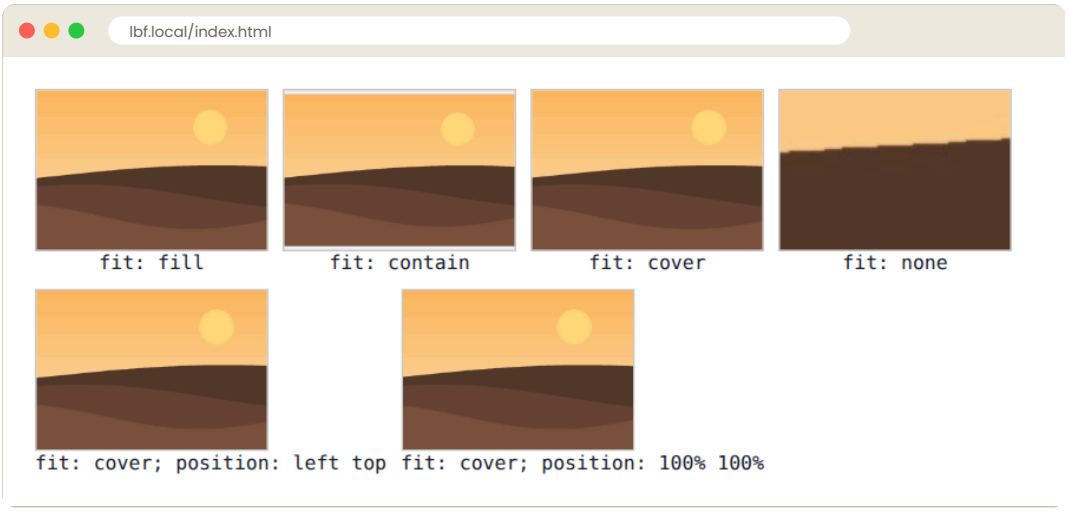
HOW THE BROWSER DISPLAYS IT

17.2.6 object-fit and object-position

For replaced elements (`img`, `video`), `object-fit` decides how the content fits the element's box, like background-size: `fill` (default, stretches), `contain`, `cover`, `none` (natural size), `scale-down`. `object-position` chooses the alignment (and therefore the crop): `top`, `30% 70%`.

OBJECT-FIT AND OBJECT-POSITION ON A 130 × 90 BOX CSS

```
.o { display: inline-block; text-align: center; font: 11px ui-monospace, monospace; margin: 4px; } .o
img { width: 130px; height: 90px; background: #eef1f8; display: block; border: 1px solid #ccc; }
```



HOW THE BROWSER DISPLAYS IT

17.3 Scrolling

17.3.1 scroll-behavior: smooth

`html { scroll-behavior: smooth; }` animates scrolling when the user follows an in-page link (`#section`) or a script calls `scrollTo()` / `scrollIntoView()`, instead of jumping. Wrap it in `@media (prefers-reduced-motion: no-preference)` so people who asked for less motion keep instant jumps.

SMOOTH SCROLLING, RESPECTFULLY CSS

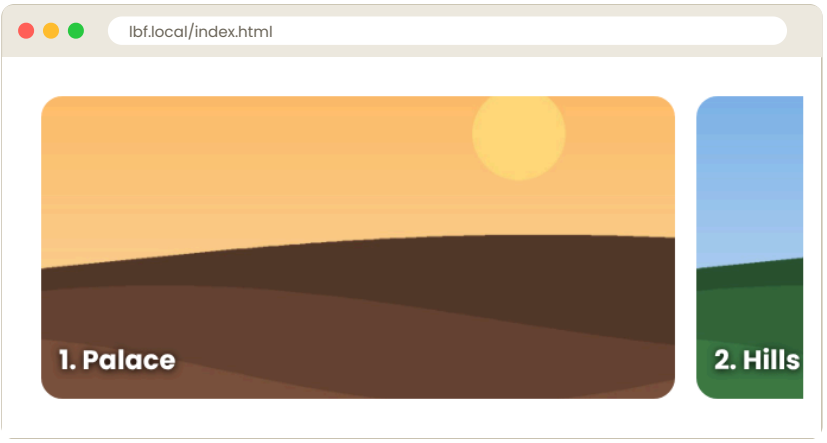
```
@media (prefers-reduced-motion: no-preference) {
  html { scroll-behavior: smooth; }
}
```

17.3.2 Scroll snapping

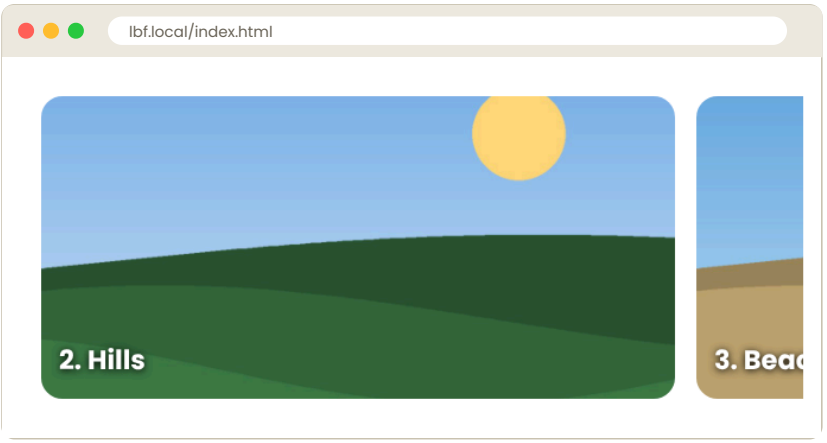
Scroll snapping makes a scroll container settle on defined positions after the user scrolls, like a native carousel or full-page sections, with no JavaScript. The container gets `scroll-snap-type: x mandatory` (or `y`, and `proximity` for gentler snapping); each child gets `scroll-snap-align: start | center | end`, and optionally `scroll-snap-stop: always` to prevent skipping items on fast swipes.

A SNAPPING CAROUSEL CSS

```
.carousel { display: flex; gap: 12px; overflow-x: auto; scroll-snap-type: x mandatory; padding: 8px;
width: 420px;
          scroll-padding-inline: 8px; overscroll-behavior-x: contain; scrollbar-width: thin; }
.slide { flex: 0 0 80%; scroll-snap-align: start; border-radius: 12px; height: 150px; background:
center / cover;
        display: flex; align-items: end; padding: 10px; color: #fff; font: 600 15px "Poppins", sans-
serif; text-shadow: 0 1px 4px #000; }
```



INITIAL POSITION



AFTER SCROLLING 250PX: THE BROWSER SNAPPED TO THE START OF SLIDE 2

COMPUTED IN CHROMIUM

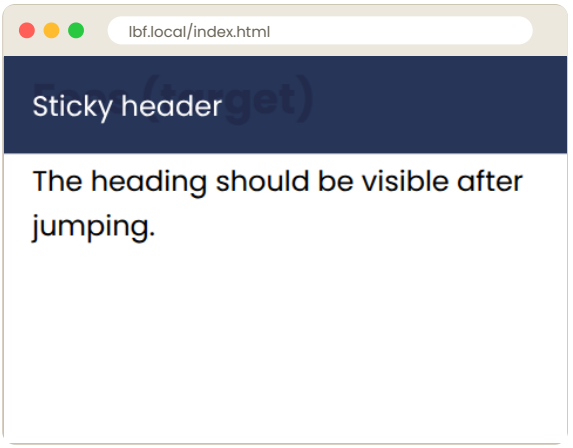
```
requested scrollLeft 250 -> snapped to 368
```

17.3.3 scroll-margin and scroll-padding

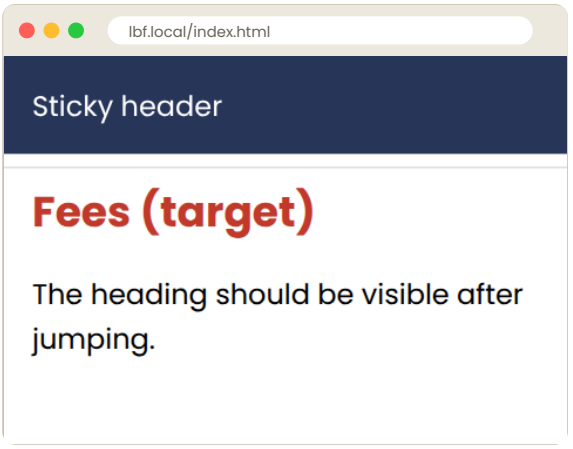
When you jump to `#section` on a page with a sticky header, the heading lands **under** the header. `scroll-padding-top` on the scroll container (usually `html`) reserves space at the top for all snapping and anchor scrolling; `scroll-margin-top` on a target element adds space for just that element.

ANCHORS THAT CLEAR A STICKY HEADER CSS

```
html { scroll-padding-top: 80px; }          /* height of the sticky header + breathing room */
h2[id] { scroll-margin-top: 1rem; }          /* extra space above headings that are link targets */
```



WITHOUT SCROLL-PADDING: AFTER JUMPING TO #FEES IN A 220PX-TALL WINDOW, THE HEADING IS HIDDEN UNDER THE HEADER



WITH HTML { SCROLL-PADDING-TOP: 64PX }: THE HEADING LANDS JUST BELOW THE HEADER

17.3.4 overscroll-behavior

When a scrollable area (a chat window, a modal, a side drawer) reaches its end, further scrolling normally "chains" to the page behind it, which is disorienting. `overscroll-behavior: contain` stops scroll chaining (and the pull-to-refresh or bounce effects on mobile); `none` also prevents the bounce on the element itself. Axis-specific versions are `overscroll-behavior-x` / `-y`.

CONTAINING SCROLL CSS

```
.chat-messages { overflow-y: auto; overscroll-behavior: contain; }
.modal__body   { overflow-y: auto; overscroll-behavior-y: contain; }
body.app       { overscroll-behavior-y: none; }    /* disable pull-to-refresh in a web app */
```

17.3.5 Styling scrollbars

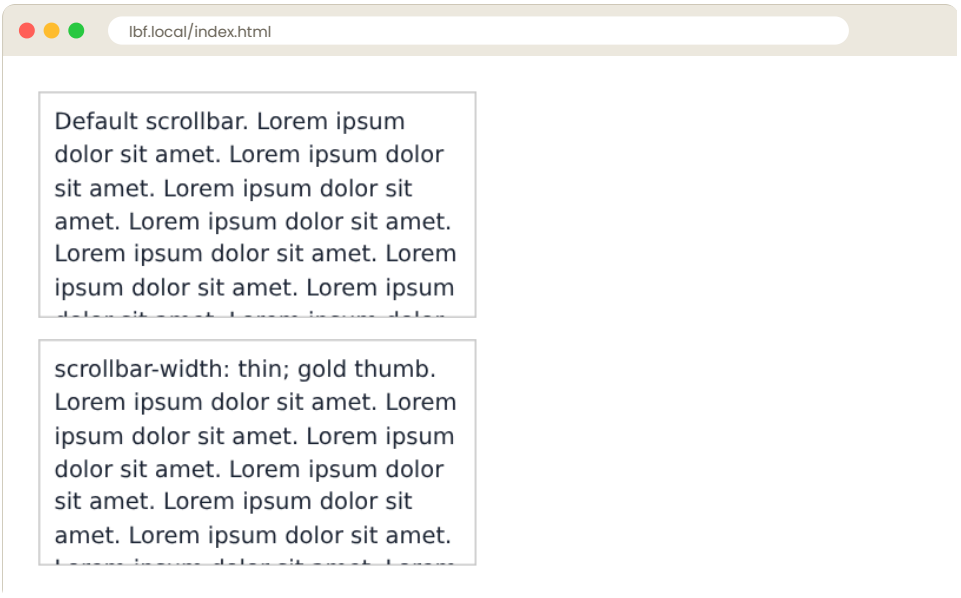
The standard properties (Baseline 2024 with Safari 26, earlier in Chromium and Firefox) are `scrollbar-width: auto | thin | none` and `scrollbar-color: thumb track`. `scrollbar-gutter: stable` reserves space for the scrollbar so content does not jump when it appears. Older WebKit-only pseudo-elements (`::-webkit-scrollbar`, `::-webkit-scrollbar-thumb`) allow more detailed styling in Chromium and Safari. Never hide scrollbars on content that must be scrollable without another visible cue.

STANDARD SCROLLBAR STYLING CSS

```css
.box { width: 230px; height: 110px; overflow-y: scroll; border: 1px solid #ccc; padding: 8px; display: inline-block; margin: 6px; vertical-align: top; font-size: 13px; }
.styled { scrollbar-width: thin; scrollbar-color: #c9a227 #eef1f8; }
.hidden { scrollbar-width: none; }
```

INDEX.HTML HTML

```html
<div class="box">Default scrollbar. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. </div><div class="box styled">scrollbar-width: thin; gold thumb. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet. </div>
```



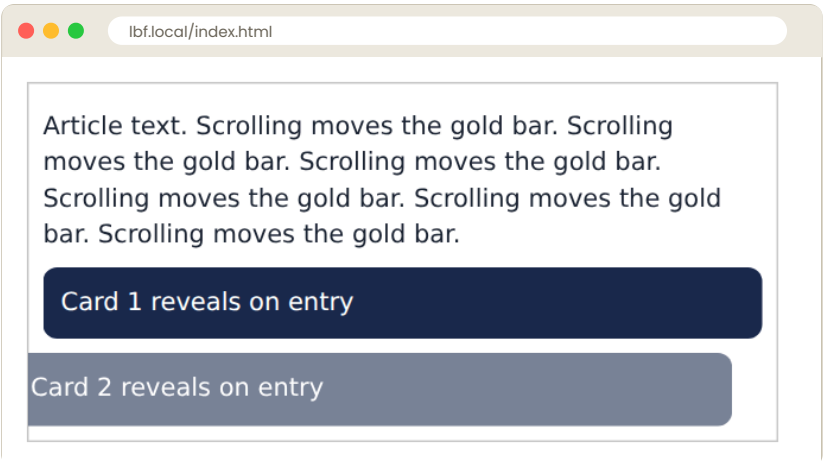
HOW THE BROWSER DISPLAYS IT

17.3.6 Scroll-driven animations (animation-timeline: scroll() and view())

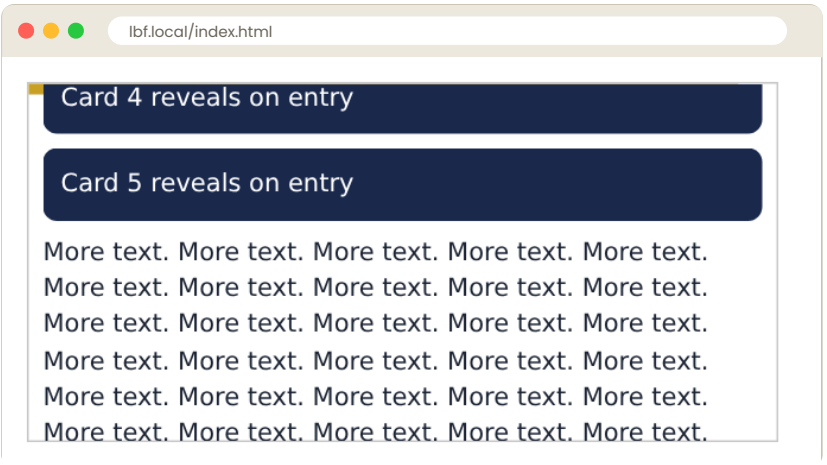
Scroll-driven animations (Chromium 115+, Safari 26; Firefox behind a flag in 2026) link a keyframe animation's progress to **scrolling** instead of time, with no JavaScript and smooth performance. `animation-timeline: scroll()` follows the scroll position of a scroll container (a reading-progress bar); `animation-timeline: view()` follows an element's progress through the viewport (reveal-on-scroll effects). `animation-range` limits which part of the timeline is used, for example `entry 0% cover 40%`.

READING PROGRESS AND REVEAL ON SCROLL CSS

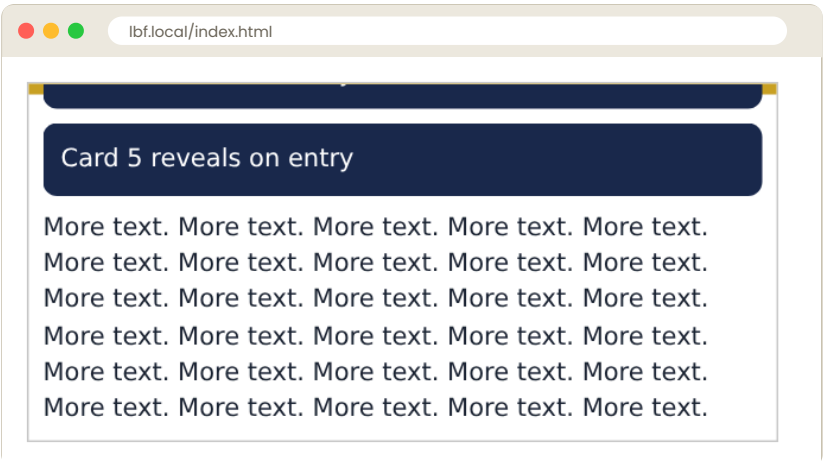
```css
.scroller { height: 200px; overflow-y: auto; border: 1px solid #ccc; position: relative; width: 420px; font-size: 14px; }
.progress { position: sticky; top: 0; height: 6px; background: #c9a227; transform-origin: left;
            animation: grow linear both; animation-timeline: scroll(nearest block); }
@keyframes grow { from { transform: scaleX(0); } to { transform: scaleX(1); } }
.reveal { animation: fade-in linear both; animation-timeline: view(); animation-range: entry 0% cover 35%; background: #1a2a4f; color: #fff; padding: 10px; border-radius: 8px; margin: 8px; }
@keyframes fade-in { from { opacity: 0; translate: -40px 0; } }
p { margin: 8px; }
```



SCROLLTOP 0: NO PROGRESS



SCROLLTOP 260: BAR ABOUT HALFWAY; CARDS FADING IN AS THEY ENTER



SCROLLTOP AT THE END: FULL BAR

COMPUTED IN CHROMIUM

```
timeline: ScrollTimeline
scrolled 50% -> animation progress 50%
```

17.4 Other modern features

17.4.1 View transitions (overview)

The **View Transitions API** animates between two states of a page, or between two pages, with a cross-fade by default and shared-element "morphs" for elements given the same `view-transition-name`. For single-page updates, JavaScript wraps the DOM change in `document.startViewTransition(() => update())` (Baseline 2025). For multi-page sites, one CSS rule enables transitions between same-origin page loads (Chromium 126+ and Safari 18.2+). Animations are customised with the `::view-transition-old()` and `::view-transition-new()` pseudo-elements.

VIEW TRANSITIONS CSS

```css
/* Multi-page sites: opt in on both pages */
@view-transition { navigation: auto; }

/* The thumbnail on the list page and the hero image on the detail page morph into each other */
.course-thumb, .course-hero { view-transition-name: course-image; }   /* one element per page */

/* Customise the default cross-fade */
::view-transition-old(root) { animation: 300ms ease-out both fade-out; }
::view-transition-new(root) { animation: 300ms ease-out both fade-in; }
@media (prefers-reduced-motion: reduce) { ::view-transition-group(*) { animation: none; } }
```

STARTING A SAME-DOCUMENT TRANSITION JavaScript

```javascript
// Single-page update
button.addEventListener("click", () => {
  if (!document.startViewTransition) return renderNextSlide();   // fallback
  document.startViewTransition(() => renderNextSlide());
});
```

17.4.2 Anchor positioning for tooltips and popovers (overview)

Anchor positioning (Chromium 125+, Safari 26, Firefox 147+ in early 2026) positions an element relative to another element anywhere in the page, without JavaScript libraries like Popper or Floating UI. The anchor gets `anchor-name: --x`; the positioned element uses `position-anchor: --x` and either `position-area` (a 3×3 grid around the anchor, such as `top center`) or the `anchor()` function in inset properties. `position-try-fallbacks` flips it when there is not enough room (for example below instead of above). Popovers and tooltips are its main use.

ANCHOR POSITIONING CSS

```css
.trigger { anchor-name: --help; margin: 70px 0 10px 150px; font: 600 14px "Poppins", sans-serif; }
.tip { position: absolute; position-anchor: --help; position-area: top center; margin-bottom: 8px;
       background: #1a2a4f; color: #fff; padding: 6px 10px; border-radius: 6px; font-size: 13px;
white-space: nowrap; }
.tip2 { position: absolute; position-anchor: --help; position-area: right center; margin-left: 8px;
        background: #c9a227; color: #1a2a4f; padding: 6px 10px; border-radius: 6px; font-size: 13px; }
```

INDEX.HTML HTML

```html
<button class="trigger">Fee details</button><div class="tip">Tethered above with position-area</div>
<div class="tip2">and to the right</div>
```



BOTH BOXES ARE POSITIONED RELATIVE TO THE BUTTON BY CSS ALONE

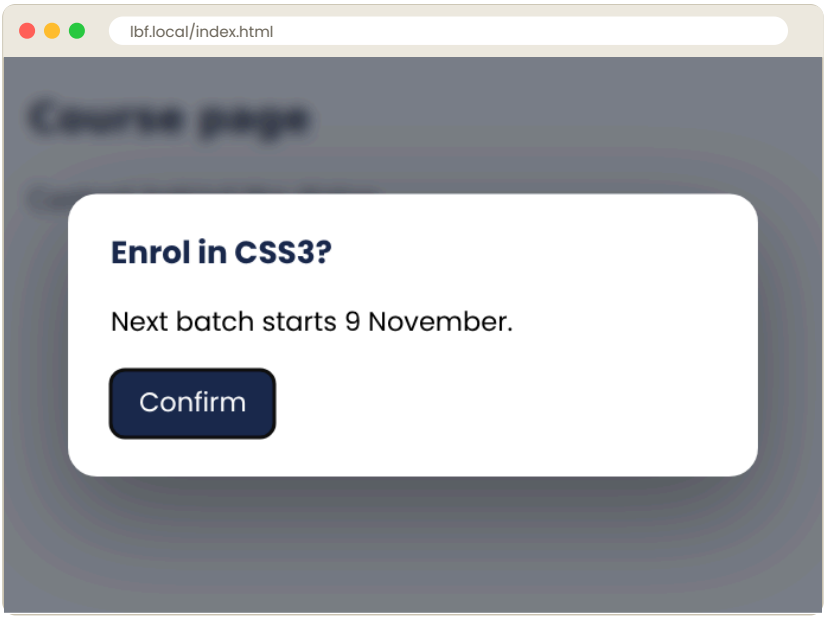
17.4.3 Styling <dialog> and popovers

Native `<dialog>` and `popover` elements (HTML course, Module 14) are rendered in the **top layer** with a `::backdrop`. Style the box, the backdrop and the open state (`dialog[open]`, `:popover-open`), and animate opening and closing with `@starting-style` and `allow-discrete` transitions of `display` and `overlay` (Module 13).

DIALOG.CSS

CSS

```
dialog { border: 0; border-radius: 16px; padding: 22px 24px; width: min(90vw, 340px); box-shadow: 0
20px 60px rgb(0 0 0 / .35);
         font-family: "Poppins", sans-serif; opacity: 1; scale: 1;
         transition: opacity .3s, scale .3s, display .3s allow-discrete, overlay .3s allow-discrete; }
dialog::backdrop { background: rgb(15 22 38 / .55); backdrop-filter: blur(4px); transition: background
.3s, display .3s allow-discrete, overlay .3s allow-discrete; }
@starting-style { dialog[open] { opacity: 0; scale: .9; } dialog[open]::backdrop { background:
transparent; } }
dialog h3 { margin: 0 0 6px; color: #1a2a4f; }
dialog button { background: #1a2a4f; color: #fff; border: 0; border-radius: 8px; padding: 8px 16px;
font: inherit; }
body { min-height: 280px; }
```



THE DIALOG AFTER SHOWMODAL() AND ITS 0.3 S ENTRY TRANSITION

17.4.4 Checking support with Baseline and Can I Use

Before using a new feature, check its status: MDN pages and caniuse.com show **Baseline** badges ("Widely available", "Newly available", "Limited availability") and per-browser version tables; web.dev

publishes the yearly Baseline feature list; the `@supports` rule and `CSS.supports()` test at run time. The status of this module's features at the time of writing:

Feature	Status (2026)	Safe usage
oklch(), color-mix(), accent-color, filters, clip-path, object-fit, scroll snap	Baseline widely or newly available	use freely
Relative colour syntax, light-dark(), backdrop-filter, masks, scrollbar-color	Baseline newly available	use; consider a simple fallback
View transitions (same-document)	Baseline newly available (2025)	use as enhancement (feature-detect in JS)
Cross-document view transitions, anchor positioning	most engines; check Can I Use	enhancement with fallback
Scroll-driven animations	Chromium and Safari; Firefox flag	enhancement: content must work without it

CSS.supports() IN CHROMIUM 141

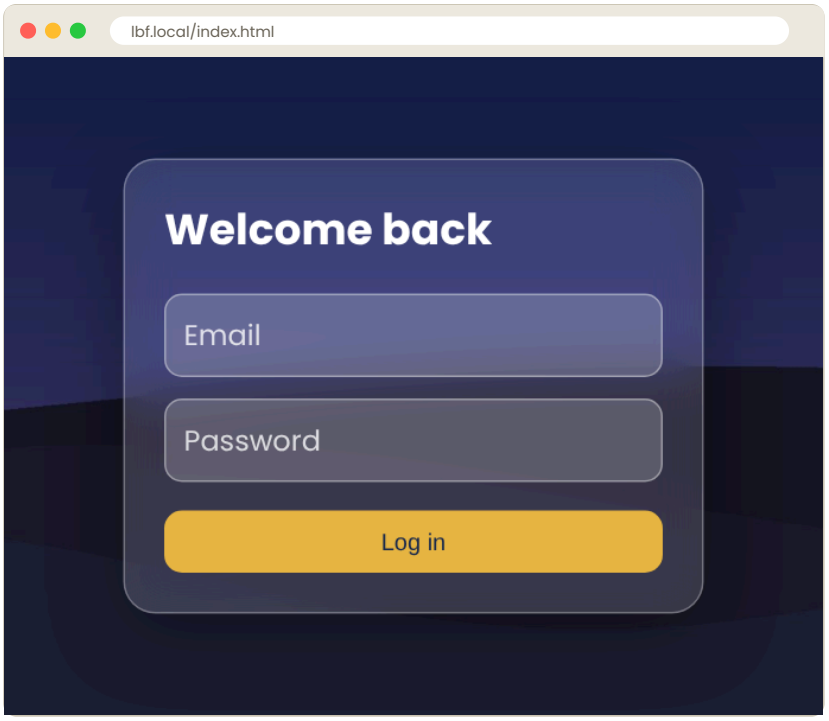
```
color: oklch(50% 0.1 200)                  true
color: color-mix(in oklch, red, blue)      true
color: rgb(from red r g b / 50%)           true
color: light-dark(#fff, #000)              true
backdrop-filter: blur(4px)                 true
animation-timeline: view()                 true
position-area: top center                  true
view-transition-name: hero                 true
scrollbar-color: red blue                  true
```

PRACTICE CHECKPOINT

A **glassmorphism login card**, a **scroll-snapping testimonial carousel**, and a **reading-progress bar** driven by a scroll-driven animation.

GLASS-LOGIN.CSS: SAMPLE SOLUTION (PART 1) CSS

```css
body { margin: 0; font-family: "Poppins", sans-serif; }
.scene { min-height: 330px; display: grid; place-items: center; background: url("night.jpg") center /
cover; padding: 20px; }
.login { width: 280px; padding: 22px; border-radius: 18px; color: #fff;
         background: rgb(255 255 255 / .14); border: 1px solid rgb(255 255 255 / .3);
         backdrop-filter: blur(14px) saturate(1.5); -webkit-backdrop-filter: blur(14px) saturate(1.5);
         box-shadow: 0 10px 40px rgb(0 0 0 / .35); }
@supports not (backdrop-filter: blur(1px)) { .login { background: rgb(26 42 79 / .92); } }
.login h2 { margin: 0 0 12px; }
.login input { width: 100%; box-sizing: border-box; margin: 6px 0; padding: 10px; border-radius: 10px;
font: inherit;
               border: 1px solid rgb(255 255 255 / .4); background: rgb(255 255 255 / .2); color:
#fff; }
.login input::placeholder { color: rgb(255 255 255 / .8); }
.login button { width: 100%; margin-top: 10px; padding: 10px; border: 0; border-radius: 10px; font:
600 15px inherit;
                background: oklch(80% 0.14 85); color: #1a2a4f; }
```



THE GLASSMORPHISM LOGIN CARD

The testimonial carousel reuses the snapping recipe of 17.3.2 with `scroll-snap-align: center` and one testimonial per slide, and the reading-progress bar is the `animation-timeline: scroll()` recipe of 17.3.6 attached to the root scroller: `animation-timeline: scroll(root)` on a `position: fixed` bar at the top of the page.

READING-PROGRESS.CSS: SAMPLE SOLUTION (PART 3) CSS

```css
.reading-progress {
  position: fixed; inset: 0 0 auto 0; height: 4px; z-index: 100;
  background: linear-gradient(90deg, #1a2a4f, #c9a227); transform-origin: 0 50%;
  animation: progress linear both; animation-timeline: scroll(root);
}
@keyframes progress { from { transform: scaleX(0); } to { transform: scaleX(1); } }
```

MODULE 18 · ADVANCED

CSS architecture and methodologies

Minimum duration **1 day** | Level **Advanced**

As projects grow, a clear architecture keeps CSS predictable and maintainable for a whole team. This module explains the problems that appear at scale (global scope, specificity wars, dead code), the main methodologies (BEM, OOCSS, SMACSS, ITCSS and utility-first CSS as a bridge to Tailwind CSS), how to organise files into base, layout, components and utilities with design systems and tokens, and an overview of scoped styles in frameworks: CSS Modules, CSS-in-JS and the scoped styles of Vue and Angular.

IN THIS MODULE

18.1 Common problems at scale
18.2 Methodologies
18.3 Organising files
18.4 Scoped styles in frameworks (overview)

18.1 Common problems at scale

18.1.1 Global scope, specificity wars, dead code

A 200-line stylesheet written by one person is easy to manage. A 20,000-line stylesheet edited by ten people over three years is not, unless it follows rules. Three problems appear in almost every large codebase:

Problem	What happens	Typical symptom
Global scope	every selector applies to the whole page; a rule written for one page (`.title { ... }`) leaks into others	"I changed the blog title and the checkout page broke"
Specificity wars	overrides need ever more specific selectors, then ids, then !important	`#main .content div.box h2.title { color: red !important; }`
Dead code	nobody dares delete a rule because nobody knows where it is used	stylesheets only grow; 50-70% of shipped CSS is unused (Coverage panel, Module 20)
Inconsistency	each developer invents values	37 shades of grey, 14 font sizes, 23 different paddings
Coupling to markup	selectors mirror the DOM (`.sidebar ul li a`)	restructuring HTML breaks styles

GLOBAL SCOPE IN ACTION

CSS

```css
/* Written for the blog page */
.title { color: #c0392b; font-size: 28px; text-transform: uppercase; }
/* Written months later by someone else for product cards */
.card .title { font-size: 16px; }
```

INDEX.HTML

HTML

```
<h2 class="title">Blog post title</h2><div class="card" style="border:1px solid
#ddd;padding:8px;border-radius:8px;width:220px"><h3 class="title">Product name</h3><p>The product
title inherited red uppercase from the blog rule.</p></div>
```



HOW THE BROWSER DISPLAYS IT

Every methodology below attacks these problems with the same ideas: **name components uniquely**, **keep specificity flat and low**, **separate concerns into layers**, **reuse values through tokens**, and **colocate styles with the component** so they can be deleted with it.

18.2 Methodologies

18.2.1 BEM (Block, Element, Modifier)

BEM, created at Yandex, is the most widely used naming convention. A **Block** is an independent component (`.card`); an **Element** is a part that only makes sense inside its block, joined with two underscores (`.card__title`); a **Modifier** is a variant or state, joined with two hyphens (`.card--featured`, `.card__button--disabled`). Every selector is a single class, so specificity is always (0,1,0) and styles never depend on HTML nesting.



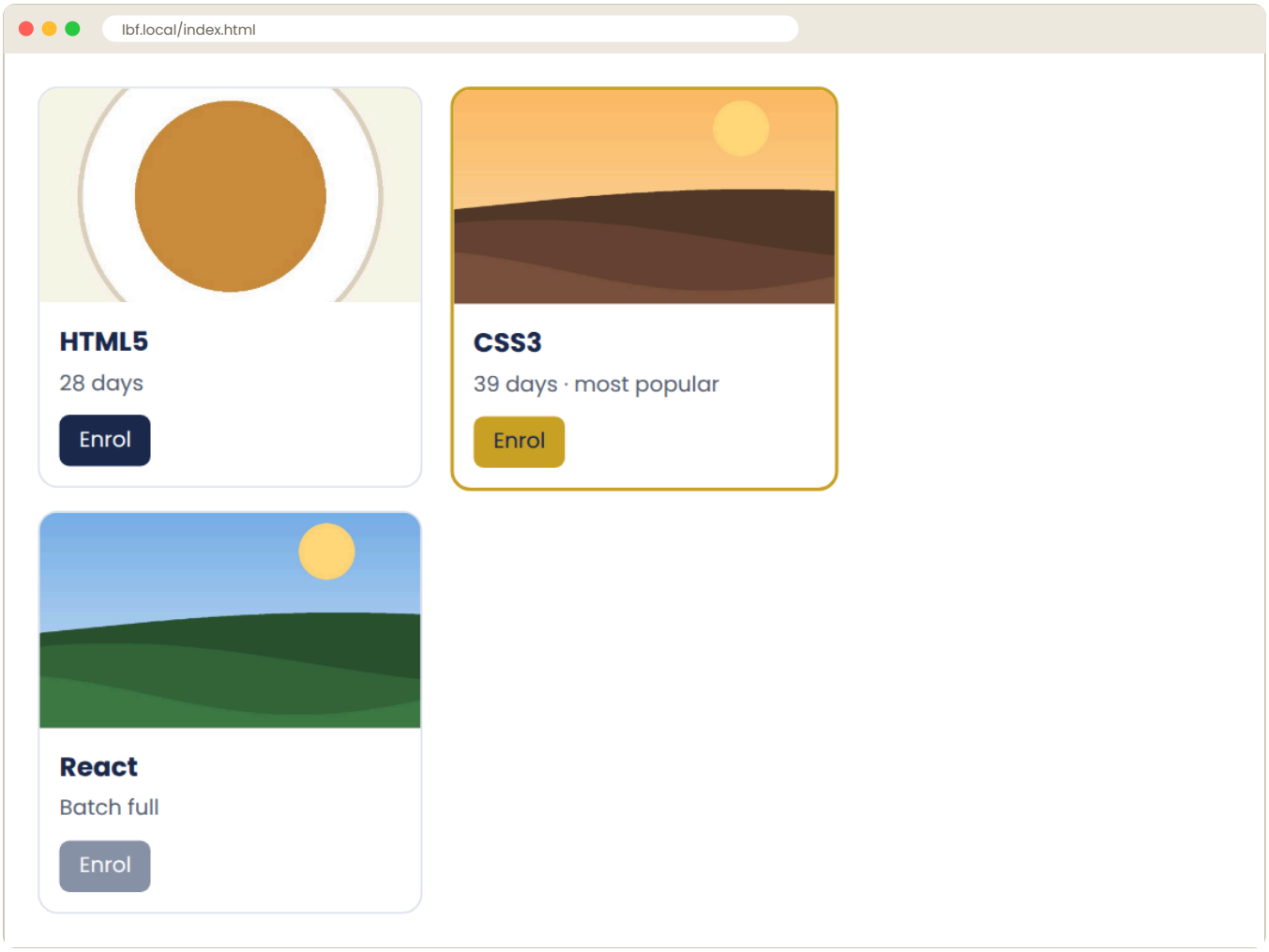
THE THREE PARTS OF A BEM NAME

CARD.CSS (BEM) CSS

```css
.card { background: #fff; border: 1px solid #dde2ee; border-radius: 12px; overflow: hidden; width: 230px; display: inline-block; vertical-align: top; margin: 6px; font-family: "Poppins", sans-serif; }
.card__image { width: 100%; aspect-ratio: 16 / 9; object-fit: cover; display: block; }
.card__body { padding: 12px; }
.card__title { margin: 0 0 4px; font-size: 16px; color: #1a2a4f; }
.card__text { margin: 0 0 10px; font-size: 13px; color: #5c6475; }
.card__button { display: inline-block; padding: 6px 12px; border-radius: 6px; background: #1a2a4f; color: #fff; text-decoration: none; font-size: 13px; }
.card--featured { border: 2px solid #c9a227; }
.card--featured .card__button { background: #c9a227; color: #1a2a4f; }   /* modifier affecting an element */
.card__button--disabled { opacity: .5; pointer-events: none; }
```

CARD.HTML HTML

```html
<article class="card">
  <img class="card__image" src="food-dosa.jpg" alt="">
  <div class="card__body">
    <h3 class="card__title">HTML5</h3>
    <p class="card__text">28 days</p>
    <a class="card__button" href="#">Enrol</a>
  </div>
</article>
<article class="card card--featured">
  <img class="card__image" src="mysuru-palace.jpg" alt="">
  <div class="card__body">
    <h3 class="card__title">CSS3</h3>
    <p class="card__text">39 days · most popular</p>
    <a class="card__button" href="#">Enrol</a>
  </div>
</article>
<article class="card">
  <img class="card__image" src="hills.jpg" alt="">
  <div class="card__body">
    <h3 class="card__title">React</h3>
    <p class="card__text">Batch full</p>
    <a class="card__button card__button--disabled" href="#">Enrol</a>
  </div>
</article>
```



A BLOCK WITH ELEMENTS AND MODIFIERS

BEM rule	Example	Why
Elements belong to a block, never to another element	`.card__title`, not `.card__body__title`	keeps names short; the DOM can change
A modifier is used together with the base class	`class="card card--featured"`	the modifier only holds the differences
No type or id selectors, no deep nesting	`.card__title`, not `.card h3`	flat (0,1,0) specificity
Nested blocks are separate blocks	a `.button` inside a card is `class="button card__button"` (a "mix")	components stay reusable
State classes may use is-/has-	`.card.is-loading`	common in JavaScript-driven states

18.2.2 OOCSS and SMACSS (overview)

OOCSS (Object-Oriented CSS, Nicole Sullivan, 2009) introduced two principles that most later methods inherit: **separate structure from skin** (layout and dimensions in one class, colours and borders in another, so they can be combined) and **separate container from content** (a heading looks the same wherever it appears; never style `.sidebar h3`). Its famous example is the **media object**, an image next to text, reused across Facebook.

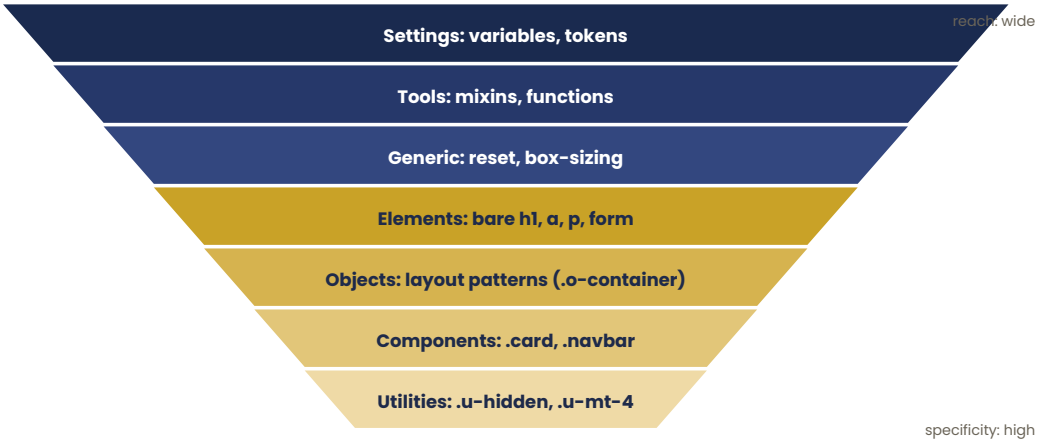
STRUCTURE VS SKIN CSS

```
/* OOCSS: structure and skin as separate, combinable classes */
.btn        { display: inline-block; padding: .5rem 1rem; border-radius: 6px; }   /* structure */
.btn-primary{ background: #1a2a4f; color: #fff; }                                 /* skin      */
.btn-ghost  { background: transparent; border: 1px solid currentColor; }          /* skin      */
/* <a class="btn btn-primary">  <a class="btn btn-ghost"> */
```

SMACSS (Scalable and Modular Architecture for CSS, Jonathan Snook, 2011) sorts rules into five categories with naming hints: **Base** (element defaults), **Layout** (`.l-header`, `.l-grid`), **Module** (reusable components), **State** (`.is-active`, `.is-hidden`) and **Theme**.

18.2.3 ITCSS layering

ITCSS (Inverted Triangle CSS, Harry Roberts) orders the whole codebase from **generic, far-reaching, low-specificity** rules at the top to **explicit, narrow, high-specificity** rules at the bottom. Because each layer only overrides the previous ones, conflicts disappear. It maps directly onto native `@layer` (Module 16).



THE ITCSS INVERTED TRIANGLE: GENERIC TO EXPLICIT

18.2.4 Utility-first CSS (bridge to Tailwind CSS)

Utility-first CSS flips the approach: instead of writing a new class for each component, you compose designs from many tiny single-purpose classes in the HTML (`p-4`, `rounded-lg`, `text-sm`, `bg-blue-600`). There is almost no custom CSS to name, organise or delete; the design system lives in the utilities' fixed scales; and specificity is always flat. The cost is long class lists in markup, which component frameworks (React, Vue, Angular) mitigate by keeping the markup in one reusable component. **Tailwind CSS** generates exactly the utilities a project uses, from a configurable token set (LBF TailwindCSS course).

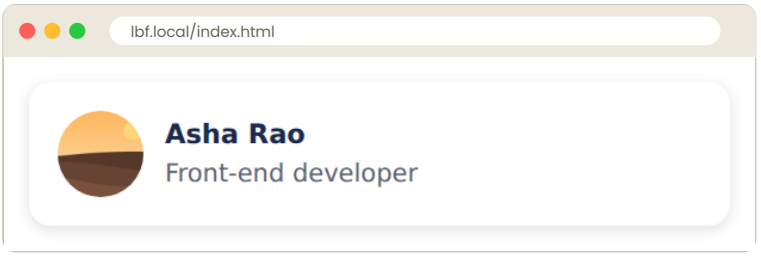
A TINY UTILITY SET (TAILWIND-STYLE NAMES) CSS

```
.flex { display: flex; } .items-center { align-items: center; } .gap-3 { gap: .75rem; }
.p-4 { padding: 1rem; } .rounded-xl { border-radius: .75rem; } .shadow { box-shadow: 0 2px 8px rgb(0 0
0 / .12); }
.bg-white { background: #fff; } .text-sm { font-size: .875rem; } .font-bold { font-weight: 700; }
.text-navy { color: #1a2a4f; } .text-muted { color: #5c6475; } .w-12 { width: 3rem; } .h-12 { height:
3rem; }
.rounded-full { border-radius: 9999px; } .object-cover { object-fit: cover; } .max-w-sm { max-width:
24rem; }
```

INDEX.HTML

HTML

```
<div class="flex items-center gap-3 p-4 rounded-xl shadow bg-white max-w-sm">
  <img class="w-12 h-12 rounded-full object-cover" src="mysuru-palace.jpg" alt="">
  <div>
    <p class="font-bold text-navy" style="margin:0">Asha Rao</p>
    <p class="text-sm text-muted" style="margin:0">Front-end developer</p>
  </div>
</div>
```



A COMPONENT COMPOSED ENTIRELY FROM UTILITY CLASSES

	Component classes (BEM)	Utility-first
Where styling lives	in CSS files	in the HTML class attribute
Naming effort	a name for every part	almost none
CSS size over time	grows with every feature	stays roughly constant
Consistency	by discipline (tokens)	enforced by the fixed scales
Readability of HTML	clean	long class lists
Best with	any stack, content sites	component frameworks, design systems

18.3 Organising files

18.3.1 Folder structure: base, layout, components, utilities

Split the CSS into small files by role, imported in cascade order into one entry file (with `@import ... layer()` natively, Sass partials, or a bundler such as Vite that combines and minifies them for production; Module 19). One component per file, named after the block, makes dead code easy to find and delete.

A LAYERED CSS FOLDER STRUCTURE text

```
css/
├── main.css                # entry: declares layers and imports everything
├── settings/
│   └── tokens.css          # colours, spacing, typography, shadows
├── base/
│   ├── reset.css
│   ├── typography.css      # h1-h6, p, a, lists
│   └── forms.css
├── layout/
│   ├── container.css
│   ├── grid.css
│   └── site-header.css
├── components/
│   ├── button.css
│   ├── card.css
│   ├── navbar.css
│   └── modal.css
└── utilities/
    └── utilities.css       # .visually-hidden, .flow > * + *, .text-center
```

MAIN.CSS CSS

```css
/* main.css */
@layer settings, base, layout, components, utilities;

@import url("settings/tokens.css")     layer(settings);
@import url("base/reset.css")          layer(base);
@import url("base/typography.css")     layer(base);
@import url("layout/container.css")    layer(layout);
@import url("components/button.css")   layer(components);
@import url("components/card.css")     layer(components);
@import url("utilities/utilities.css") layer(utilities);
/* In production a bundler inlines these imports into one file. */
```

18.3.2 Design systems and design tokens

A **design system** is the single source of truth for a product's UI: principles, tokens (Module 15), components with their states and variants, usage guidelines, accessibility rules and code. Well-known public examples are Google's Material Design, IBM Carbon, Atlassian Design System, GitHub Primer and the GOV.UK Design System. Tokens connect design and code: designers change a token value in Figma, a pipeline exports it, and every component updates.



FROM DESIGN TOKENS TO CODE

TOKENS.JSON (STYLE DICTIONARY FORMAT) text

```
{
  "color": {
    "brand":   { "value": "#1a2a4f" },
    "accent":  { "value": "#c9a227" },
    "text":    { "value": "{color.brand.value}" }
  },
  "space": { "sm": { "value": "8px" }, "md": { "value": "16px" }, "lg": { "value": "24px" } },
  "radius": { "card": { "value": "12px" } }
}
```

GENERATED TOKENS.CSS CSS

```
:root {
  --color-brand: #1a2a4f;
  --color-accent: #c9a227;
  --color-text: #1a2a4f;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --radius-card: 12px;
}
```

18.4 Scoped styles in frameworks (overview)

Component frameworks solve global scope automatically by **scoping** each component's styles to that component. Three approaches dominate:

18.4.1 CSS Modules

A **CSS Module** is a CSS file (named `*.module.css`) whose class names are made **locally unique** by the build tool (Vite, webpack, Next.js). In JavaScript you import the file as an object that maps your class names to the generated ones, so `.title` in two components can never clash.

CARD.MODULE.CSS CSS

```
/* Card.module.css */
.card { border-radius: 12px; padding: 1rem; }
.title { color: #1a2a4f; }
```

USING IT IN A COMPONENT main.js

```
// Card.jsx (React)
import styles from "./Card.module.css";

export function Card({ title }) {
  return <article className={styles.card}><h3 className={styles.title}>{title}</h3></article>;
}
```

RENDERED HTML AND GENERATED CSS (VITE, REPRESENTATIVE)

```
<article class="_card_x7f2k_1"><h3 class="_title_x7f2k_5">CSS3</h3></article>

/* generated CSS */
._card_x7f2k_1 { border-radius: 12px; padding: 1rem; }
._title_x7f2k_5 { color: #1a2a4f; }
```

18.4.2 CSS-in-JS

CSS-in-JS libraries (styled-components, Emotion) write styles inside JavaScript, often as tagged template literals, and inject them with generated class names at run time or build time. Styles can use component props directly. Run-time CSS-in-JS adds JavaScript work on every render and does not suit React Server Components, so newer libraries (vanilla-extract, Panda CSS, StyleX, Linaria) extract static CSS at build time.

STYLED-COMPONENTS JavaScript

```
import styled from "styled-components";

const Button = styled.button`
  padding: .5rem 1rem;
  border-radius: 8px;
  background: ${props => (props.$primary ? "#1a2a4f" : "transparent")};
  color: ${props => (props.$primary ? "#fff" : "#1a2a4f")};
`;
// <Button $primary>Enrol</Button>
```

18.4.3 Scoped styles in Vue and Angular

Vue single-file components support `<style scoped>`: the compiler adds a unique data attribute (such as `data-v-7ba5bd90`) to the component's elements and rewrites selectors to require it. Angular scopes component styles by default (**emulated** view encapsulation) with generated `_ngcontent-xxx` attributes; it can also use real Shadow DOM or no encapsulation. Both offer escape hatches for styling child components (`:deep()` in Vue, `::ng-deep` in Angular, the latter deprecated).

VUE SCOPED STYLES index.html

```
<!-- Card.vue -->
<template>
  <article class="card"><h3 class="title">{{ title }}</h3></article>
</template>

<style scoped>
.card { border-radius: 12px; }
.title { color: #1a2a4f; }
</style>
```

COMPILED OUTPUT (REPRESENTATIVE)

```
<article data-v-7ba5bd90 class="card"><h3 data-v-7ba5bd90 class="title">CSS3</h3></article>

.card[data-v-7ba5bd90] { border-radius: 12px; }
.title[data-v-7ba5bd90] { color: #1a2a4f; }
```

Approach	Scoping mechanism	Used by
BEM / naming conventions	discipline: unique names	any project
@layer, @scope (native)	cascade control in the browser	modern plain CSS
CSS Modules	generated unique class names at build time	React, Next.js, Vite projects
CSS-in-JS	generated classes, styles in JS	React (styled-components, Emotion)
Vue scoped / Angular emulated	generated attributes	Vue, Angular

Approach	Scoping mechanism	Used by
Shadow DOM	true encapsulation in the browser	Web Components, Angular ShadowDom
Utility-first	no custom selectors to collide	Tailwind CSS projects

PRACTICE CHECKPOINT

Refactor the portfolio site's CSS into a **BEM-named, layered file structure** with **design tokens**.

BEFORE CSS

```
/* BEFORE: one style.css, global and nested */
#header ul li a { color: white; padding: 10px; }
#header ul li a.active { color: gold; }
.projects div h3 { color: #1a2a4f; font-size: 18px; }
.projects div.featured h3 { color: red !important; }
```

AFTER: TOKENS, BEM, LAYERS CSS

```
/* settings/tokens.css */
:root { --color-brand: #1a2a4f; --color-accent: #c9a227; --space-2: .5rem; --space-3: .75rem; --text-lg: 1.125rem; }

/* components/site-nav.css */
.site-nav__link { color: #fff; padding: var(--space-2) var(--space-3); }
.site-nav__link--active { color: var(--color-accent); }

/* components/project-card.css */
.project-card__title { color: var(--color-brand); font-size: var(--text-lg); }
.project-card--featured .project-card__title { color: var(--color-accent); }

/* main.css */
@layer settings, base, layout, components, utilities;
@import url("settings/tokens.css") layer(settings);
@import url("components/site-nav.css") layer(components);
@import url("components/project-card.css") layer(components);
```

Checklist for the refactor: no id selectors, no !important (except in utilities), no selector longer than two classes, every colour/space/size from a token, one file per block, and every file imported into a layer.

MODULE 19 · ADVANCED

Preprocessors, frameworks and tooling

Minimum duration **2 days** | Level **Advanced**

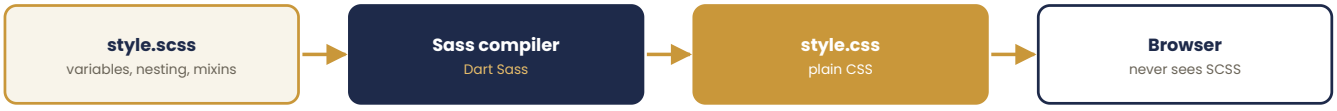
Tools extend CSS with extra features and automate tasks like prefixing and minifying. This module covers Sass/SCSS in depth (installing and compiling, variables, nesting, partials with @use and @forward, mixins, functions and operators, @extend and placeholder selectors, @if, @each and @for, and what native CSS now does instead), PostCSS with Autoprefixer and minification, using CSS with the Vite build tool, an overview of Bootstrap, Tailwind CSS and component libraries, and linting and formatting with Stylelint and Prettier. Sass and build-tool output is shown as the compiled result those tools produce; Prettier was run for real.

IN THIS MODULE

19.1 Sass/SCSS
19.2 PostCSS
19.3 Build tools
19.4 CSS frameworks (overview)
19.5 Linting and formatting

19.1 Sass/SCSS

Sass (Syntactically Awesome Style Sheets, since 2006) is a **preprocessor**: you write in an extended language and a compiler turns it into plain CSS that browsers understand. It added variables, nesting, mixins, functions, loops and modules long before CSS had any of them, and it is still widely used (Bootstrap is written in Sass). Its main syntax, **SCSS** (files ending `.scss`), is a superset of CSS: every valid CSS file is valid SCSS. The older indented syntax (`.sass`) drops braces and semicolons.



A PREPROCESSOR RUNS AT BUILD TIME

19.1.1 Installing and compiling Sass

The official implementation is **Dart Sass**, distributed on npm as `sass` (the old LibSass/node-sass is deprecated). Install it in a project, then compile once, watch for changes, or let a build tool such as Vite compile `.scss` imports automatically (19.3).

INSTALLING AND RUNNING DART SASS terminal

```
npm install --save-dev sass
npx sass scss/main.scss css/main.css                # compile once
npx sass --watch scss:css                           # recompile the folder on every save
npx sass scss/main.scss css/main.min.css --style=compressed --no-source-map
```

TERMINAL (REPRESENTATIVE)

```
added 17 packages in 3s
Compiled scss/main.scss to css/main.css.
[2026-10-08 10:42] Compiled scss/main.scss to css/main.css.
Sass is watching for changes. Press Ctrl-C to stop.
```

TIP

VS Code extensions such as "Live Sass Compiler" compile on save without a terminal, which is convenient when starting out.

19.1.2 Variables

Sass variables start with `$`. They are replaced by their values at **compile time**: the browser never sees them, so they cannot change with media queries, inherit, or be modified by JavaScript (that is what CSS custom properties are for). They are ideal for build-time constants and for computing values. `!default` sets a value only if the variable is not already set, which is how libraries let you override their settings.

STYLE.SCSS style.scss

```
$brand: #1a2a4f;
$accent: #c9a227;
$radius: 10px;
$space: 8px;
$font-stack: "Poppins", system-ui, sans-serif;
$breakpoint-md: 768px !default;

.button {
  font-family: $font-stack;
  background: $brand;
  border-radius: $radius;
  padding: $space ($space * 2);
}
```

COMPILED CSS (sass style.scss)

```
.button {
  font-family: "Poppins", system-ui, sans-serif;
  background: #1a2a4f;
  border-radius: 10px;
  padding: 8px 16px;
}
```

19.1.3 Nesting

Sass nesting works like native nesting (Module 16) with one extra power: `&` can be **concatenated** to build BEM names (`&__title` → `.card__title`), something native CSS cannot do. The compiler always outputs flat selectors.

STYLE.SCSS

style.scss

```scss
.card {
  padding: 16px;
  border-radius: 12px;

  &__title { font-size: 1.25rem; }          // .card__title
  &__text  { color: #5c6475; }              // .card__text
  &--featured { border: 2px solid gold; }   // .card--featured
  &:hover { box-shadow: 0 8px 20px rgba(0, 0, 0, .15); }

  .dark & { background: #1a2338; }          // .dark .card

  @media (min-width: 768px) {
    padding: 24px;
  }
}
```

COMPILED CSS (sass style.scss)

```css
.card {
  padding: 16px;
  border-radius: 12px;
}
.card__title {
  font-size: 1.25rem;
}
.card__text {
  color: #5c6475;
}
.card--featured {
  border: 2px solid gold;
}
.card:hover {
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.15);
}
.dark .card {
  background: #1a2338;
}
@media (min-width: 768px) {
  .card {
    padding: 24px;
  }
}
```

NOTE

Sass supports `//` single-line comments (removed from the output) as well as `/* */` (kept, except in compressed mode).

19.1.4 Partials and @use / @forward

A **partial** is a Sass file whose name starts with an underscore (`_variables.scss`); it is not compiled on its own, only loaded by other files. `@use "variables";` loads a module once and exposes its members with a **namespace** (the file name: `variables.$brand`), or with a custom one (`@use "variables" as v;` → `v.$brand`, or `as *` for none). `@forward` re-exports a module's members so one "index" file can bundle many partials. The old `@import` is deprecated in Sass (it made everything global) and will be removed in Dart Sass 3.

A SASS PROJECT STRUCTURE text

```
scss/
├── main.scss
├── abstracts/
│   ├── _index.scss        // @forward "variables"; @forward "mixins";
│   ├── _variables.scss
│   └── _mixins.scss
├── base/_reset.scss
└── components/
    ├── _button.scss
    └── _card.scss
```

MODULES WITH @USE AND @FORWARD SCSS

```scss
// abstracts/_variables.scss
$brand: #1a2a4f;
$accent: #c9a227;

// abstracts/_index.scss
@forward "variables";
@forward "mixins";

// components/_button.scss
@use "../abstracts" as a;
.button { background: a.$brand; @include a.focus-ring; }

// main.scss
@use "base/reset";
@use "components/button";
@use "components/card";
```

19.1.5 Mixins and @include

A **mixin** is a reusable block of declarations (and nested rules), optionally with parameters and default values; `@include` pastes it in. Mixins are perfect for media query shortcuts, focus styles, truncation and other repeated patterns. `@content` lets the caller pass a block of styles into the mixin.

STYLE.SCSS

style.scss

```scss
@mixin respond($min) {
  @media (min-width: $min) { @content; }
}
@mixin truncate($lines: 1) {
  @if $lines == 1 {
    white-space: nowrap; overflow: hidden; text-overflow: ellipsis;
  } @else {
    display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: $lines; overflow: hidden;
  }
}
@mixin focus-ring($color: #c9a227) {
  &:focus-visible { outline: 3px solid $color; outline-offset: 2px; }
}

.title  { @include truncate; }
.blurb  { @include truncate(3); }
.button { @include focus-ring; }
.grid {
  display: grid;
  @include respond(768px) { grid-template-columns: 1fr 1fr; }
}
```

COMPILED CSS (sass style.scss)

```css
.title {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

.blurb {
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 3;
  overflow: hidden;
}

.button:focus-visible {
  outline: 3px solid #c9a227;
  outline-offset: 2px;
}

.grid {
  display: grid;
}
@media (min-width: 768px) {
  .grid {
    grid-template-columns: 1fr 1fr;
  }
}
```

19.1.6 Functions and operators

Sass evaluates arithmetic (`+ - * / %` via `math.div()` for division), comparisons and boolean logic at compile time. Built-in modules provide functions: `sass:math` (`math.div`, `math.round`, `math.percentage`), `sass:color` (`color.adjust`, `color.scale`, `color.mix`), `sass:map`, `sass:list`, `sass:string`. You can write your own with `@function` and `@return`.

STYLE.SCSS style.scss

```scss
@use "sass:math";
@use "sass:color";

$base: 16px;
$brand: #1a2a4f;

@function rem($px) {
  @return math.div($px, $base) * 1rem;
}

.heading {
  font-size: rem(28px);
  margin-bottom: rem(12px);
  width: math.percentage(math.div(2, 3));
  color: $brand;
  border-color: color.adjust($brand, $lightness: 30%);
  background: color.mix($brand, white, 10%);
}
```

COMPILED CSS (sass style.scss)

```css
.heading {
  font-size: 1.75rem;
  margin-bottom: 0.75rem;
  width: 66.6666666667%;
  color: #1a2a4f;
  border-color: #4168c1;
  background: #e8eaed;
}
```

19.1.7 @extend and placeholder selectors

`@extend .selector;` makes one selector inherit all the rules of another by **adding it to their selector lists** in the output. A **placeholder selector** (`%name`) is never output by itself; it exists only to be extended. @extend produces smaller CSS than a mixin when many selectors share the same rules, but it can create surprising selector combinations across files and does not work inside media queries across boundaries, so many teams prefer mixins.

STYLE.SCSS style.scss

```scss
%button-base {
  display: inline-block;
  padding: 8px 16px;
  border-radius: 8px;
  font-weight: 600;
}
.btn-primary { @extend %button-base; background: #1a2a4f; color: #fff; }
.btn-ghost   { @extend %button-base; background: none; border: 1px solid currentColor; }
```

COMPILED CSS (sass style.scss)

```css
.btn-ghost, .btn-primary {
  display: inline-block;
  padding: 8px 16px;
  border-radius: 8px;
  font-weight: 600;
}

.btn-primary {
  background: #1a2a4f;
  color: #fff;
}

.btn-ghost {
  background: none;
  border: 1px solid currentColor;
}
```

19.1.8 Control directives: @if, @each, @for

Sass can generate CSS with logic: `@if` / `@else if` / `@else` for conditions, `@each` to loop over lists and maps, `@for $i from 1 through 5` for numeric ranges, and `@while`. Interpolation `#{...}` inserts values into selectors and property names. This is how frameworks generate colour variants and spacing utilities from a few lines.

STYLE.SCSS (WITH @USE "SASS:MATH" AT THE TOP)

style.scss

```scss
$colors: (primary: #1a2a4f, accent: #c9a227, danger: #c0392b);
$spacers: (0: 0, 1: 4px, 2: 8px, 3: 16px);

@each $name, $value in $colors {
  .btn-#{$name} { background: $value; }
  .text-#{$name} { color: $value; }
}

@each $key, $size in $spacers {
  .mt-#{$key} { margin-top: $size; }
}

@for $i from 1 through 3 {
  .col-#{$i} { width: math.div(100%, 3) * $i; }
}
```

COMPILED CSS (sass style.scss)

```css
.btn-primary {
  background: #1a2a4f;
}
.text-primary {
  color: #1a2a4f;
}
.btn-accent {
  background: #c9a227;
}
.text-accent {
  color: #c9a227;
}
.btn-danger {
  background: #c0392b;
}
.text-danger {
  color: #c0392b;
}

.mt-0 {
  margin-top: 0;
}
.mt-1 {
  margin-top: 4px;
}
.mt-2 {
  margin-top: 8px;
}
.mt-3 {
  margin-top: 16px;
}

.col-1 {
  width: 33.3333333333%;
}
.col-2 {
  width: 66.6666666667%;
}
.col-3 {
  width: 100%;
}
```

19.1.9 Sass vs modern native CSS

Many reasons to adopt Sass have been absorbed by CSS itself. Sass remains useful for compile-time loops and maps, mixins, module organisation and large legacy codebases; new small projects can often use plain modern CSS plus a bundler.

Need	Sass	Native CSS today
Variables	`$brand` (compile time)	`--brand` + `var()` (run time, themable, dynamic)
Nesting	yes, with `&__suffix` concatenation	yes (Baseline 2023), no concatenation
Maths	compile-time operators and functions	`calc()`, `min()`, `clamp()`, `round()`, trig: computed live
Colour manipulation	`color.adjust()`, `mix()`	`color-mix()`, relative colour syntax
Splitting files	`@use` partials, one output file	`@import` (slow at run time) or a bundler
Mixins and loops	yes	no (@mixin/@function are being specified, not shipped)
Scoping / ordering	conventions	`@layer`, `@scope`

19.2 PostCSS

PostCSS is a tool that parses CSS into a tree and runs **plugins** over it: it is not a language but a platform. Autoprefixer, cssnano, postcss-preset-env (use future CSS today), Tailwind CSS (up to v3) and many linters are PostCSS plugins. Vite, Next.js and most build tools run PostCSS automatically if a `postcss.config.js` exists.

POSTCSS.CONFIG.JS

JavaScript

```
// postcss.config.js
export default {
  plugins: {
    autoprefixer: {},
    cssnano: { preset: "default" },
  },
};
```

19.2.1 Autoprefixer

Years ago, browsers shipped experimental features behind **vendor prefixes** (`-webkit-`, `-moz-`, `-ms-`), and authors had to write every variant by hand. **Autoprefixer** adds exactly the prefixes needed for the browsers you target, using data from Can I Use, and removes outdated ones. Targets are set in a **Browserslist** query (in package.json or `.browserslistrc`), for example `"> 0.5%, last 2 versions, not dead"`.

AUTOPREFIXER WITH TARGETS "> 0.5%, LAST 2 VERSIONS, SAFARI >= 14"

style.css

```
/* input */
.title {
  user-select: none;
  background-clip: text;
  mask-image: linear-gradient(#000, transparent);
}
::placeholder { color: gray; }
```

OUTPUT (REPRESENTATIVE; EXACT PREFIXES DEPEND ON THE TARGETS)

```
.title {
  -webkit-user-select: none;
     -moz-user-select: none;
          user-select: none;
  -webkit-background-clip: text;
          background-clip: text;
  -webkit-mask-image: linear-gradient(#000, transparent);
          mask-image: linear-gradient(#000, transparent);
}
::-moz-placeholder { color: gray; }
::placeholder { color: gray; }
```

19.2.2 Minification

Minification removes whitespace, comments and redundancy (shortening colours, merging rules, dropping the last semicolon) to reduce file size. **cssnano** (a PostCSS plugin) and **Lightning CSS** (a very fast Rust-based tool used by Vite and Parcel) are the common minifiers. Combined with gzip or Brotli compression on the server, stylesheets typically shrink by 80-90 percent over the network.

BEFORE AND AFTER CSSNANO

style.css

```
/* Card component */
.card {
  background-color: #ffffff;
  margin: 0px 0px 16px 0px;
  color: rgb(26, 42, 79);
}
.card:hover { color: #1A2A4F; }
```

MINIFIED OUTPUT

```
.card{background-color:#fff;color:#1a2a4f;margin:0 0 16px}.card:hover{color:#1a2a4f}
```

19.3 Build tools

19.3.1 Using CSS with Vite

Vite is the most popular modern front-end build tool (used by Vue, React, Svelte and plain projects). In development it serves files instantly with hot module replacement: save a CSS file and the page updates without reloading. For production (`npm run build`) it bundles and minifies CSS (with Lightning CSS or esbuild), adds content hashes to file names for caching, and processes PostCSS automatically. Sass works by installing the `sass` package; CSS Modules work by naming files `*.module.css`.

CREATING A VITE PROJECT terminal

```
npm create vite@latest lbf-site -- --template vanilla
cd lbf-site
npm install
npm install --save-dev sass
npm run dev
```

TERMINAL (REPRESENTATIVE)

```
  VITE v7.1.5  ready in 312 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

MAIN.JS JavaScript

```
// main.js - CSS and SCSS are imported from JavaScript; Vite injects them
import "./styles/main.scss";
import "./styles/utilities.css";
```

PRODUCTION BUILD terminal

```
npm run build
```

TERMINAL (REPRESENTATIVE)

```
vite v7.1.5 building for production...
✓ 14 modules transformed.
dist/index.html                  0.48 kB | gzip: 0.31 kB
dist/assets/index-B3x9Kq2c.css   8.92 kB | gzip: 2.41 kB
dist/assets/index-Dk2Lw8sP.js    1.06 kB | gzip: 0.62 kB
✓ built in 412ms
```

The hashed file name (`index-B3x9Kq2c.css`) changes whenever the content changes, so the server can tell browsers to cache it for a year without users ever getting stale CSS.

19.4 CSS frameworks (overview)

19.4.1 Bootstrap: grid, components, utilities

Bootstrap (since 2011, version 5.3 current) is the most widely used component framework. It provides a 12-column responsive **grid** built on Flexbox (`container`, `row`, `col-md-6`), ready-made **components** (navbar, buttons, cards, modals, forms, alerts, dropdowns; some need its JavaScript), and **utility classes** for spacing, display and colour (`mt-3`, `d-flex`, `text-center`). It is customised by overriding its Sass variables or CSS custom properties. It is fast for admin panels and prototypes, but sites look "Bootstrap-like" unless themed.

A BOOTSTRAP CARD GRID HTML

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet">

<div class="container py-4">
  <div class="row g-3">
    <div class="col-12 col-md-6 col-lg-4">
      <div class="card h-100 shadow-sm">
        <img src="css3.jpg" class="card-img-top" alt="">
        <div class="card-body d-flex flex-column">
          <h5 class="card-title">CSS3</h5>
          <p class="card-text text-muted">39 days</p>
          <a href="#" class="btn btn-primary mt-auto">Enrol</a>
        </div>
      </div>
    </div>
  </div>
</div>
```

19.4.2 Tailwind CSS: utility-first approach

Tailwind CSS (version 4 since 2025) provides thousands of utilities generated from a design-token theme; it scans your templates and outputs only the classes you use. Version 4 is configured in CSS (`@import "tailwindcss";` and `@theme { --color-brand: ...; }`), uses cascade layers and modern colours (oklch), and integrates with Vite through a plugin. Responsive and state variants are prefixes: `md:grid-cols-3`, `hover:bg-blue-700`, `dark:bg-slate-900`. The LBF TailwindCSS course covers it fully.

THE SAME CARD WITH TAILWIND CSS HTML

```
<div class="grid gap-4 p-4 sm:grid-cols-2 lg:grid-cols-3">
  <article class="flex flex-col rounded-xl bg-white shadow-sm ring-1 ring-slate-200">
    <img class="aspect-video w-full rounded-t-xl object-cover" src="css3.jpg" alt="">
    <div class="flex flex-1 flex-col p-4">
      <h3 class="text-lg font-semibold text-slate-900">CSS3</h3>
      <p class="text-sm text-slate-500">39 days</p>
      <a href="#" class="mt-auto rounded-lg bg-indigo-600 px-4 py-2 text-center text-white hover:bg-
indigo-700">Enrol</a>
    </div>
  </article>
</div>
```

Each Tailwind class maps to plain CSS you already know. A few, as Tailwind v4 generates them:

Utility	CSS it generates (simplified)
`flex`	`display: flex`
`p-4`	`padding: calc(var(--spacing) * 4)` = 1rem
`rounded-xl`	`border-radius: var(--radius-xl)` = 0.75rem
`text-sm`	`font-size: var(--text-sm)` = 0.875rem plus a line height
`sm:grid-cols-2`	`@media (width >= 40rem) { grid-template-columns: repeat(2, minmax(0, 1fr)) }`
`hover:bg-indigo-700`	`&:hover { @media (hover: hover) { background-color: var(--color-indigo-700) } }`

19.4.3 Component libraries and when to use a framework

Component libraries go further and provide complete, accessible, interactive components for a specific framework: MUI (Material UI) and Chakra UI for React, shadcn/ui (copy-in components built on Radix and Tailwind), Vuetify and PrimeVue for Vue, Angular Material for Angular, DaisyUI and Flowbite on top of Tailwind.

Situation	Good choice
Learning CSS, portfolio projects	plain modern CSS (shows you understand the fundamentals)
Admin dashboard or internal tool, fast delivery	Bootstrap or a component library
Custom-designed product with a design system	Tailwind CSS or your own tokens + CSS Modules
React/Vue/Angular app with complex widgets (date pickers, data grids)	a component library for those widgets
Small marketing site	plain CSS or Tailwind; avoid shipping a full framework

WATCH OUT
A framework does not replace CSS knowledge: debugging layout, overriding framework styles, theming and accessibility all require understanding the cascade, box model, Flexbox and Grid.

19.5 Linting and formatting

19.5.1 Stylelint

Stylelint is the standard CSS linter: it finds errors (invalid properties, unknown units, duplicate selectors, invalid hex colours) and enforces conventions (no ids, a naming pattern for classes, a maximum nesting depth, no `!important`, lower-case hex). Start from a shared config such as `stylelint-config-standard` (or `-standard-scss`) and add your own rules; editor extensions show problems as you type, and `--fix` corrects many automatically.

.STYLELINTRC.JSON (BEM CLASS NAMES, NO IDS, NO !IMPORTANT) text

```
{
  "extends": ["stylelint-config-standard"],
  "rules": {
    "selector-max-id": 0,
    "declaration-no-important": true,
    "max-nesting-depth": 2,
    "selector-class-pattern": "^[a-z][a-z0-9]*(?:-[a-z0-9]+)*(?:__[a-z0-9]+(?:-[a-z0-9]+)*)?(?:--[a-z0-9]+(?:-[a-z0-9]+)*)?$",
    "color-hex-length": "short"
  }
}
```

RUNNING STYLELINT terminal

```
npx stylelint "src/**/*.css"
```

TERMINAL (REPRESENTATIVE)

```
src/components/card.css
   3:3   ✖  Unexpected unknown property "colr"                          property-no-unknown
   8:1   ✖  Expected selector "#sidebar a" to have no more than 0 ID selectors   selector-max-id
  12:15  ✖  Unexpected !important                                       declaration-no-important
  15:1   ✖  Expected class selector ".CardTitle" to match pattern        selector-class-pattern
  19:10  ✖  Unexpected invalid hex color "#1a2a4"                        color-no-invalid-hex

5 problems (5 errors, 0 warnings)
```

19.5.2 Prettier

Prettier formats CSS, SCSS and Less consistently: one declaration per line, consistent spacing, quotes, lower-case hex colours (when configured), and line wrapping. Use Stylelint for code-quality rules and Prettier for formatting; modern Stylelint configs no longer include stylistic rules, so they do not conflict. Prettier really ran on the messy file below:

MESSY.CSS text

```
.card{background:#FFF;padding:16px 24px;border-radius:12px;box-shadow:0 2px 4px rgba(0,0,0,.1),0 8px 24px rgba(0,0,0,.12)}
.card__title{font:700 1.25rem/1.2 "Poppins",sans-serif;color:#1A2A4F}
@media (min-width:768px){.card{padding:24px 32px}}
```

PRETTIER FORMATTING A MESSY STYLESHEET terminal

```
prettier messy.css
```

TERMINAL

```
$ prettier messy.css
.card {
  background: #fff;
  padding: 16px 24px;
  border-radius: 12px;
  box-shadow:
    0 2px 4px rgba(0, 0, 0, 0.1),
    0 8px 24px rgba(0, 0, 0, 0.12);
}
.card__title {
  font:
    700 1.25rem/1.2 "Poppins",
    sans-serif;
  color: #1a2a4f;
}
@media (min-width: 768px) {
  .card {
    padding: 24px 32px;
  }
}
```

PACKAGE.JSON SCRIPTS

text

```
{
  "scripts": {
    "lint:css": "stylelint \"src/**/*.{css,scss}\"",
    "format": "prettier --write \"src/**/*.{css,scss,html,js}\""
  }
}
```

PRACTICE CHECKPOINT

Rebuild one page's styles in **SCSS** with **partials**, **mixins** and a **loop that generates spacing utilities**, compiled through **Vite with Autoprefixer**.

SAMPLE SOLUTION: SCSS PARTIALS

SCSS

```scss
// src/styles/abstracts/_tokens.scss
$brand: #1a2a4f;
$accent: #c9a227;
$space-unit: 4px;
$spacers: (0: 0, 1: 1, 2: 2, 3: 4, 4: 6, 5: 8);     // multiples of $space-unit
$breakpoints: (md: 768px, lg: 1024px);

// src/styles/abstracts/_mixins.scss
@use "tokens" as t;
@use "sass:map";
@mixin up($bp) { @media (min-width: map.get(t.$breakpoints, $bp)) { @content; } }
@mixin focus-ring { &:focus-visible { outline: 3px solid t.$accent; outline-offset: 2px; } }

// src/styles/utilities/_spacing.scss
@use "../abstracts/tokens" as t;
@each $key, $mult in t.$spacers {
  .mt-#{$key} { margin-top: t.$space-unit * $mult; }
  .mb-#{$key} { margin-bottom: t.$space-unit * $mult; }
  .p-#{$key}  { padding: t.$space-unit * $mult; }
}

// src/styles/components/_button.scss
@use "../abstracts/tokens" as t;
@use "../abstracts/mixins" as m;
.button {
  display: inline-block; padding: 8px 16px; border-radius: 8px;
  background: t.$brand; color: #fff; user-select: none;
  @include m.focus-ring;
  @include m.up(md) { padding: 12px 24px; }
  &--accent { background: t.$accent; color: t.$brand; }
}

// src/styles/main.scss
@use "components/button";
@use "utilities/spacing";
```

VITE + AUTOPREFIXER WIRING

JavaScript

```
// postcss.config.js  (Vite runs it automatically)
export default { plugins: { autoprefixer: {} } };

// package.json
{ "browserslist": ["> 0.5%", "last 2 versions", "not dead"] }

// src/main.js
import "./styles/main.scss";
```

BUILDING

terminal

```
npm run build && head -c 400 dist/assets/index-*.css
```

TERMINAL (REPRESENTATIVE)

```
✓ built in 380ms
.button{display:inline-block;padding:8px 16px;border-radius:8px;background:#1a2a4f;color:#fff;-webkit-user-select:none;-moz-user-select:none;user-select:none}.button:focus-visible{outline:3px solid #c9a227;outline-offset:2px}@media (min-width:768px){.button{padding:12px 24px}}.button--accent{background:#c9a227;color:#1a2a4f}.mt-0{margin-top:0}.mb-0{margin-bottom:0}.p-0{padding:0}.mt-1{margin-top:4px}...
```

MODULE 20 · ADVANCED

Performance, accessibility and debugging

Minimum duration **1 day** Level **Advanced**

Professional CSS loads fast, works for everyone and behaves the same across browsers. This module covers how browsers render (style, layout, paint, composite), avoiding layout thrashing and expensive properties, critical CSS and render-blocking stylesheets, removing unused CSS, font loading performance, content-visibility and contain; accessibility in CSS (WCAG contrast ratios, visible focus styles, hiding content visually but not from screen readers, respecting user preferences, never conveying meaning with colour alone); cross-browser compatibility with vendor prefixes, progressive enhancement and fallbacks; and debugging with the DevTools Styles, Computed, Layout and Animations panels and the most common CSS bugs. Measurements were taken in Chromium.

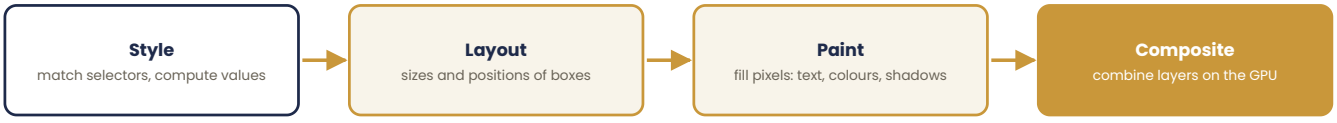
IN THIS MODULE

20.1 Performance
20.2 Accessibility in CSS
20.3 Cross-browser compatibility
20.4 Debugging CSS

20.1 Performance

20.1.1 How browsers render: style, layout, paint, composite

To turn HTML and CSS into pixels, the browser builds the DOM and the CSSOM, then runs a pipeline for every frame that changes. Knowing which stage a CSS change triggers explains why some animations are smooth and others stutter.



THE RENDERING PIPELINE FOR EACH FRAME

If you change…	The browser must redo	Cost
`width`, `height`, `margin`, `padding`, `top`, `font-size`, `display`, adding text	style → layout → paint → composite	high: may re-layout the whole page
`color`, `background`, `box-shadow`, `border-color`, `visibility`	style → paint → composite	medium
`transform`, `opacity` (on a layer), `filter` in many cases	style → composite	low: can run on the GPU off the main thread

At 60 frames per second the browser has about **16.7 ms** per frame for all of this plus JavaScript; on 120 Hz screens, about 8 ms. Frames that take longer are dropped, which users perceive as jank.

20.1.2 Avoiding layout thrashing and expensive properties

Layout thrashing happens when JavaScript alternately **writes** styles and **reads** layout values (`offsetHeight`, `getBoundingClientRect()`) in a loop: every read forces the browser to recalculate layout synchronously. Batch all reads, then all writes (or use `requestAnimationFrame`). This script measures both patterns in Chromium on 300 elements:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<div id="w"></div>
```

MEASURING LAYOUT THRASHING main.js

```
const w = document.getElementById("w");
for (let i = 0; i < 300; i++) w.insertAdjacentHTML("beforeend", '<div class="b"
style="height:10px;width:50%">b</div>');
const boxes = [...document.querySelectorAll(".b")];
function thrash() {                       // read, write, read, write...
  for (const b of boxes) { const h = b.offsetHeight; b.style.height = (h + 1) + "px"; }
}
function batched() {                      // all reads first, then all writes
  const hs = boxes.map(b => b.offsetHeight);
  boxes.forEach((b, i) => b.style.height = (hs[i] + 1) + "px");
}
function time(fn) { const t = performance.now(); for (let k = 0; k < 20; k++) fn(); return
performance.now() - t; }
const a = time(thrash), b = time(batched);
console.log("interleaved read/write:", a.toFixed(1), "ms");
console.log("batched reads, then writes:", b.toFixed(1), "ms");
console.log("batched was about", Math.round(a / b) + "x faster");
```

BROWSER CONSOLE

```
interleaved read/write: 641.3 ms
batched reads, then writes: 44.4 ms
batched was about 14x faster
```

Timings vary between machines and runs; the ratio is what matters.

- Animate `transform` and `opacity`, not `top`, `width` or `margin` (Module 14).
- Be careful with large `box-shadow`, `filter: blur()` and `backdrop-filter` on big or many elements, especially when animated or scrolled: they are expensive to paint.
- Avoid `* { transition: all ... }` and transitions of `all`: they animate layout properties you did not intend.
- Selector performance rarely matters today; DOM size and expensive paints matter much more. Keep the DOM under about 1,500 elements where possible.

20.1.3 Critical CSS and render-blocking stylesheets

Stylesheets in the head are **render-blocking**: the browser shows nothing until they have downloaded and been parsed, so users stare at a blank screen on slow connections. Strategies:

- Keep the main stylesheet small, served with compression (Brotli/gzip) and long-term caching (hashed file names, Module 19).
- Inline the **critical CSS** (the styles needed for the content visible on first load) in a `<style>` element, and load the rest without blocking. Tools such as `critical` or `beasties` extract it automatically.
- Use `media` attributes so non-matching stylesheets (print, large screens) do not block rendering.

- Avoid `@import` chains in CSS (Module 2), and preload late-discovered resources such as fonts.

CRITICAL CSS PATTERN HTML

```
<head>
  <style>/* critical CSS: header, hero, layout above the fold (a few KB) */ ...</style>

  <!-- full stylesheet without blocking first paint -->
  <link rel="preload" href="/css/main.css" as="style" onload="this.onload=null;
this.rel='stylesheet'">
  <noscript><link rel="stylesheet" href="/css/main.css"></noscript>

  <link rel="stylesheet" href="/css/print.css" media="print">   <!-- never blocks screen rendering -->
</head>
```

20.1.4 Removing unused CSS

Sites accumulate CSS that no page uses, especially when they include a whole framework. Chrome DevTools' **Coverage** panel (Ctrl+Shift+P, "Show Coverage", then reload) shows how many bytes of each stylesheet were used, with unused rules marked red in the Sources panel. Removing them can be done by hand, by deleting components' CSS together with the components (colocation, Module 18), or with tools such as **PurgeCSS**, which scans your templates for class names; Tailwind does this automatically. Be careful with class names built dynamically in JavaScript: safelist them.



THE COVERAGE PANEL

20.1.5 Font loading performance

- Use **WOFF2** and only the weights and styles you need; prefer one variable font over many static files.
- **Subset** fonts to the characters you use (Latin only, or Latin plus Devanagari) with `unicode-range` or a subsetting tool; Google Fonts does this automatically.
- `font-display: swap` shows text immediately in a fallback font; `optional` avoids layout shift entirely (Module 4).
- **Preload** the one or two most important font files: `<link rel="preload" href="/fonts/poppins-600.woff2" as="font" type="font/woff2" crossorigin>`.
- Reduce the layout shift when the web font replaces the fallback with metric overrides on a fallback `@font-face`: `size-adjust`, `ascent-override`, `descent-override`.
- Consider the **system font stack** (`system-ui`) for interfaces: zero download.

A METRIC-MATCHED FALLBACK FONT CSS

```css
@font-face {
  font-family: "Poppins Fallback";
  src: local("Arial");
  size-adjust: 112%;            /* make Arial's metrics match Poppins, so the swap does not move text */
  ascent-override: 93%;
  descent-override: 31%;
}
body { font-family: "Poppins", "Poppins Fallback", sans-serif; }
```

20.1.6 content-visibility and contain

`contain` tells the browser that an element's subtree is independent of the rest of the page (`layout`, `paint`, `size`, `style`, or `content` = layout + paint + style), so changes inside it do not trigger work outside it. `content-visibility: auto` (Baseline 2024) goes further: the browser **skips rendering** off-screen sections entirely until they approach the viewport, which can dramatically reduce initial rendering time on long pages. Pair it with `contain-intrinsic-size` to reserve an estimated height so the scrollbar does not jump.

SKIPPING OFF-SCREEN RENDERING CSS

```css
.article-section {
  content-visibility: auto;
  contain-intrinsic-size: auto 800px;   /* estimated height; "auto" remembers the real one once rendered */
}
.widget { contain: content; }           /* changes inside never relayout the page */
```

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<style>.cv { content-visibility: auto; contain-intrinsic-size: auto 500px; } section { width: 600px; }
</style>
```

MEASURING CONTENT-VISIBILITY IN CHROMIUM main.js

```javascript
function build(cls) {
  const host = document.createElement("div");
  host.innerHTML = Array.from({ length: 400 }, (_, i) =>
    `<section class="${cls}"><h2>Section ${i}</h2>${"<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. </p>".repeat(12)}</section>`).join("");
  document.body.append(host);
  const t = performance.now(); host.getBoundingClientRect(); document.body.offsetHeight; const ms = performance.now() - t;
  host.remove(); return ms;
}
const normal = build("plain"), cv = build("cv");
console.log("first layout, 400 sections, normal:                ", normal.toFixed(1), "ms");
console.log("first layout, 400 sections, content-visibility: auto:", cv.toFixed(1), "ms");
```

BROWSER CONSOLE

```
first layout, 400 sections, normal:                 56.9 ms
first layout, 400 sections, content-visibility: auto: 2.6 ms
```

Absolute timings depend on the machine; the off-screen sections with content-visibility: auto are not laid out at all.

20.2 Accessibility in CSS

20.2.1 Colour contrast (WCAG ratios)

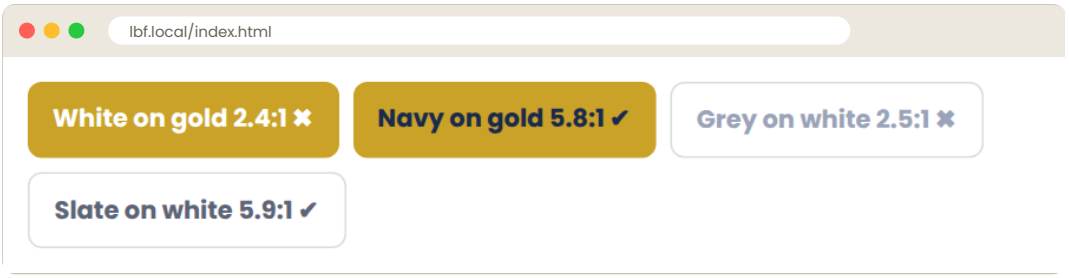
WCAG 2.2 level AA requires a contrast ratio of at least **4.5:1** for normal text, **3:1** for large text (at least 24px, or 18.66px bold) and **3:1** for meaningful non-text elements (input borders, icons, focus indicators, chart lines). AAA requires 7:1 and 4.5:1. Check with DevTools (the colour picker shows the ratio and AA/AAA lines), the Lighthouse accessibility audit, or WebAIM's contrast checker. Common failures: light grey placeholder or helper text, white text on gold or light blue buttons, text over photos without an overlay.

FAILING AND PASSING COMBINATIONS CSS

```
.row { display: flex; gap: 8px; flex-wrap: wrap; } .s { padding: 10px 14px; border-radius: 8px; font:
600 14px "Poppins", sans-serif; }
```

INDEX.HTML HTML

```
<div class="row"><span class="s" style="background:#c9a227;color:#fff">White on gold 2.4:1 ✖</span>
<span class="s" style="background:#c9a227;color:#1a2a4f">Navy on gold 5.8:1 ✔</span><span class="s"
style="background:#fff;color:#9aa3b8;border:1px solid #ddd">Grey on white 2.5:1 ✖</span><span
class="s" style="background:#fff;color:#5c6475;border:1px solid #ddd">Slate on white 5.9:1 ✔</span>
</div>
```



HOW THE BROWSER DISPLAYS IT

THE RATIOS IN THE FIGURE, COMPUTED WITH THE WCAG FORMULA main.js

```
const lum = h => h.match(/\w\w/g).map(x => parseInt(x, 16) / 255).map(v => v <= 0.03928 ? v / 12.92 :
((v + 0.055) / 1.055) ** 2.4).reduce((s, v, i) => s + v * [0.2126, 0.7152, 0.0722][i], 0);
const ratio = (a, b) => { const [x, y] = [lum(a), lum(b)].sort((m, n) => n - m); return ((x + 0.05) /
(y + 0.05)).toFixed(2); };
for (const [f, b] of [["ffffff", "c9a227"], ["1a2a4f", "c9a227"], ["9aa3b8", "ffffff"], ["5c6475",
"ffffff"]]) console.log(`#${f} on #${b}: ${ratio(f, b)}:1`);
```

BROWSER CONSOLE

```
#ffffff on #c9a227: 2.42:1
#1a2a4f on #c9a227: 5.83:1
#9aa3b8 on #ffffff: 2.53:1
#5c6475 on #ffffff: 5.94:1
```

20.2.2 Visible focus styles

Keyboard users must always see which element has focus (WCAG 2.4.7, and 2.4.11 in WCAG 2.2 for focus not obscured). Never remove outlines without a replacement. Use `:focus-visible` for a strong, consistent ring that keyboard users see and mouse users do not; make it at least 2px thick and 3:1 against adjacent colours; `outline-offset` separates it from the element. Make sure sticky headers and cookie banners do not cover the focused element (`scroll-padding`, Module 17).

A GLOBAL FOCUS STYLE CSS

```css
:focus-visible {
  outline: 3px solid #c9a227;
  outline-offset: 2px;
  border-radius: 4px;
}
/* On dark backgrounds, a double ring stays visible on any colour */
.dark :focus-visible { outline: 3px solid #fff; box-shadow: 0 0 0 6px #1a2a4f; }
```

20.2.3 Hiding content visually but not from screen readers

Some content is for screen-reader users only: a label for an icon-only button, "(opens in a new tab)", a skip link before it is focused, extra context like "Read more *about the CSS3 course*". `display: none` and `visibility: hidden` hide content from **everyone**, including assistive technology. The standard **visually-hidden** pattern keeps it in the accessibility tree:

THE VISUALLY-HIDDEN UTILITY (AS USED BY BOOTSTRAP, TAILWIND'S SR-ONLY AND MOST DESIGN SYSTEMS) CSS

```css
.visually-hidden {
  position: absolute !important;
  width: 1px !important;
  height: 1px !important;
  padding: 0 !important;
  margin: -1px !important;
  overflow: hidden !important;
  clip-path: inset(50%) !important;
  white-space: nowrap !important;
  border: 0 !important;
}
/* Becomes visible when focused (skip links) */
.visually-hidden-focusable:not(:focus):not(:focus-within) { /* same declarations as above */ }
```

WHAT A SCREEN READER GETS: VISUALLY-HIDDEN VS DISPLAY: NONE index.html

```html
<style>.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden; clip-path:
inset(50%); white-space: nowrap; }
.hidden { display: none; }</style>
<button><svg aria-hidden="true" width="16" height="16"></svg><span class="visually-hidden">Search
courses</span></button>
<button><svg aria-hidden="true" width="16" height="16"></svg><span class="hidden">Search
courses</span></button>
<a href="/css3">Read more<span class="visually-hidden"> about the CSS3 course</span></a>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- button "Search courses"
- button
- link "Read more about the CSS3 course":
  - /url: /css3
```

COMPUTED IN CHROMIUM

```
button with visually-hidden text -> innerText seen by accessibility: "Search courses"
button with display:none text     -> innerText: "" (no accessible name!)
```

20.2.4 Respecting user preferences (motion, contrast, colour scheme)

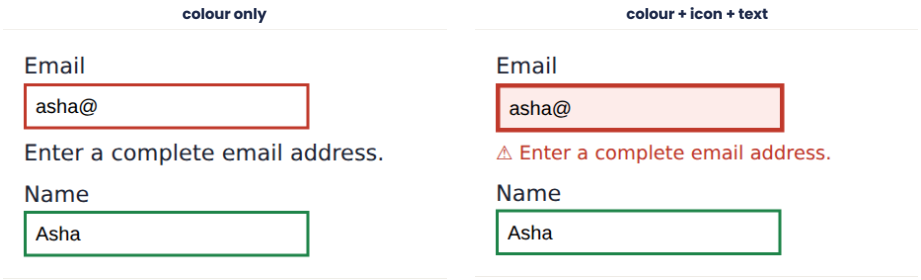
Preference	Media feature	What to do
Reduce motion	`prefers-reduced-motion: reduce`	remove parallax, large movement, autoplaying animation; keep subtle fades (Modules 12, 14)
Dark mode	`prefers-color-scheme: dark`	provide a dark theme with good contrast; set `color-scheme` (Modules 12, 15, 17)
More contrast	`prefers-contrast: more`	stronger borders and text, no translucency
Forced colours (Windows contrast themes)	`forced-colors: active`	do not rely on backgrounds or box-shadows for meaning; use real borders and system colours
Reduced transparency	`prefers-reduced-transparency: reduce`	replace glass effects with solid backgrounds (Chromium)
Text size and zoom	(user setting, no query)	use rem/em, avoid fixed heights, test at 200% zoom and 320px width (WCAG 1.4.10 reflow)

SUPPORTING FORCED COLOURS CSS

```css
@media (forced-colors: active) {
  .btn { border: 2px solid ButtonText; }          /* backgrounds may be removed: keep a border */
  .badge { forced-color-adjust: none; }           /* opt out only when colours carry meaning   */
  .icon { fill: CanvasText; }
}
```

20.2.5 Don't convey meaning with colour alone

Around 300 million people have colour vision deficiency. Red/green status, coloured chart series, "required fields in red" and links distinguished only by colour fail WCAG 1.4.1. Add a second cue: text, an icon, a pattern, an underline, a different shape or position.



THE LEFT VERSION ONLY CHANGES A BORDER COLOUR; THE RIGHT ADDS AN ICON, A THICKER BORDER AND A MESSAGE

A quick check: view the page in DevTools > Rendering > "Emulate vision deficiencies" (protanopia, deuteranopia, achromatopsia) or in greyscale (`filter: grayscale(1)` on html). If information disappears, add another cue.

20.3 Cross-browser compatibility

20.3.1 Vendor prefixes

Vendor prefixes (`-webkit-` for Chrome and Safari, `-moz-` for Firefox, `-ms-` for old Internet Explorer and Edge, `-o-` for old Opera) marked experimental implementations. Browsers now ship new features behind flags instead, so prefixes are mostly history; a few remain necessary or useful (`-webkit-background-clip:`

`text` for older Safari, `-webkit-line-clamp`, `-webkit-backdrop-filter` for older Safari, `-webkit-text-fill-color`, `-webkit-mask`). Let **Autoprefixer** (Module 19) add them from your Browserslist targets rather than writing them by hand, and always put the unprefixed property last.

20.3.2 Progressive enhancement and fallbacks

Write a solid baseline that works everywhere, then layer newer features on top. CSS makes this natural because browsers **ignore** declarations they do not understand:

FALLBACKS BY CASCADE AND @SUPPORTS CSS

```
.card {
  background: #1a2a4f;                              /* 1. fallback colour                    */
  background: oklch(30% 0.08 265);                  /* 2. ignored by old browsers            */
  width: 90%;                                       /* fallback                              */
  width: min(90%, 40rem);                           /* modern                                */
}
.grid { display: flex; flex-wrap: wrap; }           /* baseline layout                       */
@supports (display: grid) {                          /* enhancement                           */
  .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr)); }
}
.hero { min-height: 100vh; min-height: 100dvh; }     /* vh fallback, dvh where supported      */
.title { color: #1a2a4f; }                           /* readable even if the next part fails  */
@supports (background-clip: text) {
  .title { background: linear-gradient(90deg, #1a2a4f, #c9a227); background-clip: text; color:
transparent; }
}
```

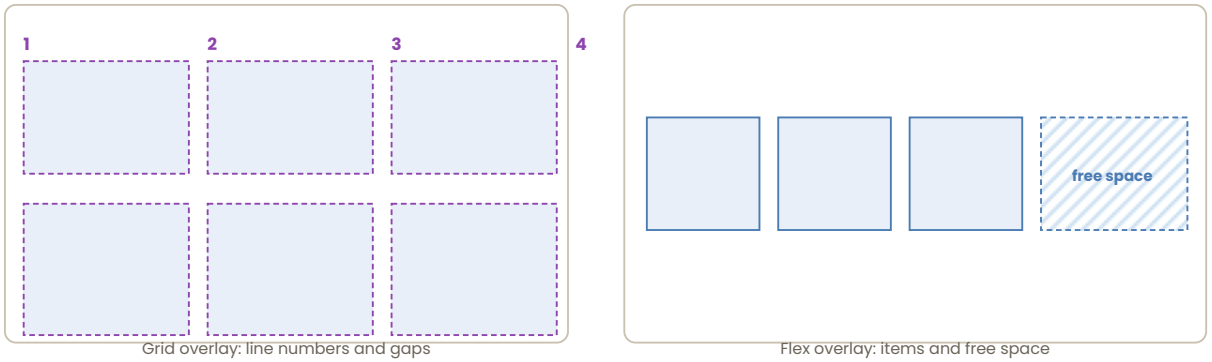
- Decide your browser support from your audience's analytics (in India, Chrome on Android and Samsung Internet dominate; check Safari on iPhone).
- Check features on Can I Use and MDN's Baseline badges (Module 17).
- Test in Chrome, Firefox and Safari (or a WebKit-based browser such as GNOME Web, or Playwright's WebKit), and on a real mid-range Android phone.
- Enhancements must not hide content: if a scroll-driven animation or a view transition is unsupported, the content must still be visible and usable.

20.4 Debugging CSS

20.4.1 DevTools: Styles, Computed, Layout (Flexbox and Grid overlays), Animations panels

Panel	Use it to
Elements > **Styles**	see every matching rule in cascade order, crossed-out overrides, toggle and edit declarations, force :hover/:focus states (:hov), add classes (.cls), find the source file and line
Elements > **Computed**	see final values and the box model diagram; expand a property to see which rule won and why; filter by name
Elements > **Layout**	list every grid and flex container; toggle overlays with line numbers, names, track sizes, gaps and free space
Animations (More tools)	capture CSS animations and transitions, replay at 10% or 25% speed, scrub the timeline, edit timings and easing curves

Panel	Use it to
Rendering	paint flashing, layout shift regions, layer borders, emulate media (print, dark mode, reduced motion, vision deficiencies)
Coverage	unused CSS bytes (20.1.4)
Performance	record a trace: long tasks, style recalculation and layout costs, dropped frames
CSS Overview (Chrome)	every colour, font, media query and unused declaration in the page; contrast issues



Grid overlay: line numbers and gaps

Flex overlay: items and free space

WHAT THE DEVTOOLS GRID AND FLEX OVERLAYS SHOW (SCHEMATIC)

20.4.2 Common bugs: overflow, collapsing margins, z-index, unexpected wrapping

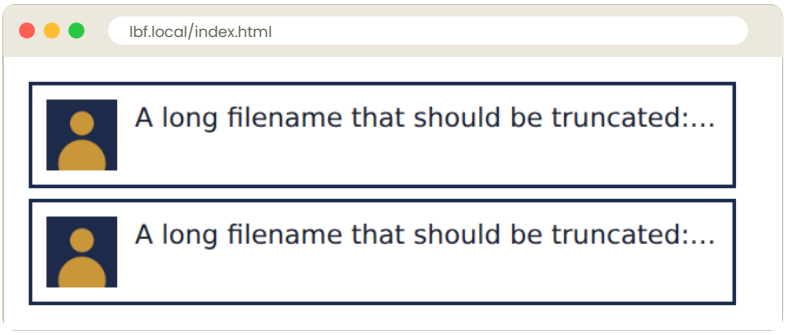
Symptom	Usual cause	Fix
Horizontal scrollbar on phones	an element wider than the viewport: fixed width, `100vw` plus a scrollbar, long URL, negative margins, an image without max-width	find it with `* { outline: 1px solid red }` or the DevTools "scroll" badge; use max-width: 100%, overflow-wrap: anywhere, min-width: 0
Unexpected gap above a section	margin collapsing: a child's margin escapes its parent (Module 5)	padding on the parent, display: flow-root, or use gap
z-index does nothing / dropdown behind content	element not positioned; or trapped in a lower stacking context created by transform, opacity or z-index on an ancestor (Module 7)	raise the ancestor, remove the context, isolation: isolate on the right component
Flex item overflows / text does not truncate	min-width: auto on flex items (cannot shrink below content)	min-width: 0 on the item
Items wrap unexpectedly	width + padding + border exceed 100% (content-box), or inline-block whitespace	box-sizing: border-box; use flex/grid with gap
Image has a small gap below it	images are inline and sit on the text baseline	display: block or vertical-align: middle
Sticky does not stick	an ancestor has overflow hidden/auto, the parent is not taller, or no top value	remove overflow or use clip, set top: 0
height: 100% has no effect	the parent has no definite height	use min-height on the chain, Flexbox/Grid stretch, or dvh
Style "not applied"	selector typo, specificity, invalid value, wrong file path, cache	check Styles for the rule and strike-throughs; hard-reload (Ctrl+Shift+R)

TWO CLASSIC BUGS AND THEIR FIXES CSS

```css
.row { display: flex; gap: 10px; width: 380px; border: 2px solid #1a2a4f; padding: 8px; margin: 6px 0;
}
.row img { width: 40px; height: 40px; }
.text { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.fixed .text { min-width: 0; }
.imgs img { border: 2px solid #c0392b; }
.imgs.fixed img { display: block; }
```

INDEX.HTML HTML

```html
<div class="row"><img src="avatar.png" alt=""><span class="text">A long filename that should be
truncated: lbf-css3-module-20-performance-notes-final.pdf</span></div>
<div class="row fixed"><img src="avatar.png" alt=""><span class="text">A long filename that should be
truncated: lbf-css3-module-20-performance-notes-final.pdf</span></div>
```



TOP: THE FLEX ITEM REFUSES TO SHRINK AND OVERFLOWS THE BOX; BOTTOM: MIN-WIDTH: 0 LETS THE ELLIPSIS WORK

PRACTICE CHECKPOINT

Run a **Lighthouse audit** on the portfolio site, fix **contrast and focus issues**, **remove unused CSS** and improve the **performance score**.

BEFORE: TYPICAL FINDINGS ON A FIRST PORTFOLIO style.css

```
DevTools > Lighthouse > Mode: Navigation > Device: Mobile > Categories: Performance, Accessibility >
Analyze page load
```

LIGHTHOUSE (REPRESENTATIVE)

```
Performance   71        Accessibility  83

Largest Contentful Paint            3.9 s     hero image not prioritised, render-blocking CSS
Total Blocking Time                 180 ms
Cumulative Layout Shift             0.14      web font swap, image without dimensions
Opportunities
  Eliminate render-blocking resources       bootstrap.min.css, main.css       saves 0.9 s
  Reduce unused CSS                         bootstrap.min.css 214 KiB unused  saves 0.6 s
  Serve images in next-gen formats          hero.jpg                           saves 0.5 s
Accessibility
  Background and foreground colors do not have a sufficient contrast ratio   (6 elements)
  [user-scalable="no"] is used in the <meta name="viewport"> element
```

Fix	CSS work
Contrast	darken helper text (#9aa3b8 → #5c6475), navy text on gold buttons instead of white, overlay on hero text

Fix	CSS work
Focus	global :focus-visible ring (20.2.2); remove any outline: none
Unused CSS	drop the full Bootstrap import, keep only the grid you used (or rewrite it with CSS Grid); run Coverage again
Render blocking	inline critical CSS, defer the rest; preload the heading font
Layout shift	width/height or aspect-ratio on images; font-display: optional or a metric-matched fallback
Viewport	remove user-scalable=no

NOTE

A realistic target after these fixes is Performance 90+ and Accessibility 100 on the mobile preset; run Lighthouse several times in an Incognito window, because scores vary by a few points between runs.

MODULE 21 · ADVANCED

Projects and interview preparation

Minimum duration **3 days** | Level **Advanced**

The final module brings everything together in portfolio projects and prepares you for front-end interviews. It specifies six capstone projects (a responsive portfolio with dark mode, a landing page in the style of a well-known product site, an admin dashboard with Grid, e-commerce listing and detail pages, a pure-CSS animation showcase and an overview of responsive email), with requirements and a rendered reference build for the key screens; practice platforms (Frontend Mentor and CSS Battle); a core-topics checklist, the common live tasks (centre a div, build a layout, make a component responsive) with model solutions, answers to frequent questions, and how to explain design decisions; and the next steps: Tailwind CSS, JavaScript and a front-end framework.

IN THIS MODULE

21.1 Capstone projects
21.2 Practice challenges
21.3 Interview preparation
21.4 Next steps

21.1 Capstone projects

Each project lists what a reviewer will check. All must be **mobile-first and responsive** from 320px to 1440px, pass keyboard and contrast checks, honour `prefers-reduced-motion`, use design tokens (custom properties), and be deployed with a live URL and a repository with a README that explains your decisions.

Project	Main skills	Days
1. Responsive portfolio with dark mode	tokens, theming, Grid, Flexbox, responsive nav, transitions	0.5
2. Landing page in the style of a product site	visual fidelity, typography, gradients, shadows, sections, animations	0.5
3. Admin dashboard with Grid	grid-template-areas, responsive grid, cards, tables, charts in CSS	0.5
4. E-commerce listing and product detail	card grids, filters layout, image galleries, forms, states	0.75
5. Pure CSS animations showcase	keyframes, transitions, 3D, scroll-driven, @property, reduced motion	0.5
6. Responsive email template (overview)	tables, inline styles, email-client limits	0.25

21.1.1 Responsive personal portfolio with dark mode

- Sections: hero with name and role, projects grid (each card with image, title, tags and links), skills, about, contact form, footer.
- Theme tokens on `:root`; dark theme via `prefers-color-scheme` **and** a toggle button that sets `data-theme` and remembers the choice (Module 15).

- Mobile hamburger navigation below 640px; sticky header with `scroll-padding-top`; smooth scrolling only without reduced-motion.
- Project grid with `repeat(auto-fit, minmax(min(260px, 100%), 1fr))`, cards lifting on hover and focus.
- Print stylesheet; Lighthouse Performance 90+ and Accessibility 100.

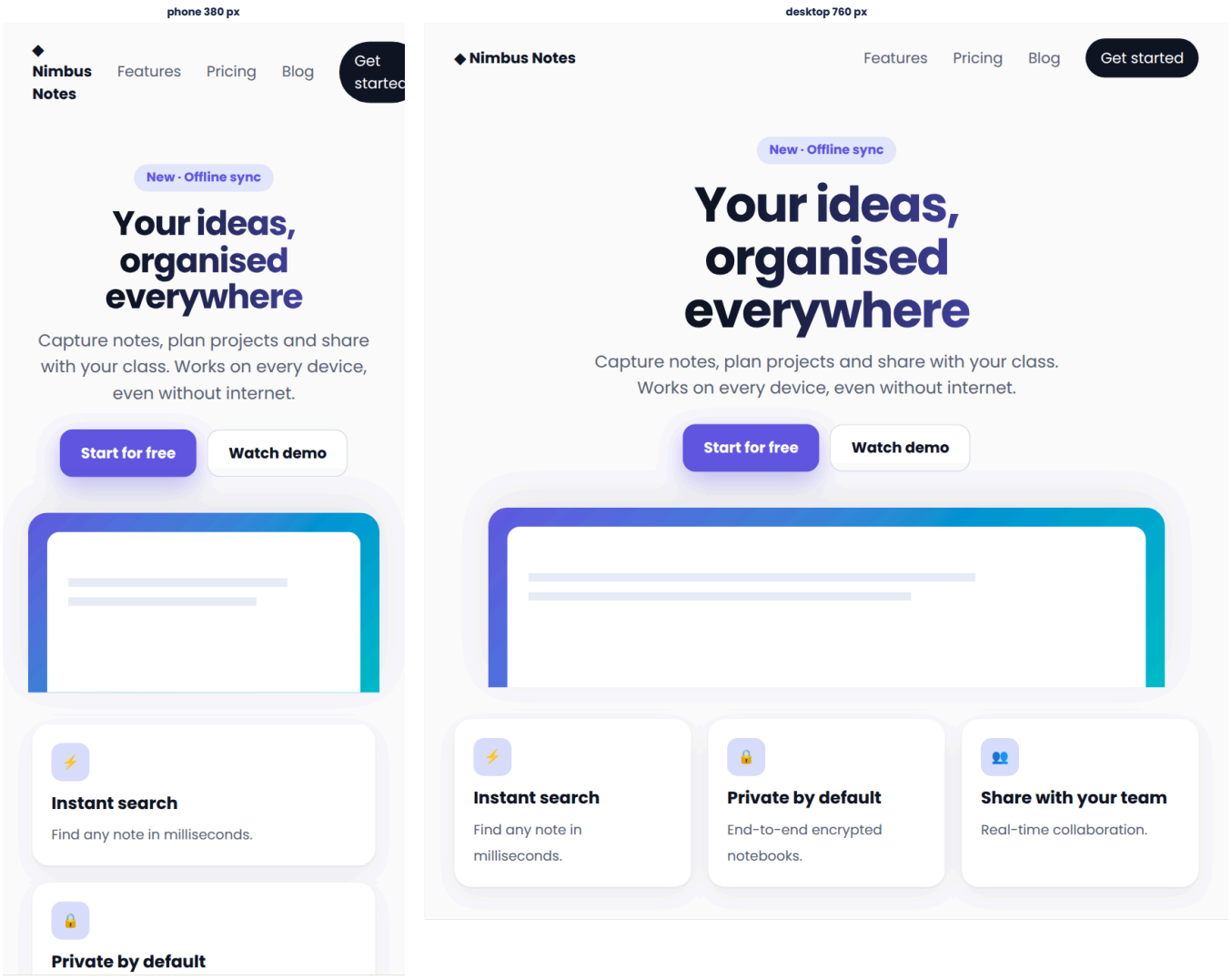
Module 12's checkpoint is a complete starting point for this project (hamburger menu, responsive grid, dark mode and print styles).

21.1.2 Landing page clone of a well-known product site

Recreating the layout of a polished product page (the kind of marketing page used by popular apps, laptops or payment services) is excellent practice in visual precision: spacing scales, type hierarchy, gradients, shadows, section rhythm and responsive behaviour. Use the real page only as a reference for layout and technique; build it with your own placeholder brand name, text and images, since brand assets and copy belong to their owners. The reference build below uses a fictional product, "Nimbus Notes":

LANDING.CSS (REFERENCE BUILD, FICTIONAL BRAND) CSS

```
:root { --ink: #0f1626; --muted: #5c6475; --brand: oklch(55% 0.2 280); --brand-2: oklch(70% 0.17 200);
--bg: #fbfbfe; }
* { box-sizing: border-box; } body { margin: 0; font-family: "Poppins", sans-serif; color: var(--ink);
background: var(--bg); }
.nav { display: flex; align-items: center; gap: 24px; padding: 14px 28px; font-size: 14px; }
.nav .logo { font-weight: 700; margin-right: auto; } .nav a { color: var(--muted); text-decoration:
none; }
.nav .cta { background: var(--ink); color: #fff; padding: 8px 14px; border-radius: 999px; }
.hero { text-align: center; padding: 40px 24px 0; }
.eyebrow { display: inline-block; padding: 4px 12px; border-radius: 999px; font-size: 12px; font-
weight: 600;
           background: color-mix(in oklch, var(--brand), white 85%); color: var(--brand); }
.hero h1 { font-size: clamp(2rem, 1rem + 4vw, 3.4rem); line-height: 1.08; letter-spacing: -0.03em;
margin: 14px auto 10px; max-width: 16ch; text-wrap: balance;
           background: linear-gradient(90deg, var(--ink) 30%, var(--brand)); -webkit-background-clip:
text; background-clip: text; color: transparent; }
.hero p { color: var(--muted); max-width: 46ch; margin: 0 auto 22px; }
.actions { display: flex; gap: 10px; justify-content: center; }
.btn { padding: 12px 20px; border-radius: 12px; font-weight: 600; text-decoration: none; font-size:
14px; }
.btn-primary { background: var(--brand); color: #fff; box-shadow: 0 8px 24px color-mix(in oklch, var(-
-brand), transparent 60%); }
.btn-secondary { background: #fff; color: var(--ink); box-shadow: inset 0 0 0 1px #dde2ee; }
.shot { margin: 34px auto 0; max-width: 640px; height: 170px; border-radius: 18px 18px 0 0;
        background: linear-gradient(135deg, var(--brand), var(--brand-2)); position: relative; box-
shadow: 0 -10px 40px rgb(0 0 0 / .08); }
.shot::before { content: ""; position: absolute; inset: 18px 18px 0; border-radius: 12px 12px 0 0;
background:
  linear-gradient(#fff 0 0) 20px 20px / 40% 10px no-repeat, linear-gradient(#e9ebf5 0 0) 20px 44px /
70% 8px no-repeat,
  linear-gradient(#e9ebf5 0 0) 20px 62px / 60% 8px no-repeat, #fff; }
.features { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 16px;
padding: 30px 28px; }
.feature { background: #fff; border-radius: 16px; padding: 18px; box-shadow: 0 1px 2px rgb(0 0 0 /
.05), 0 8px 24px rgb(15 22 38 / .06); }
.feature b { display: block; margin: 8px 0 4px; } .feature span { font-size: 13px; color: var(--
muted); }
.icon { width: 36px; height: 36px; border-radius: 10px; background: color-mix(in oklch, var(--brand),
white 80%); display: grid; place-items: center; }
```



LANDING PAGE REFERENCE BUILD

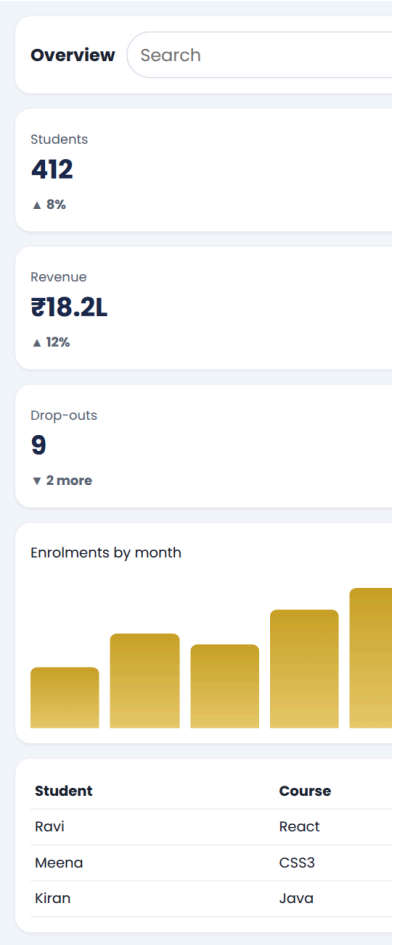
21.1.3 Admin dashboard layout with Grid

- Areas: sidebar navigation, top bar with search and profile, KPI cards, a chart area, a recent-activity table, a notifications panel.
- Desktop: `grid-template-areas`; tablet: sidebar collapses to icons; phone: single column with the sidebar as an off-canvas menu.
- KPI cards with a trend badge (green/red **and** an arrow icon); a bar chart drawn with CSS (heights from custom properties); a responsive table that scrolls horizontally on phones.
- Light and dark themes; visible focus; reduced motion.

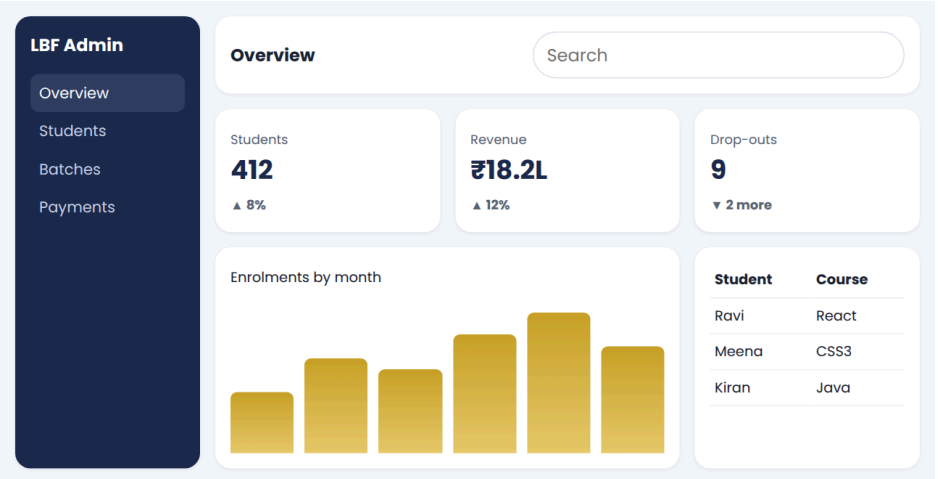
DASHBOARD.CSS (REFERENCE BUILD) CSS

```
* { box-sizing: border-box; } body { margin: 0; font-family: "Poppins", sans-serif; background:
#f4f6fb; color: #1d2433; }
.app { display: grid; min-height: 100vh; gap: 14px; padding: 14px;
  grid-template-columns: 1fr; grid-template-areas: "top" "k1" "k2" "k3" "chart" "table"; }
@media (min-width: 720px) {
  .app { grid-template-columns: 170px repeat(3, 1fr); grid-template-rows: auto auto 1fr;
         grid-template-areas: "side top top top" "side k1 k2 k3" "side chart chart table"; }
  .side { display: block !important; }
}
.app > * { background: #fff; border-radius: 14px; padding: 14px; box-shadow: 0 1px 3px rgb(26 42 79 /
.08); }
.side { grid-area: side; display: none; background: #1a2a4f !important; color: #c9d0e4; }
.side b { color: #fff; display: block; margin-bottom: 14px; } .side a { display: block; padding: 7px
8px; border-radius: 8px; color: inherit; text-decoration: none; font-size: 14px; }
.side a[aria-current] { background: rgb(255 255 255 / .1); color: #fff; }
.top { grid-area: top; display: flex; align-items: center; gap: 10px; } .top input { margin-left:
auto; padding: 8px 12px; border: 1px solid #dde2ee; border-radius: 999px; font: inherit; }
.k1 { grid-area: k1; } .k2 { grid-area: k2; } .k3 { grid-area: k3; }
.kpi span { font-size: 12px; color: #5c6475; } .kpi b { display: block; font-size: 24px; color:
#1a2a4f; }
.trend { font-size: 12px; font-weight: 600; } .up { color: #1e8449; } .down { color: #c0392b; }
.chart { grid-area: chart; } .bars { display: flex; align-items: end; gap: 10px; height: 140px;
margin-top: 10px; }
.bars i { flex: 1; height: calc(var(--v) * 1%); background: linear-gradient(#c9a227, #e7c96a); border-
radius: 6px 6px 0 0; }
.table { grid-area: table; font-size: 13px; overflow-x: auto; } table { width: 100%; border-collapse:
collapse; } td, th { text-align: left; padding: 6px 4px; border-bottom: 1px solid #eef1f8; }
```

phone 360 px



desktop 860 px



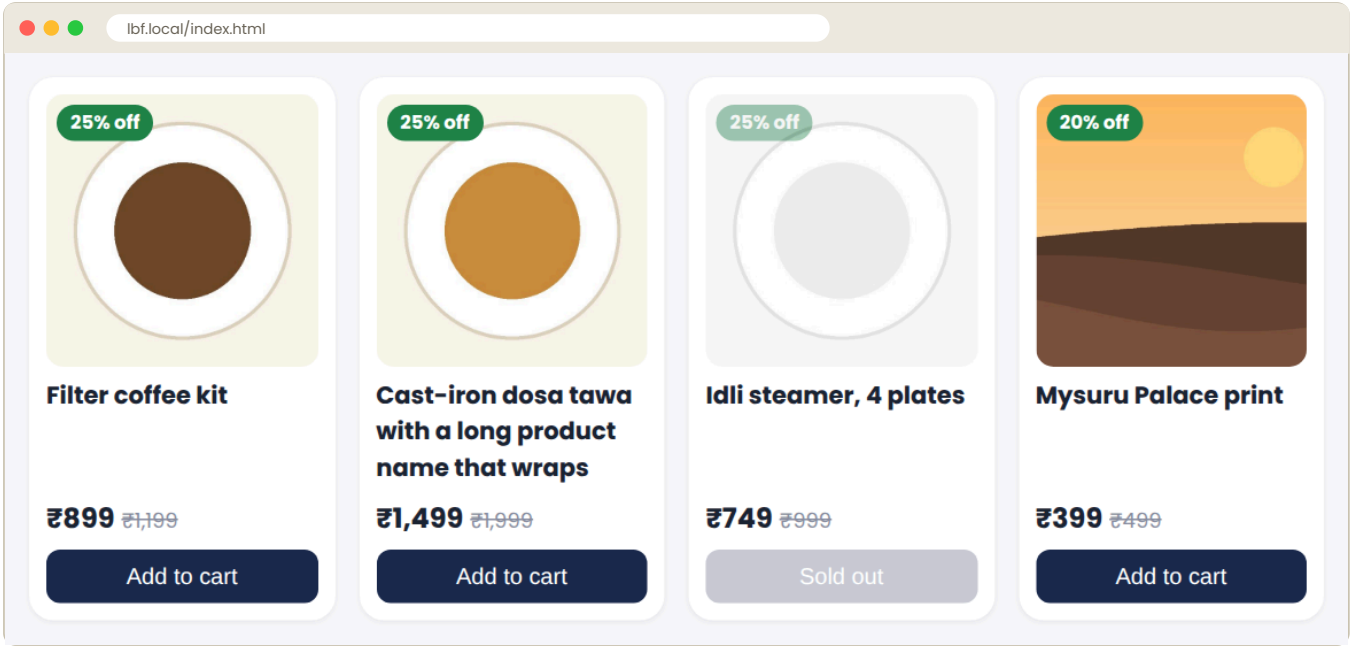
DASHBOARD: STACKED ON PHONES, NAMED GRID AREAS FROM 720PX

21.1.4 E-commerce product listing and product detail pages

- **Listing**: filters sidebar (collapsible on phones), sort bar, product grid with image, name, rating, price (with struck-through MRP and discount badge), "Add to cart" with hover and focus states, out-of-stock state via a modifier or `:has()`.
- **Detail**: image gallery (main image and thumbnails, scroll-snap on phones), title, price block, size selector built from radio buttons styled with `:checked`, quantity input, sticky "Add to cart" bar on phones (`position: sticky; bottom: 0` with safe-area padding), tabs or accordion for description and reviews.
- Consistent product-card heights with subgrid; prices with `font-variant-numeric: tabular-nums`.

LISTING.CSS (REFERENCE BUILD) CSS

```
* { box-sizing: border-box; } body { margin: 0; padding: 14px; font-family: "Poppins", sans-serif;
background: #f7f7fa; color: #1d2433; }
.grid { display: grid; gap: 14px; grid-template-columns: repeat(auto-fill, minmax(170px, 1fr)); }
.product { display: grid; grid-row: span 4; grid-template-rows: subgrid; gap: 6px; background: #fff;
border-radius: 14px; padding: 10px; position: relative;
          box-shadow: 0 1px 3px rgb(0 0 0 / .06); transition: translate .2s, box-shadow .2s; }
.product:hover, .product:focus-within { translate: 0 -3px; box-shadow: 0 10px 24px rgb(0 0 0 / .1); }
.product img { width: 100%; aspect-ratio: 1; object-fit: cover; border-radius: 10px; }
.product h3 { margin: 0; font-size: 14px; }
.price { font-variant-numeric: tabular-nums; font-weight: 700; } .price s { color: #8a8fa0; font-
weight: 400; font-size: 12px; margin-left: 4px; }
.off { position: absolute; top: 16px; left: 16px; background: #1e8449; color: #fff; font-size: 11px;
font-weight: 700; padding: 2px 8px; border-radius: 999px; }
.add { border: 0; border-radius: 8px; padding: 8px; background: #1a2a4f; color: #fff; font: 600 13px
inherit; }
.add:focus-visible { outline: 3px solid #c9a227; outline-offset: 2px; }
.product:has(.sold-out) img { filter: grayscale(1); opacity: .6; }
.product:has(.sold-out) .add { background: #c9ccd6; pointer-events: none; }
```



PRODUCT LISTING: SUBGRID KEEPS PRICES AND BUTTONS ALIGNED; THE SOLD-OUT CARD IS STYLED THROUGH :HAS()

21.1.5 Pure CSS animations showcase

A single page that demonstrates, with a short explanation beside each demo: a loader set, a button-hover collection, a 3D flip card and a cube, a typewriter heading, a marquee, staggered entrance animations, a scroll-driven progress bar and reveal, an animated gradient border with `@property`, and a

view-transition between two states. Every animation must stop or simplify under `prefers-reduced-motion: reduce`, and the page must provide a visible "Pause animations" toggle that sets `animation-play-state: paused` through a class. Modules 13, 14, 15 and 17 contain every building block.

21.1.6 Responsive email template (overview)

HTML email is its own world: many clients (notably Outlook on Windows, which renders with Microsoft Word, and some webmail apps) strip `<style>` blocks or ignore modern CSS. Professional emails therefore use **tables** for layout (no Flexbox or Grid), **inline styles** on every element (tools such as Juice or the build step of frameworks inline them), widths around 600px with fluid percentages, web-safe fonts with fallbacks, images with `alt` and explicit widths, and `@media` queries only as an enhancement for clients that support them. Frameworks such as **MJML** and **React Email** generate this markup from simpler components. Test with Litmus or Email on Acid.

THE SKELETON OF A ROBUST HTML EMAIL — HTML

```
<table role="presentation" width="100%" cellpadding="0" cellspacing="0" style="background:#f4f6fb;">
  <tr><td align="center" style="padding:24px 12px;">
    <table role="presentation" width="600" cellpadding="0" cellspacing="0" style="width:100%;max-
width:600px;background:#ffffff;border-radius:12px;">
      <tr><td style="padding:24px;font-family:Arial,Helvetica,sans-serif;color:#1a2a4f;">
        <h1 style="margin:0 0 12px;font-size:24px;">Your CSS3 batch starts Monday</h1>
        <p style="margin:0 0 20px;font-size:16px;line-height:1.5;color:#5c6475;">Bring your laptop and
the syllabus.</p>
        <a href="https://example.com/batch" style="display:inline-
block;background:#1a2a4f;color:#ffffff;padding:12px 20px;border-radius:8px;text-decoration:none;font-
weight:bold;">View batch details</a>
      </td></tr>
    </table>
  </td></tr>
</table>
```

21.2 Practice challenges

21.2.1 Frontend Mentor challenges

Frontend Mentor (frontendmentor.io) provides real-world designs (mobile and desktop mock-ups, a style guide, assets) for components and full pages, graded by difficulty from Newbie to Guru. You build them, submit a live URL and repository, and receive automated accessibility and HTML reports plus community feedback. Good starting sequence: QR code component, Blog preview card, Social links profile, Recipe page, Product preview card, Results summary, then multi-page projects. Submitted solutions make excellent portfolio entries.

21.2.2 CSS Battle

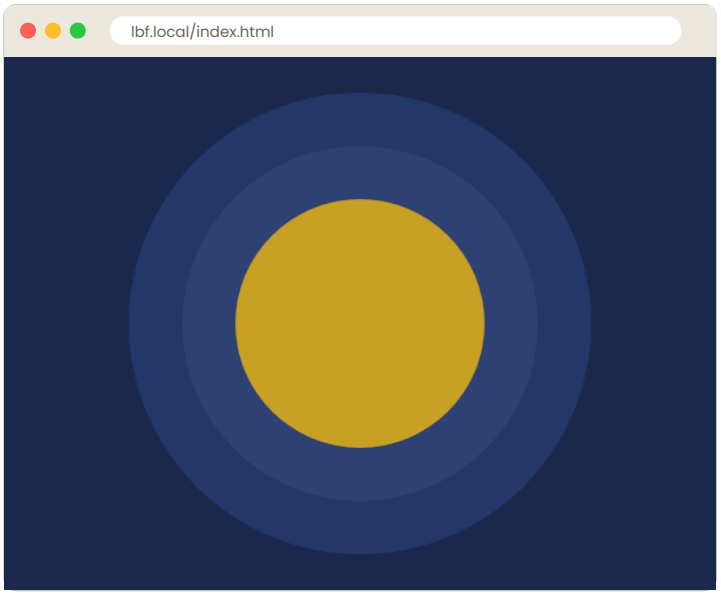
CSS Battle (cssbattle.dev) gives you a target image; you reproduce it pixel-perfectly in a 400×300 frame using HTML and CSS, scored on accuracy and on the **fewest characters**. Daily targets sharpen your understanding of positioning, box-shadow tricks, gradients, border-radius and clip-path. Code-golf solutions are deliberately unreadable; the skill they build (seeing how any shape can be made with CSS) transfers to real work.

A CSS BATTLE-STYLE TARGET IN TWO RULES CSS

```
body { margin: 0; background: #1a2a4f; width: 400px; height: 300px; display: grid; place-items:
center; }
div { width: 140px; height: 140px; border-radius: 50%; background: #c9a227;
      box-shadow: 0 0 0 30px #2e4377, 0 0 0 60px #26386a; }
```

INDEX.HTML HTML

```
<div></div>
```



THREE RINGS FROM ONE ELEMENT: BOX-SHADOW WITH SPREAD AND NO BLUR

21.3 Interview preparation

21.3.1 Core topics checklist

Topic	Be able to explain and demonstrate	Module
Box model	content/padding/border/margin, box-sizing, margin collapsing	5
Specificity and the cascade	calculating (A,B,C), origin, !important, order, @layer, :where()	8, 16
Positioning	static/relative/absolute/fixed/sticky, containing block	7
Flexbox vs Grid	one vs two dimensions, when to use each, key properties	10, 11
Responsive design	mobile-first, media vs container queries, fluid type, viewport meta	12, 16
Units	px, em, rem, %, vw/vh, dvh, ch, fr	4
Pseudo-classes vs pseudo-elements	:hover, :nth-child, :has() vs ::before, ::marker	9
z-index and stacking contexts	what creates a context, why z-index 9999 fails	7
Animations and performance	transitions vs keyframes, transform/opacity, reduced motion	13, 14, 20

Topic	Be able to explain and demonstrate	Module
Custom properties	var(), scope, theming, vs Sass variables	15, 19
Accessibility	contrast, focus-visible, visually-hidden, colour alone	20

21.3.2 Common tasks: centre a div, build a layout live, make a component responsive

Task 1: "Centre a div horizontally and vertically." Give the modern answer first, then alternatives, and say when each fits:

CENTRING ANSWERS CSS

```css
/* 1. Grid on the parent - the shortest */
.parent { display: grid; place-items: center; min-height: 100vh; }

/* 2. Flexbox on the parent */
.parent { display: flex; justify-content: center; align-items: center; }

/* 3. Absolute positioning (overlays, unknown size) */
.parent { position: relative; }
.child  { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }

/* 4. Horizontal only, block with a width */
.child { width: 300px; margin-inline: auto; }
```

Task 2: "Build this layout live: header, sidebar, main, footer; sidebar below main on phones." Talk while typing: semantic HTML first, then mobile-first CSS, then one breakpoint.

LIVE LAYOUT ANSWER CSS

```css
body { margin: 0; font-family: system-ui, sans-serif; }
.page { min-height: 100vh; display: grid; gap: 8px; padding: 8px;
        grid-template-rows: auto 1fr auto auto; grid-template-areas: "header" "main" "aside" "footer";
}
@media (min-width: 700px) {
  .page { grid-template-columns: 220px 1fr; grid-template-rows: auto 1fr auto;
          grid-template-areas: "header header" "aside main" "footer footer"; }
}
header { grid-area: header; } main { grid-area: main; } aside { grid-area: aside; } footer { grid-
area: footer; }
.page > * { background: #eef1f8; border-radius: 8px; padding: 12px; }
```



SIDEBAR AFTER MAIN ON PHONES, BESIDE IT ON DESKTOPS

Task 3: "Make this card responsive." Mention that you would use a container query so the card adapts wherever it is placed, with a media-query fallback if needed; use `aspect-ratio` and `object-fit` for the

image, `clamp()` for the heading, `min-width: 0` for text that should truncate, and test at 320px and 200% zoom.

Frequently asked questions. Interviewers often follow the live tasks with conceptual questions. Short model answers:

Q. What is the box model?
Every element is a rectangle made of content, padding, border and margin. With the default box-sizing: content-box, width sets only the content and padding and border are added; with border-box, width includes them. Vertical margins between blocks collapse to the larger one.

Q. How is specificity calculated?
As three columns (A,B,C): ids, then classes/attributes/pseudo-classes, then types/pseudo-elements, compared left to right. Inline styles beat all selectors; !important beats everything except a more specific !important. :where() is zero, :is()/:not()/:has() take their most specific argument. Ties go to the rule that comes last.

Q. Difference between display: none, visibility: hidden and opacity: 0?
display: none removes the element from layout and the accessibility tree; visibility: hidden keeps its space but hides it from sight and assistive technology; opacity: 0 keeps space and interactivity (it can still be clicked and focused).

Q. relative vs absolute vs fixed vs sticky?
relative stays in flow and shifts visually, and creates a containing block; absolute leaves the flow and positions against the nearest positioned ancestor; fixed positions against the viewport; sticky behaves as relative until a scroll threshold, then sticks within its parent.

Q. Flexbox or Grid?
Flexbox for one-dimensional, content-driven arrangements (navbars, toolbars, centring, components); Grid for two-dimensional, layout-driven structures (page layouts, card grids, dashboards). They are often nested.

Q. em vs rem?
em is relative to the element's own font size (the parent's for font-size) and compounds when nested; rem is relative to the root font size and does not compound. Use rem for font sizes and global spacing, em for component-internal spacing that should scale with the component's text.

Q. Pseudo-class vs pseudo-element?
A pseudo-class (one colon) selects an element in a state or position: :hover, :focus-visible, :nth-child(2). A pseudo-element (two colons) styles a part of an element or creates a box: ::before, ::after, ::first-line, ::placeholder.

Q. Why does z-index not work?
Either the element is not positioned (and not a flex/grid item), or it is inside a stacking context (created by an ancestor's transform, opacity below 1, filter, or positioned z-index) that is itself below the other element. z-index only competes within the same stacking context.

Q. What are CSS custom properties and how do they differ from Sass variables?
Custom properties (--x, read with var()) are real properties: they cascade, inherit, can change in media queries and states, and be read and written from JavaScript at run time. Sass variables are replaced at compile time and do not exist in the browser.

Q. What does "mobile-first" mean?
Write base styles for small screens and add complexity with min-width media queries, rather than starting from the desktop and undoing styles with max-width queries.

Q. How do you make text scale smoothly?
clamp(min, preferred, max) with a preferred value combining rem and vw (or cqi for components), e.g. font-size: clamp(1.5rem, 1rem + 3vw, 3rem).

Q. Which properties should you animate for performance?
transform (translate, rotate, scale) and opacity, which can be handled by the compositor; avoid animating width, height, top, left and margin, which trigger layout every frame.

Q. What is a BFC (block formatting context)?
An isolated layout region in which floats are contained and child margins do not collapse with the outside. Created by display: flow-root, overflow other than visible, floats, absolute positioning, inline-block, flex and grid items.

Q. What does :has() do?
It selects an element based on its descendants or following siblings, e.g. .card:has(img) or form:has(:user-invalid): effectively a parent selector, Baseline since 2023.

Q. Container queries vs media queries?
Media queries respond to the viewport; container queries respond to the size of an ancestor declared with container-type, so components adapt to wherever they are placed.

Q. How do you hide something visually but keep it for screen readers?
With a visually-hidden utility: absolute positioning, 1px size, overflow hidden, clip-path: inset(50%), white-space: nowrap. Never display: none for content screen readers need.

21.3.3 Explaining your design decisions

Interviewers care less about whether you know every property than whether you make **reasoned** choices. For each portfolio project, prepare a two-minute walkthrough that covers:

Decision	What to say (example)
Architecture	"Tokens on :root, BEM components in separate files, @layer order reset → base → components → utilities, so I never needed !important."
Layout choice	"Grid for the page and card grid with auto-fit and minmax, so it needs no breakpoints; Flexbox inside cards to push the button to the bottom."
Responsiveness	"Mobile-first, two breakpoints chosen where the content broke, a container query on the product card because it appears in both the grid and the sidebar."
Accessibility	"Contrast checked at 4.5:1, :focus-visible ring, visually-hidden labels on icon buttons, reduced-motion alternatives; Lighthouse 100."
Performance	"Animated only transform and opacity, preloaded the heading font with font-display: swap, removed 200 KB of unused framework CSS; LCP 1.6 s on mobile."
Trade-off	"I used plain CSS rather than Tailwind to show fundamentals; on a team project I would choose Tailwind for consistency and speed."

- Have the live site and DevTools ready: show the grid overlay, the cascade layers in the Styles pane, the dark mode toggle, the reduced-motion emulation.
- Mention one thing you would improve next and why.
- If you do not know an answer, reason aloud from fundamentals (box model, cascade, formatting contexts): it shows how you would debug in real work.

21.4 Next steps

21.4.1 Tailwind CSS

With solid CSS fundamentals, **Tailwind CSS** becomes a productivity multiplier: every utility maps to a property you now understand, its design tokens mirror the custom-property systems from Module 15, and its responsive and state variants are the media queries and pseudo-classes from Modules 9 and 12. The LBF TailwindCSS course is the natural next step.

21.4.2 JavaScript and a front-end framework

JavaScript adds behaviour: toggling classes and custom properties (the bridge between scripts and styles used in Modules 12 and 15), fetching data and rendering lists of components. Then a **framework** such as **React**, **Angular** or **Vue** organises the UI into components, each with its scoped styles (Module 18), where everything in this book applies directly.



THE LBF FRONT-END PATH

PRACTICE CHECKPOINT

Build and deploy **three portfolio-quality responsive projects** and complete **five Frontend Mentor challenges**.

Before calling a project done	Check
Responsive	320px to 1440px; no horizontal scroll; 200% zoom; landscape phone
Accessible	Lighthouse Accessibility 100; keyboard-only run; contrast 4.5:1; focus visible; reduced motion
Fast	Lighthouse Performance 90+ (mobile); images sized and lazy; fonts optimised; no unused framework CSS
Maintainable	tokens; BEM or utility conventions; no ids or !important in components; Stylelint and Prettier clean
Cross-browser	tested in Chrome, Firefox and Safari (or WebKit); fallbacks for newer features
Published	live URL, repository with README (screenshots, decisions, what you learned)

Congratulations on completing CSS3: Styling, Layout and Modern CSS. You can now turn any design into responsive, accessible, fast and maintainable CSS, and reason about the cascade, layout and performance like a professional front-end developer.